

# **RK-XC7Z020 LINUX 开发教程**

| 文档变更记录 |            |    |
|--------|------------|----|
| 版本     | 时间         | 描述 |
| V0.1   | 2025.04.04 | 初版 |

## 目录

|     |                                |     |
|-----|--------------------------------|-----|
| 一.  | RK 7020 Linux 教程环境配置.....      | 5   |
| 1.  | 安装虚拟机软件 VMware .....           | 5   |
| 2.  | 创建 Ubuntu 虚拟机.....             | 12  |
| 3.  | 安装 Petalinux .....             | 19  |
| 4.  | NFS 服务软件安装 .....               | 25  |
| 二.  | Petalinux 设计流程.....            | 28  |
| 1.  | Zynq-7000 嵌入式软件栈概述 .....       | 28  |
| 2.  | Petalinux 工具的设计流程概述.....       | 29  |
| 3.  | 使用 Petalinux 定制 Linux 系统 ..... | 30  |
| 4.  | 设置离线编译.....                    | 37  |
| 5.  | 设置本地内核源码以及 u-boot 源码 .....     | 43  |
| 6.  | 修改设备树、编译 petalinux 工程.....     | 50  |
| 7.  | 生成 BOOT 文件.....                | 52  |
| 8.  | 运行 Linux .....                 | 54  |
| 9.  | Petalinux 使用流程总结.....          | 56  |
| 10. | 创建和使用 BSP 包 .....              | 57  |
| 三.  | 添加开机自启动脚本.....                 | 59  |
| 1.  | 准备工作.....                      | 59  |
| 2.  | 自启动脚本.....                     | 59  |
| 四.  | 使用 Vitis 开发 Linux 应用.....      | 65  |
| 1.  | 创建基于 Vitis 的 Linux 平台工程.....   | 65  |
| 2.  | 以太网的使用.....                    | 70  |
|     | 外网连接测试（有路由器） .....             | 71  |
|     | 电脑直连测试（无路由器） .....             | 73  |
| 3.  | 通过 NFS 共享运行 .....              | 75  |
| 五.  | 程序“Hello World”交叉编译 .....      | 76  |
| 1.  | Petalinux 的交叉编译工具链.....        | 76  |
| 2.  | 编译根文件系统并构建 SDK.....            | 77  |
| 3.  | SDK 中的交叉编译工具链.....             | 80  |
| 4.  | 编写“Hello World”代码.....         | 82  |
| 5.  | 交叉编译 arm 程序 .....              | 84  |
| 六.  | GPIO 控制 LED.....               | 87  |
| 1.  | 使用 Shell 控制 LED 亮灭.....        | 88  |
| 2.  | 使用 C 语言控制 LED 的亮灭 .....        | 90  |
| 七.  | Petalinux 下的 HDMI 显示 .....     | 95  |
| 1.  | 建立 vivado 工程.....              | 95  |
| 2.  | Petalinux 配置 .....             | 104 |
| 3.  | 配置 Linux 内核 .....              | 105 |
| 4.  | 编写设备树.....                     | 109 |
| 5.  | 运行结果.....                      | 112 |
| 八.  | SD 卡根文件系统.....                 | 114 |
| 1.  | Petalinux 配置 .....             | 114 |

|     |                                      |     |
|-----|--------------------------------------|-----|
| 2.  | 制作 SD 卡文件系统.....                     | 117 |
| 3.  | 同步根文件系统到 SD 卡 EXT 分区.....            | 121 |
|     | 使用 Petalinux 工程编译后生成的根文件系统.....      | 121 |
|     | 使用 Debain 根文件系统.....                 | 123 |
| 九.  | 制作 QSPI Flash 启动的 Linux.....         | 126 |
| 1.  | Petalinux 配置.....                    | 126 |
| 2.  | 下载至 Flash 验证 Linux 启动.....           | 129 |
| 十.  | Petalinux 板载 LCD 显示 Linux 命令行实验..... | 132 |
| 1.  | 建立 vivado 工程.....                    | 132 |
| 2.  | Petalinux 中配置.....                   | 135 |
| 3.  | 编写设备树.....                           | 139 |
| 4.  | SPI LCD 显示命令行输出测试.....               | 140 |
| 十一. | Petalinux 中 USB HOST 实验.....         | 142 |
| 1.  | 建立 vivado 工程.....                    | 142 |
| 2.  | Petalinux 中配置.....                   | 143 |
| 3.  | USB HOST 功能测试.....                   | 144 |



# 一. RK 7020 Linux 教程环境配置

RK 7020 Linux 教程主要是讲解 Ubuntu 系统，包括如何在虚拟机上安装 Ubuntu 操作系统，安装好以后 Ubuntu 的设置、基本操作等等。在正式进行嵌入式开发之前肯定是要先学习下 Ubuntu 系统如何使用的，由于 Ubuntu 系统功能很庞大，如果要详细讲解的话一本书都讲不完，因此本篇只做 Ubuntu 系统入门讲解，能掌握基本的 ZYNQ Linux 开发所需的技能即可，如果想详细的学习 Ubuntu 系统的话可以参考其他的书籍，比如经典的《鸟哥的 linux 私房菜》，《鸟哥的 linux 私房菜》这本书使用的 CentOS 操作系统，但是 Ubuntu 下完全可以使用。

**特别提醒：**本教程使用的是 Ubuntu 22.04.5 LTS 64位操作系统（ubuntu-22.04.5-desktop-amd64），如果使用其他版本，可能有不可预料的错误，请保持版本一致，请不要升级系统。

Vivado与Vitis的版本均采用的是2023.1，Petalinux版本也采用的是23.1，Vivado、Vitis与Petalinux的版本都是对应的。如果采用不同的版本需要自行修改Vivado，Vitis，Petalinux的版本。

Petalinux并不是一个特殊Linux内核，而是一套开发环境配置的工具，降低Uboot、Linux内核、根文件系统的配置的工作量，可以从Vivado的导出硬件信息自动完成相关软件的配置。

这里要特意强调一下，Petalinux对系统版本和设置有严格要求，如果使用其他版本操作系统，使用中遇到问题请自行百度解决。

本章教程参考Xilinx官方文档UG1144（<https://docs.xilinx.com/r/en-US/ug1144-petalinux-tools-reference-guide/Overview>）进行安装，如果本教程安装失败，请参考Xilinx的ug1144文档进行安装。

## 1. 安装虚拟机软件 VMware

RK7020 Linux 教程主要是涉及到嵌入式 Linux 开发，一般需要一台 Linux 操作系统主机，用来编译 Linux 的 u-boot 或者 Linux-kernel。这通常需要在 Windows 与 Linux 系统下进行切换，如果是安装物理机的情况下，只能在两个系统中进行二选一进入系统。因此，在 Windows 操作系统的电脑上安装个虚拟机，再在虚拟机上安装 Linux 操作系统是最便捷的方法了。

（1）本教程使用 VMware Workstation 软件来做虚拟机。笔者使用的是 VMware Workstation Pro 15，如果读者使用 Windows11 系统，则可以下载 VMware Workstation Pro 16 或者更高版本。VMware Workstation 软件可以在 VMware 官网下载，下载地址：[Fusion and Workstation | VMware](https://www.vmware.com/cn/fusion-and-workstation)，当前最新的版本是 VMware Workstation Pro 17，下载 Windows 版本的，如下图所示：

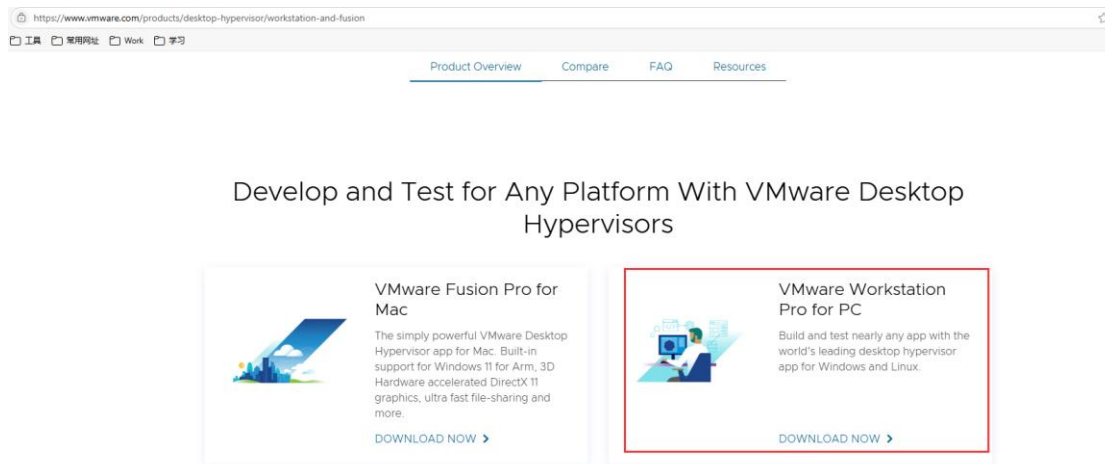


图 1-1 官网下载 VMware 软件

VMware 于 2024 年 5 月 14 日推出了桌面虚拟化平台的最新版本 Workstation Pro 17.5.2 和 Fusion Pro 13.5.2，并且宣布这两个产品对个人使用完全免费。至此可以免费安装使用 VMware，而无需使用密钥激活或者支付任何费用。

接下来以安装 VMware 15 Pro 为例，安装虚拟机环境，其他版本的虚拟机环境也可以参考该方式安装。

(2) VMware Workstation 的安装和普通软件安装一样，双击下载的 exe 文件进入安装界面，如下图所示：

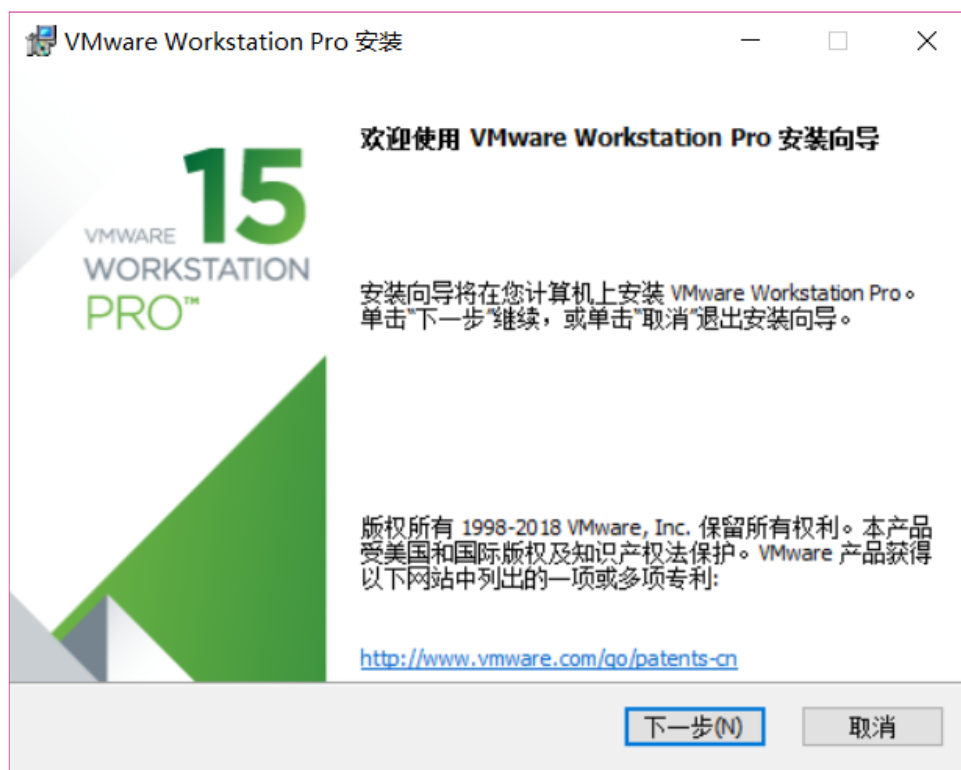


图 1-2 VMware 安装界面

(3) 点击图 1-2 中的“下一步”，进入图 1-3 所示步骤：

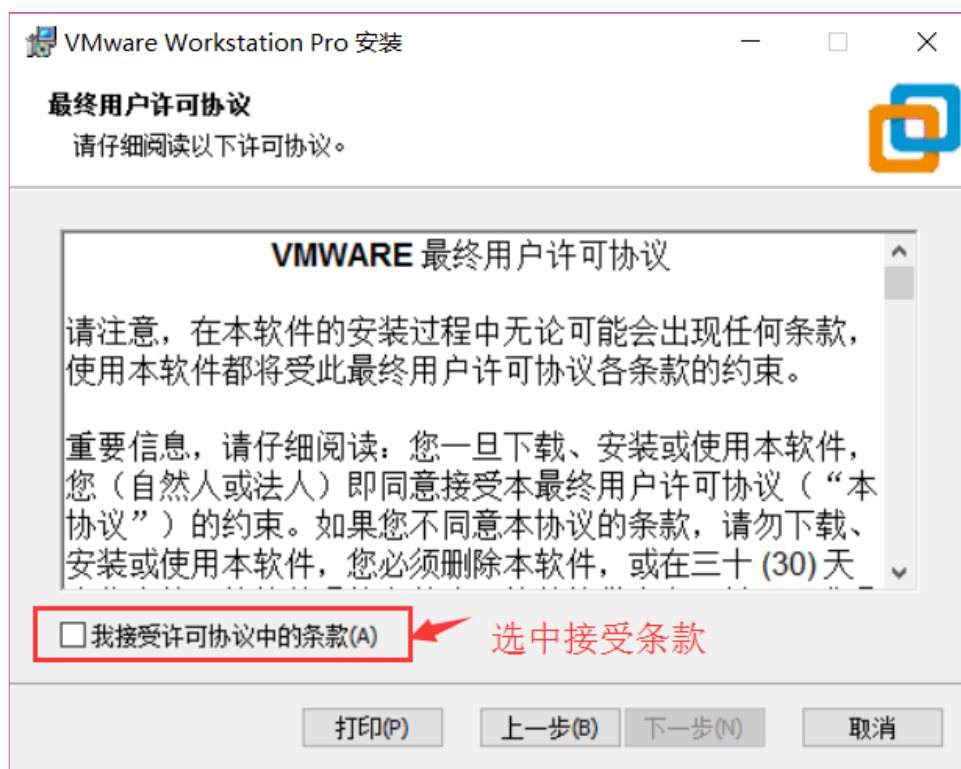


图 1-3 VMware 条款

(4) 先选择图 1-3 中的“我接受许可协议中的条款”，然后在选择“下一步”，进入图 1-4 所示步骤：

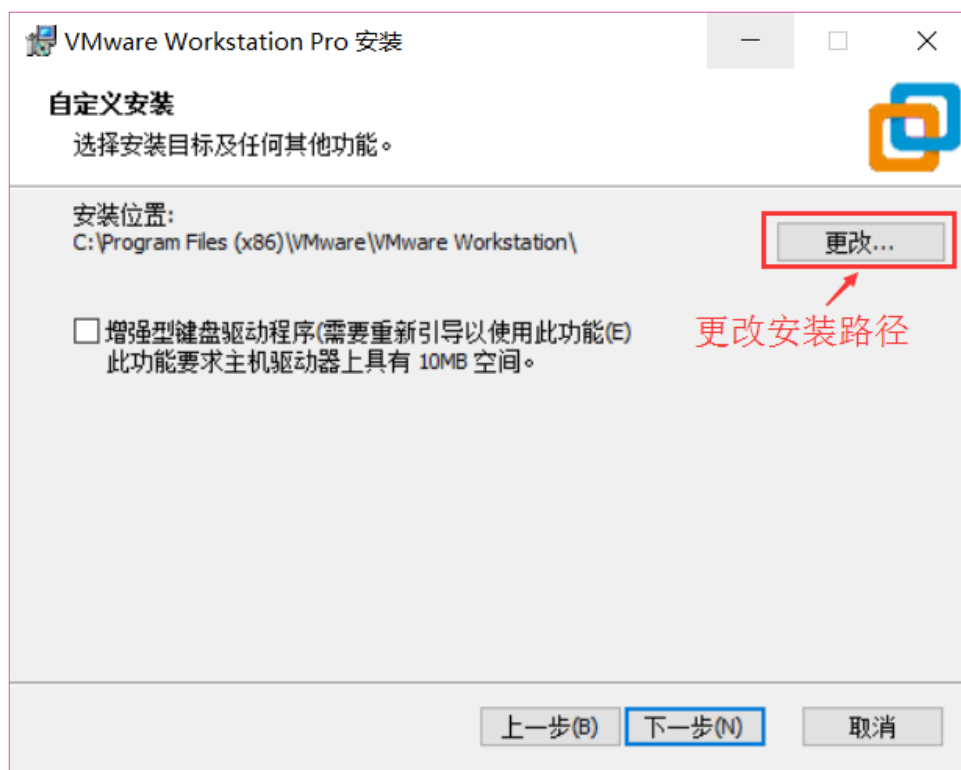


图 1-4 选择安装路径

(5) 在上图中选择软件的安装路径，点击“更改”按钮，然后根据自己的实际需要选择合适路径即可，安装路径如下图所示：

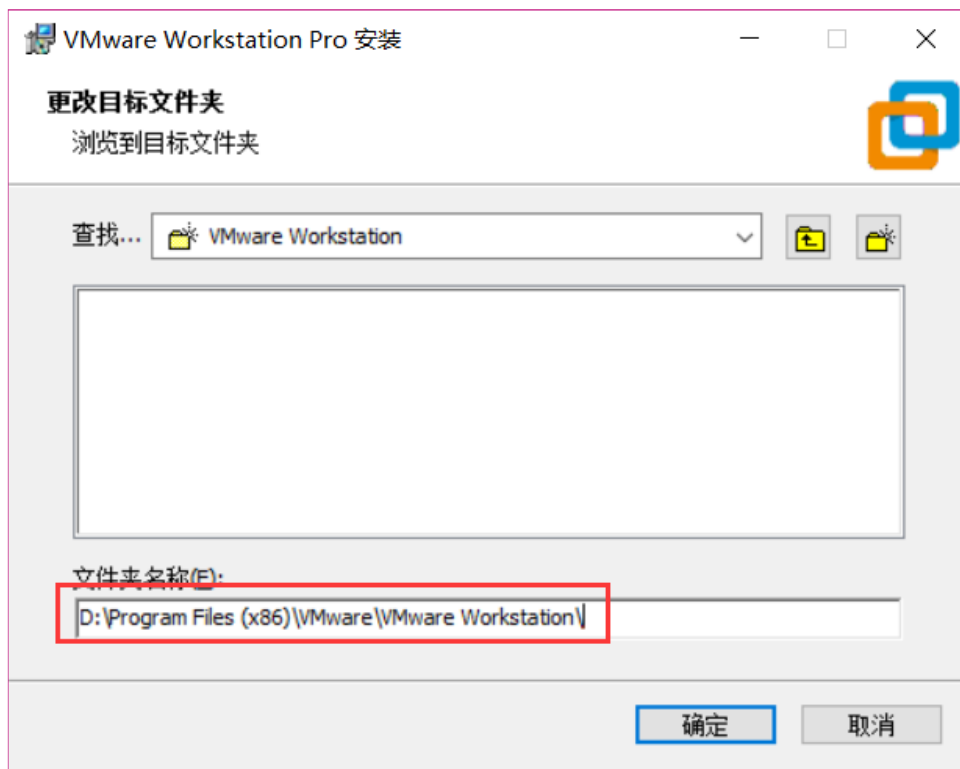


图 1-5 安装路径

(6) 选择好路径以后点击图 1-5 中的“确定”按钮，然后回到图 1-4 所示界面，点击图 1-4 中的“下一步”，进入图 1-6 所示界面：

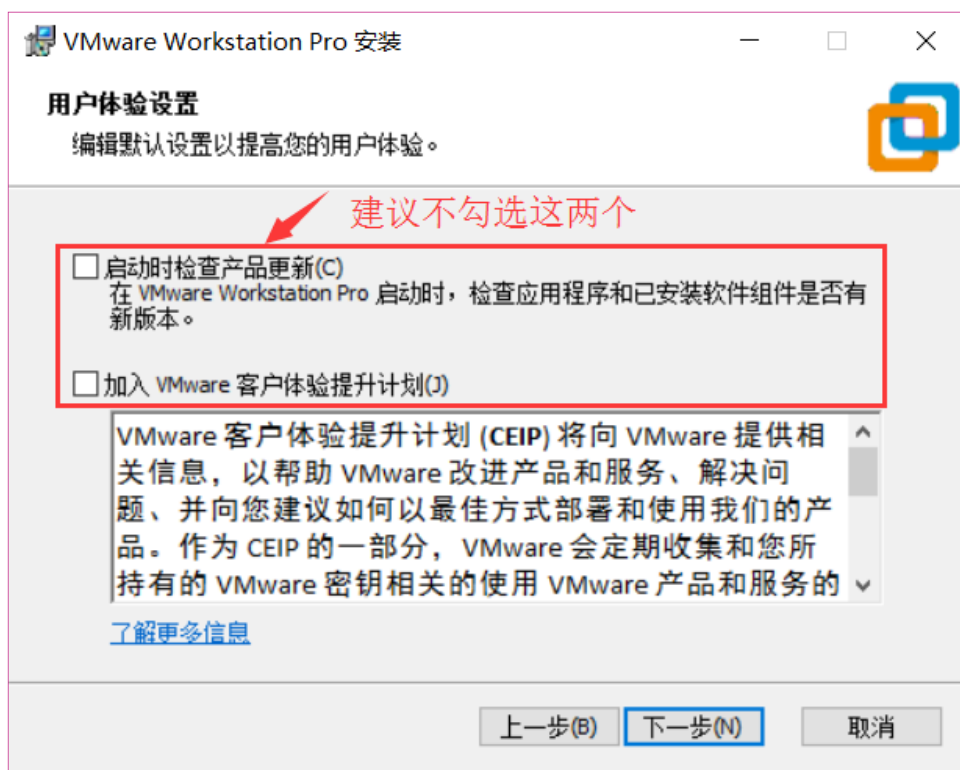


图 1-6 检查更新界面

(7) 在图 1-6 中，会有两个复选框，默认都是选中的，建议不要选中！然后点击该图中的“下一步”按钮，进入下图所示界面。

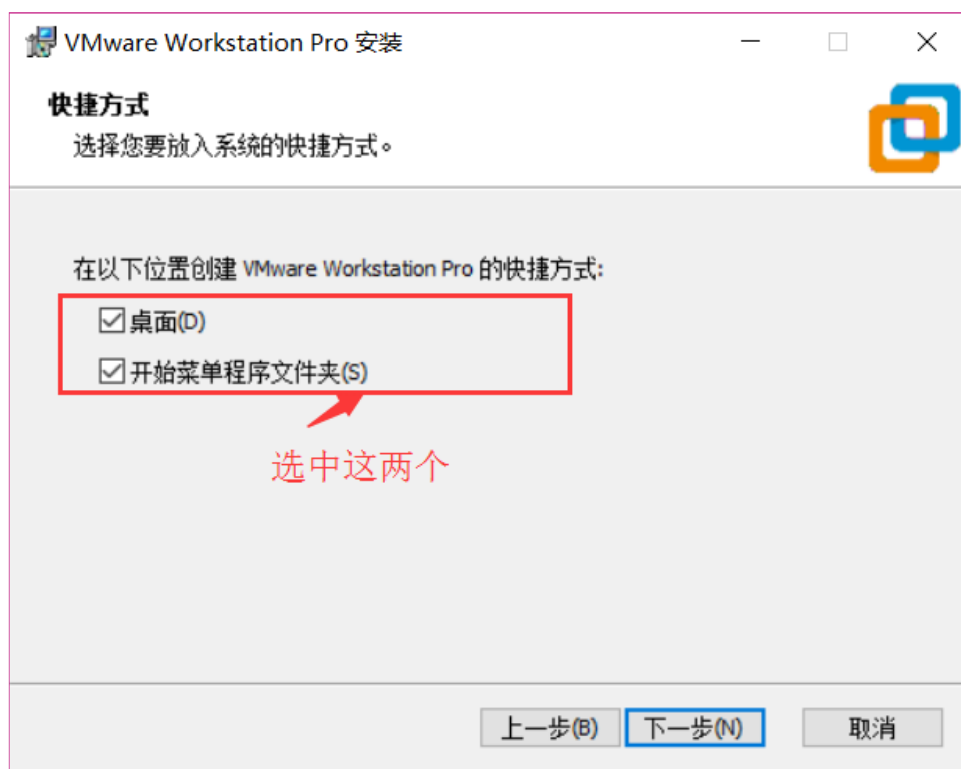


图 1-7 快捷方式设置

(8) 在图 1-7 中有两个选项，都选中，这样在安装完成以后就会在开始菜单和桌面上有 VMware 的图标，选中以后点击该图中的“下一步”，进入图 1-8 界面。

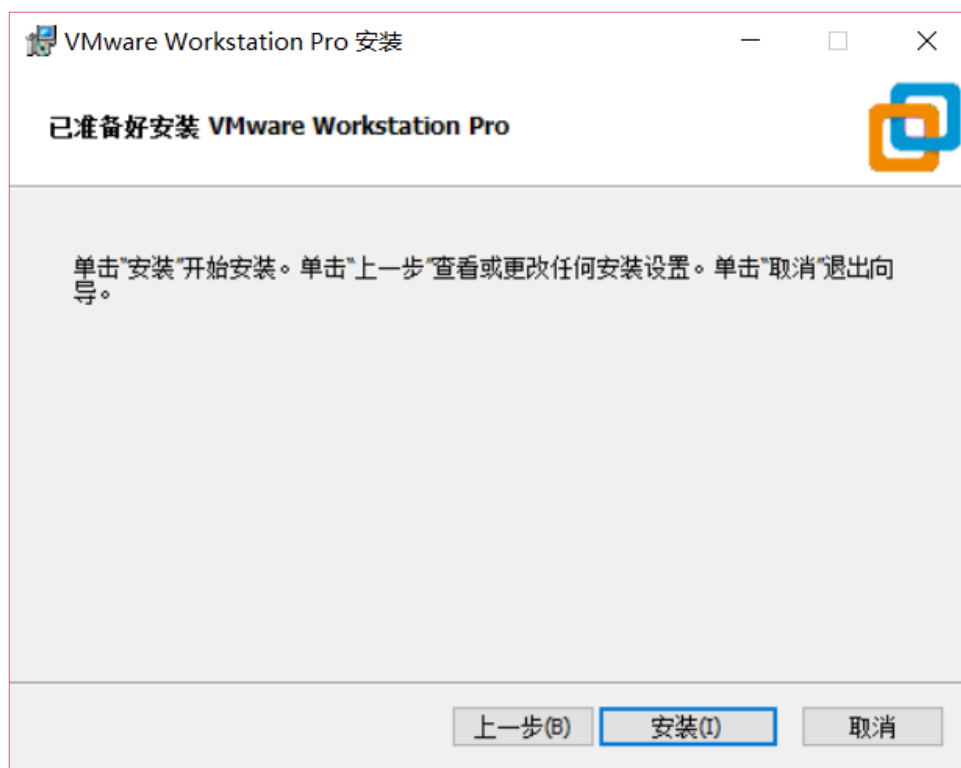


图 1-8 安装确定界面

(9) 前面几步已经设置好安装参数了，如果许需要修改安装参数的话就点击图 1-8 中的“安装”按钮开始安装 VMware，安装过程如下图所示：

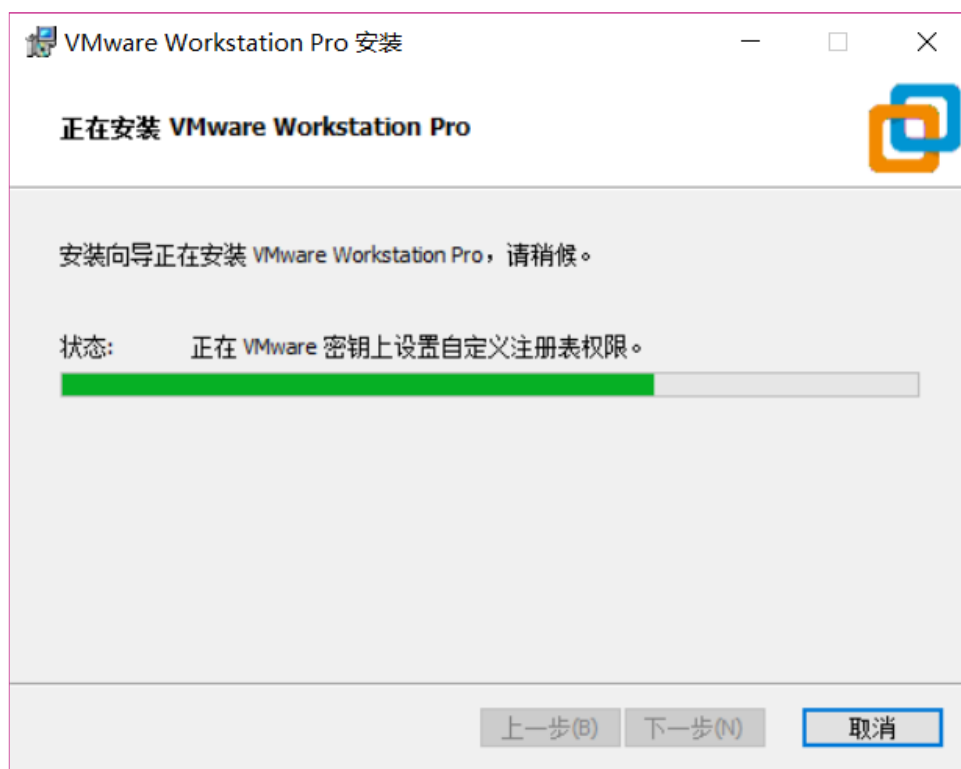


图 1-9 安装进行中

(10) 图 1-9 就是安装过程，耐心等待几分钟，等待安装完成，安装完成以后会有如图 1-10 所示提示：



图 1-10 安装完成

(11) 点击图 1-10 中的“完成”按钮，完成 VMware 的安装，安装完成以后就会在桌面上出现 VMware Workstation Pro 的图标，如下图所示：

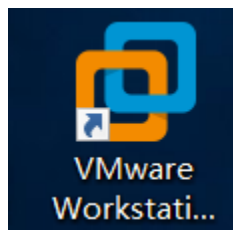


图 1-11 VMware 桌面图标

(12) 双击图 1-11 中的图标打开 VMware 软件，在使用时会出现，如下图所示，选择个人用途即可：

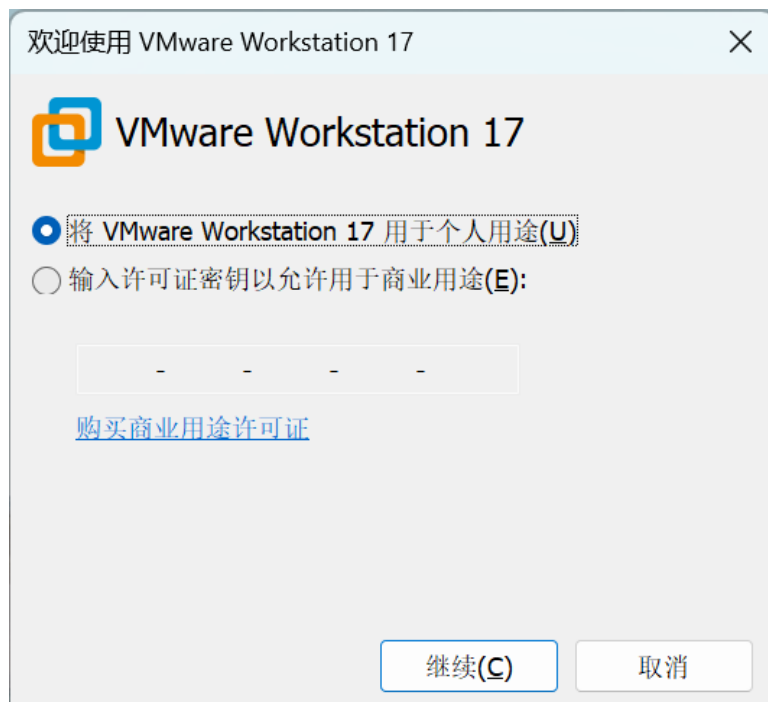


图 1-12 选择个人用途

(13) 点击图 1-12 的“继续”按钮，VMware 软件正式打开，界面如下图所示：

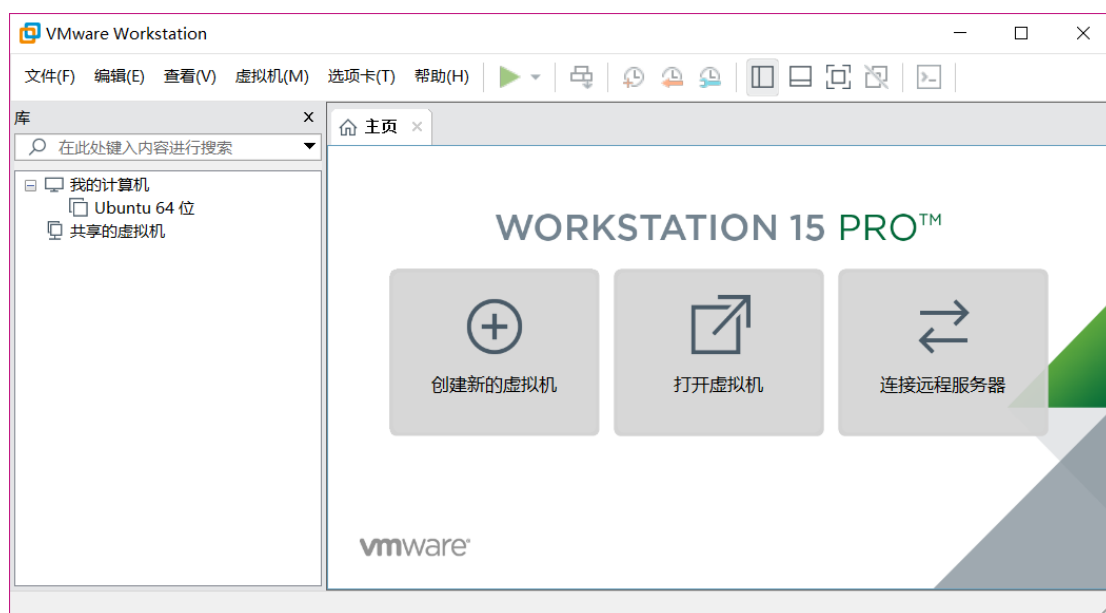


图 1-13 VMware Workstation 主界面

至此，虚拟机软件 VMware 安装成功。

## 2. 创建 Ubuntu 虚拟机

(1) 安装好 VMware 以后我们就可以在 VMware 上创建一个虚拟机，打开 VMware，选择：文件->新建虚拟机，如图 1-14 所示：



图 1-14 新建虚拟机

(2) 打开图 1-15 所示创建虚拟机向导界面：



图 1-15 创建虚拟机向导

(3) 选择 Ubuntu ISO 所在的位置，选择完毕后，点击下一步：





图 1-16 选择 Ubuntu ISO 镜像所在的位置

(4) 随后输入用户名及密码，如下图所示：

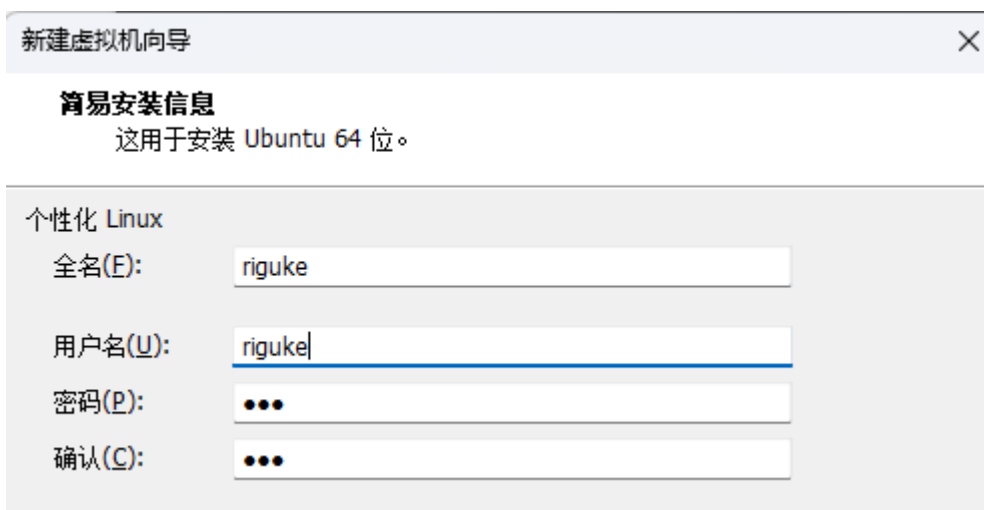


图 1-17 输入 Ubuntu 的用户名及密码

(5) 选择虚拟机的安装位置，如下图所示：

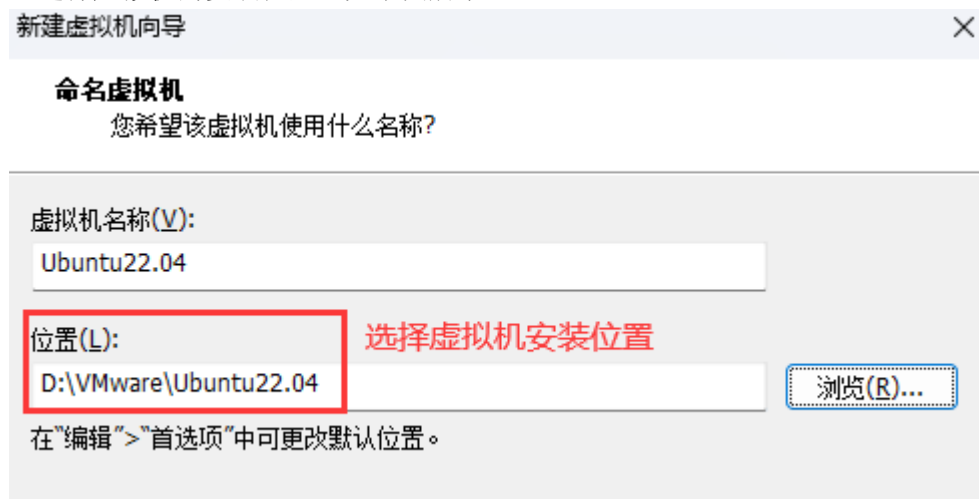


图 1-18 选择 Ubuntu 虚拟机的安装位置

(6) 设置最大的磁盘大小为 300G，因为需要在虚拟机里安装软件，这里预留空间大一些。用户可以根据自己的硬盘空间选择合适的空间尺寸，建议大于等于 300G。如下图所示：

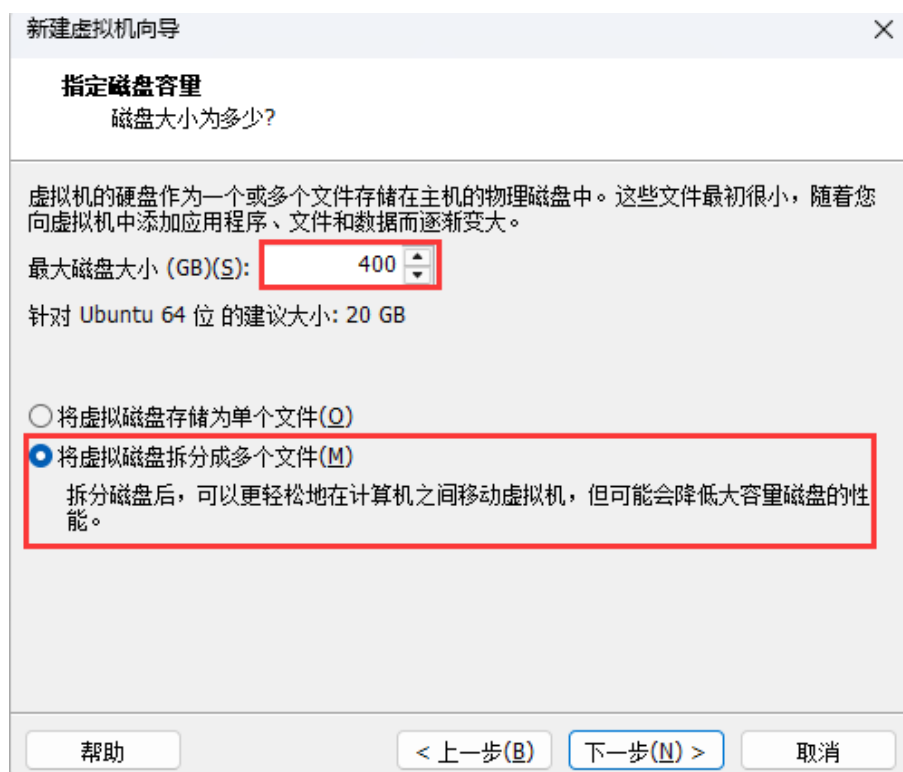


图 1-19 分配虚拟机的空间大小

(7) 可以根据修改修改内存大小和处理器核心，网络适配器选项，网络连接选择 NAT 模式，内存建议设置为 4G，否则编译会出错。自定义硬件，如下图所示：

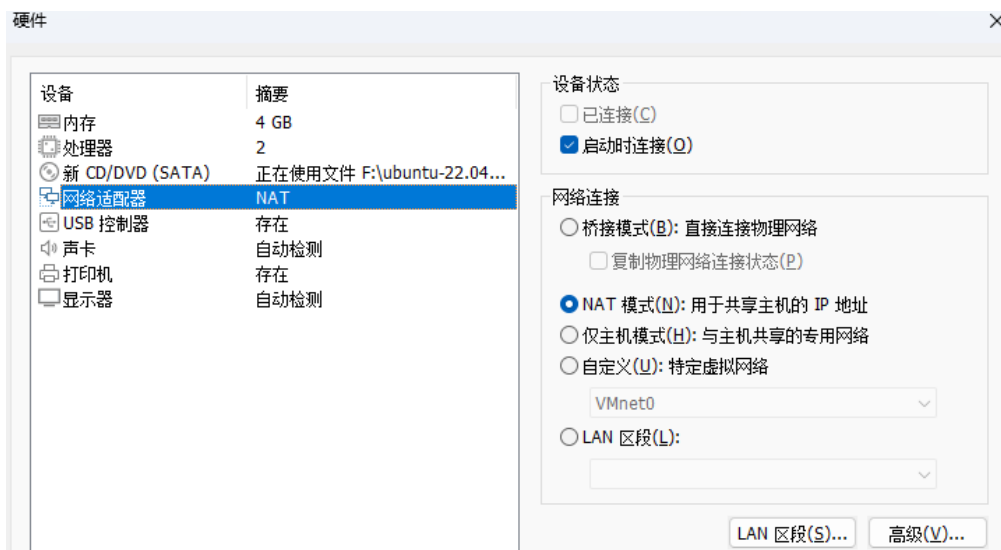


图 1-20 自定义当前 Ubuntu 的硬件信息

**特别注意：**该虚拟机内存最好为 16GB，采用 4GB 可能会编译不出来 Peatalinux 工程，上述以 4GB 为安装教程。笔者因为这个坑，编译了好几次，最后由于内存问题选用物理机进行 Petalinux 编译。

(8) 点击完成就开始安装 Ubuntu，并安装完成以后进入系统，安装完成如下图所示：

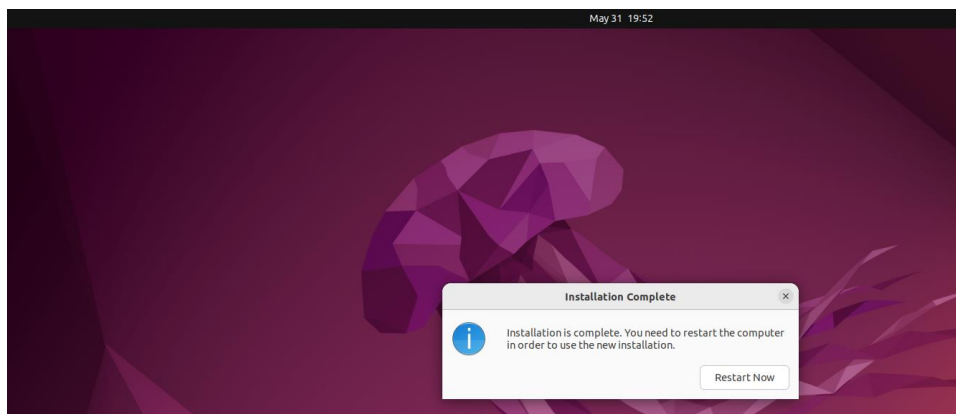


图 1-21 虚拟机安装 Ubuntu 完成

(9) 安装完毕后，系统是英文版本的，可以在“Setting”中找到“Region & Language”中讲系统语言修改为“Chinese”，修改语言完成后如下图所示：



图 1-22 修改为中文界面

(10) 为了以后安装软件方便，要设置一下软件源，点击应用菜单，搜索“软件和更新”，如下图所示：

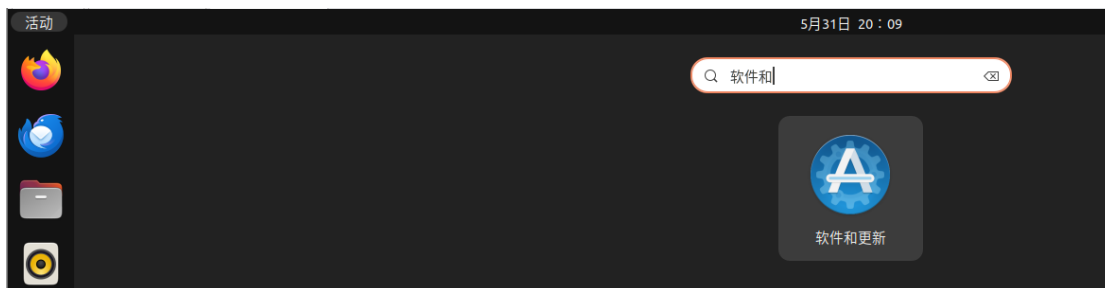


图 1-23 搜索软件和更新

(11) 在“下载至”中选择“其他...”，如下图所示：



图 1-24 修改软件源

(12) 点击“选择最佳服务器”，可以测试出一个最快的服务器，然后选择“选择服务器”，这些操作都是基于虚拟机能够连接互联网的情形。

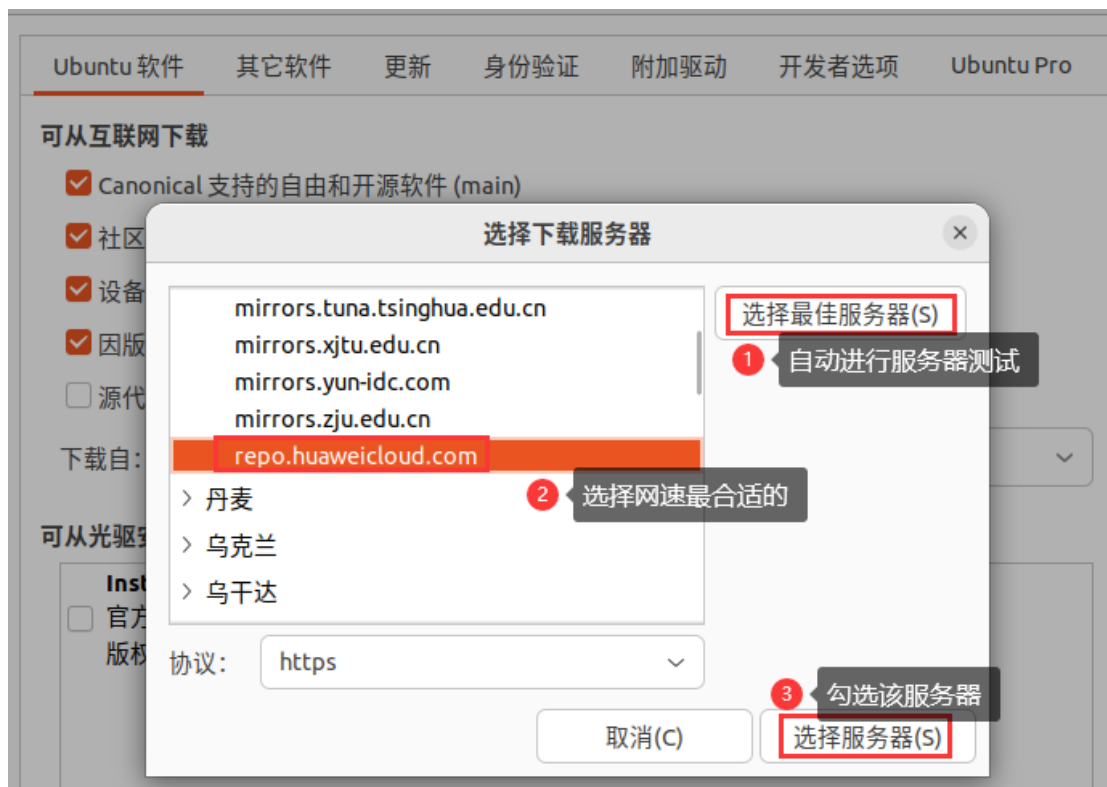


图 1-25 选择最佳的服务器

(13) 输入密码，完成软件源修改，如下图所示：

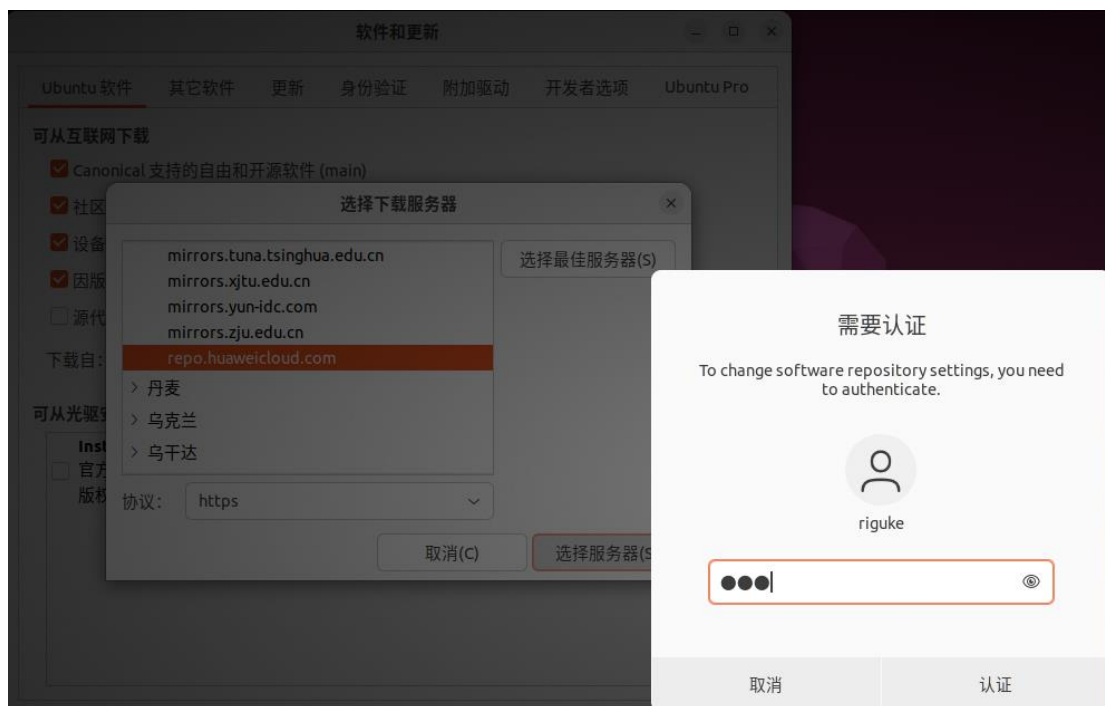


图 1-26 修改软件源

(14) 设置 bash 为默认 sh，打开终端输入以下命令，如下图所示：

命令为：sudo dpkg-reconfigure dash



图 1-27 终端输入指令

(15) 将 dash 设置为默认的 dash，如下图所示：



图 1-28 将 dash 设置为默认的 dash

### 3. 安装 Peatlinux

(1) 这里先给大家说明一下，Petalinux (2023.1) 的安装包文件比较大，有 3.18G 左右，如果大家网速太慢或者不想浪费时间下载，在开发板光盘资料中提供了 Petalinux 安装包软件，读者可以直接使用，在光盘路径：“RK-ZYNQ7020-F 开发板网盘资料 -> 7.开发软件 -> Petalinux-> petalinux-v2023.1-05012318-installer.run”，如果读者想亲自下载，可以到 Xilinx 的官方网站：[下载](#)。进行下载，如下图所示（注：需要注册 Xilinx 账号并登陆才能下载）：



图 1-29 官方下载 Peatlinux 安装包

(2) 下载完成之后就获得了 petalinux 的安装文件 petalinux-v2023.1-05012318-installer.run, 如下图所示:



图 1-30 Petalinux 安装包

(3) 安装 Petalinux 之前, 需要为 Ubuntu 系统安装一些必要的运行软件以及依赖库, 所以大家需要确保 Ubuntu 能够正常上网, 打开 Ubuntu Terminal 终端执行如下命令 (该命令是一行, 不要分开成多行):

命令为: `sudo apt-get install iproute2 gawk python3 python build-essential gcc git make net-tools libncurses5-dev tftpd zlib1g-dev libssl-dev flex bison libselinux1 gnupg wget git-core diffstat chrpath socat xterm autoconf libtool tar unzip texinfo zlib1g-dev gcc-multilib automake zlib1g:i386 screen pax gzip cpio python3-pip python3-pexpect xz-utils debianutils iputils-ping python3-git python3-jinja2 libegl1-mesa libsdl1.2-dev pylint3`

执行结果如下图所示:

```
sqd@sqd-virtual-machine:~$ sudo apt-get install iproute2 gawk python3 python build-essential gcc git make net-tools libncurses5-dev tftpd zlib1g-dev libssl-dev flex bison libselinux1 gnupg wget git-core diffstat chrpath socat xterm autoconf libtool tar unzip texinfo zlib1g-dev gcc-multilib automake zlib1g:i386 screen pax gzip cpio python3-pip python3-pexpect xz-utils debianutils iputils-ping python3-git python3-jinja2 libegl1-mesa libsdl1.2-dev pylint3
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
注意, 选中 'git' 而非 'git-core'
debianutils 已经是最新版 (4.8.4)。
diffstat 已经是最新版 (1.61-1build1)。
libselinux1 已经是最新版 (2.7-2build2)。
python3-pexpect 已经是最新版 (4.2.1-1)。
python3 已经是最新版 (3.6.7-1~18.04)。
将会同时安装下列软件:
autotools-dev blt cpp cpp-7 dh-python dirmngr dpkg-dev fakeroot g++ g++-7
gcc-7 gcc-7-base gcc-7-multilib gcc-8-base gcc-8-base:i386 gir1.2-snapd-1
git-man gnupg-l10n gnupg-utils gpg gpg-agent gpg-wks-client gpg-wks-server
qqconf qqgsm qgv lib32asan4 lib32atomic1 lib32cilkrt5 lib32gcc-7-dev
```

图 1-31 运行结果

(4) 除了上面使用命令的方法安装依赖库, 也可以使用 Xilinx 提供的脚本 plnx-env-setup.sh 安装。该脚本可以从 [Widget](#) 处下载, 如下图所示:

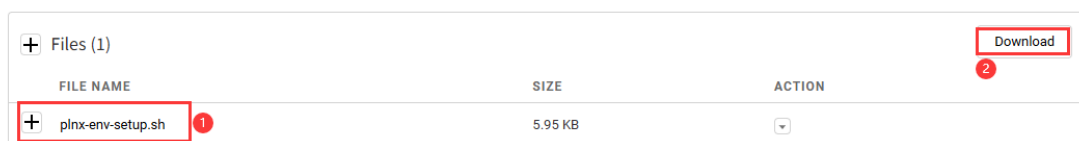
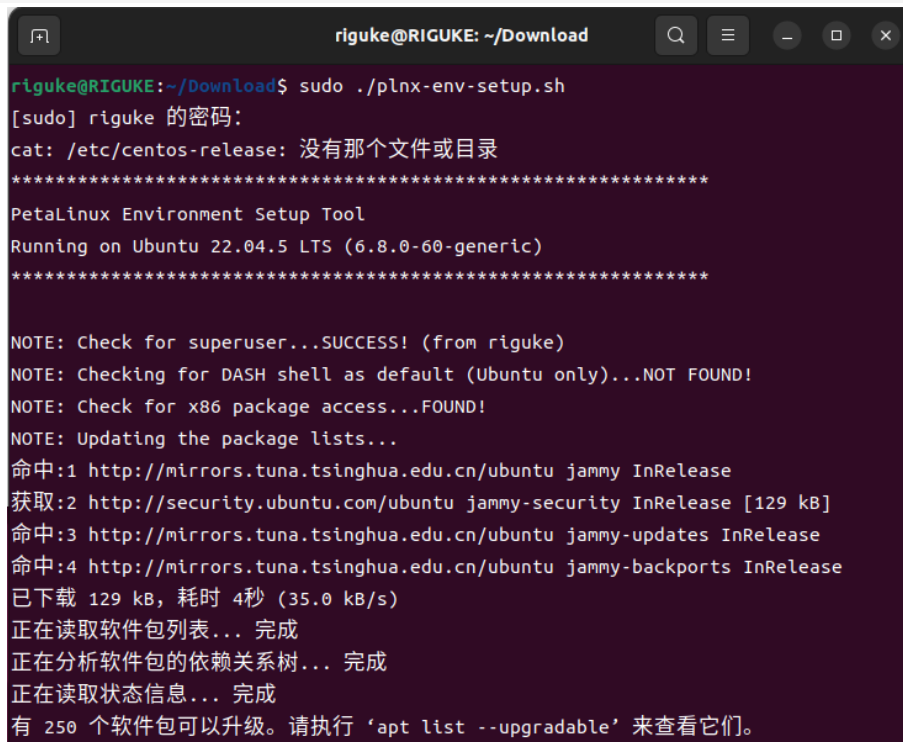


图 1-32 使用脚本安装依赖库



(5) 将下载后的脚本拷贝到 Ubuntu 虚拟机中，打开 Ubuntu Terminal 终端执行输入如下命令以执行此脚本：

命令为：sudo ./plnx-env-setup.sh



```

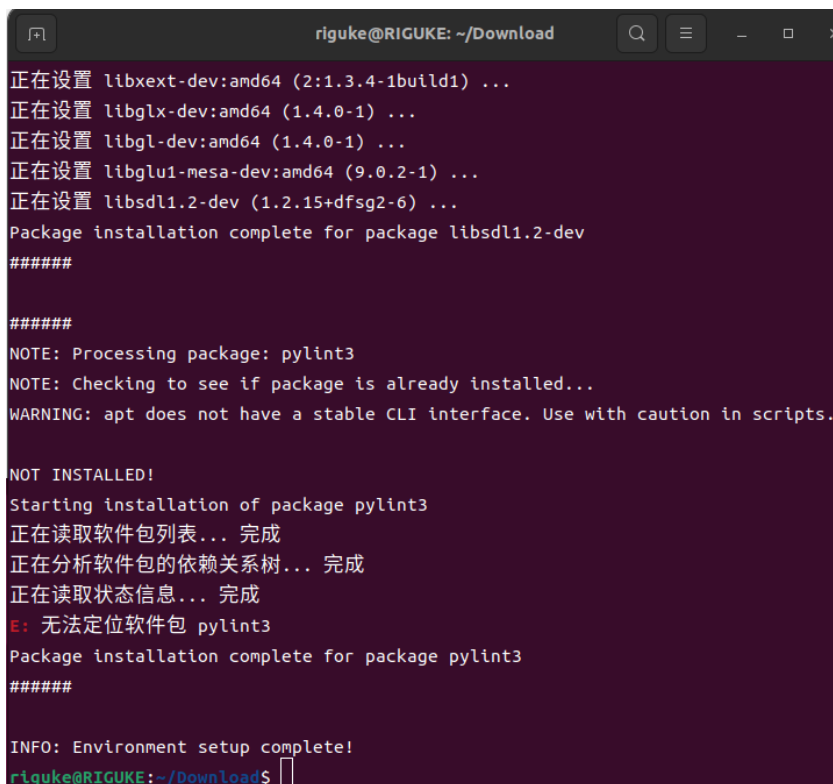
riguke@RIGUKE: ~/Download
riguke@RIGUKE:~/Download$ sudo ./plnx-env-setup.sh
[sudo] riguke 的密码:
cat: /etc/centos-release: 没有那个文件或目录
*****
PetaLinux Environment Setup Tool
Running on Ubuntu 22.04.5 LTS (6.8.0-60-generic)
*****

NOTE: Check for superuser...SUCCESS! (from riguke)
NOTE: Checking for DASH shell as default (Ubuntu only)...NOT FOUND!
NOTE: Check for x86 package access...FOUND!
NOTE: Updating the package lists...
命中:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy InRelease
获取:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
命中:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates InRelease
命中:4 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-backports InRelease
已下载 129 kB, 耗时 4秒 (35.0 kB/s)
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
有 250 个软件包可以升级。请执行 'apt list --upgradable' 来查看它们。

```

图 1-33 Ubuntu 安装必要依赖库及软件

这里需要等待大概 1-2 分钟，等待软件以及库安装完成之后即可进入下一步。安装完成后如下图所示：



```

riguke@RIGUKE: ~/Download
正在设置 libxext-dev:amd64 (2:1.3.4-1build1) ...
正在设置 libglx-dev:amd64 (1.4.0-1) ...
正在设置 libgl-dev:amd64 (1.4.0-1) ...
正在设置 libglu1-mesa-dev:amd64 (9.0.2-1) ...
正在设置 libstdc++6-dev (1.2.15+dfsg2-6) ...
Package installation complete for package libstdc++6-dev
#####

#####
NOTE: Processing package: pylint3
NOTE: Checking to see if package is already installed...
WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

NOT INSTALLED!
Starting installation of package pylint3
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
E: 无法定位软件包 pylint3
Package installation complete for package pylint3
#####

INFO: Environment setup complete!
riguke@RIGUKE:~/Download$

```

图 1-34 脚本方式安装库

(6) 安装 Petalinux 就要考虑安装位置了, 对于 Petalinux 这种体积庞大的工具, 我们将其放在/opt目录下。在/opt目录下新建专门存放 Petalinux 的文件夹, 如/opt/pkg/petalinux/2023.1, 在终端输入以下命令即可:

```
命令为: sudo chown -R $USER:$USER /opt
        mkdir -p /opt/pkg/petalinux
```

chown 命令将/opt目录的属主和属组更改为当前的用户名, 如笔者的 Ubuntu 用户名为 riguke, 执行的 chown 命令就相当于“sudo chown -R riguke:riguke /opt”, 然后通过 mkdir 创建安装目录。现在将 petalinux 安装在/opt/pkg/petalinux 目录下, 在安装包所在位置下开启终端, 并输入如下命令:

```
命令为: ./petalinux-v2023.1-05012318-installer.run -d /opt/pkg/petalinux
```

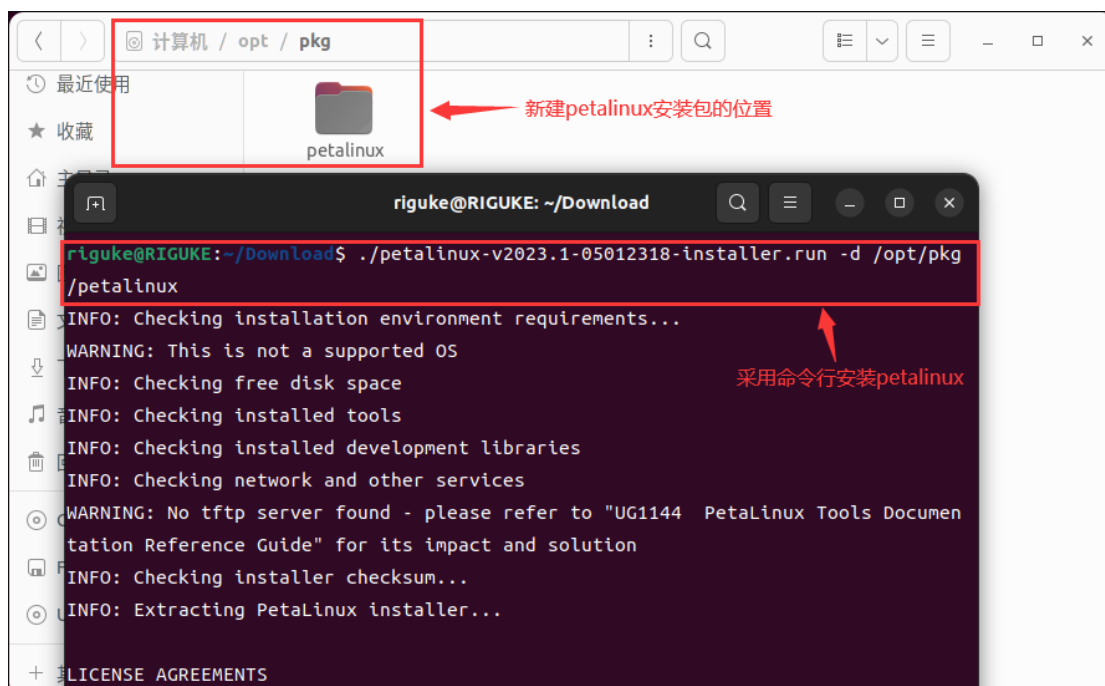


图 1-35 安装 petalinux

运行上述命令后, 需要等待一段时间, 当出现“Press Enter to display the license agreements”字样时, 如下图所示:

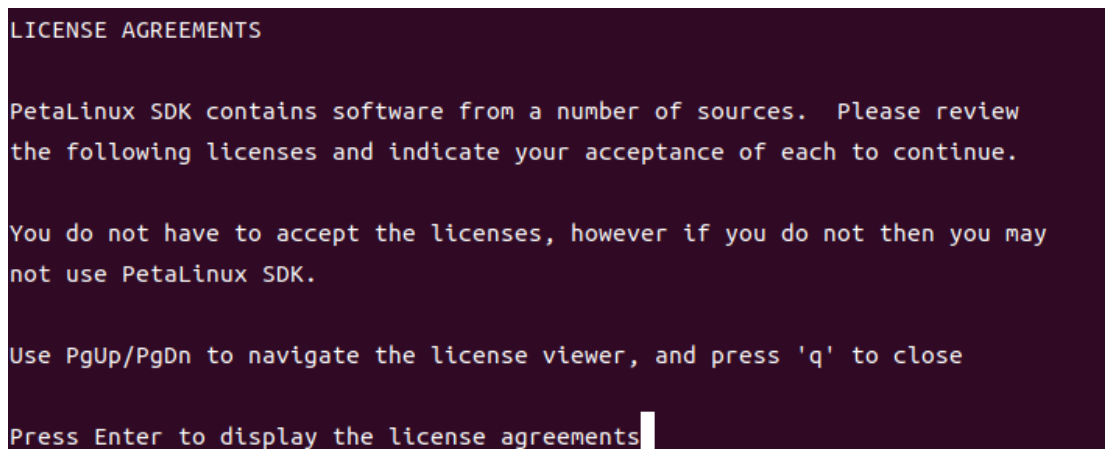


图 1-36 按下回车显示许可协议

从显示的意思可以知道, 让我们按下回车键显示软件许可协议, 按下键盘上的回车键 Enter, 显示协议内容如下:

```

XILINX, INC.
END USER LICENSE AGREEMENT FOR PETALINUX TOOLS

CAREFULLY READ THIS END USER LICENSE AGREEMENT FOR PETALINUX TOOLS ("AGREEMENT")
. BY CLICKING THE "ACCEPT" OR "AGREE" BUTTON, ENTERING <93>YES<94> OR <93>Y<94>
TO ACCEPT THIS AGREEMENT, OR OTHERWISE ACCESSING, DOWNLOADING, INSTALLING OR US
ING THE SOFTWARE, YOU AGREE ON BEHALF OF LICENSEE TO BE BOUND BY THIS AGREEMENT.

IF LICENSEE DOES NOT AGREE TO ALL OF THE TERMS AND CONDITIONS OF THIS AGREEMENT,
DO NOT CLICK THE "ACCEPT" OR "AGREE" BUTTON, ENTER <93>YES<94> OR <93>Y<94>, OR
ACCESS, DOWNLOAD, INSTALL OR USE THE SOFTWARE.

1.      Definitions

"Bitstream" means a machine-executable, binary form of a core used to program a
Xilinx Device.

"Licensee" means the individual, corporation or other legal entity who has downl
oaded and installed the Software.

"User" means a specific human being who is identified by Licensee as a person wh
o is authorized to use the applicable Software on behalf of Licensee. In cases
/tmp/tmp.00D52FitVe/Petalinux_EULA.txt

```

图 1-37 许可协议内容

这些内容如果你感兴趣可以看看，此处我们就不详细看了，直接按下键盘上的 Q 键退出，回到之前的界面之后会出现一个选择项，询问我们是否接受 Xilinx 最终用户协议。

```

Use PgUp/PgDn to navigate the license viewer, and press 'q' to close

Press Enter to display the license agreements
Do you accept Xilinx End User License Agreement? [y/N] > y

```

图 1-38 Xilinx 最终用户协议

这个显然是没得选，必须接受，否则无法进行下面的安装；输入 y 按回车接受。除了 Xilinx 最终用户协议之外，还有一个协议也需要大家接受，会依次显示在终端上，同理也是输入 Y 按下回车接受。

```

Press Enter to display the license agreements
Do you accept Xilinx End User License Agreement? [y/N] > y
Do you accept Third Party End User License Agreement? [y/N] > y
INFO: Installing PetaLinux...

```

图 1-39 第三方最终用户许可协议

接受所有协议之后 Petalinux 安装工具便会继续安装，直到安装完成。安装完成之后，进入到安装目录下，目录内容如下图所示：

```
riguke@RIGUKE:~$ ls
公共的 模板 视频 图片 文档 下载 音乐 桌面 Download snap
riguke@RIGUKE:~$ cd /opt/pkg/petalinux/
riguke@RIGUKE:/opt/pkg/petalinux$ ls
components doc etc settings.csh settings.sh tools
riguke@RIGUKE:/opt/pkg/petalinux$
```

图 1-40 petalinux 安装目录内容

(7) 在正式使用 Petalinux 工具之前，需要先运行 Petalinux 安装目录下的 settings.sh 脚本文件设置 Petalinux 工作环境，settings.sh 脚本用于 bash，还有一个 settings.csh 用于 C shell，如下图所示：

```
riguke@RIGUKE:~$ cd /opt/pkg/petalinux/
riguke@RIGUKE:/opt/pkg/petalinux$ ls
components doc etc settings.csh settings.sh tools
riguke@RIGUKE:/opt/pkg/petalinux$
```

图 1-41 settings.sh 脚本文件

(8) 一般默认情况下是使用 bash 作为登录 shell，所以 source 脚本文件 settings.sh 对 petalinux 所需的运行环境进行配置，命令如下：

命令为：source settings.sh

需要注意的是该命令只对当前终端有效，重新打开终端后需要重新执行这一步。执行结果如下图所示：

```
riguke@RIGUKE:~$ cd /opt/pkg/petalinux/
riguke@RIGUKE:/opt/pkg/petalinux$ ls
components doc etc settings.csh settings.sh tools
riguke@RIGUKE:/opt/pkg/petalinux$ source settings.sh
PetaLinux environment set to '/opt/pkg/petalinux'
WARNING: This is not a supported OS
INFO: Checking free disk space
INFO: Checking installed tools
INFO: Checking installed development libraries
INFO: Checking network and other services
WARNING: No tftp server found - please refer to "UG1144 2023.1 PetaLinux Tools D
ocumentation Reference Guide" for its impact and solution
riguke@RIGUKE:/opt/pkg/petalinux$
```

图 1-42 运行 settings.sh 脚本文件

接下来验证下工作环境是否已设置，在终端输入如下命令：

命令为：source settings.sh

```
riguke@RIGUKE:/opt/pkg/petalinux$ echo $PETALINUX
/opt/pkg/petalinux
riguke@RIGUKE:/opt/pkg/petalinux$
```

图 1-43 显示 PETALINUX 变量

显示 Petalinux 的安装目录，表明工作环境已设置。现在可以使用 Petalinux 工具了。

鉴于每次打开终端使用 Petalinux 都需要设置相应环境变量，为了方便，将设置 Petalinux 环境变量的命令设置成别名，这样我们使用起来就方便些。设置别名方法的很简单，在终端输入如下命令：

命令为: `echo "alias sptl='source $PETALINUX/settings.sh'" >> ~/.bashrc`

以后我们打开终端后, 输入 `sptl` 就可以设置 `Petalinux` 的环境变量了, 无需输入长长的路径。`sptl` 的记忆法是 `Source PetaLinux` 的环境变量。提醒: 在使用 `linux` 的时候要善用别名但不要滥用别名。

按上述操作, `Petalinux` 的环境基本就配置完毕了, 那么就可以使用 `Petalinux` 进行嵌入式开发了。

## 4. NFS 服务软件安装

NFS (Network FileSystem, 网络文件系统) 是由 SUN 公司发展, 并于 1984 年推出的技术, 用于在不同机器, 不同操作系统之间通过网络互相分享各自的文件。NFS 设计之初就是为了在不同的系统间使用, 所以它的通讯协议设计与主机及操作系统无关。

NFS 分服务器和客户机, 当使用远端文件时只要用 `mount` 命令就可把远端 NFS 服务器上的文件系统挂载在本地文件系统之下, 操作远程文件与操作本地文件没有不同。NFS 服务器所共享文件或目录记录在 `/etc/exports` 文件中。

嵌入式 Linux 开发中, 会经常使用 NFS, 目标系统通常作为 NFS 客户机使用, Linux 主机作为 NFS 服务器。在目标系统上通过 NFS, 将服务器的 NFS 共享目录挂载到本地, 可以直接运行服务器上的文件。在调试系统驱动模块以及应用程序, NFS 都是十分必要的, 并且 Linux 还支持 NFS 根文件系统, 能直接从远程 NFS root 启动系统, 这对嵌入式 Linux 根文件系统裁剪和集成也是十分有必要的。

(1) 通过以下命令安装 NFS 服务器

命令为: `sudo apt-get install nfs-kernel-server`

```

riguke@riguke: ~/桌面
riguke@riguke:~/桌面$ sudo apt-get install nfs-kernel-server
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
将会同时安装下列软件:
  keyutils libevent-core-2.1-7 libnfsidmap1 nfs-common rpcbind
建议安装:
  open-iscsi watchdog
下列【新】软件包将被安装:
  keyutils libevent-core-2.1-7 libnfsidmap1 nfs-common nfs-kernel-server
  rpcbind
升级了 0 个软件包, 新安装了 6 个软件包, 要卸载 0 个软件包, 有 0 个软件包未被升级
。
需要下载 615 kB 的归档。
解压缩后会消耗 2,235 kB 的额外空间。
您希望继续执行吗? [Y/n] Y

```

图 1-43 终端安装 NFS 服务器



(2) 新建一个 work 目录做为 NFS 的一个工作目录，以后可以把交叉编译的程序放在这个目录里，开发板可以很方便共享到这个目录里的文件。

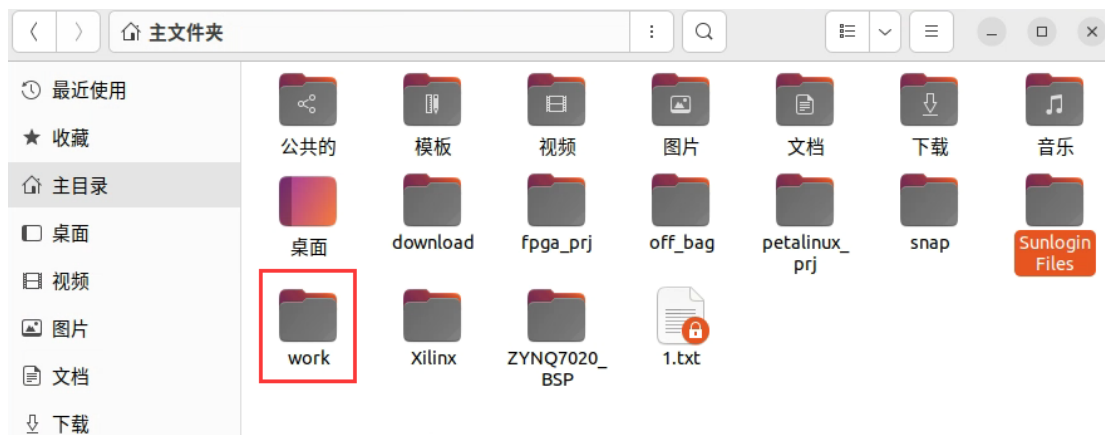


图 1-43 新建一个 NFS 服务器工作目录

(3) 使用下面命令编辑/etc/exports 文件，配置 NFS 服务路径。

命令为：sudo gedit /etc/exports



图 1-44 编辑/etc/exports 文件

(4) 在打开的文件尾部添加/home/riguke/work\*(rw,sync,no\_root\_squash,no\_subtree\_check)，配置/home/riguke/work 目录为 NFS 的一个工作目录。

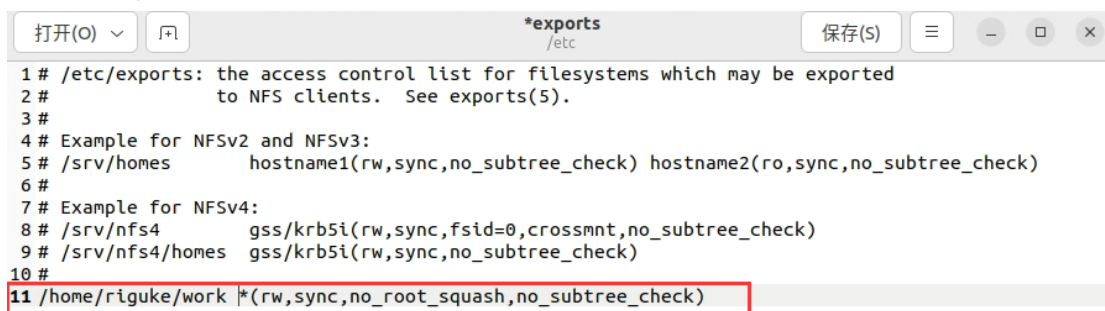


图 1-45 配置 work 目录为 NFS 的工作目录

/home/riguke/work 目录是要刚才创建的 NFS 的目录，也就是上面命令 pwd 输出的内容，\*代表允许所有的网络段访问，rw 是可读写权限，sync 是文件同步写入存储器，no\_root\_squash 是 nfs 客户端分享目录使用者的权限。如果客户端使用的是 root 用户，那么对于该共享目录而言，该客户端就具有 root 权限。

(5) 执行下面命令重启 rpcbind 服务。nfs 是一个 RPC 程序，使用它前，需要映射好端口，通过 rpcbind 设定

命令为：sudo /etc/init.d/rpcbind restart

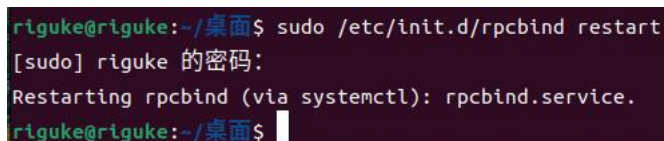


图 1-45 重启 rpcbind 服务

(6) 执行下面命令重启 NFS 服务

命令为: `sudo /etc/init.d/nfs-kernel-server restart`

```
riguke@riguke:~/桌面$ sudo /etc/init.d/rpcbind restart
[sudo] riguke 的密码:
Restarting rpcbind (via systemctl): rpcbind.service.
riguke@riguke:~/桌面$ sudo /etc/init.d/nfs-kernel-server restart
Restarting nfs-kernel-server (via systemctl): nfs-kernel-server.service.
riguke@riguke:~/桌面$
```

图 1-45 重启 NFS 服务

(7) 检查 NFS 服务器中挂载的目录

命令为: `showmount -e`

```
riguke@riguke:~/桌面$ showmount -e
Export list for riguke:
/home/riguke/work *
```

图 1-46 检查 NFS 服务器中挂载的目录

(8) 通过下面命令挂载 NFS, 在本机将 NFS 工作路径挂载在/mnt 目录

命令为: `sudo mount -t nfs 127.0.0.1:/home/riguke/work /mnt`

```
riguke@riguke:~/桌面$ sudo mount -t nfs 127.0.0.1:/home/riguke/work/ /mnt/
riguke@riguke:~/桌面$
```

图 1-47 本机将 NFS 工作路径挂载在/mnt 目录

(9) 进入/mnt,新建一个 test 目录测试一下, 可以在/home/alinx/work 目录同步看到 test 文件夹。

命令为: `cd /mnt`

`mkdir test`

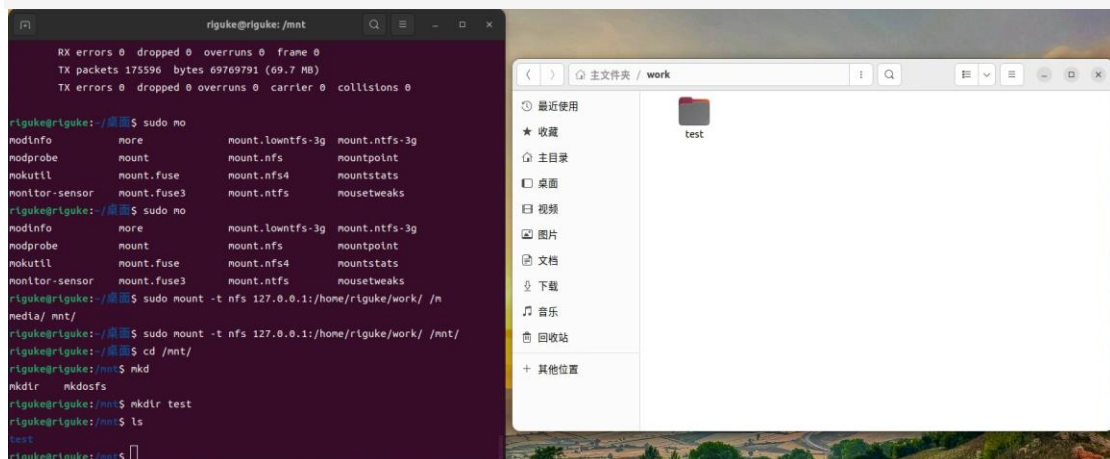


图 1-48 测试 NFS 服务器

## 二. Petalinux 设计流程

PetaLinux 工具提供了在 Xilinx 处理系统上自定义、构建和部署嵌入式 Linux 解决方案所需的一切。该解决方案旨在提高设计生产力，可与 Xilinx 硬件设计工具一起使用，以简化针对 Zynq SoC 的 Linux 系统的开发。本章以使用 Petalinux 定制 Linux 系统为例，实战 Petalinux 的设计流程，看下 Petalinux 如何简化 Linux 系统的开发。

### 1. Zynq-7000 嵌入式软件栈概述

要想在 Zynq-7000 SoC 中搭建运行 Linux，首先需要先简单的了解下其嵌入式软件栈，如下图所示：



图 2-1 Zynq-7000 嵌入式软件栈

处理器系统引导是一个分两个阶段的过程。第一个阶段是一个内部 BootROM，它存储 stage-0 的引导代码。BootROM 在 CPU 0 上执行，CPU 1 执行等待事件(WFE)指令。BootROM 还配置必要的外围设备，以开始从其中一个引导设备获取第一阶段引导加载程序（FSBL）引导代码。可编程逻辑（PL）不是由 BootROM 配置的。第二种状态是 FSBL 引导代码。这通常存储在闪存、SD 卡中，或者可以通过 JTAG 下载。BootROM 代码将 FSBL 引导代码从选定的非易失性存储器复制到片上存储器（OCM）。加载到 OCM 中的 FSBL 的大小限制为 192KB。在 FSBL 开始执行后，完整的 256 KB 可用。有一个可选的第二阶段引导加载程序，它是可选的，由用户设计。常见的第二阶段引导加载程序是 U-boot。

#### （10）FSBL

Zynq-7000 的第一阶段引导加载程序（FSBL）使用硬件比特流（如果存在）配置 FPGA，并将操作系统（OS）映像或第二阶段引导加载器映像从非易失性存储器（NAND/SD/eMC/QSPI）加载到存储器（DDR/OCM）。它支持多个分区，每个分区可以是代码映像、位流或通用数据。如果需要，可以对这些分区中的每个分区进行身份验证和/或解密。

#### （11）U-Boot

U-Boot，通用引导加载程序的缩写，是一种开源的主引导加载程序，用于嵌入式设备，



以引导 Linux 社区中经常使用的设备操作系统内核。Xilinx 在 Zynq-7000 设备中使用 U-Boot 作为第二阶段引导加载程序。

### (12) Linux

Linux，全称 GNU/Linux，是一种免费使用和自由传播的类 UNIX 操作系统，是我们本开发指南的重点。

以上就简单的介绍了 Zynq-7000 嵌入式软件栈，如果没有看懂没关系，笔者这里简单的概括下。Zynq-7000 上电后，首先由 BootROM 对 Zynq 设备进行初始启动，然后引导加载 FSBL 到 OCM 并启动 FSBL；FSBL 启动后将 U-Boot 加载到 DDR 并启动 U-Boot；U-Boot 启动后加载 Linux 系统镜像到 DDR 并启动 Linux，至此整个 Linux 系统启动完成。

综上，就是需要在 Zynq-7000 运行 Linux 系统所需要搭建的软件栈。如果这些软件栈由我们一个个手工搭建，任务量极其庞大，所幸的是，Xilinx 推出了 Petalinux 开发工具，可以很方便快捷的完成这些软件栈的搭建，从而加快 Linux 的使用和开发。

## 2. Petalinux 工具的设计流程概述

通常 PetaLinux 工具遵循顺序设计流程模型，如下表所示：

表 2-1 petalinux 设计流程表

| Design Flow Step                 | Tool/Workflow                         |
|----------------------------------|---------------------------------------|
| Hardware Platform Creation       | Vivado                                |
| Create PetaLinux Project         | petalinux-create -t project           |
| Initialize PetaLinux Project     | petalinux-config --get-hw-description |
| Configure System-Level Options   | petalinux-config                      |
| Create User Components           | petalinux-create -t COMPONENT         |
| Configure the Linux Kernel       | petalinux-config -c kernel            |
| Configure the Root Filesystem    | petalinux-config -c rootfs            |
| Build the System                 | petalinux-build                       |
| Package for Deploying the System | petalinux-package                     |
| Boot the System for Testing      | petalinux-boot                        |

从上表可以看到，使用 Vivado 搭建好硬件平台后，通过几个命令就完成了 Linux 系统的定制，极其方便。

需要说明的是以上设计流程不是按部就班的每一步都执行一遍，可以根据使用场景有选择的执行。一般的设计流程如下：

- (1) 通过 Vivado 创建硬件平台，得到 xsa 文件；
- (2) 运行 `source <petalinux 安装路径>/settings.sh`，设置 Petalinux 运行环境；
- (3) 通过 `petalinux-create -t project` 创建 petalinux 工程；
- (4) 使用 `petalinux-config --get-hw-description`，将 xsa 文件导入到 petalinux 工程当中并配置 petalinux 工程；
- (5) 使用 `petalinux-config -c kernel` 配置 Linux 内核；
- (6) 使用 `petalinux-config -c rootfs` 配置 Linux 根文件系统；
- (7) 配置设备树文件；
- (8) 使用 `petalinux-build` 编译整个工程；
- (9) 使用 `petalinux-package --boot` 制作 BOOT.BIN 启动文件；
- (10) 制作 SD 启动卡，将 BOOT.BIN 和 image.ub 以及根文件系统部署到 SD 卡中；
- (11) 将 SD 卡插入开发板，并将开发板启动模式设置为从 SD 卡启动；

(12) 开发板连接串口线并上电启动，串口上位机打印启动信息，登录进入 Linux 系统。

### 3. 使用 Petalinux 定制 Linux 系统

现在使用 Petalinux 定制 Linux 系统为例，来实战下 Petalinux 的设计流程，体验 Petalinux 对 Linux 系统开发的简便之处。

在《RK\_7020\_Vitis》中我们创建了很多 Vivado 工程，相信大家对 vivado 工程的创建已经非常熟悉了，为了节省时间，这里统一使用 ZYNQ7020 中开启 Uart, QSPI, USB, Ethernet, SD Card，通过基本外设来熟悉 Petalinux 的使用，ZYNQ 7020 外设开启如下图所示：

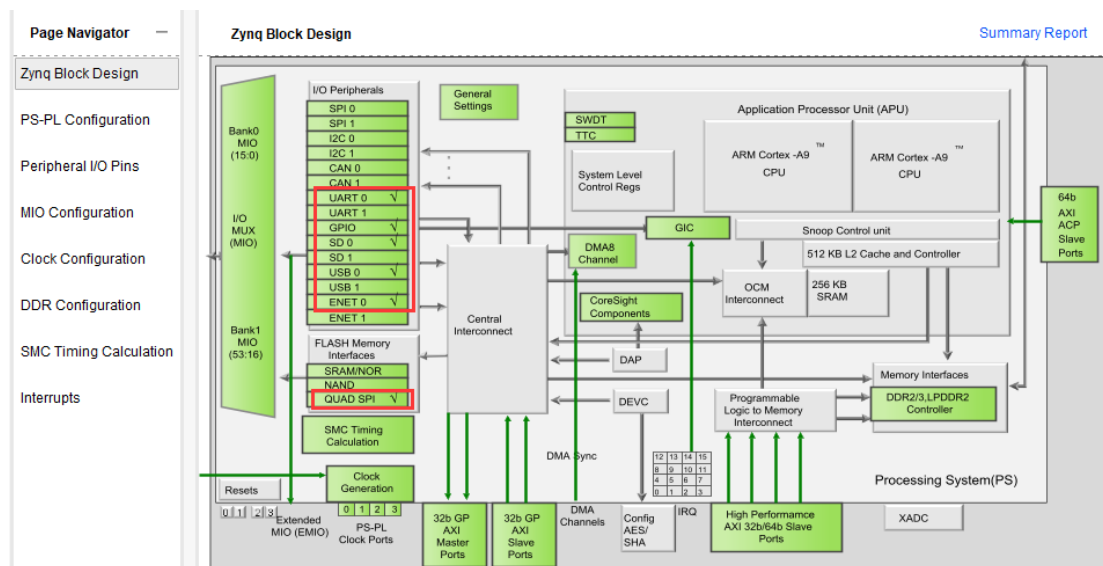


图 2-2 ZYNQ 7020 开启外设

该配置 Petalinux 工程使用的 Vivado 工程与 Vitis 教程中一致，提供 TCL 文件，方便在使用时能够快速配置外设。

新建工程后，导出 XSA 文件，用于新建 Petalinux 工程，需要注意的是，该工程只有 PS 端外设，因为在导出选项时，不要选择“include bitstream”。

导出文件如下图所示：

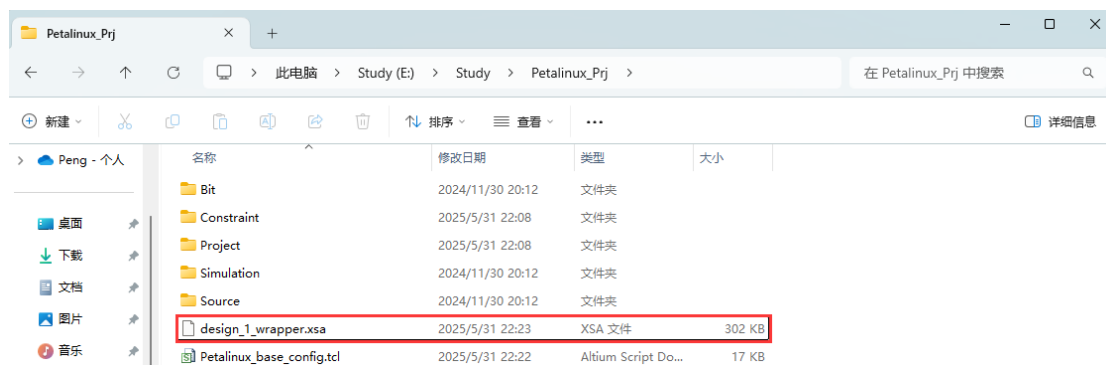


图 2-2 Vivado 工程目录

该目录下有一个 design\_1\_wrapper.xsa 文件，该文件包含着 Vivado 工程所对应的硬件平台信息，Petalinux 根据这些信息来配置 fsbl、uboot、内核等。

将该文件拷贝到 Ubuntu 系统下，譬如笔者在 Ubuntu 系统用户根目录下创建了一个 petalinux\_prj 的目录，并在该目录下创建一个 01\_petalinux\_base/hardware 目录用于存放 design\_1\_wrapper.xsa 文件。将 design\_1\_wrapper.xsa 文件拷贝到 01\_petalinux\_base/hardware

目录中，如下图所示：

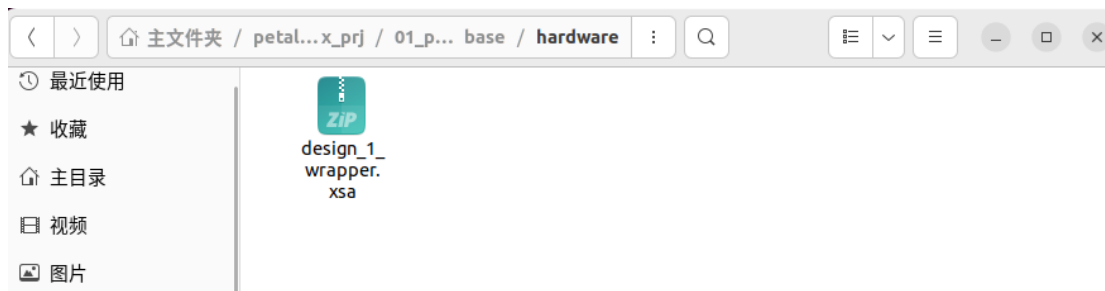


图 2-3 将 xsa 文件复制到 01\_petalinux\_base/hardware 目录下

上述将 XSA 文件拷贝到 Ubuntu 下后，后续就可以进行 Petalinux 的工程建立：

#### (1) 创建 petalinux 工程

进入 01\_petalinux\_base 文件夹后，在终端输入以下命令：

命令为：petalinux-create -t project --template zynq -n petalinux

```
riguke@RIGUKE:~/petalinux_prj/01_petalinux_base$ petalinux-create -t project --template zynq -n petalinux
```

图 2-4 终端创建 petalinux 工程

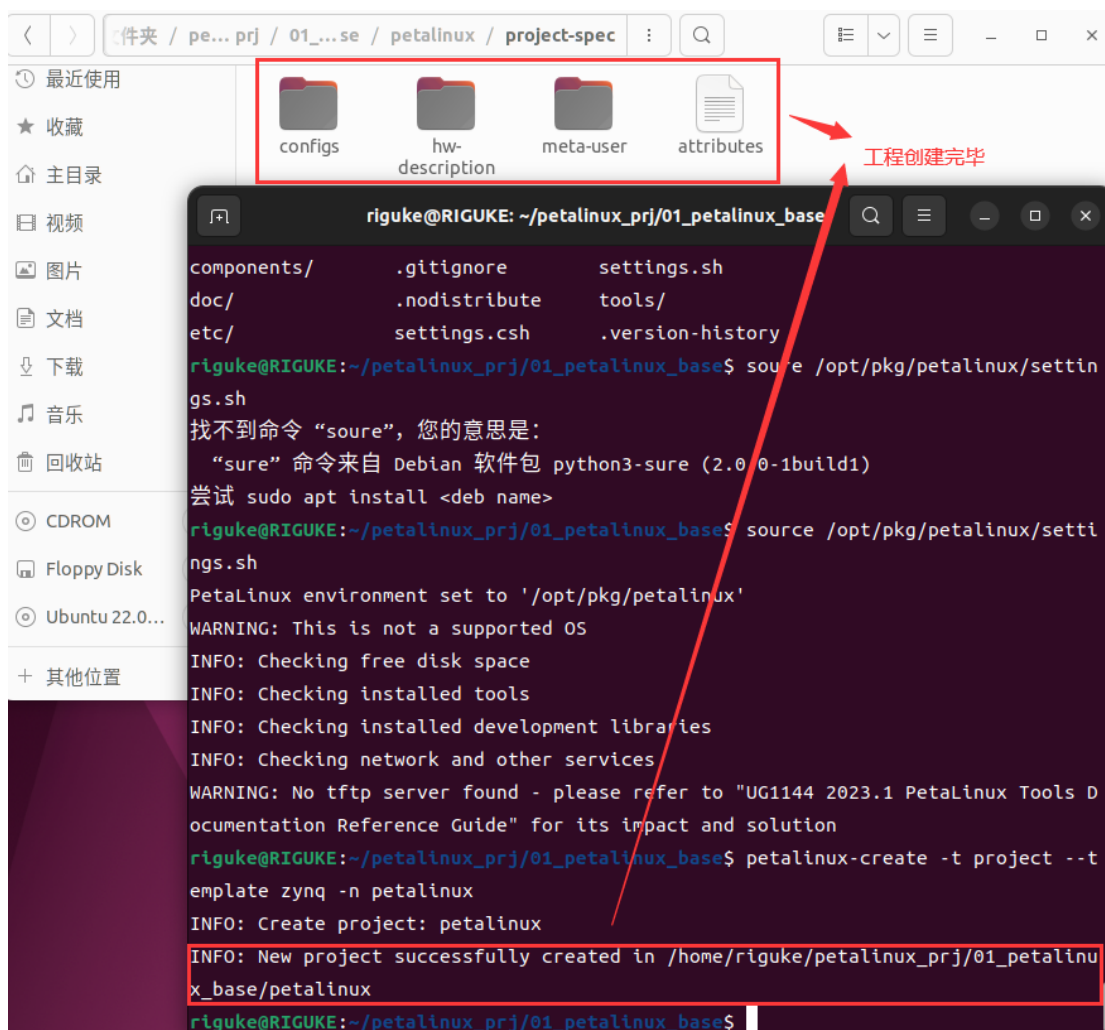


图 2-5 工程创建完毕

(2) 首次配置 Petalinux 工程是将 xsa 文件导入到 Petalinux 工程中，Petalinux 工具会解析 xsa 文件并弹出配置窗口。在终端中输入如下命令配置 Petalinux 工程：

命令为：petalinux-config --get-hw-description ../hardware/

即进入到 petalinux 工程文件夹，并对 petalinux 工程进行配置。“petalinux-config --get-hw-description”命令后面的文件夹就是我们复制到 Ubuntu 系统下 design\_1\_wrapper.xsa 文件所在的位置。如果后面修改了 Vivado 工程，重新生成 xsa 文件后，可以重新执行“petalinux-config --get-hw-description < xsa 文件所在的位置>”以重新配置 Petalinux 工程。

执行结果如下图所示：

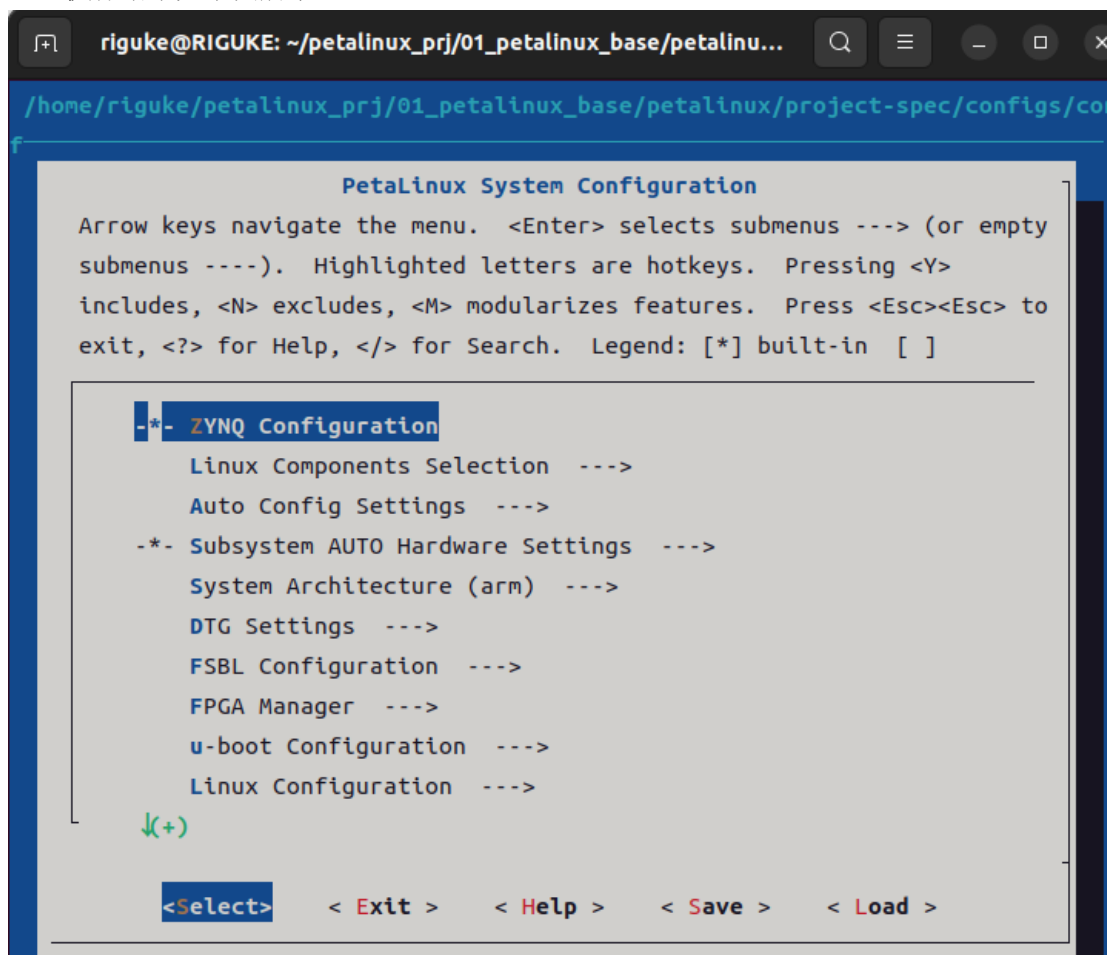


图 2-6 petalinux 工程配置窗口

需要注意的是该窗口不可以使用鼠标操作，只能通过键盘操作，界面上方的英文就是简单的操作说明，操作方法如下：

通过键盘上的“↑”和“↓”键来选择要配置的菜单，按下“Enter”键进入子菜单。菜单中蓝色高亮的首字母就是此菜单的热键，在键盘上按下此高亮字母对应的键可以快速选中对应的菜单。选中子菜单以后按下“Y”键就会将相应的配置选项写入配置文件中，菜单前面变为“<\*>”。按下“N”键不编译相应的代码，按下“M”键就会将相应的代码编译为模块，菜单前面变为“<M>”。按两下“Esc”键退出，也就是返回到上一级，按下“?”键查看此菜单的帮助信息，按下“/”键打开搜索框，可以在搜索框输入要搜索的内容。

(3) 首先按键盘上的下方方向键移动到“Linux Components Selection”，然后按键盘上的“Enter”进入子菜单，子菜单内容如下图所示：

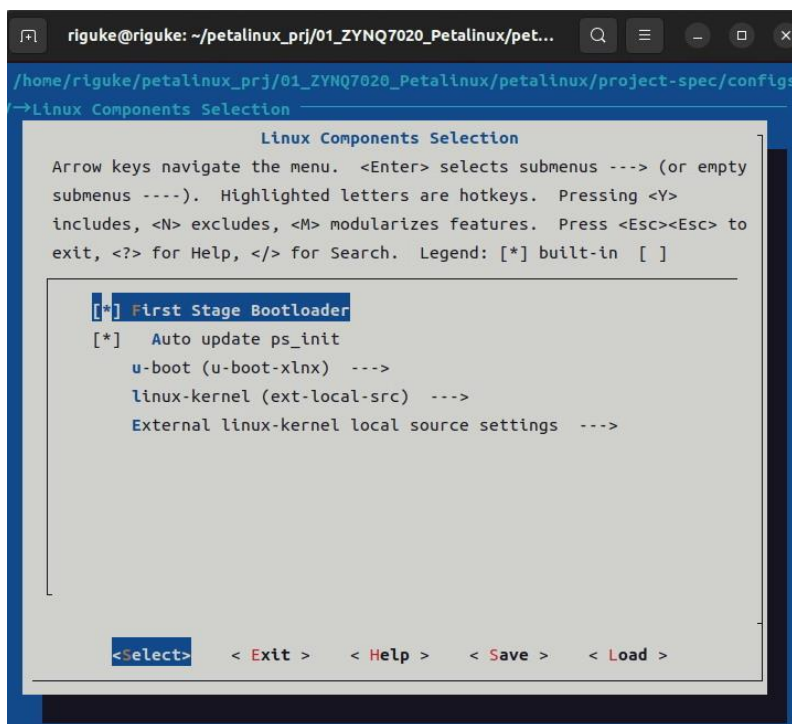


图 2-7 Linux Components Selection 子菜单

上图中括号里的“\*”表示已使能配置。第二项和第三项表示会自动生成 Vitis 教程中固化实验中使用的 fsbl.elf 文件和自动更新 ps\_init。下面两个选项用来配置 u-boot 和 linux-kernel 的来源。

需要注意的是，上述在使用默认的连接会从 Github 上下载文件，如果网络不是很好的话，建议将安装包下载至本地，具体方法在下一节去描述。

(4) 在配置界面中，使用键盘上的“上下”方向键把蓝色的条移动到“Auto Config Settings”选项上，按回车进入该选项，“Auto Config Settings”菜单主要就是选择是否使能 Device tree 自动配置，这里使用默认配置，无需更改，就不看了。具体配置如下图所示：

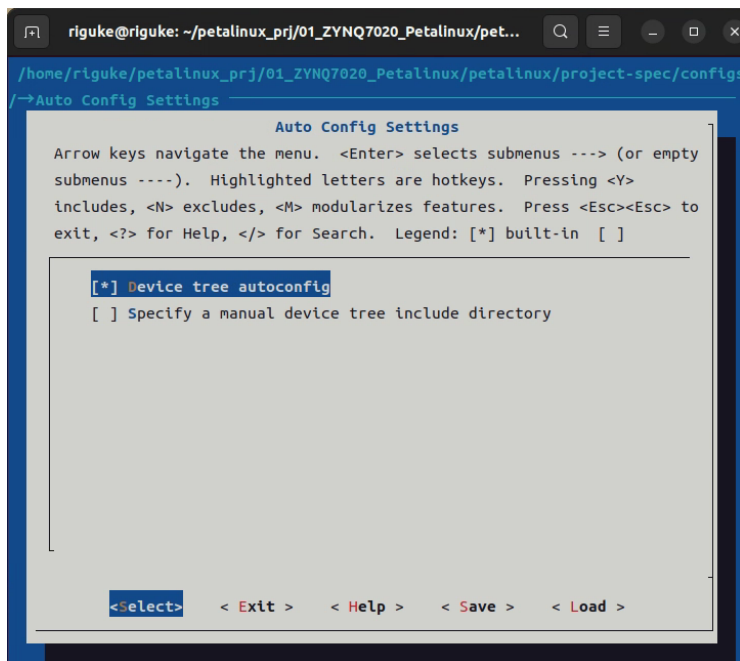


图 2-8 Auto Config Settings 子菜单



(5) “Subsystem AUTO Hardware Settings”子菜单的内容如下图所示：

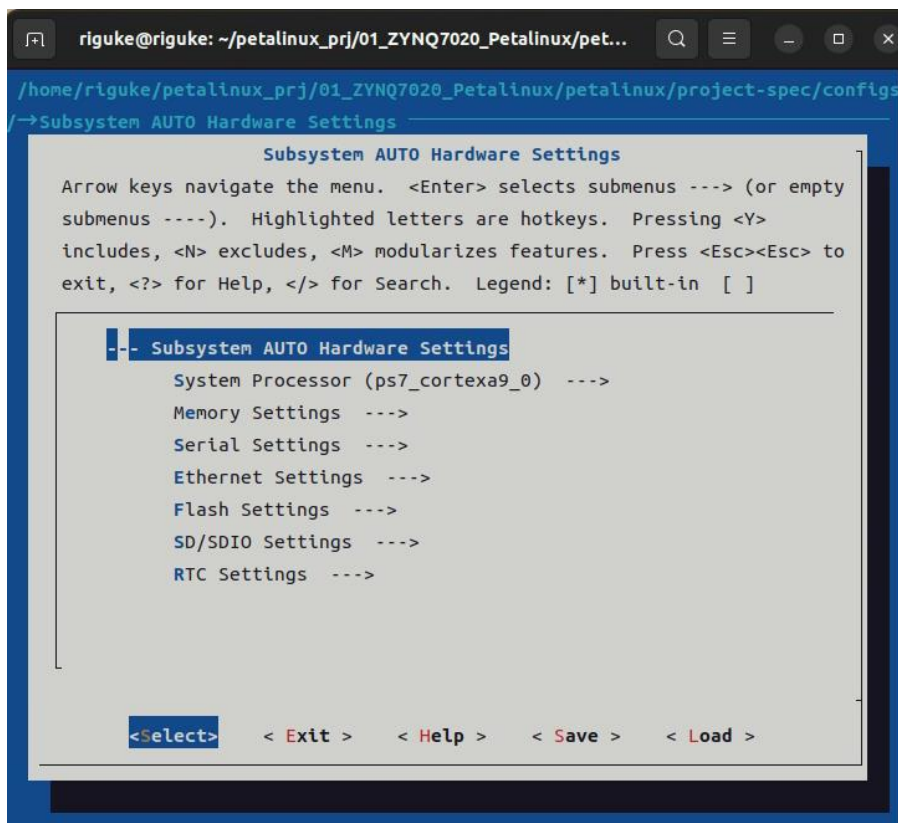


图 2-9 Subsystem AUTO Hardware Settings 子菜单

进入到该界面的各个外设子菜单中，可以发现都已经设置好了默认配置，这些默认配置是根据 xsa 文件的信息自动配置的，基本上无需我们手动配置

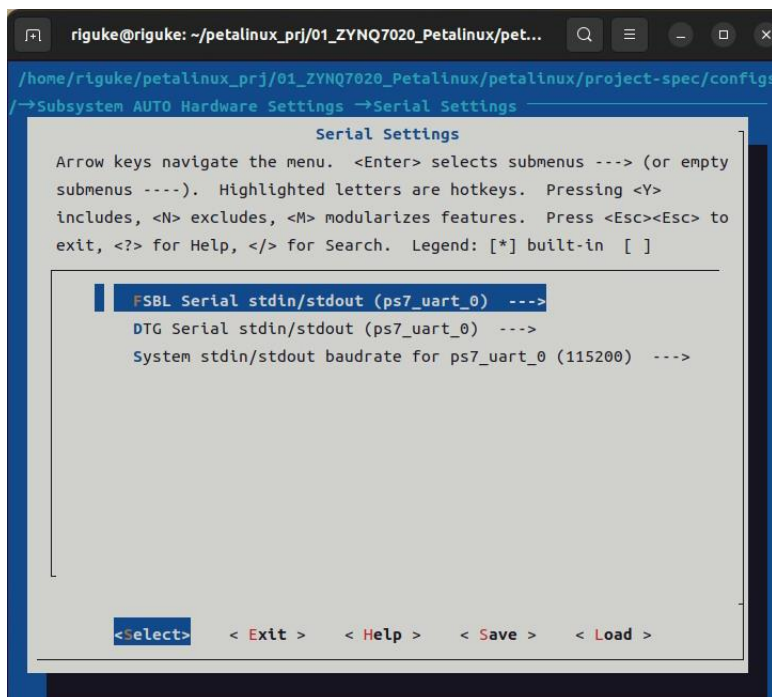


图 2-10 配置串口

(6) 返回到主界面（按二次“ESC”按键），“DTG Settings”、FSBL 配置菜单“FSBL Configuration”、FPGA 管理器菜单“FPGA Manager”、u-boot 配置菜单“u-boot Configuration”

和 linux 配置菜单“Linux Configuration”一般保持默认即可。

(7) 进入“Image Packaging Configuration”子菜单，如下图所示：

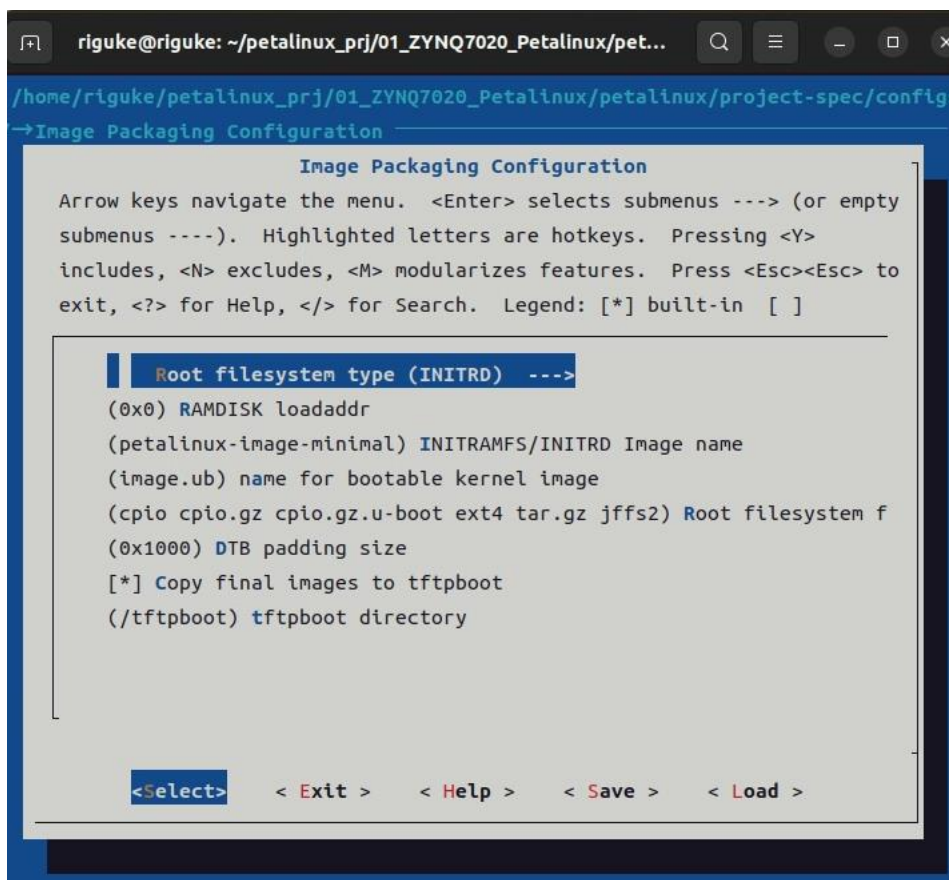


图 2-11 Image Packaging Configuration 子菜单

第一个选项便是根文件系统的类型的配置，默认为 INITRD，一般默认即可。如果我们需运行 Ubuntu 或 Debian 的根文件系统时，就需要配置成 EXT4(SD/eMMC/SATA/USB)，NFS 挂载启动需要配置成 NFS。

注：INITRD 类型的根文件系统每次重新启动 linux 系统都是全新的、未改动过的，也就是说启动系统后进行的所有修改掉电后就全部丢失了，再次重新启动还是之前未修改过的根文件系统，选择“EXT4”可以将根文件系统放在 SD 卡、eMMC 的 ext4 分区，这样启动系统后进行的所有修改掉电后就不会丢失了。后面会讲解如何将根文件系统放在 SD 卡的 ext4 分区。本章还是先从简单的 INITRD 类型用起。

另外从该界面我们可以看到，有“Copy final images to tftpboot”选项，当在 Ubuntu 的根文件下创建一个名为 tftpboot 的文件夹时，工程生成镜像后会自动将相关文件复制到 tftpboot 目录中。

回到主界面，“Firmware Version Configuration”可以用来修改定制的 linux 系统的主机名和产品名，默认与该 Petalinux 工程同名，如果需要可修改。“Yocto Settings”进行与 Yocto 相关的设置，这里就不做介绍了，一般保持默认即可。

按键盘上的右方向键（即右箭头），移动到底部的“Save”，按键盘上的“Enter”键，进入如下图所示的保存配置文件界面：

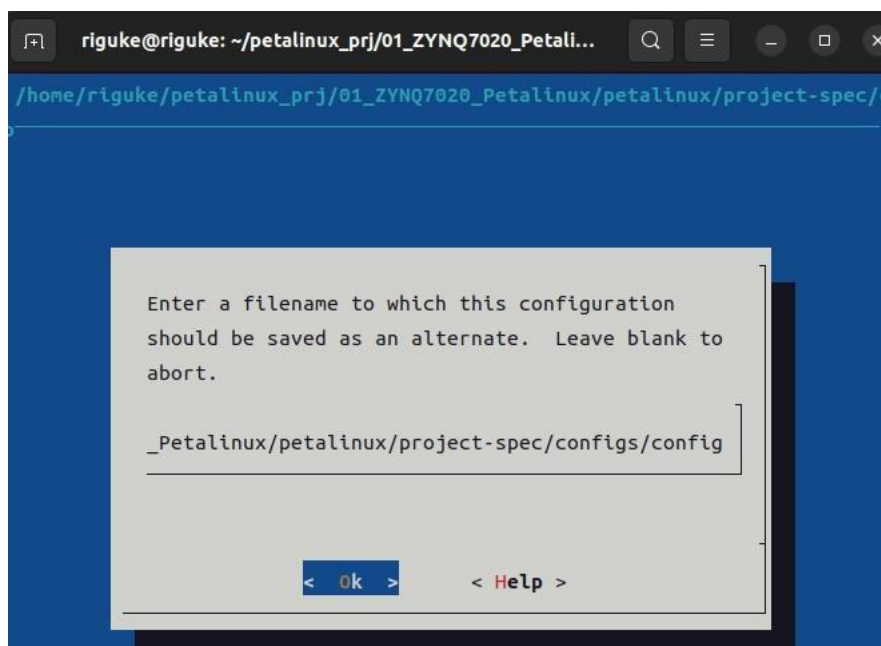


图 2-12 保存配置文件界面

再次按键盘上的“Enter”键确认，直接返回到主界面，到这里配置修改就完成了。最后按两次键盘上的“Esc”退出配置窗口，Petalinux工具开始自动配置工程。

(8) 这一步可能需要几分钟才能完成。这是因为 PetaLinux 会根据“Auto Config Settings -->”和“Subsystem AUTO Hardware Settings --->”来解析 xsa 文件，以获取更新设备树、U-Boot 配置文件和内核配置文件所需的硬件信息等待一段时间后，完成 Petalinux 工程的配置，如下图所示：

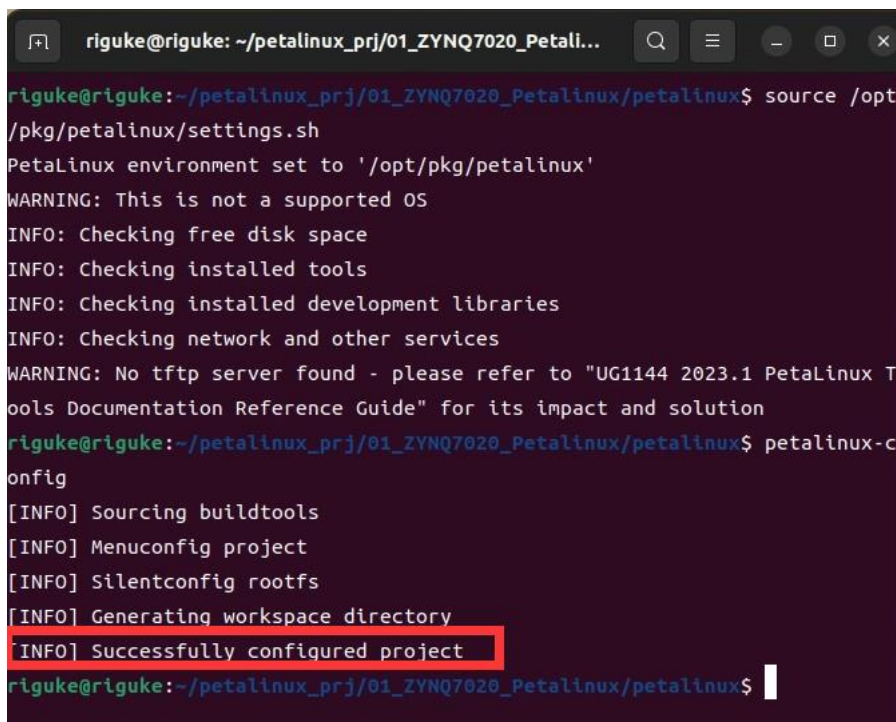


图 2-13 完成 petalinux 工程的配置

需要注意得是：第一次配置使用的指令并不是图 2-13 显示得指令，该指令是已经配置过，可以使用该指令再次进入 Petalinux 配置。



## 4. 设置离线编译

Petalinux 编译工程默认需要联网下载一些资源，如果网络不佳会编译非常缓慢甚至编译失败。为了解决这个问题，可以预先下载两个离线资源包，然后设置 Petalinux 离线编译。

这两个资源包在光盘路径：“RK-ZYNQ7020-F 开发板网盘资料 -> 7.开发软件 -> Petalinux”，该离线编译包也可以从 Xilinx 官网获取：[Downloads](#) 下载图中红框标出来的两个文件。注意，一定要下载 Petalinux 版本对应的文件，我们使用的 Petalinux 版本是 2023.1，就需要找到 2023.1 标签中的这两个文件下载。

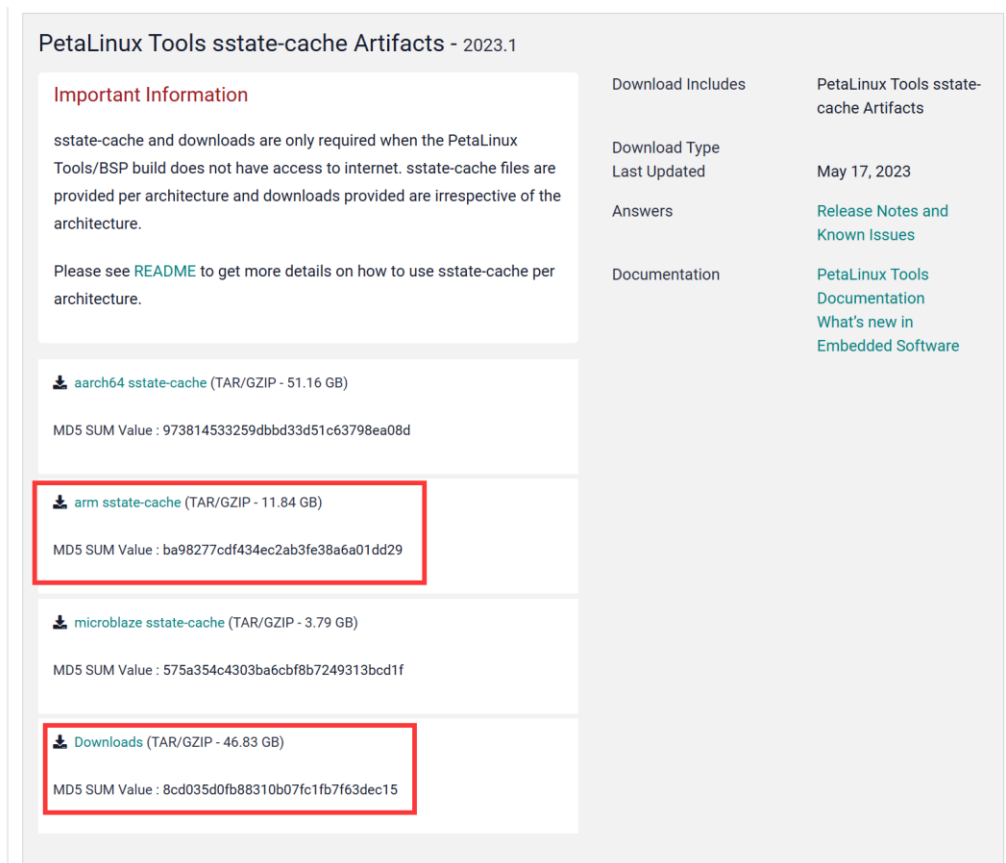


图 2-14 官网下载离线资源包

(1) 把资源包拷贝到 Linux 中并解压。这里我解压到/home/riguke/off\_bag 路径下，如下图所示：

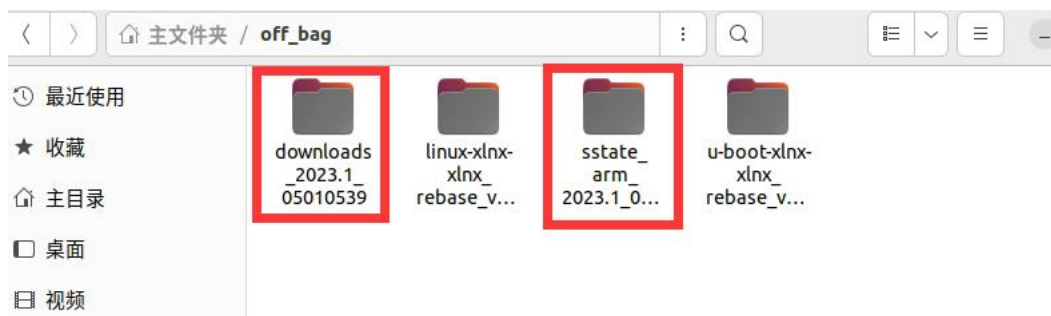


图 2-15 解压官网离线安装包

(2) 在 petalinux 工程中打开终端，输入命令  
命令为：petalinux-config

然后会弹出如下配置界面：

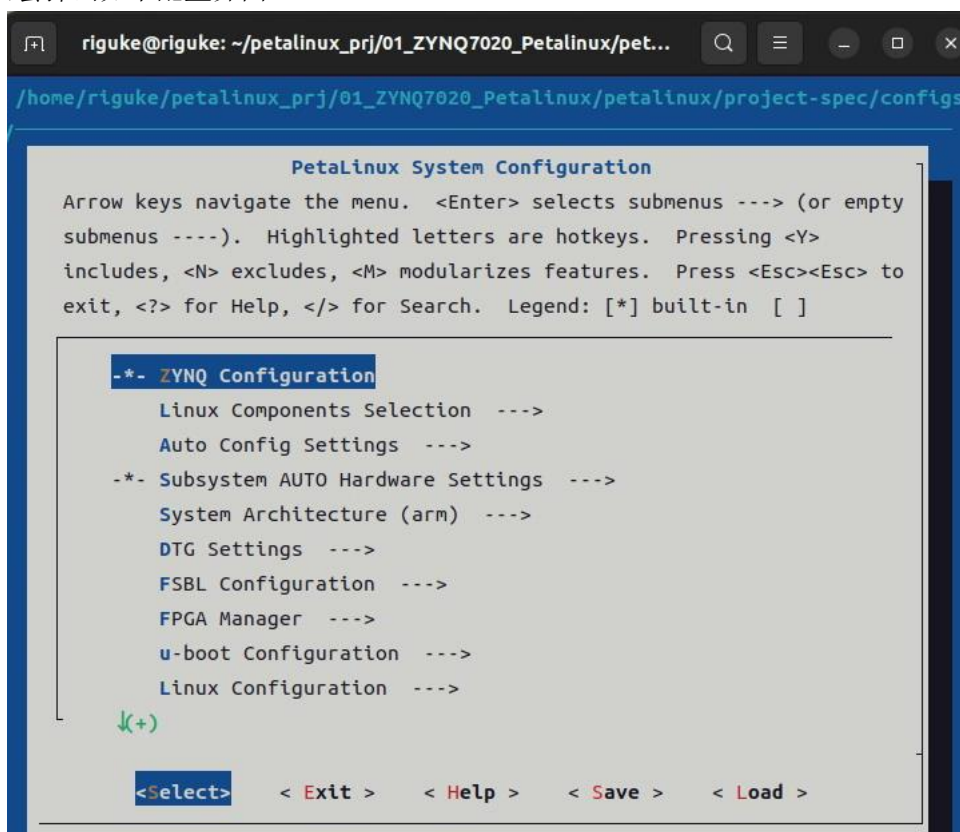


图 2-16 进入 petalinux config

(3) 选择最下面的“Yocto Settings --->”选项。

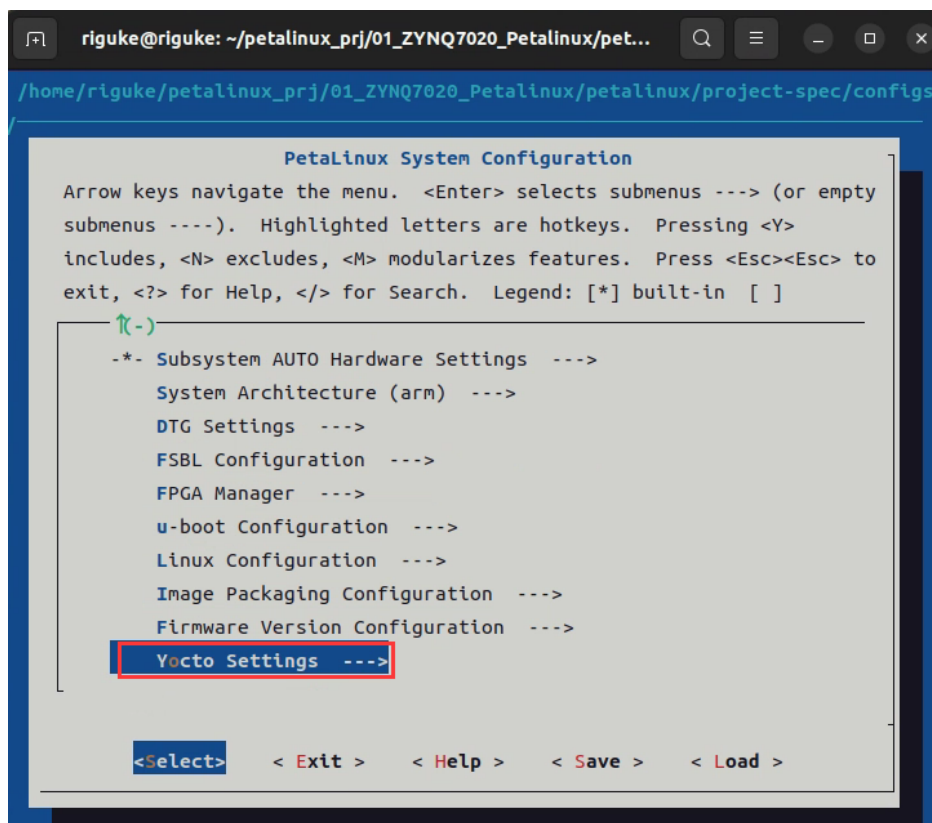


图 2-16 选择 Yocto Setting

(4) 在“Yocto Settings --->”界面中，取消选择“Enable Network sstate feeds”，选择“Enable BB NO NETWORK”选项，修改成如下图的状态：

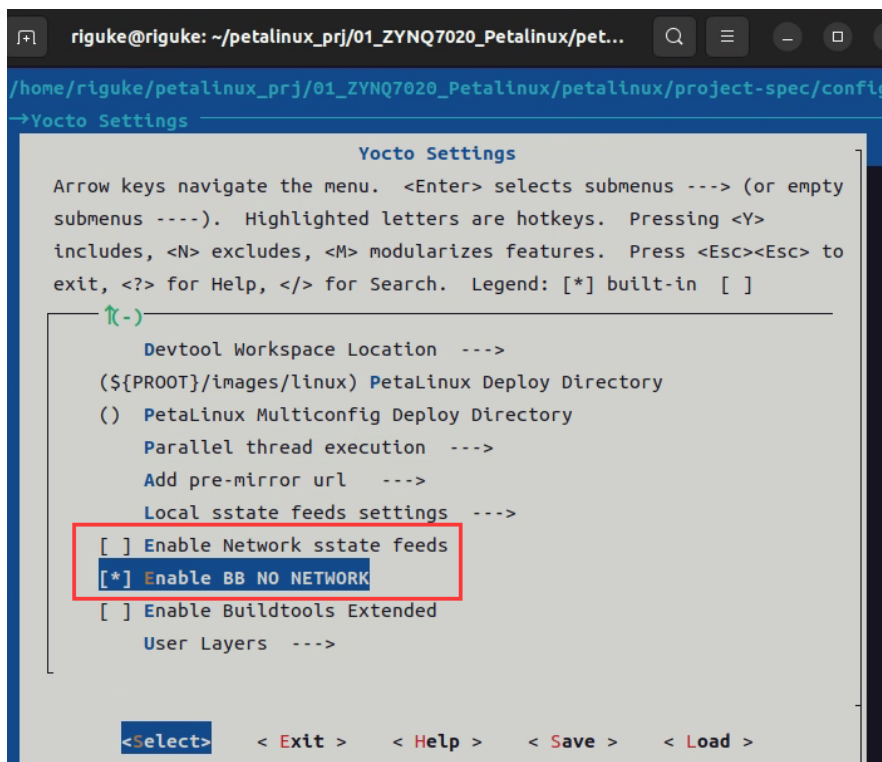


图 2-17 修改为不使用网络编译

(5) 设置“sstate\_arm”包的路径，选择“Local sstate feeds settings --->”选项

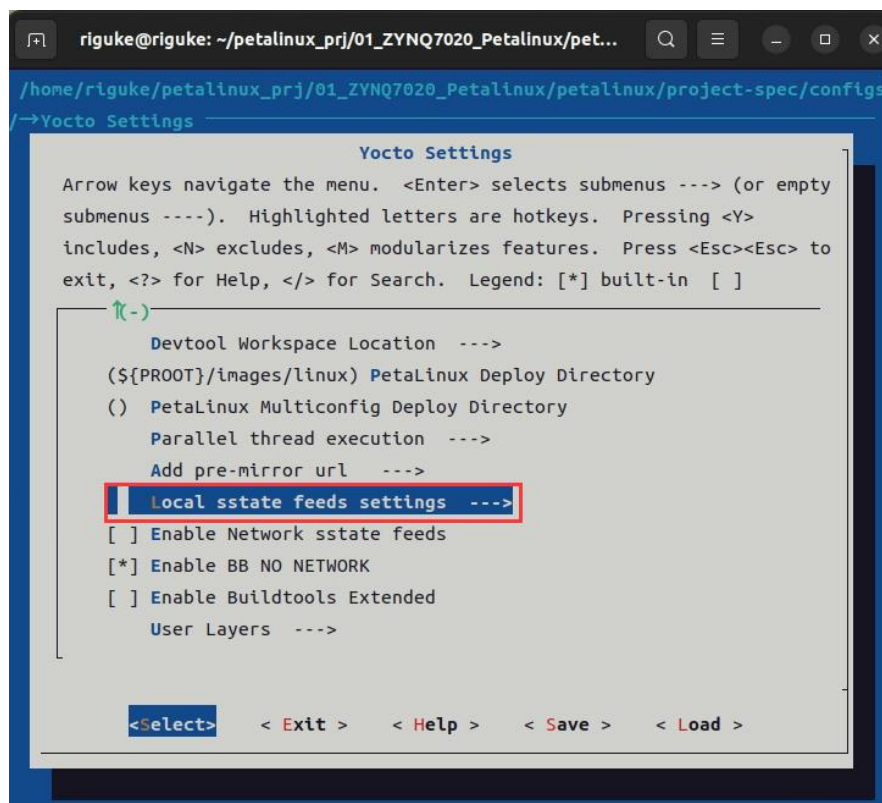


图 2-18 设置“sstate\_arm”包的路径

在之后的界面中直接按回车

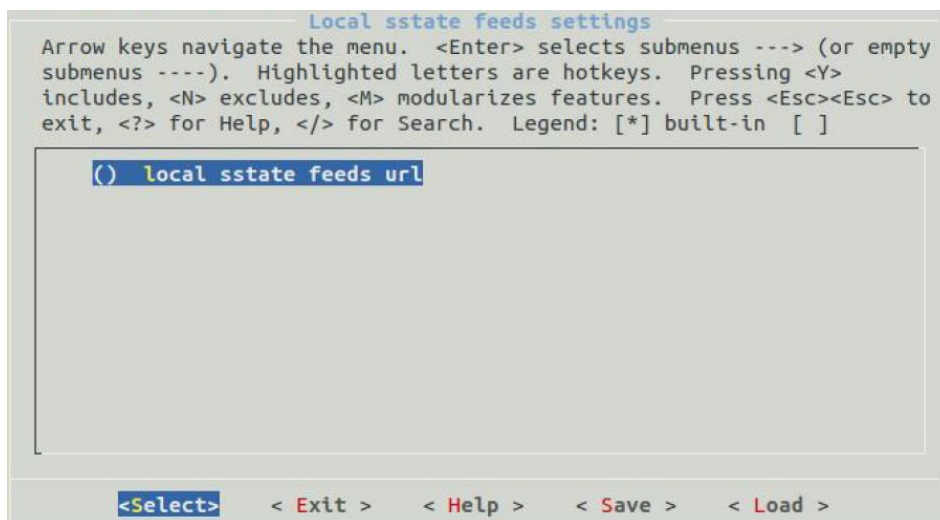


图 2-19 选择 sstate 包的位置

然后在弹窗中输入解压后的“arm”包的路径，注意：红色部分是自己电脑的路径需与自己环境相匹配。

命令为：`/home/riguke/off_bag/sstate_arm_2023.1_05010539/arm`

设置好之后按回车键选择< ok >选项。

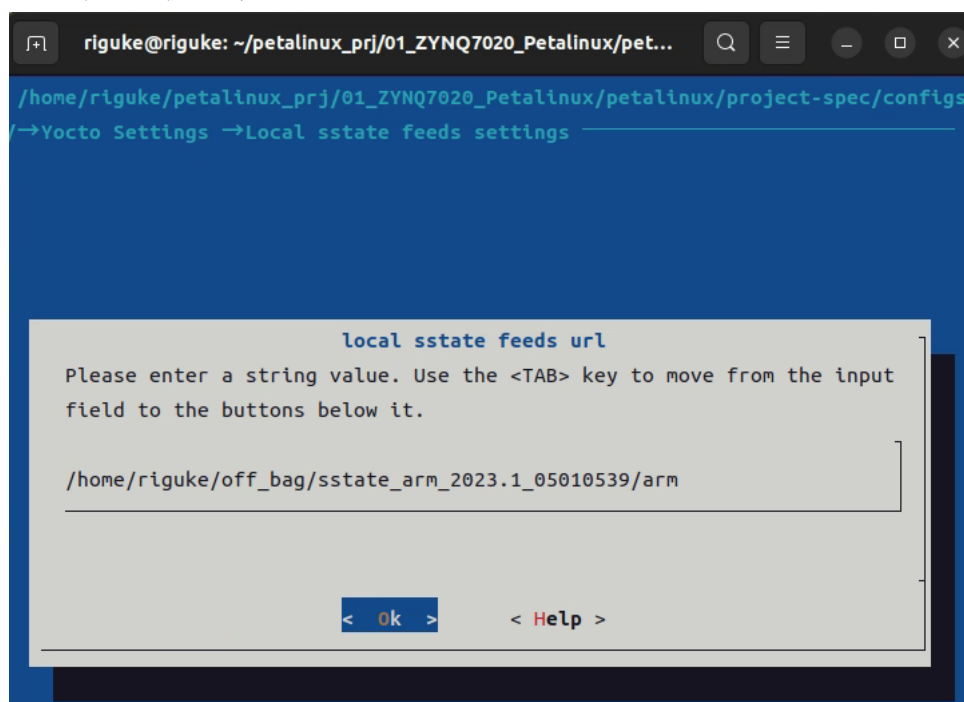


图 2-20 输入 sstate 包的位置

路径中的内容如下图：

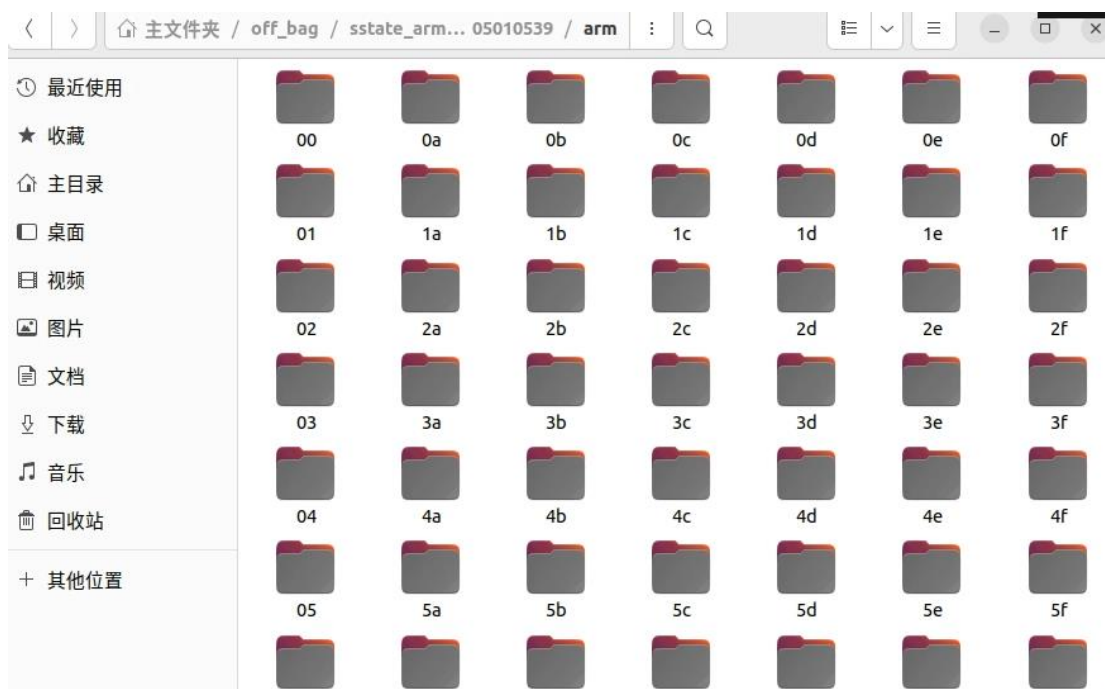


图 2-21 sstate 包内容

(6) 设置 “downloads” 包的路径，先选择< Exit >选项返回到下图的界面，然后选择“A dd pre-mirrorurl --->”选项

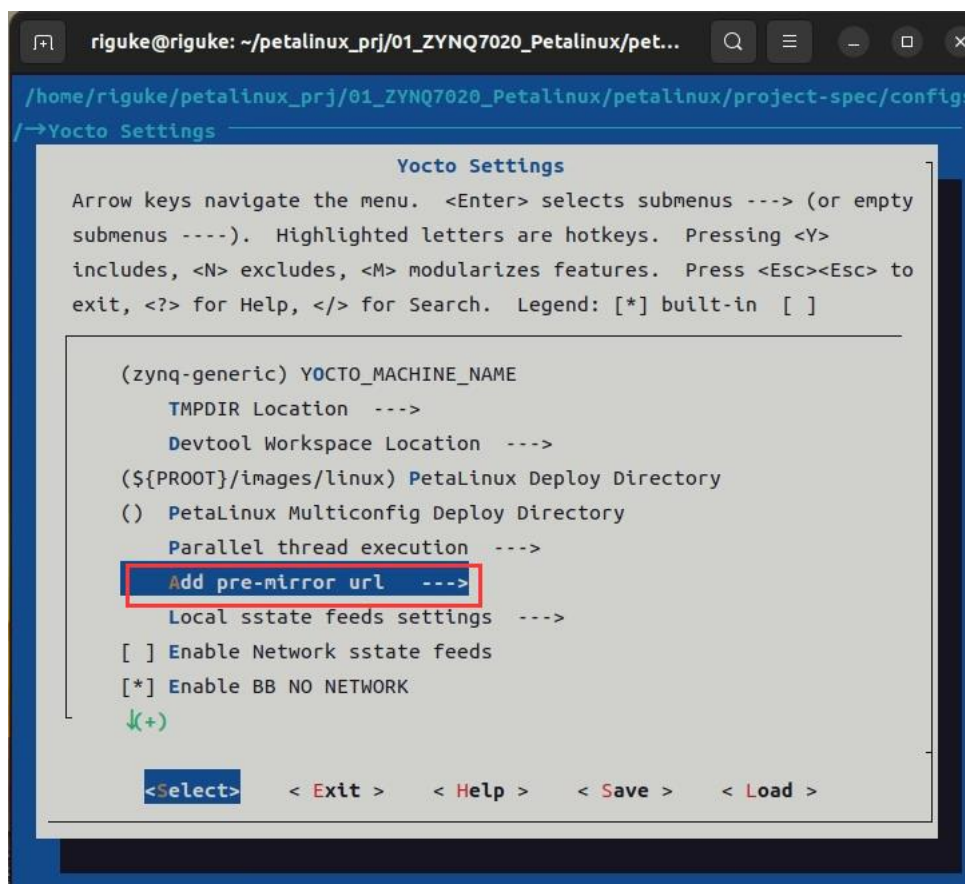


图 2-21 选择 Download 文件位置

修改这个选项里面原有的路径为: file:///home/riguke/off\_bag/downloads\_202



## 3.1\_05010539//downloads

注意路径以及路径格式，然后选择< ok >。

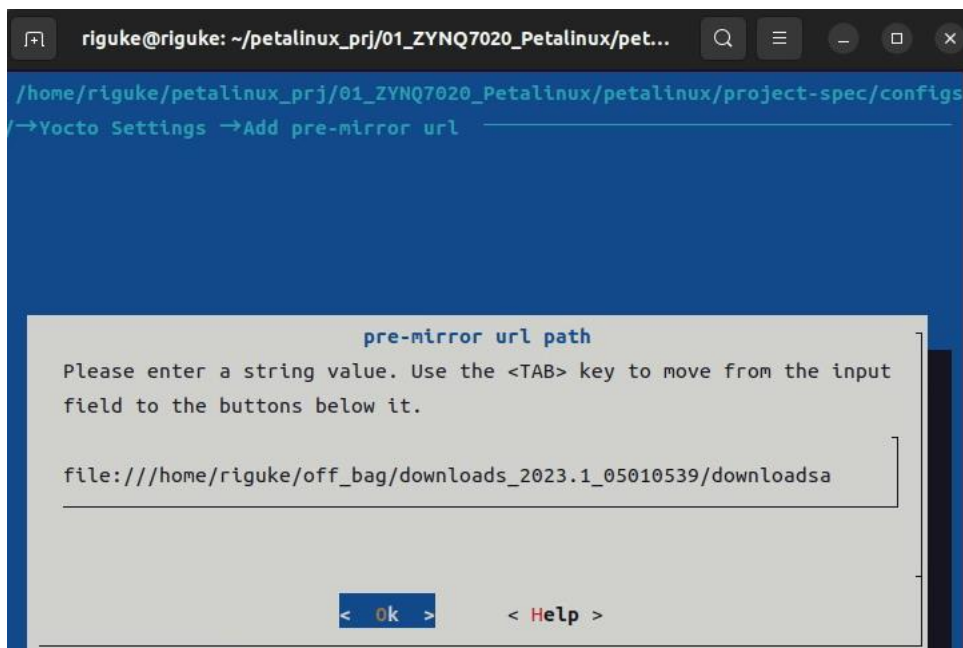


图 2-22 输入 Download 离线包的位置

路径中的内容如下图：

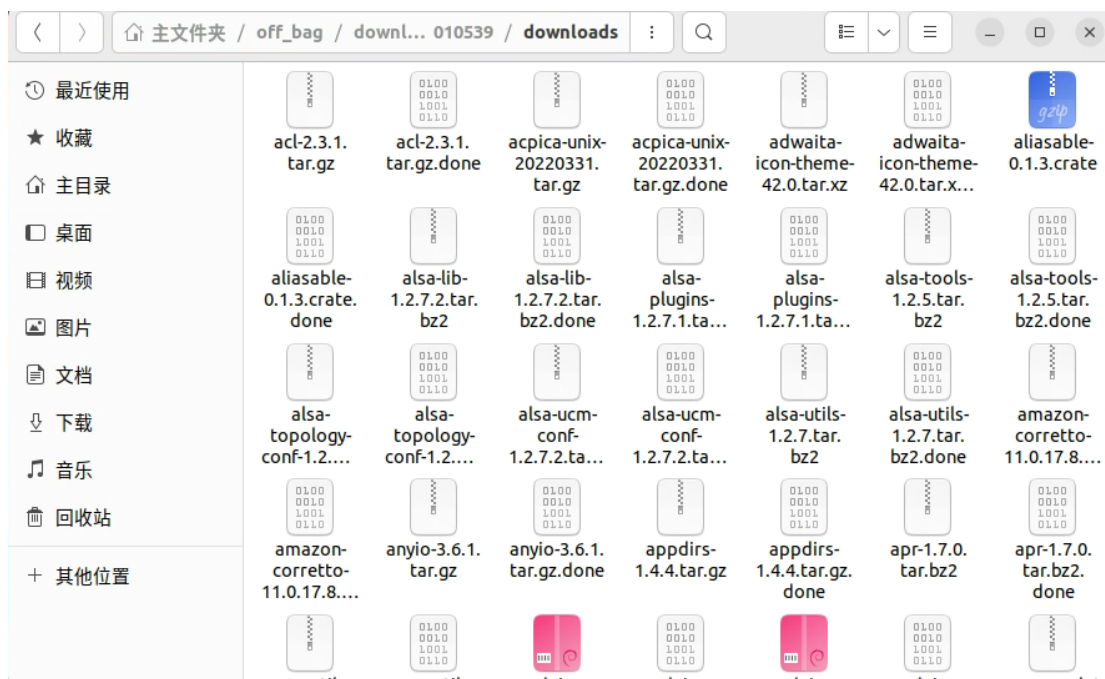


图 2-23 Download 离线包内容

(7) 选择< Save >选项，然后选择< Exit >选项退出配置界面即可。

```
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux$ petalinux-config
[INFO] Sourcing buildtools
[INFO] Menuconfig project
[INFO] Generating kconfig for rootfs
[INFO] Silentconfig rootfs
[INFO] Adding user layers
[INFO] Generating machine conf file
[INFO] Generating plnxtool conf file
[INFO] Generating workspace directory
[INFO] Successfully configured project
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux$
```

图 2-24 设置本地离线包配置成功

## 5. 设置本地内核源码以及 u-boot 源码

在选项 Linux Components Selection 中可以配置 uboot 和 Linux 内核的来源，按回车键可进入下一个选择界面。uboot 和 Linux 内核默认是 github 上下载的，需要 Linux 主机连接互联网才能下载。本实验将对应的内核都下载到了本地。内核源码和 u-boot 源码由 Xilinx 提供移植好的版本，可以到 Xilinx 的 Github 主页下载：[GitHub - Xilinx/linux-xlnx: The official Linux kernel from Xilinx](https://github.com/Xilinx/linux-xlnx)。在资料中提供了修改后的内核源码，而 **Linux 教程后续章节的内容** 都需要用到我们资料中的内核源码。

如果有修改内核源码以及 u-boot 源码的需要，我们可以设置使用本地的内核源码和 u-boot 源码。

注意：①一定要使用和 petalinux 对应版本；②如果多个 petalinux 工程都需要使用本地源码，那么每一个 petalinux 工程都需要单独对应内核源码和 u-boot 源码，否则其中一个 petalinux 工程修改了源码，就会影响到其他的工程。

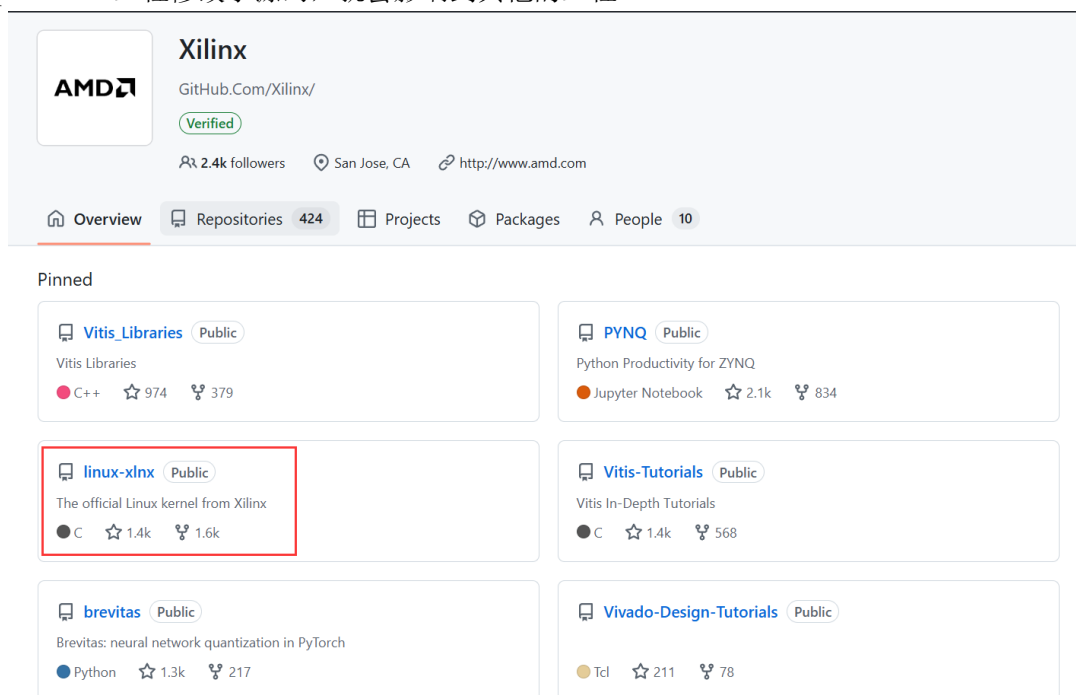


图 2-25 Github 上 Linux 内核源码



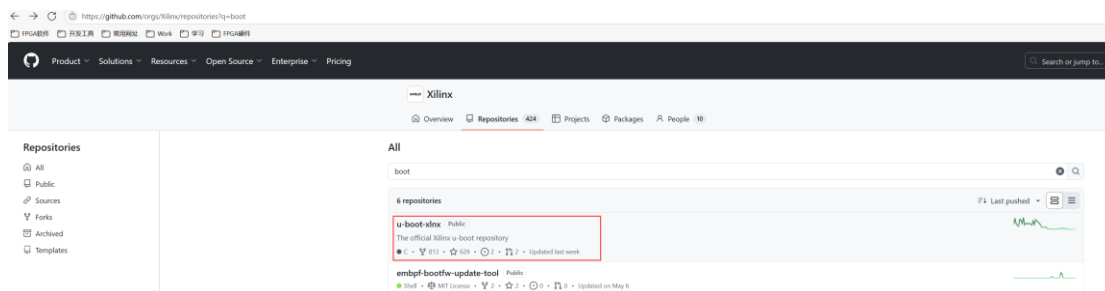


图 2-26 Github 上 u-boot 源码

我们提供的资源包中也包含了 xilinx 的内核源码和 u-boot 源码，可以直接使用。

(1) 把内核源码和 u-boot 源码拷贝到虚拟机中并解压。这里解压到/home/riguke/off\_bag 路径中，如下图所示：

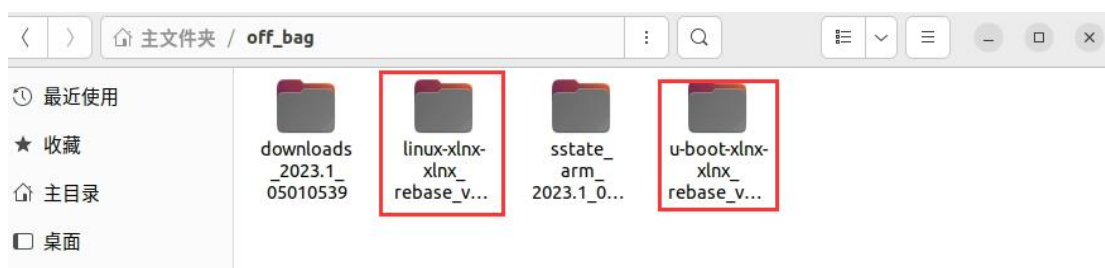


图 2-27 Linux 内核与 u-boot 源码

(2) 在 petalinux 工程中打开终端，输入命令

命令为: `petalinux-config`

弹出如下配置界面：

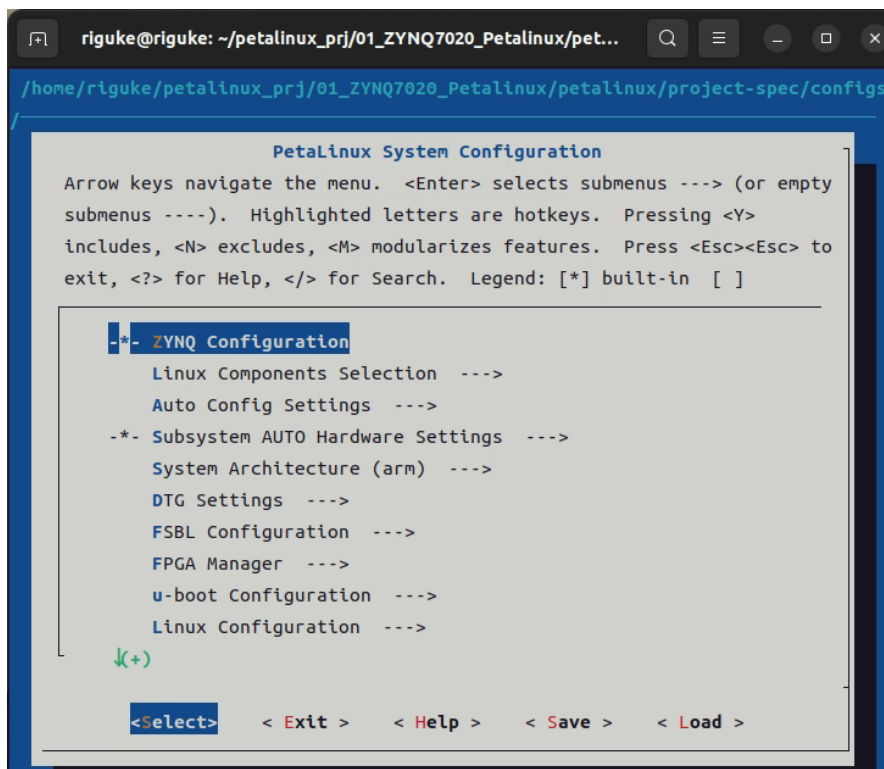


图 2-28 进入 Petalinux 配置

(3) 设置内核源码的路径，选择“Linux Components Selection --->”选项

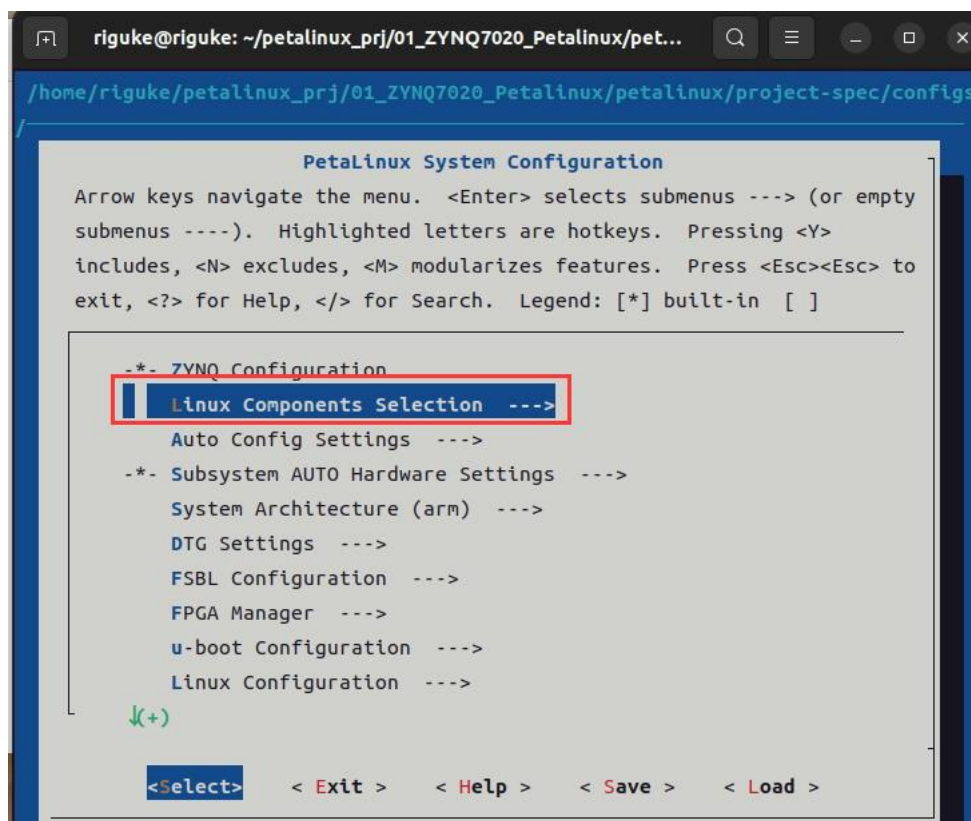


图 2-28 选择“Linux Components Selection --->”

然后选择“linux-kernel (linux-xlnx) --->”选项

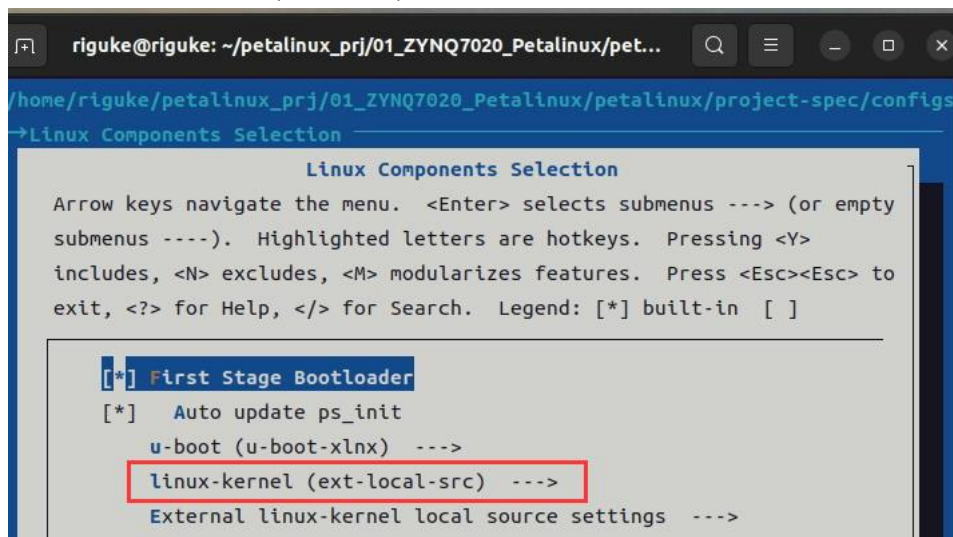


图 2-29 选择“linux-kernel (linux-xlnx) --->”

接下来的界面中选择最下面的“( ) ext-local-src”选项，

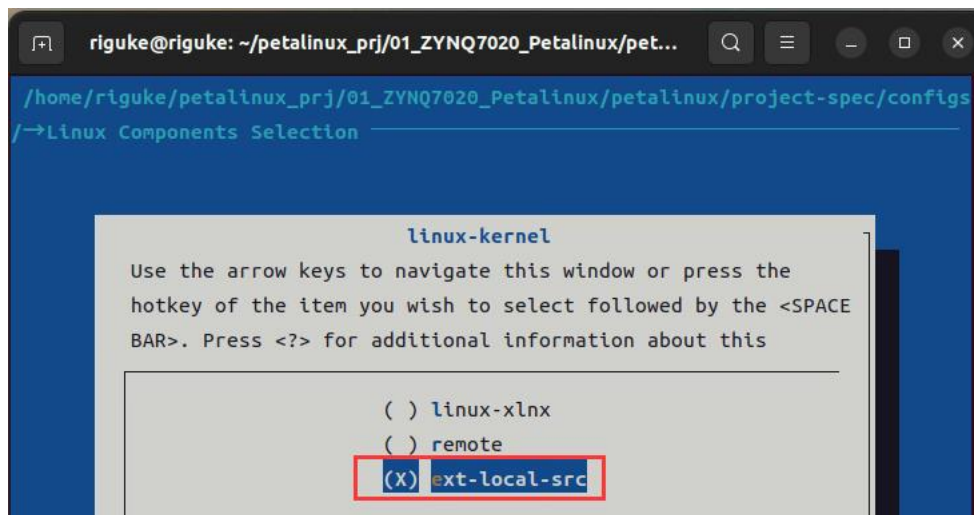


图 2-30 选择 “( ) ext-local-src”

然后刚才的界面中最下面会多出一行选项，按回车选择

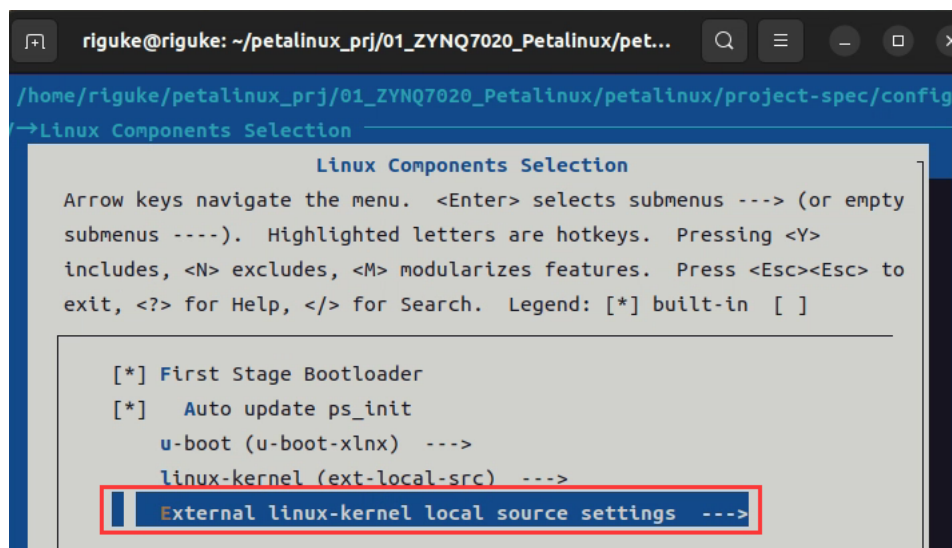


图 2-30 选择 Linux 内核路径

接下来就可以设置内核源码路径了，选择 “” 选项，输入内核源码路径

命令为：/home/riguke/off\_bag/linux-xlnx-xlnx\_rebase\_v6.1\_LTS\_2023.1

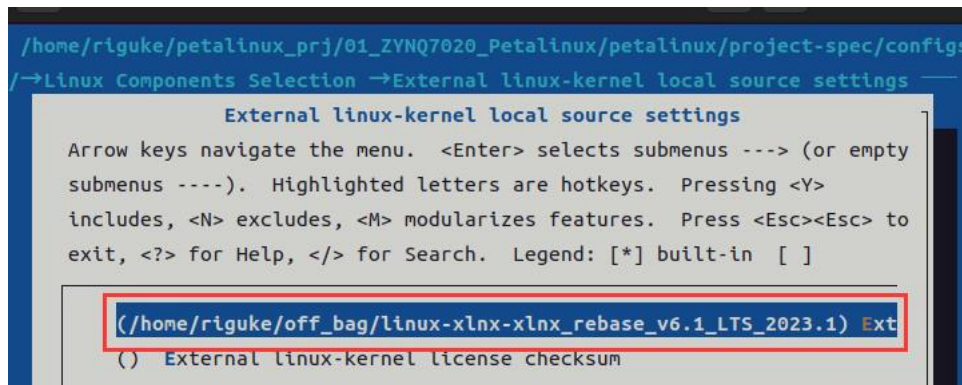


图 2-31 选择第一个路径

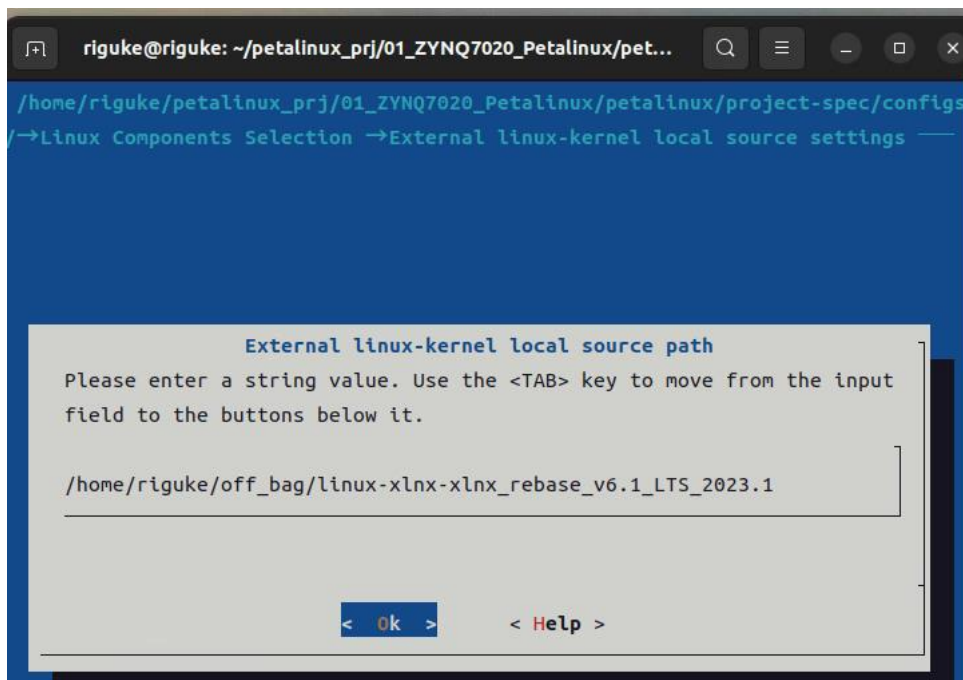


图 2-32 输入 Linux 内核路径

随后选择 ok

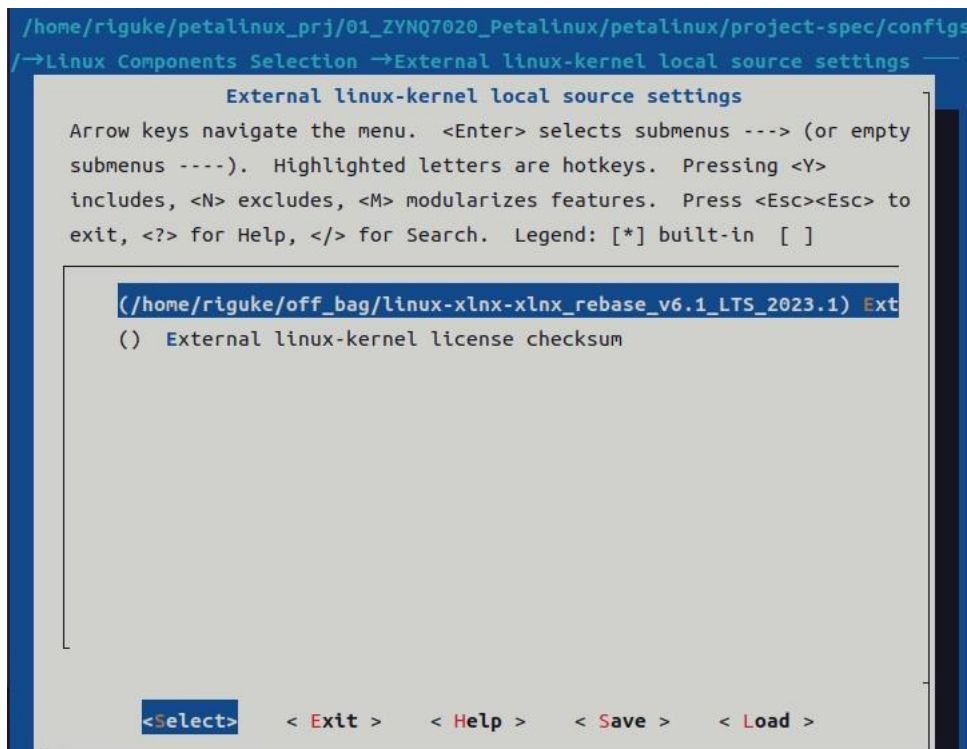


图 2-33 确认 Linux 内核路径

然后选择< Save >保存一下（如果不需要使用 u-boot 源码，选择<Exit>选项退出配置界面即可）。

（4）设置 u-boot 源码的路径，接着上面的步骤选择< Exit >选项回到下图的界面，选择“u-boot(u-boot-xlnx) --->”选项



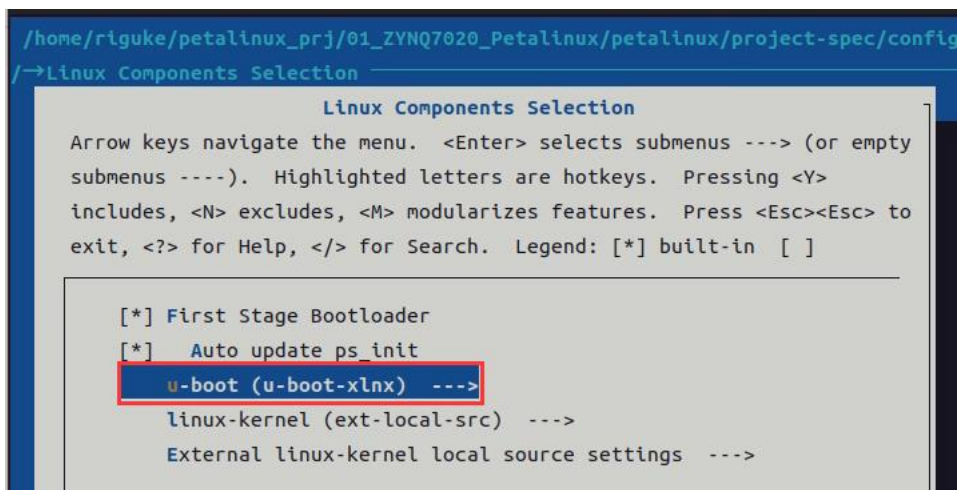


图 2-34 选择 u-boot 路径

选择 “() ext-local-src” 选项

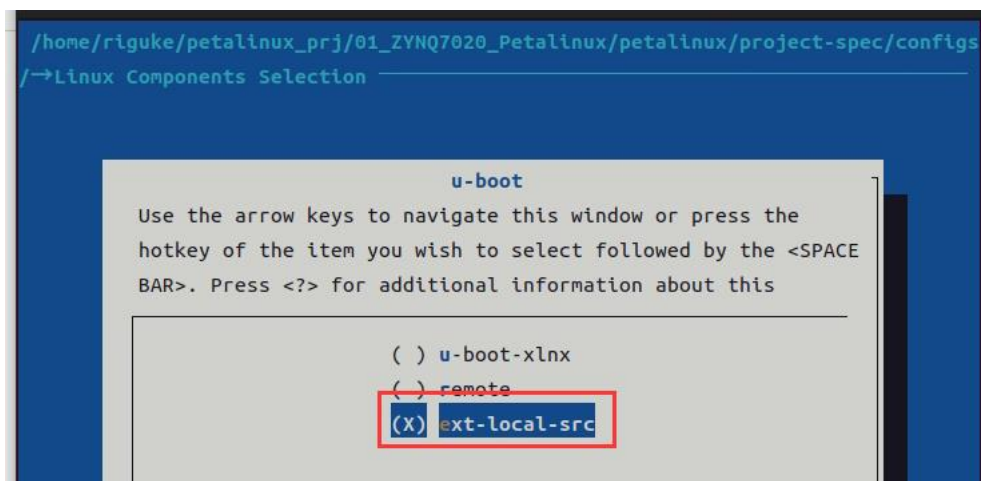


图 2-34 修改 u-boot 路径

然后选择多出来的这一行 “External u-boot local source settings --->” 选项

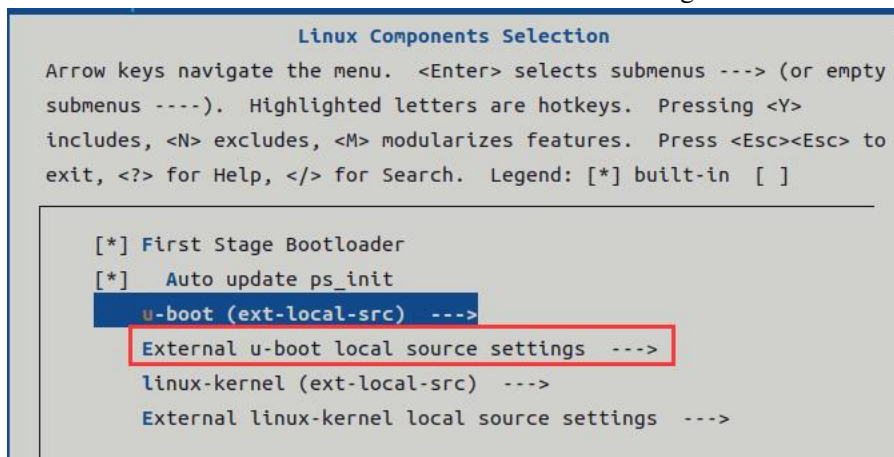


图 2-35 增加一行 u-boot 路径

选择 “() External u-boot local source path” 选项设置 u-boot 源码路径,

命令为: /home riguke/off\_bag/u-boot-xlnx-xlnx\_rebase\_v2023.01\_2023.1

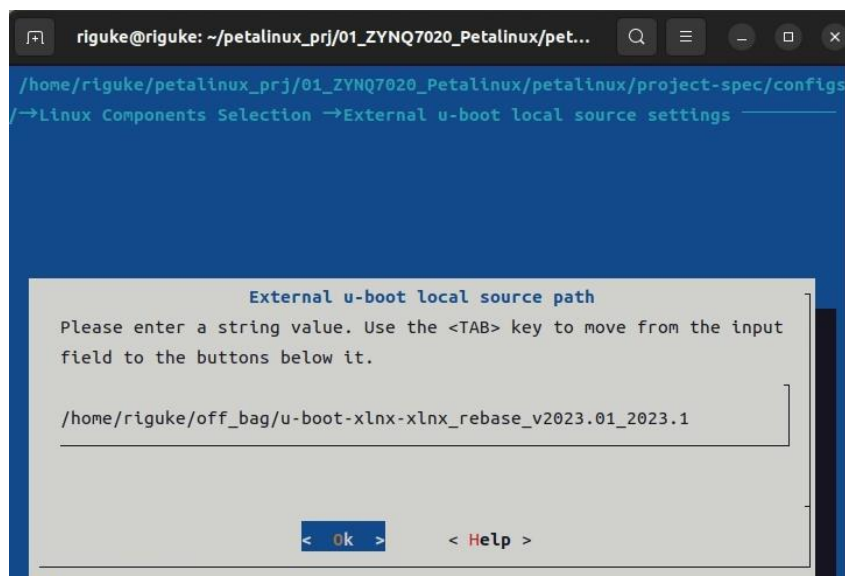


图 2-36 输入 u-boot 路径

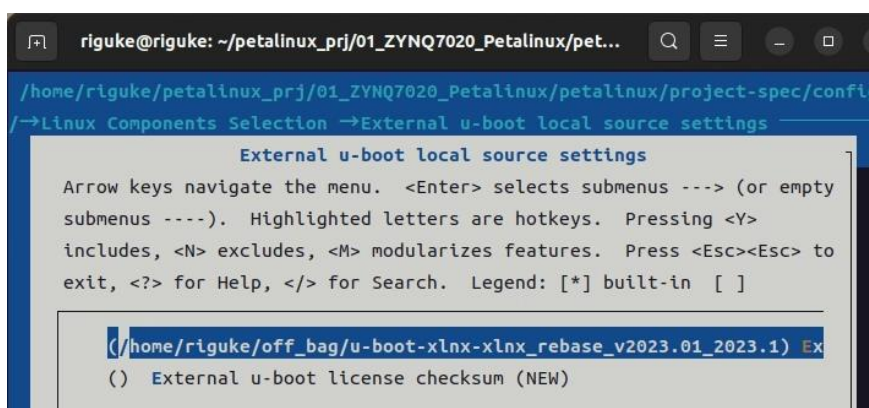


图 2-37 确认输入路径

然后选择< Save >选项再选择< Exit >选项退出配置界面即可。

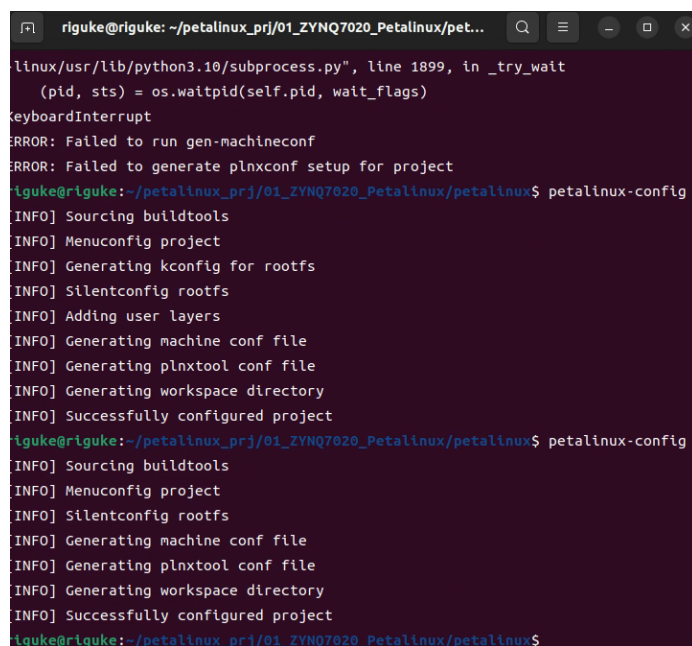


图 2-38 配置路径成功

## 6. 修改设备树、编译 petalinux 工程

关于设备树的概念，这里先不做介绍。设备树的概念源自于 Linux 内核当中，当然其实在 U-Boot 当中也已经使用了。如果需要配置设备树，可以编辑当前 petalinux 工程目录下的 project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi 文件



图 2-39 设备树文件

(1) 上图文件夹的路径中，“Home/peta\_prj\_linuxPsBase”是我们使用 mkdir 命令创建的路径，从“petalinux”开始往后的路径以及路径中的文件是由 petalinux-create 命令生成的。我们进入到 petalinux 工程目录下的 project-spec/meta-user/recipes-bsp/device-tree/files，是 petalinux 帮我们生成的自定义设备树文件，我们可以按照自己的需求来编辑这两个设备树这两个文件编辑两个或者只其中任意一个都可以，在本教程中我们选择编辑 system-user.dtsi 文件，双击打开这个文件把他修改为如下内容：



图 2-40 编写设备树

(2) 输入下面的命令，编译 petalinux 工程：

命令为：petalinux-build

注：如果没有设置离线编译，该步骤必须保证 ubuntu 系统可以连接网络，petalinux 需从 Github 上下载一些源代码。开始编译如下图所示：



```

riguke@riguke: ~/petalinux_prj/01_ZYNQ7020_Petalinux/pet...
[INFO] Generating workspace directory
[INFO] Successfully configured project
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux$ petalinux-config
-c rootfs
[INFO] Sourcing buildtools
[INFO] Silentconfig project
[INFO] Menuconfig rootfs
[INFO] Generating plnxtool conf file
[INFO] Successfully configured rootfs
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux$ petalinux-config
-c rootfs
[INFO] Sourcing buildtools
[INFO] Silentconfig project
[INFO] Menuconfig rootfs
[INFO] Generating plnxtool conf file
[INFO] Successfully configured rootfs
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux$ petalinux-build
[INFO] Sourcing buildtools
[INFO] Building project
[INFO] Silentconfig project
[INFO] Silentconfig rootfs
[INFO] Generating workspace directory
[INFO] bitbake petalinux-image-minimal

```

图 2-41 开始编译 Peatalinux 工程

编译完成后，终端如下图所示：

```

riguke@riguke: ~/petalinux_prj/01_ZYNQ7020_Petalinux/pet...
Setscene tasks: 1533 of 1533
Setscene tasks: 1533 of 1533
Setscene tasks: 1533 of 1533
Setscene tasks: 1533 of 1533
Setscene tasks: 1533 of 1533
Currently 20 running tasks (3004 of 4134) 72% |#####|
Setscene tasks: 1533 of 1533
Setscene tasks: 1533 of 1533
Setscene tasks: 1533 of 1533
Setscene tasks: 1533 of 1533
Setscene tasks: 1533 of 1533
Setscene tasks: 1533 of 1533
WARNING: xilinx-bootbin-1.0-r0 do_configure: Empty file /home/riguke/petalinux_p
rj/01_ZYNQ7020_Petalinux/petalinux/build/tmp/work/zynq_generic_7z020-xilinx-linu
x-gnueabi/xilinx-bootbin/1.0-r0/recipe-sysroot/boot/bitstream/download-zynq-gene
ric-7z020.bit, excluding from bif file
NOTE: Tasks Summary: Attempted 4134 tasks of which 0 didn't need to be rerun and
all succeeded.

Summary: There were 4 WARNING messages.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux$

```

图 2-42 成功编译 Petalinux 工程

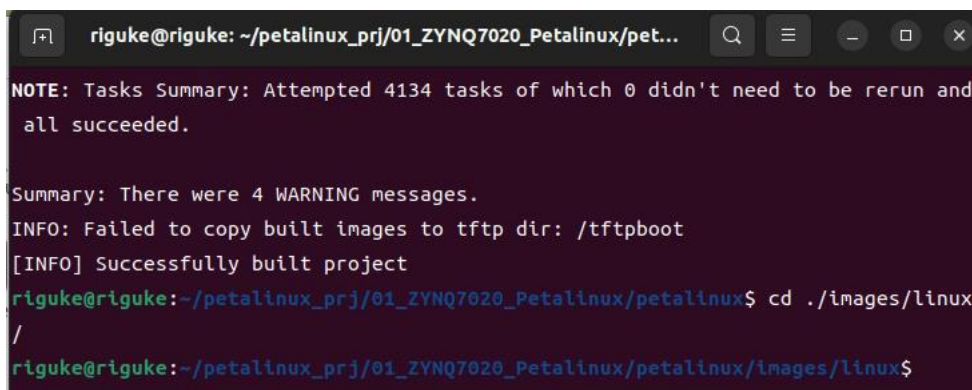
## 7. 生成 BOOT 文件

Petalinux 提供了 `petalinux-package` 命令将 PetaLinux 项目打包为适合部署的格式，其中“`petalinux-package --boot`”命令生成可引导映像，该映像可直接与 Zynq 系列设备（包括 Zynq-7000 和 Zynq UltraScale + MPSoC）或基于 MicroBlaze 的 FPGA 设计一起使用。对于 Zynq 系列设备，可引导格式为 BOOT.BIN，可以从 SD 卡引导启动。对于基于 MicroBlaze 的设计，默认格式为 MCS PROM 文件，适用于通过 Vivado 或其他 PROM 编程器进行编程。

ZYNQ 的启动文件 BOOT.BIN 一般包含 fsbl 文件、bitstream 文件和 uboot 文件。

(1) 打开一个终端，使用下面的命令，进入上面 petalinux 编译之后的结果目录中。

命令为：`cd petalinux_prj/01_ZYNQ7020_Petalinux/peatlinux/images/linux/`

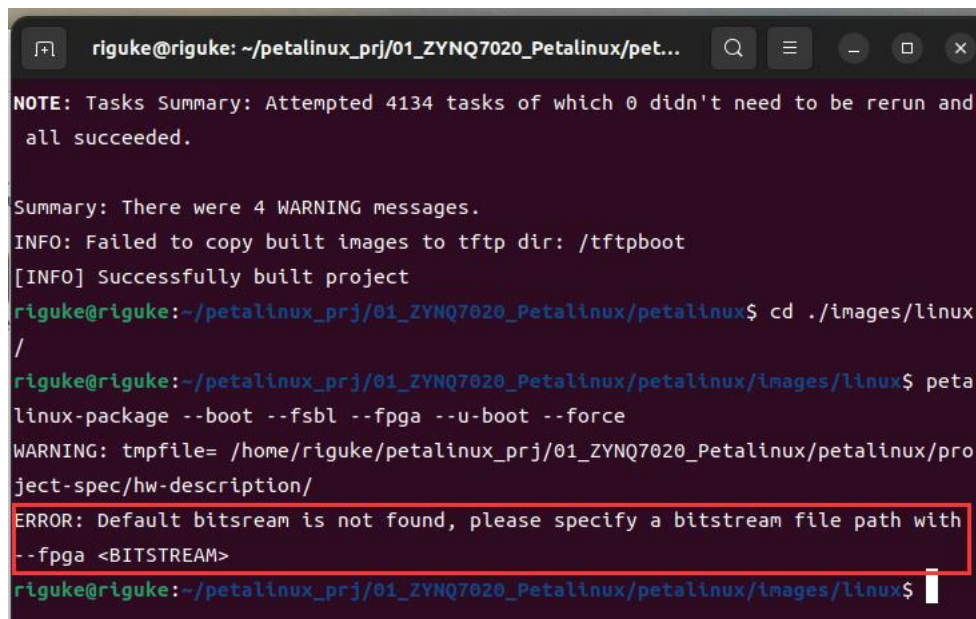


```
riguke@riguke: ~/petalinux_prj/01_ZYNQ7020_Petalinux/pet...
NOTE: Tasks Summary: Attempted 4134 tasks of which 0 didn't need to be rerun and
all succeeded.

Summary: There were 4 WARNING messages.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux$ cd ./images/linux
/
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux/images/linux$
```

图 2-43 进入 petalinux 编译之后的结果目录

(2) 使用“`petalinux-package --boot`”命令打包BOOT.bin文件。“`--u-boot`”参数会把u-boot 依赖项全部合成到BOOT.bin 中，“`--fpga`”指合成bit 文件到BOOT.bin 中(注意：如果你的xsa 文件中不包含bitstream，就不要加这个参数)，“`--fsbl`”指合成fsbl 文件到BOOT.bin 中，“`--force`”为强制合成，覆盖以前合成的BOOT.BIN。注意这条命令里的参数都是2个短横线。



```
riguke@riguke: ~/petalinux_prj/01_ZYNQ7020_Petalinux/pet...
NOTE: Tasks Summary: Attempted 4134 tasks of which 0 didn't need to be rerun and
all succeeded.

Summary: There were 4 WARNING messages.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux$ cd ./images/linux
/
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux/images/linux$ peta
linux-package --boot --fsbl --fpga --u-boot --force
WARNING: tmpfile= /home/riguke/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux/pro
ject-spec/hw-description/
ERROR: Default bitsream is not found, please specify a bitstream file path with
--fpga <BITSTREAM>
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux/images/linux$
```

图 2-44 没有 FPGA bit 文件导致打包失败

```

riguke@riguke: ~/petalinux_prj/01_ZYNQ7020_Petalinux/pet...
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux/images/linux$ petalinux-package --boot --fsbl --u-boot --force
[INFO] Sourcing buildtools
INFO: File in BOOT BIN: "/home/riguke/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux/images/linux/zynq_fsbl.elf"
INFO: File in BOOT BIN: "/home/riguke/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux/images/linux/u-boot.elf"
INFO: File in BOOT BIN: "/home/riguke/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux/images/linux/system.dtb"
INFO: Generating zynq binary package BOOT.BIN...

***** Bootgen v2023.1
**** Build date : Apr  7 2023-10:18:04
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
** Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.

[INFO] : Bootimage generated successfully

INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux/images/linux$

```

图 2-45 成功打包

(3) 进入到petalinux 工程编译结果目录中，可以看到红框中的三个文件，其中BOOT.bin文件是我们刚才生成的，其他两个文件是“petalinux-build”命令生成的。

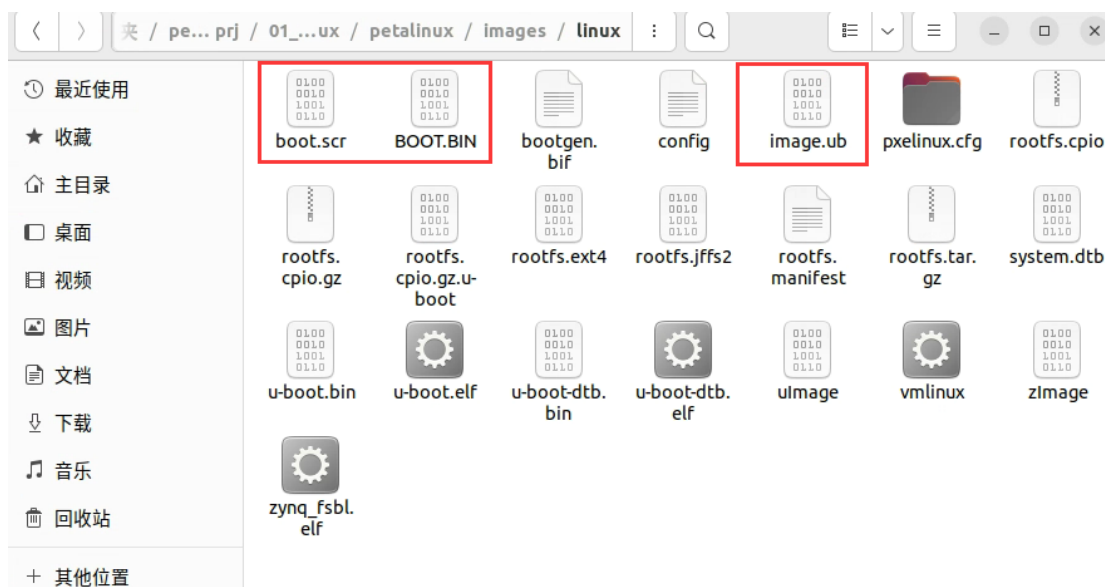


图 2-46 生成 Linux 镜像

## 8. 运行 Linux

(1) 把 SD 卡格式化为 FAT32 格式。



图 2-48 格式化 SD 卡

(2) 将之前得到的BOOT.bin、boot.scr 及image.ub 三个文件拷贝到上一步格式化的SD格式化后的SD卡中，然后弹出SD 卡。



图 2-47 复制到 SD 卡中

(3) 将 SD 卡插入板卡并设置为 SD 卡启动。



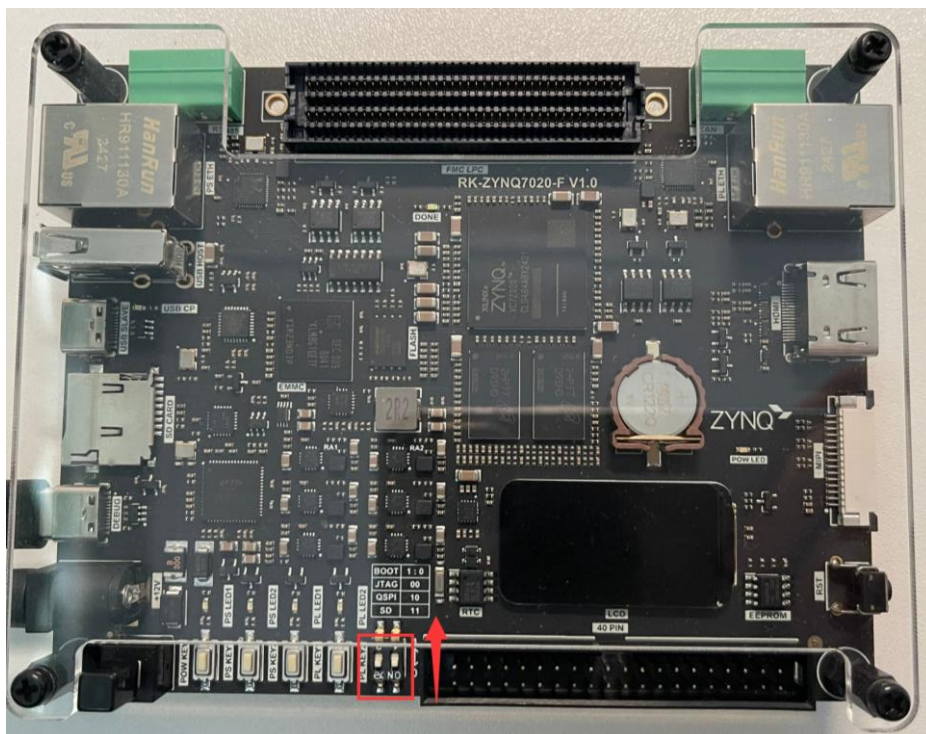


图 2-48 设置 ZYNQ 7020 为 SD 卡启动

(4) 连接串口线到开发板的“PS\_UART”口上，打开串口调试工具，将板卡上电，串口显示内容如下。

```
U-Boot 2023.01 (Jun 03 2025 - 15:29:41 +0000)

CPU:   Zynq 7z020
Silicon: v3.1
DRAM:  ECC disabled 1 GiB
Core:  20 devices, 15 uclasses, devicetree: board
Flash: 0 Bytes
NAND:  0 MiB
MMC:   mmc@e0100000: 0
Loading Environment from FAT... *** Error - No Valid Environment Area found
*** Warning - bad env area, using default environment

In:    serial@e0000000
Out:   serial@e0000000
Err:   serial@e0000000
Net:

ZYNQ GEM: e000b000, mdio bus e000b000, phyaddr 0, interface rgmii-id
eth0: ethernet@e000b000
Hit any key to stop autoboot:  0
switch to partitions #0, OK
mmc0 is current device
Scanning mmc 0:1...
Found U-Boot script /boot.scr
3016 bytes read in 17 ms (172.9 KiB/s)
## Executing script at 03000000
Trying to load boot images from mmc0
```

图 2-49 Petalinux 在 ZYNQ7020 上启动

(5) 在串口工具中输入用户名 root，密码 root，即可登录板载的 linux 系统（因为 petalinux 制作的根文件默认为 ram 类型，所以文件系统不能保存文件和配置即重启后回恢复初始状态）。

2023.1 版本的 Petalinux 是需要自定义用户名与密码的，可以依据个人喜好，进行设置

修改，为了方便，用户名和密码都设置为 root。

```
Starting rpcbind daemon...done.
starting statd: done
hwclock: can't open '/dev/misc/rtc': No such file or directory
Starting internet superserver: inetd.
NFS daemon support not enabled in kernel
Starting syslogd/klogd: done
Starting tcf-agent: OK

PetaLinux 2023.1+release-S05010539 petalinux ttyPS0

petalinux login: root
root@petalinux:~# ls
root@petalinux:~#
```

图 2-50 进入 ZYNQ Linux 系统

由于设备树只是简单写了 SD 卡，导致并不能直接看到现象，只能说明当前已经进入了 Linux 系统，不过不要紧，后续继续学习使用 Petalinux

## 9. Petalinux 使用流程总结

(1) Petalinux 工作环境：

- a. Petalinux 软件安装
- b. "/bin/sh" 需要是 bash，系统默认可能是 dash，需要修改
- c. 不能在共享文件夹创建 petalinux 工程
- d. 使用 petalinux 命令前需要设置 petalinux 环境变量：source<path-to-installed-

PetaLinux>/settings.sh

(2) 创建 Petalinux 工程：

- a. petalinux-create --type project --template <CPU\_TYPE> --name <PROJECT\_NAME>
- b. <CPU\_TYPE>可以是 zynqMP、zynq、microblaze
- c. <PROJECT\_NAME>就是工程名称

(3) 导入硬件信息：

- a. 在 petalinux 工程目录中打开终端
- b. 使用 petalinux-config --get-hw-description <HARDWARE>，<HARDWARE>是含有 xsa 文件的目录

(4) 配置 petalinux 工程：

- a. petalinux-config 配置整个 petalinux 工程，如果需要配置 uboot、kernel、rootfs 等，分别使用下面的命令：

- b. petalinux-config -c u-boot
- c. petalinux-config -c kernel
- d. petalinux-config -c rootfs

(5) 编译 petalinux 工程：

- a. petalinux-build

(6) 手动修改 petalinux 配置（这些配置文件可以直接复制到其他工程中使用）：

- a. 在“<plnx-proj-root>/project-spec/meta-user”目录保存着各种可以手动配置的文件
- b. “<plnx-proj-root>/project-spec/meta-user/recipes-bsp/u-boot/files/platform-top.h”文件用于修改 u-boot 配置



c. “<plnx-proj-root>/project-spec/meta-user/recipes-bsp/device-tree/files/”路径中的“system-user.dtsi”和“pl-custom.dtsi”文件用于自定义设备树

d. “<plnx-proj-root>/project-spec/meta-user/recipes-kernel/linux/linux-xlnx/”路径中保存了内核的配置

e. “<plnx-proj-root>/project-spec/meta-user/project-spec/meta-user/recipes-apps/”路径中保存着 app

#### (7) 清理 petalinux:

如果petalinux 编译出现各种问题，特别是我们修改一些配置后无法正常编译或者一直卡死，可以尝试使用命令“**petalinux-build -x mrproper -f**”来清理以前的编译结果。

## 10. 创建和使用 BSP 包

使用 Petalinux 的 BSP 包，我们可以方便快速的提供我们验证好的 Petalinux 工程给他人使用，或者是使用他人验证好的 Petalinux 工程。只需要掌握创建和使用 BSP 包的两条命令即可。

#### (1) 创建 bsp 包

使用下面的命令即可创建 petalinux 工程的 BSP 包到当前打开终端的路径中：

命令为：**petalinux-package --bsp -p ./petalinux/ --output chapter1.bsp**

-p 参数后面的路径是 petalinux 工程的一级路径，与工程名字相关

--output 参数后面跟的是 bsp 包的名称

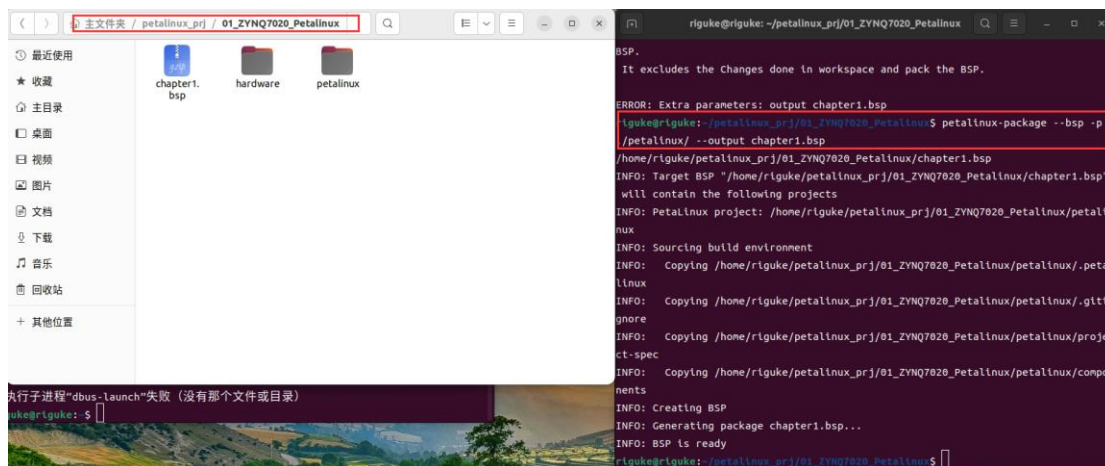


图 2-51 将 Petalinux 工程打包成为 BSP

#### (2) 使用 BSP 包

使用下面的命令即可用 BSP 包来创建 petalinux 工程到当前打开终端的路径中：

命令为：**petalinux-create -t project -n petalinux\_bsp -s ./chapter1.bsp**

-n 参数后面跟的是 petalinux 工程的名称

-s 参数后面跟的是使用的 BSP 包的路径

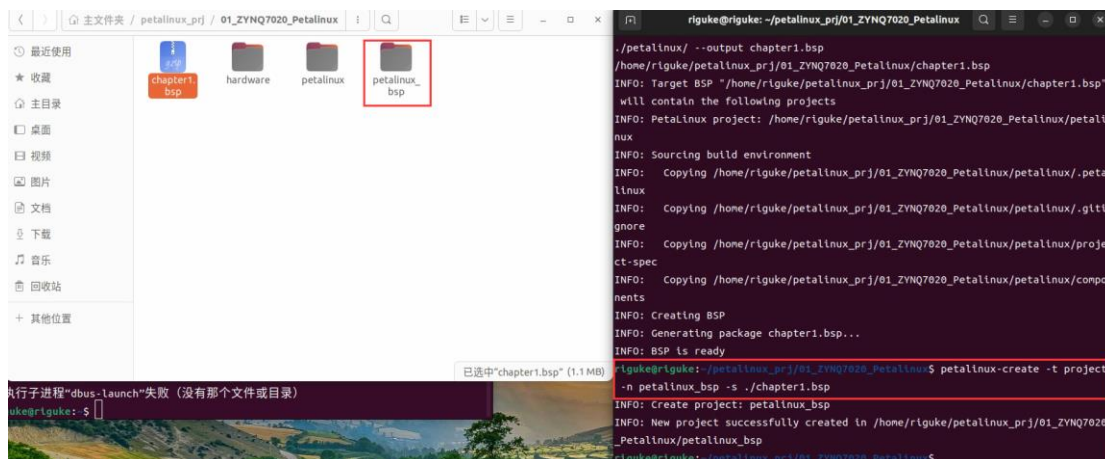


图 2-52 从 BSP 包恢复的 BSP 工程

打开BSP包创建的工程目录如下：

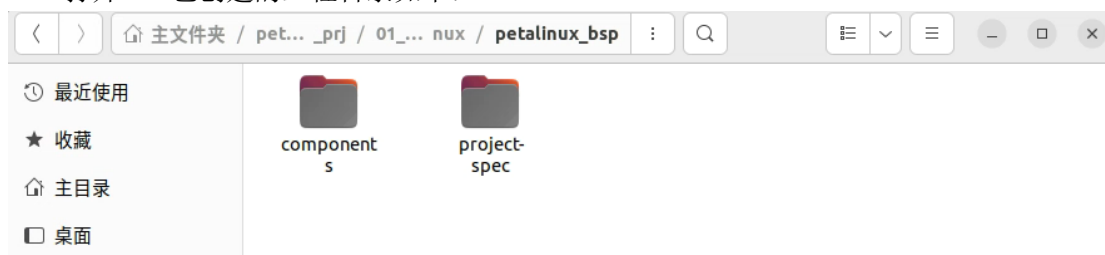


图 2-53 BSP 包工程目录

由于 Peatalinux 工程在编译的过程中会产生很多文件，占用硬盘空间比较大，因此在编译成功后，可以直接打包成 BSP 包，这样会极大的节省硬盘空间，如果制作 BSP 包的工程可以编译成功，那使用 BSP 包制作的工程也可以直接编译。

## 三. 添加开机自启动脚本

使用 Petalinux 来创建 Linux，默认的文件系统为 ramdisk 类型，这种类型的文件系统为只读类型，对该文件系统的任何修改，在系统重启后又会丢失。本章讲述如何在这种文件系统中加用户的应用和数据文件以及开机自动启动用户应用程序的方法。

### 1.准备工作

按上一章重新制作一个 Petalinux 工程，使用的硬件环境仍是上一章中 vivado 制作的硬件环境，在这就不重复介绍了。

编译完成后，就在该工程路径下打开终端，如下图所示：

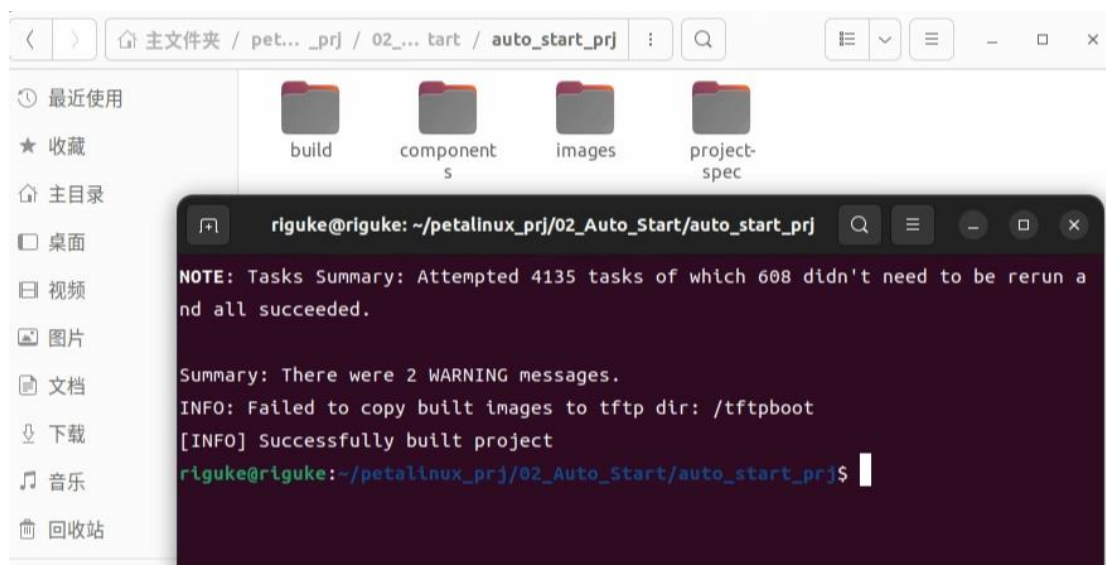


图 3-1 编译完成自启动工程

### 2.自启动脚本

(1) 输入下面的命令，用“install”模板，创建一个名为“autostart”的应用程序：

命令为: `petalinux-create -t apps --template install --name autostart --enable`

创建成功后，会提示我们文件所在路径，如下图红框中所示。

```

riguke@riguke: ~/petalinux_prj/02_Auto_Start/auto_start_prj
NOTE: Tasks Summary: Attempted 4135 tasks of which 608 didn't need to be rerun and all succeeded.

Summary: There were 2 WARNING messages.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
riguke@riguke:~/petalinux_prj/02_Auto_Start/auto_start_prj$ petalinux-create -t apps --template install --name autostart --enable
INFO: Create apps: autostart
INFO: New apps successfully created in /home/riguke/petalinux_prj/02_Auto_Start/auto_start_prj/project-spec/meta-user/recipes-apps/autostart
INFO: Enabling created component...
INFO: autostart has been enabled
riguke@riguke:~/petalinux_prj/02_Auto_Start/auto_start_prj$

```

图 3-2 创建 app 工程

(2) 我们进入刚才创建的程序所在的路径，内容如下图所示：

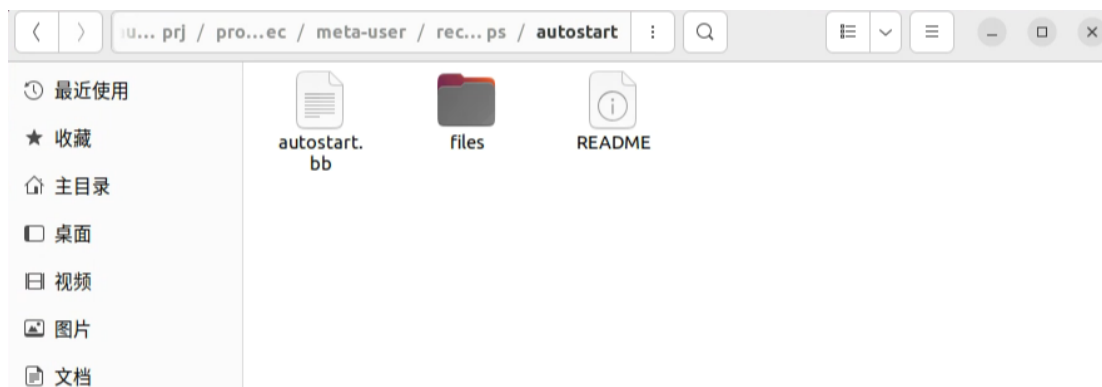


图 3-3 app 工程路径下文件内容

(3) 打开上图中的 files 文件夹下（不是上图的.bb 文件）的 autostart 文件，这是一个脚本文件，原来的内容是打印字符串“Hello Petalinux World”，我们将它改为下面的内容。

```

#!/bin/sh
echo "Hello RK XC7Z020!"
USERHOOK_SD0=/media/sd-mmcb1k0p1/autostart.sh
USERHOOK_SD1=/media/sd-mmcb1k1p1/autostart.sh
if [ -f $USERHOOK_SD0 ]; then
sh $USERHOOK_SD0 &
fi
if [ -f $USERHOOK_SD1 ]; then
sh $USERHOOK_SD1 &
fi

```

先打印“Hello RK XC7Z020”。然后判断 SD 卡或 EMMC 第一分区中是否存在“autostart.sh”脚本，如果存在就运行。

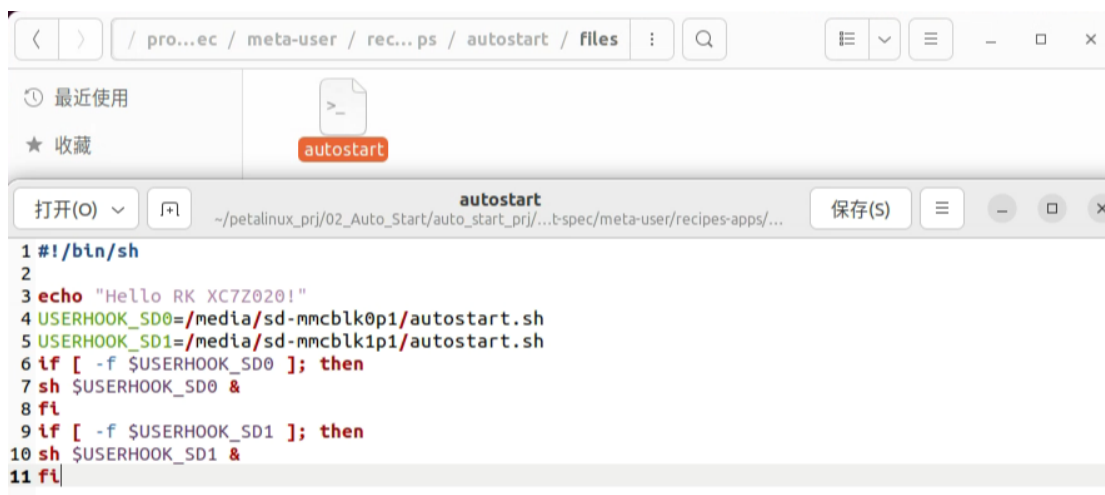


图 3-4 编写脚本内容

(4) 回到上一级目录，编辑文件 `autostart.bb` 为如下内容：

```
#
#This file is the autostart recipe.
#
SUMMARY = "Simple autostart application"
SECTION = "PETALINUX/apps"
LICENSE = "MIT"
LIC_FILES_CHKSUM =
    "file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ec
da2f7b4f302"
SRC_URI = "file://autostart \
"
S = "${WORKDIR}"
inherit update-rc.d
INITSCRIPT_NAME = "autostart"
INITSCRIPT_PARAMS = "start 99 5 ."
do_install() {
    install -d ${D}${sysconfdir}/init.d
    install -m 0755 ${S}/autostart
    ${D}${sysconfdir}/init.d/autostart
}
FILES_${PN} += "${sysconfdir}/*"
```

主要功能是复制 `autostart` 到根文件系统 `init.d` 目录，然后添加 `0755` 权限。`FILES_${PN} += "${sysconfdir}/*"`是指定哪些文件或目录应该被打包到最终生成的软件包（`${PN}`）中。



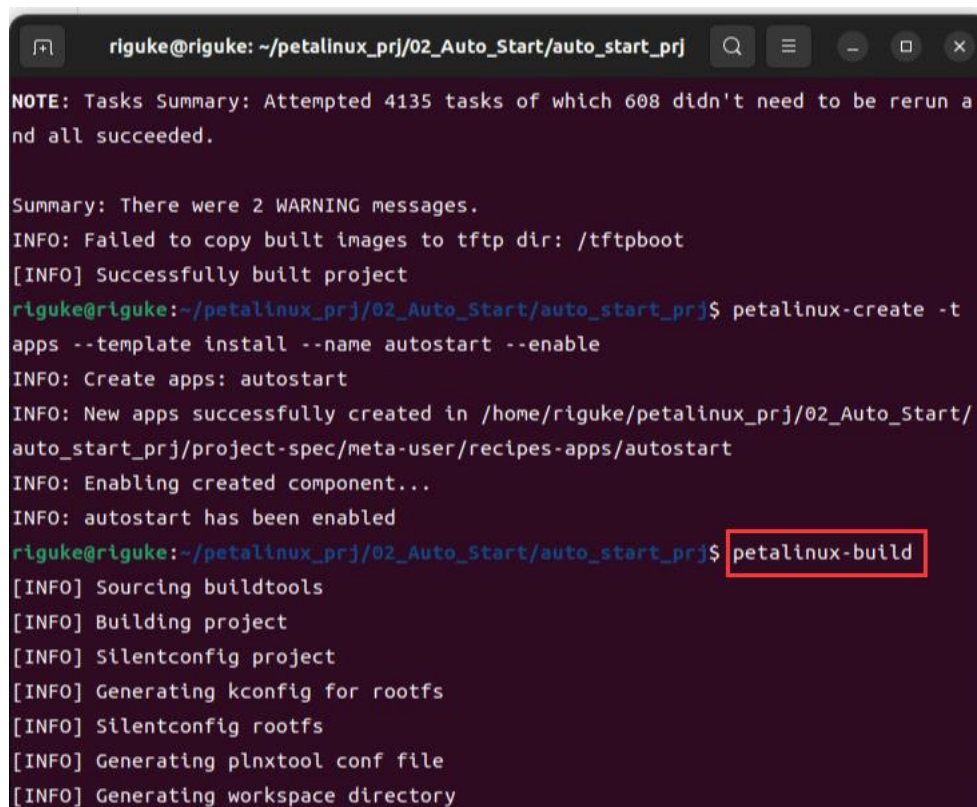
```

1 #
2 #This file is the autostart recipe.
3 #
4 SUMMARY = "Simple autostart application"
5 SECTION = "PETALINUX/apps"
6 LICENSE = "MIT"
7 LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"
8
9 SRC_URI = "file://autostart \
10
11 S = "${WORKDIR}"
12
13 inherit update-rc.d
14
15 INITSCRIPT_NAME = "autostart"
16 INITSCRIPT_PARAMS = "start 99 5 ."
17
18 do_install() {
19     install -d ${D}${sysconfdir}/init.d
20     install -m 0755 ${S}/autostart ${D}${sysconfdir}/init.d/autostart
21 }
22
23 FILES_${PN} += "${sysconfdir}/*"

```

图 3-4 复制 autostart 到根文件系统

(5) 随后，在终端中输入命令重新编译 petalinux 工程：



```

riguke@riguke: ~/petalinux_prj/02_Auto_Start/auto_start_prj
NOTE: Tasks Summary: Attempted 4135 tasks of which 608 didn't need to be rerun and all succeeded.

Summary: There were 2 WARNING messages.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
riguke@riguke:~/petalinux_prj/02_Auto_Start/auto_start_prj$ petalinux-create -t apps --template install --name autostart --enable
INFO: Create apps: autostart
INFO: New apps successfully created in /home/riguke/petalinux_prj/02_Auto_Start/auto_start_prj/project-spec/meta-user/recipes-apps/autostart
INFO: Enabling created component...
INFO: autostart has been enabled
riguke@riguke:~/petalinux_prj/02_Auto_Start/auto_start_prj$ petalinux-build
[INFO] Sourcing buildtools
[INFO] Building project
[INFO] Silentconfig project
[INFO] Generating kconfig for rootfs
[INFO] Silentconfig rootfs
[INFO] Generating plnxtool conf file
[INFO] Generating workspace directory

```

图 3-5 重新编译 petalinux 工程



```

riguke@riguke: ~/petalinux_prj/02_Auto_Start/auto_start_prj
Removing 2 stale sstate objects for arch zynq_generic_7z020: 100% || ETA: 0:00:
Removing 2 stale sstate objects for arch zynq_generic_7z020: 100% || Time: 0:00:
00
NOTE: Executing Tasks
NOTE: Tasks Summary: Attempted 4150 tasks of which 4116 didn't need to be rerun
and all succeeded.

Summary: There was 1 WARNING message.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
riguke@riguke:~/petalinux_prj/02_Auto_Start/auto_start_prj$

```

图 3-6 编译成功 petalinux 工程

(6) 随后，使用下面命令打包 BOOT.bin:

命令为: `petalinux-package --boot --u-boot --fsbl --force`

```

riguke@riguke:~/petalinux_prj/02_Auto_Start/auto_start_prj$ petalinux-package --
boot --u-boot --fsbl --force
[INFO] Sourcing buildtools
INFO: File in BOOT BIN: "/home/riguke/petalinux_prj/02_Auto_Start/auto_start_prj
/images/linux/zynq_fsbl.elf"
INFO: File in BOOT BIN: "/home/riguke/petalinux_prj/02_Auto_Start/auto_start_prj
/images/linux/u-boot.elf"
INFO: File in BOOT BIN: "/home/riguke/petalinux_prj/02_Auto_Start/auto_start_prj
/images/linux/system.dtb"
INFO: Generating zynq binary package BOOT.BIN...

***** Bootgen v2023.1
**** Build date : Apr  7 2023-10:18:04
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
** Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.

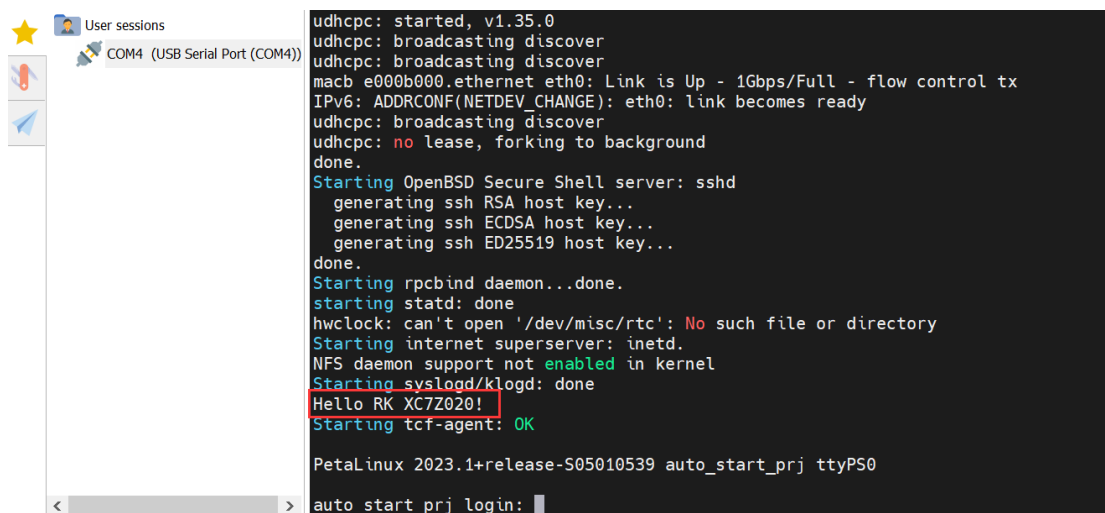
[INFO] : Bootimage generated successfully

INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
riguke@riguke:~/petalinux_prj/02_Auto_Start/auto_start_prj$

```

图 3-7 成功打包 Boot.bin

(7) 把 BOOT.bin、boot.scr 及 image.ub 三个文件拷贝到格式化 FAT 格式的 SD 卡中。连接好串口，使用这张 SD 卡启动开发板，实验结果如下所示。



```
udhcpd: started, v1.35.0
udhcpd: broadcasting discover
udhcpd: broadcasting discover
macb e000b000.ethernet eth0: Link is Up - 1Gbps/Full - flow control tx
IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready
udhcpd: broadcasting discover
udhcpd: no lease, forking to background
done.
Starting OpenBSD Secure Shell server: sshd
  generating ssh RSA host key...
  generating ssh ECDSA host key...
  generating ssh ED25519 host key...
done.
Starting rpcbind daemon...done.
Starting statd: done
hwclock: can't open '/dev/misc/rtc': No such file or directory
Starting internet superserver: inetd.
NFS daemon support not enabled in kernel
Starting syslogd/klogd: done
Hello RK XC7Z020!
Starting tcf-agent: OK

PetaLinux 2023.1+release-S05010539 auto_start_prj ttyPS0
auto_start_prj login: 
```

图 3-8 自动启动

可以看到系统启动后打印了“Hello RK 1XC7Z020”。

## 四. 使用 Vitis 开发 Linux 应用

上一章学习了在 Petalinux 搭建的 Linux 系统上基础外设的使用,本章以创建“Hello World”工程为例学习如何使用 Vitis 开发 Linux 应用以及如何让应用程序运行在 Petalinux 搭建的 Linux 系统上。运行方式以第一章中 NFS 服务方式运行。

### 1. 创建基于 Vitis 的 Linux 平台工程

(1) 在/home/riguke/petalinux\_prj 目录下新建一个文件夹,命名为:02\_Linux\_app,用于 Vitis 的工作空间:

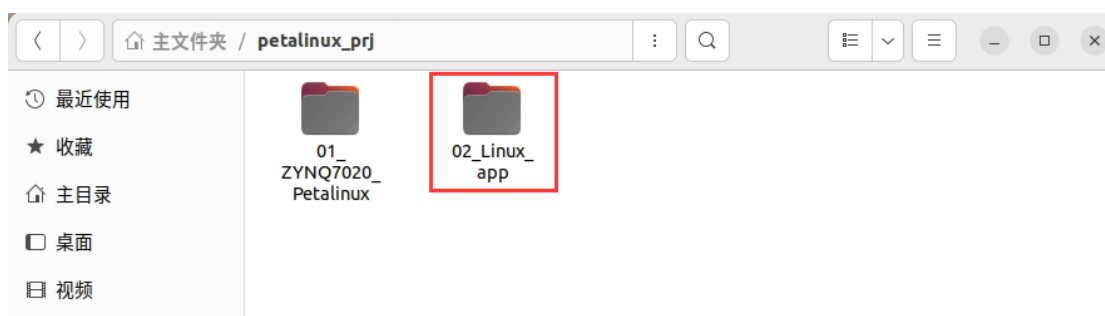


图 4-1 新建 Linux\_app 工作目录

(2) 设置 Vitis 环境变量,运行 vitis

命令为: `source /home/riguke/Xilinx/Vitis/2023.1/settings64.sh`  
`vitis`

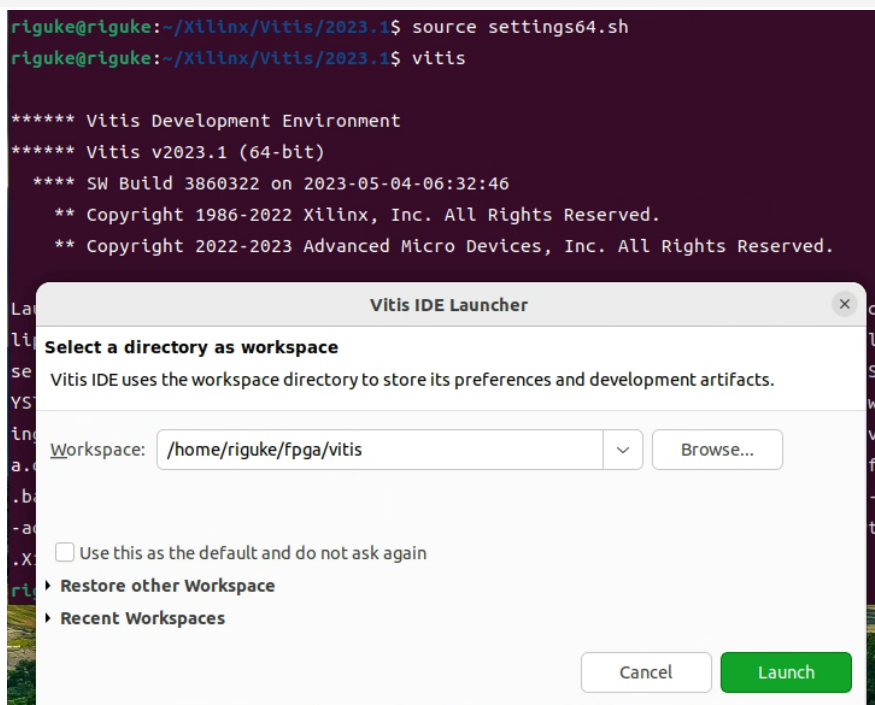


图 4-2 新建 Vitis 工程

(3) 工作空间选/home/riguke/petalinux\_prj/03\_Linux\_app

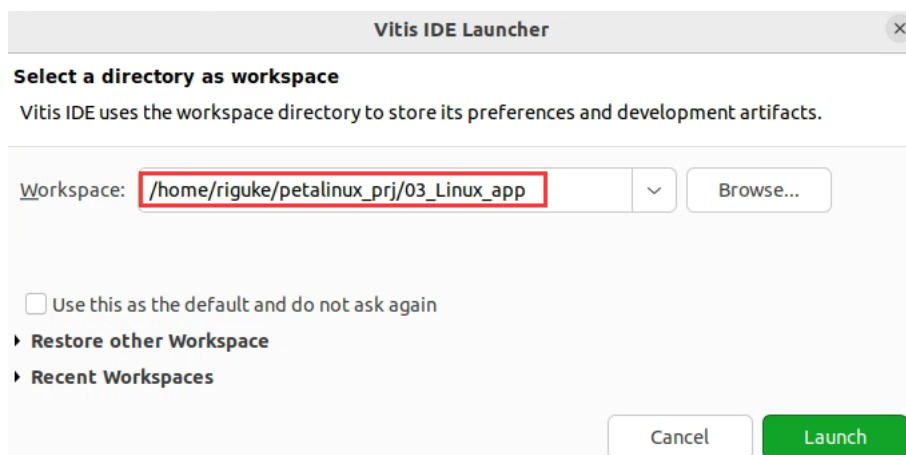


图 4-3 选择 Workspace 路径

(4) 点击“Welcome”旁边的 X，关闭欢迎界面。

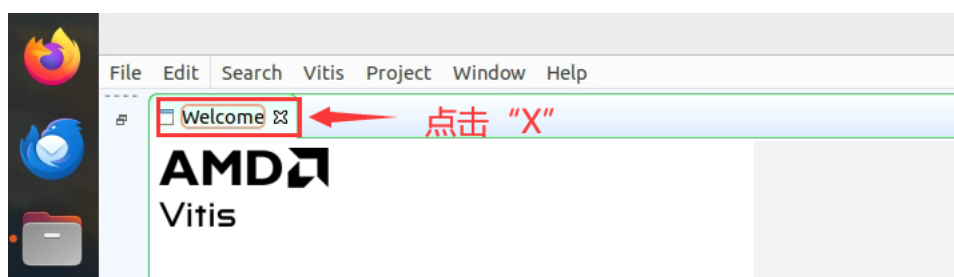


图 4-4 关闭欢迎界面

(5) 在 Explorer 空白处点击右键“New->Project...”

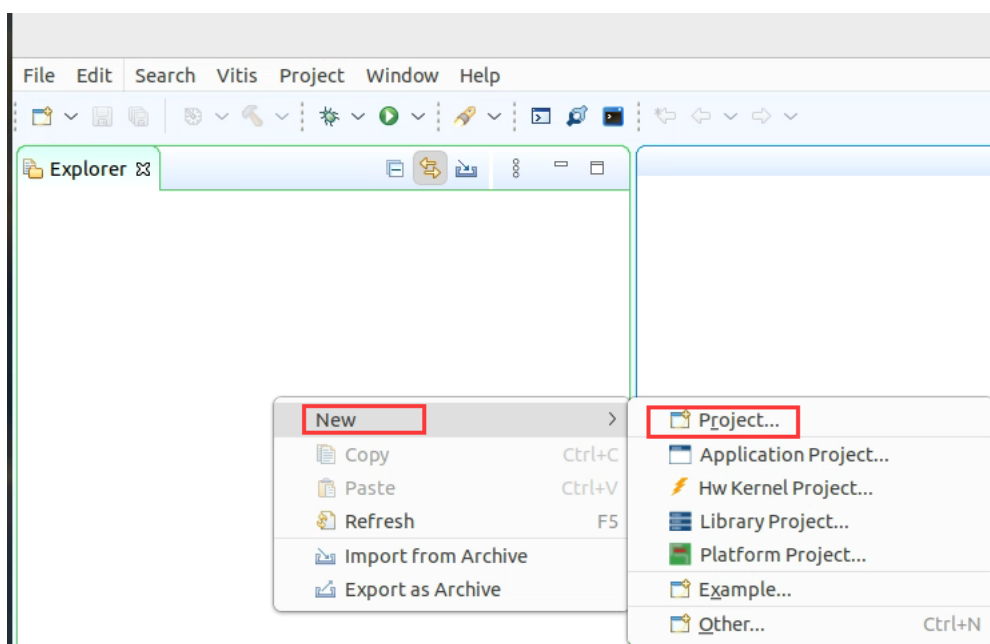


图 4-5 新建新工程

(6) 选择“C Project”，就是一个 C 语言的一个工程，点击“Next”。

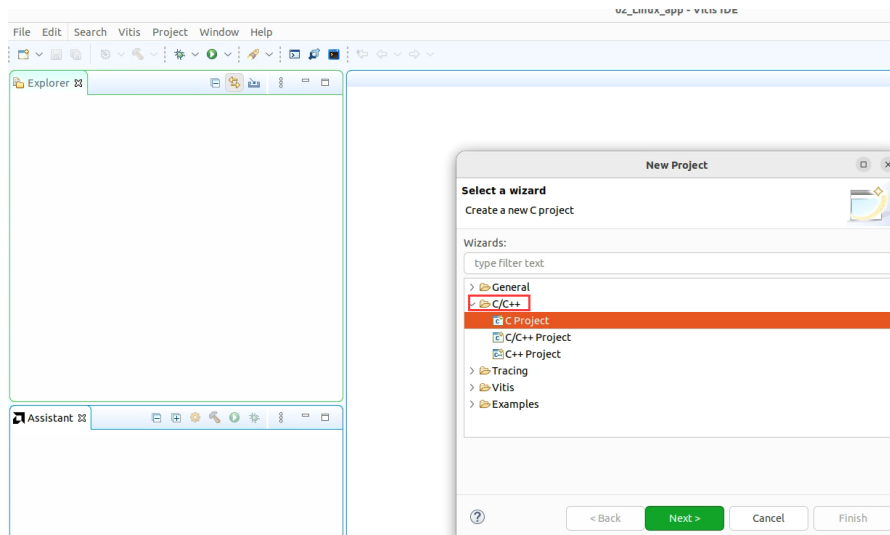


图 4-6 选择 C 语言工程

(7) 工程名填写“hello\_world”，工程类型选择 Others 文件夹里面的“Vitis ARM v8 32 Bit Linux Executable”，然后点击“Finish”

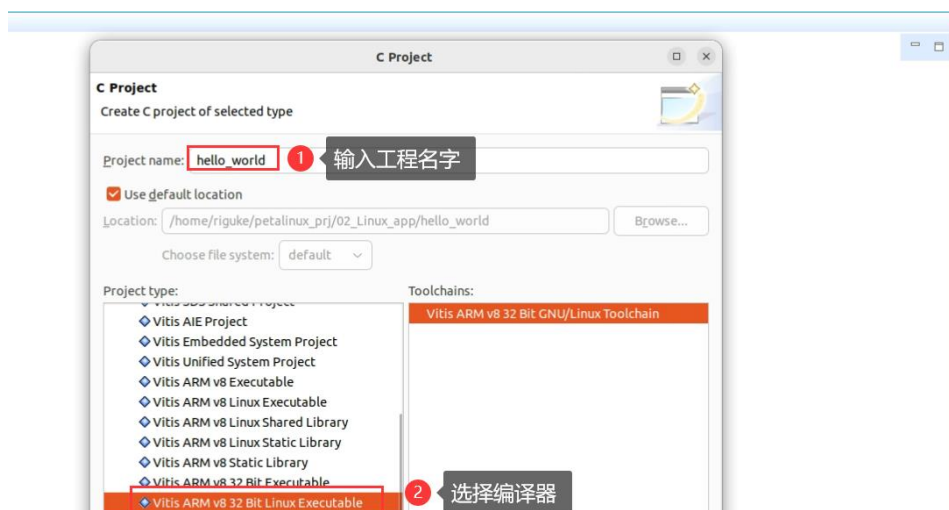


图 4-7 修改工程名，并选择编译器

(8) 如果弹出下面的信息，点“No”

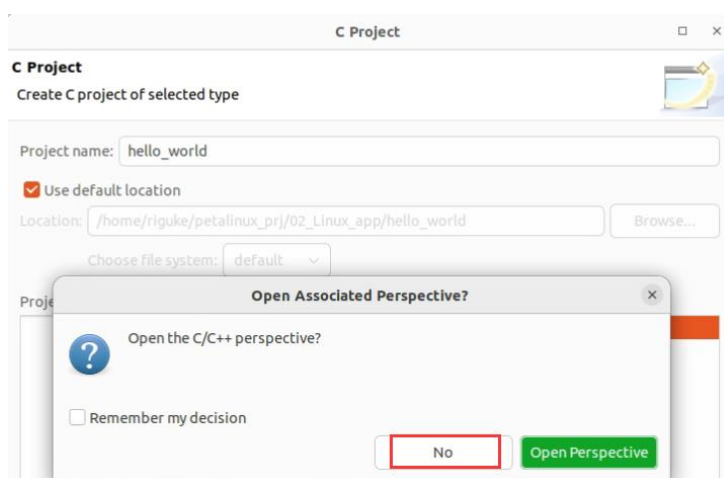


图 4-8 弹出信息，选择“No”

(9) 在“Explorer”窗口中右击 hello\_world 然后选择“New > File”

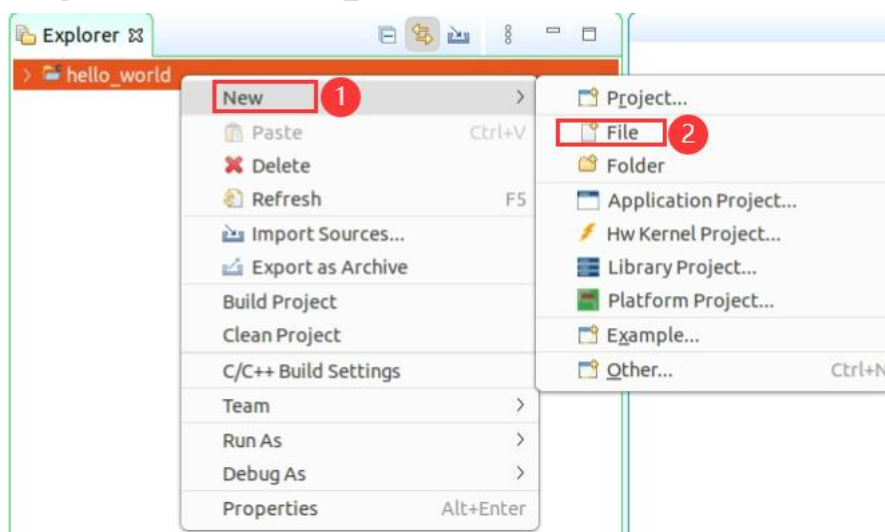


图 4-9 选择“New > File”

(10) 弹出的窗口中，“File name:”后面填写文件名,这里命名为“main.c”,然后点“Finish”。

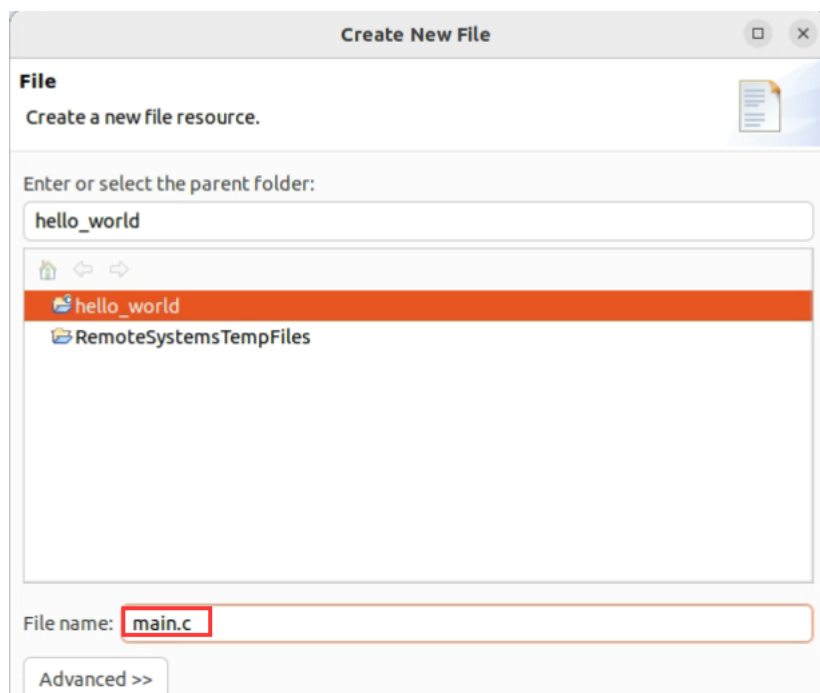


图 4-10 新建 File name

(11) 编辑 main.c 文件, 写一个简单的打印

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
    printf("hello world! Hello riguke xc7z020!!\n");
    return 0;
}
```



```

1 #include<stdio.h>
2
3 #include<stdlib.h>
4
5 int main()
6 {
7
8     printf("hello world! hello riguke xc7z020!!\n");
9
10    return 0;|
11
12 }

```

图 4-11 编写简单的打印

(12) 在“hello\_world”目录上右击，然后选择“Build Project”编译工程。

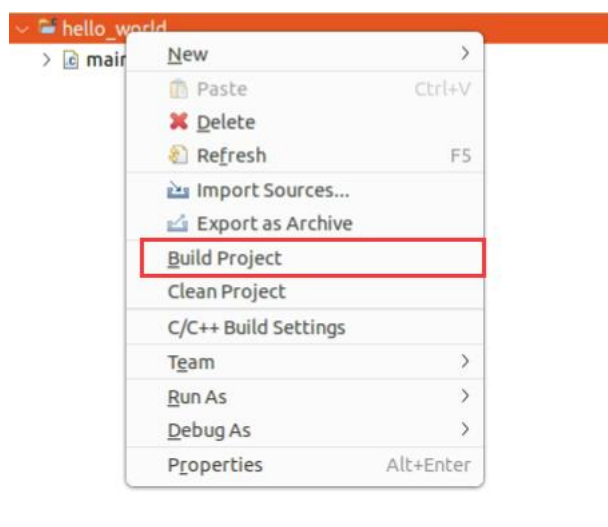


图 4-12 编译工程

(13) 将 Vitis 编译工程中产生的 elf 文件拷贝到建立的 work 文件夹中，方便开发板通过 NFS 服务器去运行编译结果。

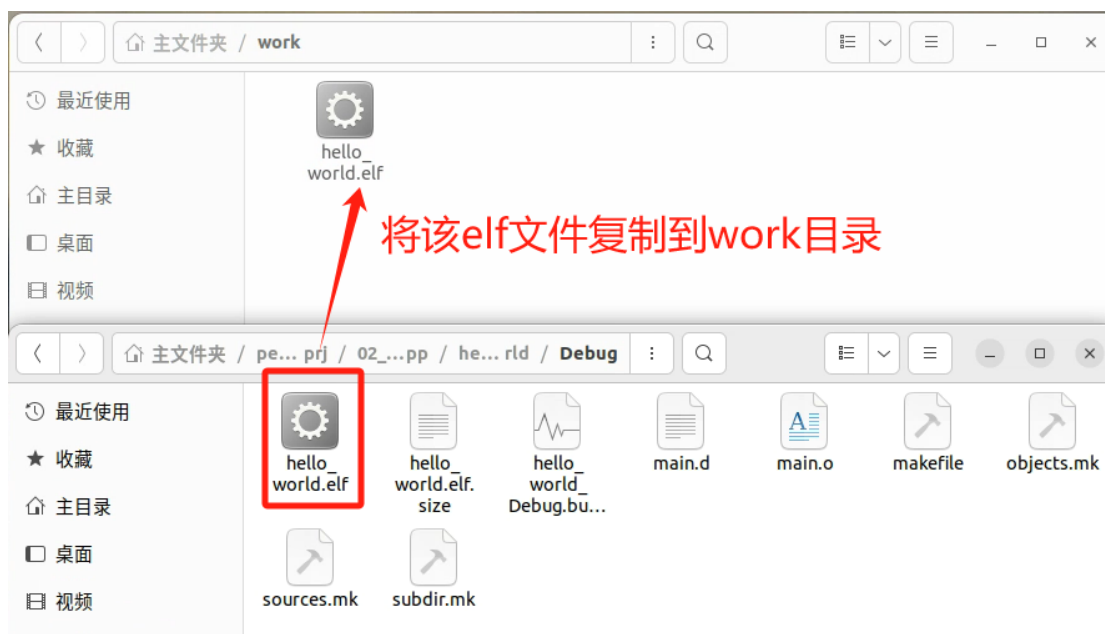


图 4-13 将编译产生的 elf 文件拷贝到 work 目录下

## 2. 以太网的使用

RK ZYNQ 7020 开发板有两路千兆以太网接口，PS\_ETH 和 PL\_ETH；由于 PS\_ETH 网口使用了 PS 端 IO 资源，而 PL\_ETH 网口使用了 PL 端 IO 资源，所以这里也把 PS\_ETH 网口称为 PS 网口、而把 PL\_ETH 网口称为 PL 网口。

连接网口的网线要使用千兆网线，譬如 CAT-5E 类网线或 CAT-6 类网线，笔者在实际测试当中，发现 CAT-5E 类网线并不稳定，所以这里推荐使用 CAT-6 类网线进行测试。

将 RK ZYNQ 7020 通过串口连接至 MobaXterm，等待 Petalinux 启动，启动完成后输入一下指令，查看系统中的所有网络设备。如下图所示：

命令为：ifconfig -a

```
petalinux login: root
root@petalinux:~# ifconfig -a
eth0      Link encap:Ethernet  HWaddr 00:0A:35:00:1E:53
          inet6 addr: fe80::20a:35ff:fe00:1e53/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:36  errors:0  dropped:0  overruns:0  frame:0
          TX packets:15  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1656 (1.6 KiB)  TX bytes:2334 (2.2 KiB)
          Interrupt:40 Base address:0xb000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:2  errors:0  dropped:0  overruns:0  frame:0
          TX packets:2  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:140 (140.0 B)  TX bytes:140 (140.0 B)

sit0      Link encap:IPv6-in-IPv4
          NOARP  MTU:1480  Metric:1
          RX packets:0  errors:0  dropped:0  overruns:0  frame:0
          TX packets:0  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

root@petalinux:~#
```

图 4-15 查看所有网络设备

图 3-13 中显示了当前系统中所有的网络设备，其中 eth0 表示开发板上的 PS 网口，由于在配置 petalinux 最小系统时候，并没有启动 PL 端网口，因此网络设备只有一个 PS 网卡。还可以直接使用 ifconfig 命令不加任何选项查看当前系统已经激活（打开）的网络设备，如下图所示：

```

root@petalinux:~# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0A:35:00:1E:53
          inet6 addr: fe80::20a:35ff:fe00:1e53/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:78 errors:0 dropped:0 overruns:0 frame:0
          TX packets:32 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:3588 (3.5 KiB)  TX bytes:7672 (7.4 KiB)
          Interrupt:40 Base address:0xb000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:2 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:140 (140.0 B)  TX bytes:140 (140.0 B)

root@petalinux:~#

```

图 4-16 查看当前打开的网络设备

上图中的 eth0 就是开发板上的 PS 网口，可以通过 ifconfig 命令来关闭或激活对应的网口。ifconfig 命令打开或关闭 PS 网口的命令如下所示：

命令为：ifconfig eth0 down #关闭 eth0（PS 网口）  
ifconfig eth0 up #打开 eth0（PS 网口）

由于只配置了 PS 端网口，所以使用命令打开 PL 端网口的命令就不输入了，打开与关闭 PL 端网口的命令与 PS 端是一致的，只需要将名称进行修改即可。

如果是用 ip 命令，则对应命令如下：

命令为：ip link set eth0 down #关闭 eth0（PS 网口）  
ip link set eth0 up #打开 eth0（PS 网口）

下面在使用 PS 网口的时候，只需要打开 PS 网口；同理使用 PL 网口的时候，需要把 PS 网口给关闭，只打开 PL 网口；在后面的使用当中，笔者以 PS 网口为例进行介绍，PL 网口使用方式相同。

## 外网连接测试（有路由器）

在测试之前，先使用网线将开发板 PS 网口连接到能够上网的路由器设备上，如果没有能够上网的路由器设备的请跳过该小节，进入下一小节，因为访问外网需要联网的路由器。

（1）执行下面的命令打开开发板的 PS 网口

命令为：ifconfig eth0 up #打开 eth0（PS 网口）  
ip link show up

```

root@petalinux:~# ifconfig eth0 up
root@petalinux:~# ip link show up
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT
    T group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP m
    ode DEFAULT group default qlen 1000
    link/ether 00:0a:35:00:1e:53 brd ff:ff:ff:ff:ff:ff
root@petalinux:~#

```

图 4-17 打开 PS 网口

(2) 接下来需要给 PS 网口分配一个 IP 地址，使用 udhcpd 命令从 DHCP 服务器中动态获取一个 IP 地址，如下所示：

```
root@petalinux:~# udhcpd -i eth0
udhcpd: started, v1.35.0
udhcpd: broadcasting discover
udhcpd: broadcasting select for 192.168.31.193, server 192.168.31.1
udhcpd: lease of 192.168.31.193 obtained from 192.168.31.1, lease time 43200
/etc/udhcpd.d/50default: Adding DNS 192.168.31.1
root@petalinux:~#
```

图 4-18 动态获取 IP 地址

从图 3-17 中可以看到，这里 PS 网口动态获取到的 IP 地址为 192.168.31.193。获取到 IP 地址之后，接下来需要测试下开发板是不是能够上网，也就是测试开发板 PS 网口是否工作正常、是否能够连接外网。当然首先确定开发板 PS 网口连接到的路由器是能够连接外网的，可以使用 ping 命令来测试开发板与另一台主机的网络连接是否通畅。

ping 命令是基于 ICMP (Internet Control Message Protocol) 协议来工作的，执行 ping 命令本地主机会向目标主机发送一份 ICMP 回显请求报文，并等待目标主机返回 ICMP 应答；因为 ICMP 协议会要求目标主机收到消息之后，必须返回 ICMP 应答消息给本地主机，如果本地主机收到了目标主机的应答，则表示两台主机之间的网络运行、网络连接是正常的。

例如在串口终端执行 ping 命令来测试开发板（本地主机）与百度 www.baidu.com 服务器（目标主机）之间的网络连接是否通畅，如下所示：

命令为：ping -c 10 www.baidu.com #测试开发板与百度服务器之间网络连接情况

```
root@petalinux:~# ping -c 10 www.baidu.com
PING www.baidu.com (110.242.70.57): 56 data bytes
64 bytes from 110.242.70.57: seq=0 ttl=53 time=44.496 ms
64 bytes from 110.242.70.57: seq=1 ttl=53 time=44.312 ms
64 bytes from 110.242.70.57: seq=2 ttl=53 time=44.180 ms
64 bytes from 110.242.70.57: seq=3 ttl=53 time=44.056 ms
64 bytes from 110.242.70.57: seq=4 ttl=53 time=44.420 ms
64 bytes from 110.242.70.57: seq=5 ttl=53 time=44.523 ms
64 bytes from 110.242.70.57: seq=6 ttl=53 time=44.399 ms
64 bytes from 110.242.70.57: seq=7 ttl=53 time=45.536 ms
64 bytes from 110.242.70.57: seq=8 ttl=53 time=45.073 ms
64 bytes from 110.242.70.57: seq=9 ttl=53 time=44.408 ms

--- www.baidu.com ping statistics ---
10 packets transmitted, 10 packets received, 0% packet loss
round-trip min/avg/max = 44.056/44.540/45.536 ms
root@petalinux:~#
```

1 从百度服务器主机接收到应道以及所需要的时间

2 统计：发送10次请求。接收10次

图 4-19 执行 ping 命令

从上图可以知道，通过开发板对百度服务器主机（IP 地址：110.242.70.57）发送了 10 次应答请求，并且每次都收到了它的应答消息（64 字节数据），并且没有数据丢失，说明开发板与百度服务器主机之间的网络运行、网络连接是 OK 的，也就意味着开发板与外网是连通的。

PL 网口的外网连接测试同理。

## 电脑直连测试（无路由器）

在开发过程中，电脑和开发板互相访问是经常需要的，这可以通过路由器来实现，连接到同一路由器的设备是可以互相访问的，如果没有路由器，也可以使用网线将开发板 PS 网口或者 PL 网口直接连接到电脑的以太网接口上，也就是电脑直连，不过这种方式不能访问外网。笔者以 PS 网口为例，用网线将开发板的 PS 网口和电脑的以太网接口相连接。

（1）连接好网线之后，需要设置电脑以太网的 IP 地址，笔者使用的是 Linux 物理机进行 IP 地址设置，因此首先进行 Linux 物理机 IP 设置。笔者设置电脑的 IP 地址如下图所示：



图 4-20 设置 Ubuntu 下 IPv4 地址

（2）配置完成后，在串口终端中与上一章中一致，打开 PS 端网口，与上一章的操作是一致的。因此这不进行重复操作。执行下面的命令设置开发板 eth0 网口的静态 IP 地址为 192.168.1.10：

命令为：`ifconfig eth0 192.168.1.10 netmask 255.255.255.0`

设置完成后，开发板的 IP 地址和电脑的 IP 就在同一网段。接下来进行 Ping 测试，看开发板和电脑能不能相互 Ping 通。

首先开发板 ping 电脑，命令如下：

命令为：`ping -c4 192.168.1.1`

```
root@petalinux:~# ping -c4 192.168.1.1
PING 192.168.1.1 (192.168.1.1): 56 data bytes
64 bytes from 192.168.1.1: seq=0 ttl=64 time=0.905 ms
64 bytes from 192.168.1.1: seq=1 ttl=64 time=0.302 ms
64 bytes from 192.168.1.1: seq=2 ttl=64 time=0.303 ms
64 bytes from 192.168.1.1: seq=3 ttl=64 time=0.445 ms

--- 192.168.1.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.302/0.488/0.905 ms
root@petalinux:~#
```

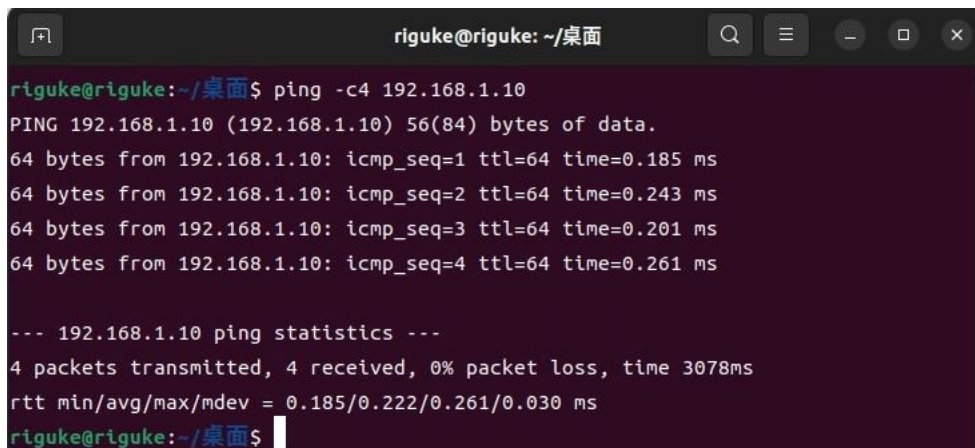
图 4-21 开发板 ping 电脑结果

如果 Ping 不通电脑，请关闭电脑的防火墙后再尝试。

（3）电脑 ping 开发板。首先打开 Ubuntu 的终端，然后输入如下命令：

命令为：`ping -c4 192.168.1.10`

结果如下：



```

riguke@riguke: ~/桌面
riguke@riguke:~/桌面$ ping -c4 192.168.1.10
PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=0.185 ms
64 bytes from 192.168.1.10: icmp_seq=2 ttl=64 time=0.243 ms
64 bytes from 192.168.1.10: icmp_seq=3 ttl=64 time=0.201 ms
64 bytes from 192.168.1.10: icmp_seq=4 ttl=64 time=0.261 ms

--- 192.168.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3078ms
rtt min/avg/max/mdev = 0.185/0.222/0.261/0.030 ms
riguke@riguke:~/桌面$

```

图 4-22 电脑 ping 开发板的结果

开发板和电脑相互 Ping 通，说明双方的通信基本上没啥问题了。

(4) 如果使用的是虚拟机 Ubuntu，虚拟机 IP 可以设置为图 3-19 所示的 IP 地址。

首先开发板 ping Ubuntu 虚拟机，命令与(3)中是一致的。Ping 的结果与上述实验是一致的，如果 ping 不通，点击虚拟机菜单栏“编辑→虚拟网络编辑器”，检查 Vmnet0 是否桥接至以太网网卡，如下图所示：

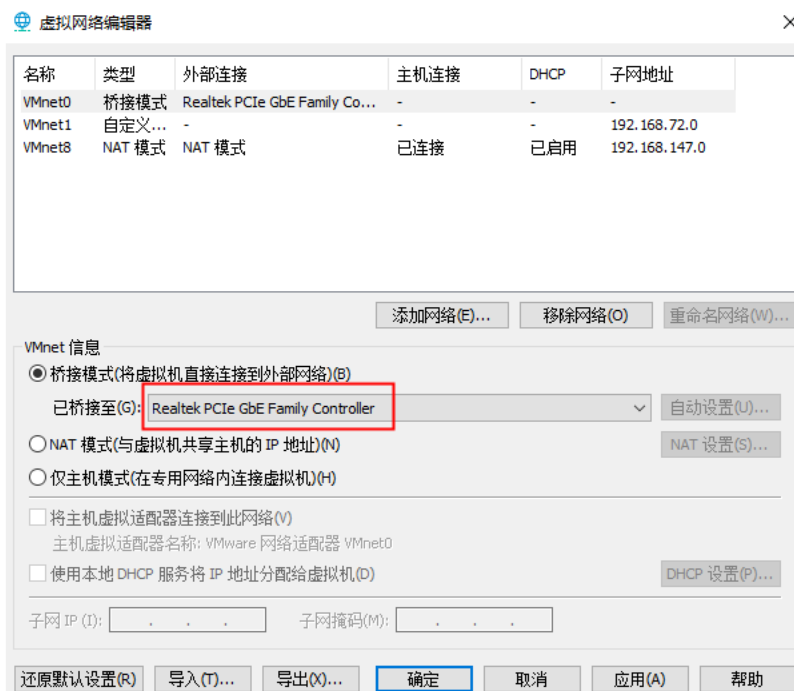


图 4-23 Vmnet0 桥接至以太网网卡



### 3. 通过 NFS 共享运行

此次实验是将开发板直接通过网线与电脑直接连接，需要配置开发板的 IP 地址

(1) 开发板插上网线 (PS ETH1)，与电脑直接连接，上电，挂载 Linux 主机 NFS，其中 192.168.1.1 为主机地址，/home/riguke/work 为 NFS 目录，/mnt 为开发板目录。这里要求主机和开发板在一个网段。

命令为：`mount -t nfs -o nolock 192.168.1.1:/home/riguke/work /mnt`

注意：上述红色的部分，是需要与自己电脑主机 IP 以及 NFS 服务器相对应，需要与自己开发环境相对应。

在本章第一节中将产生的 elf 文件拷贝至 NFS 服务器文件中，因此需要将开发板通过网线挂载至 NFS 服务器。

```
root@petalinux:~# mount -t nfs -o nolock 192.168.1.1:/home/riguke/work /mnt
root@petalinux:~#
```

图 4-24 开发板挂载至主机 NFS 服务器

(2) 进入/mnt 目录，运行 hello\_world.elf，可以看到打印出 Hello World! Hellow riguke xc7z022!!

命令为：`cd /mnt`

`./hello.elf`

```
--- 192.168.1.1 ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.393/0.457/0.522 ms
root@petalinux:~# mount -t nfs -o nolock 192.168.1.1:/home/riguke/work /mnt
root@petalinux:~# ls /mnt/
hello_world.elf
root@petalinux:~# chmod +x /mnt/hello_world.elf
root@petalinux:~# /mnt/hello_world.elf
hello world! hello riguke xc7z020!!
root@petalinux:~#
```

图 4-25 开发板上运行 Vitis app 程序结果

## 五. 程序“Hello World”交叉编译

在前面实验使用的 Petalinux 工具中包含 linux 交叉编译链，可以用来编译 uboot、kernel 和 linux 应用程序。

对于 2019.1 之前版本的 Petalinux，设置 Petalinux 工作环境变量后可以直接使用 arm 的 linux 交叉编译工具链，然而此后版本的 Petalinux 包括我们当前使用的 Petalinux 在设置环境变量后只能得到裸机的交叉编译工具链，无法获得 linux 交叉编译工具链，所以本章以构建交叉编译开发环境为例讲解如何获得 linux 交叉编译工具链。

### 1. Petalinux 的交叉编译工具链

交叉编译是编译技术发展过程中的一个重要分支。通俗的说，交叉编译就是在一个平台上生成另一个平台上的可执行代码。这里提到的平台有两方面的含义：处理器的体系结构和所运行的操作系统。

通常，程序是在一台计算机上编译，然后再分布到将要使用的其它计算机上。当主机系统（运行编译器的系统）和目标系统（产生的程序将在其上运行的系统）不兼容时，该过程就叫做交叉编译。嵌入式系统应用的日益广泛和需求推动了交叉编译技术的发展。非 X86 的嵌入式系统开发，一般都是基于 X86 的 PC 平台进行，这种开发方式就是交叉编译。嵌入式系统中的应用程序都是通过交叉编译得到的。

我们使用 Petalinux 工具编译，显示 Petalinux 工具是带有交叉编译工具链的。设置 Petalinux 的工作环境后，输入“arm”，然后按两次 TAB 键，会显示当前环境中，以“arm”开头的工具和命令，包括交叉编译的工具和命令，如下图所示：

```
arm2hpd1
arm-linux-gnueabi-hf-gdb
arm-linux-gnueabi-hf-gdb-add-index
arm-none-eabi-addr2line
arm-none-eabi-ar
arm-none-eabi-as
arm-none-eabi-c++filt
arm-none-eabi-cpp
arm-none-eabi-dwp
arm-none-eabi-elfedit
arm-none-eabi-g++
arm-none-eabi-gcc
arm-none-eabi-gcc-ar
arm-none-eabi-gcc-nm
arm-none-eabi-gcc-ranlib
arm-none-eabi-gcov
arm-none-eabi-gcov-dump
arm-none-eabi-gcov-tool
arm-none-eabi-gdb
arm-none-eabi-strip
armr5-none-eabi-addr2line
armr5-none-eabi-ar
armr5-none-eabi-as
armr5-none-eabi-c++filt
armr5-none-eabi-cpp
armr5-none-eabi-dwp
armr5-none-eabi-elfedit
armr5-none-eabi-g++
armr5-none-eabi-gcc
armr5-none-eabi-gcc-ar
armr5-none-eabi-gcc-nm
armr5-none-eabi-gcc-ranlib
armr5-none-eabi-gcov
armr5-none-eabi-gcov-dump
armr5-none-eabi-gcov-tool
armr5-none-eabi-gdb
armr5-none-eabi-gdb-add-index
armr5-none-eabi-gprof
```

图 5-1 以“arm”开头的工具和命令

其中以“arm”开头的是 arm 的交叉编译工具链，显示的 arm 交叉编译工具链只有 none，也就是裸机下的交叉编译工具链，不能编译 linux，那么用于 linux 的交叉编译工具链是啥样的呢，如下图所示：



```

riguke@riguke:/opt/petalinux/2023.1$ arm
Display all 150 possibilities? (y or n)
arm2hpd1
arm-linux-gnueabihf-addr2line
arm-linux-gnueabihf-ar
arm-linux-gnueabihf-as
arm-linux-gnueabihf-c++filt
arm-linux-gnueabihf-cpp
arm-linux-gnueabihf-dwp
arm-linux-gnueabihf-elfedit
arm-linux-gnueabihf-g++
arm-linux-gnueabihf-gcc
arm-linux-gnueabihf-gcc-ar
arm-linux-gnueabihf-gcc-nm
arm-linux-gnueabihf-gcc-ranlib
arm-linux-gnueabihf-gcov
arm-linux-gnueabihf-gcov-dump
arm-linux-gnueabihf-gcov-tool
arm-linux-gnueabihf-gdb
arm-linux-gnueabihf-gdb-add-index
arm-linux-gnueabihf-gprof

```

linux交叉编译工具和命令

图 5-2 linux 交叉编译工具链

如果我们想编译 linux 内核或 linux 应用就可以用其中的 `arm-xilinx-linux-gnueabi-gcc` 进行编译。既然设置 Petalinux 工作环境没有，那么 Petalinux 是怎么构建 linux 系统的呢？其实 Petalinux 在构建 linux 系统过程中会编译生成 linux 交叉编译工具链，然后使用其构建 linux 系统，可以在 Petalinux 工程下使用“`find . -name "arm-xilinx-linux-gnueabi-gcc"`”命令找到相应的痕迹。不过这种构建过程中编译生成的 linux 交叉编译工具链是不方便使用的，而且其功能完整性还不能得到保证，毕竟我们不知道 Petalinux 在构建完成后会怎么处理这些中间件。那么怎样才能获得实用的 linux 编译工具链呢？这就是本章的重点了。因为有了 linux 交叉编译工具链就可以摆脱对 Petalinux 的依赖，直接使用 linux 交叉编译工具链进行编译，方便快捷。

获取 linux 交叉编译工具链需要用 Petalinux 构建 SDK，然后安装 SDK。所谓的 SDK，也就是软件开发工具集，与 Petalinux 构建的根文件系统息息相关，里面不仅包含有 Petalinux 构建的根文件系统、各种库和头文件，还包含 linux 交叉编译工具链，用来编译 linux 内核及 linux 应用以使其能在 ZYNQ 开发板上运行。

## 2. 编译根文件系统并构建 SDK

如果定制号根文件系统，那么可以使用如下命令编译刚才定制根文件系统：

命令为：`petalinux-build -c rootfs`

```

zy@zy-virtual-machine:~/petalinux/Navigator_7010_v3/ALIENTEK-ZYNQ-disp-dev$ pe
talinux-build -c rootfs
INFO: Sourcing build tools
[INFO] Building rootfs
[INFO] Sourcing build environment
[INFO] Generating workspace directory
INFO: bitbake petalinux-image-minimal -c do_image_complete
Parsing recipes: 100% |#####| Time: 0:00:36
Parsing of 2995 .bb files complete (0 cached, 2995 parsed). 4265 targets, 204
skipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |#####| Time: 0:00:05
Checking sstate mirror object availability: 100% |#####| Time: 0:00:02
Sstate summary: Wanted 1499 Found 116 Missed 1383 Current 776 (7% match, 39% c
omplete)
NOTE: Executing Tasks
NOTE: Setscene tasks completed
NOTE: Tasks Summary: Attempted 6096 tasks of which 2663 didn't need to be reru
n and all succeeded.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built rootfs
zy@zy-virtual-machine:~/petalinux/Navigator_7010_v3/ALIENTEK-ZYNQ-disp-dev$

```

图 5-3 编译根文件系统

Petalinux 工程编译生成的根文件系统在 images/linux/目录下。

接下来构建 SDK，输入如下命令：

命令为：petalinux-build --sdk

该命令可构建 SDK 并将其部署在当前 Petalinux 工程的 images/linux/sdk.sh 中。编译过程中可能会遇到如下图所示的错误：

```

NOTE: Setscene tasks completed
ERROR: qt3d-native-5.13.2+gitAUTOINC+93361fla59-r0 do_fetch: Network access di
sabled through BB_NO_NETWORK (or set indirectly due to use of BB_FETCH_PREMIRR
ORONLY) but access requested with command /usr/bin/env wget -t 2 -T 30 --passi
ve-ftp --no-check-certificate -c -P /home/zy/petalinux/Navigator_7010_v3/ALIEN
TEK-ZYNQ-disp-dev/build/downloads 'http://petalinux.xilinx.com/sswreleases/rel
-v2020/downloads/git2_code.qt.io.qt.qt3d.git.tar.gz' (for url http://petalinux
.xilinx.com/sswreleases/rel-v2020/downloads/git2_code.qt.io.qt.qt3d.git.tar.gz
)
ERROR: Logfile of failure stored in: /home/zy/petalinux/Navigator_7010_v3/ALIE
NTEK-ZYNQ-disp-dev/build/tmp/work/x86_64-linux/qt3d-native/5.13.2+gitAUTOINC+9
3361fla59-r0/temp/log.do_fetch.9552
ERROR: Task (virtual:native:/home/zy/petalinux/Navigator_7010_v3/ALIENTEK-ZYNQ
-disp-dev/components/yocto/layers/meta-qt5/recipes-qt/qt5/qt3d_git.bb:do_fetch
) failed with exit code '1'

```

图 5-4 Network access disable 错误

错误的原因是缺少 qt3d 包，而工程又不能联网下载。解决方法是，运行“petalinux-config”命令，进入“Yocto Settings”选项界面，取消使能“BB\_NO\_NETWORK”，如下图所示：

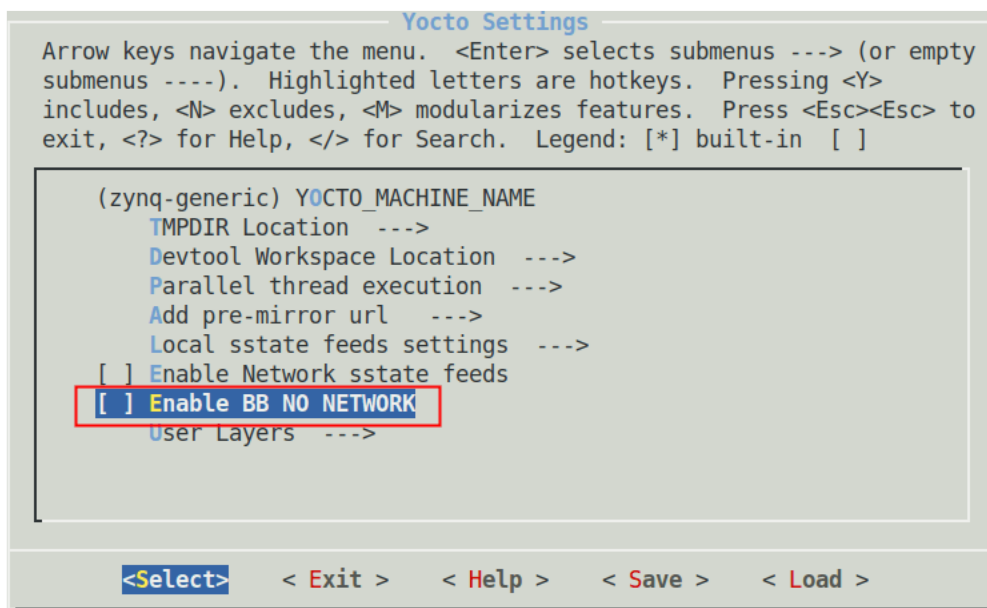


图 5-4 取消使能 BB\_NO\_NETWORK

保存配置并退出，继续构建 SDK。

注：在构建 SDK 的时候，如果你的电脑在编译的过程中卡住，建议将虚拟机的线程改成 4 线程

若编译过程中报如下错误，建议将虚拟机的内存调整到 12G 或更大。成功构建的 SDK 文件在工程的 images/linux 目录下，名为 sdk.sh，如下图所示：

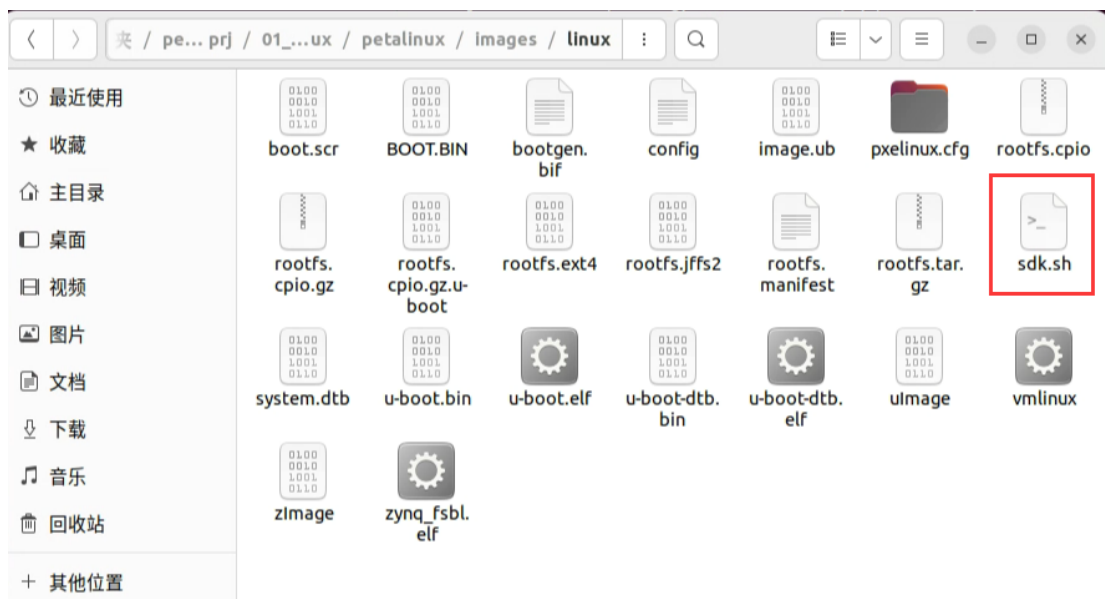


图 5-5 SDK 文件

注意：构建 SDK 太耗硬盘空间，推荐使用我们提供的已经构建好的 SDK 文件，路径为：“RK-ZYNQ7020-F 开发板网盘资料 -> 7.开发软件->Petalinux-> sdk.sh”。

从图 5-5 中可以看到，构建的 SDK 文件是一个脚本文件，需要执行才能使用，也就是安装 sdk，如果出现没有权限，安装之前可以给该脚本文件权限，命令为：

```
命令为：sudo chmod 777 ./sdk.sh
        ./sdk.sh
```

如果使用的是网盘提供的 sdk.sh 文件，需要先将其拷贝到 Ubuntu 系统中，然后执行



sdk.sh 进行安装。

安装路径默认在/opt/petalinux/2023.1 目录下，如果想安装在其他目录下，可以输入相应的路径，此处笔者保持默认，按回车键继续，接下来提示确认路径，按回车键确认，结果如下图所示：

```

riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux/images/linux$ sudo
chmod 777 ./sdk.sh
riguke@riguke:~/petalinux_prj/01_ZYNQ7020_Petalinux/petalinux/images/linux$ ./sd
k.sh
Petalinux SDK installer version 2023.1
=====
Enter target directory for SDK (default: /opt/petalinux/2023.1): 安装路径
You are about to install the SDK to "/opt/petalinux/2023.1". Proceed [Y/n]? Y
Extracting SDK.....
.....
.....done
Setting it up...done
  
```

图 5-6 SDK 安装路径设置

安装完成后提示在一个新终端中使用 SDK 前需要先执行“./opt/petalinux/2023.1/environment-setup-cortexa9t2hf-neon-xilinx-linux-gnueabi”以设置相应的环境变量，其中的“.”和 source 具有相同含义。

```

riguke@riguke:/opt/petalinux/2023.1$ source environment-setup-cortexa9t2hf-neon-
xilinx-linux-gnueabi
riguke@riguke:/opt/petalinux/2023.1$ arm
Display all 150 possibilities? (y or n)
arm2hpd1
arm-linux-gnueabihf-addr2line
arm-linux-gnueabihf-ar
arm-linux-gnueabihf-as
arm-linux-gnueabihf-c++filt
arm-linux-gnueabihf-cpp
arm-linux-gnueabihf-dwp
arm-linux-gnueabihf-elfedit
arm-linux-gnueabihf-g++
arm-linux-gnueabihf-gcc
arm-linux-gnueabihf-gcc-ar
arm-linux-gnueabihf-gcc-nm
arm-linux-gnueabihf-gcc-ranlib
  
```

图 5-7 使用 SDK 前进行环境变量设置

### 3. SDK 中的交叉编译工具链

安装完成后，进入到安装目录/opt/petalinux/2023 下，该目录下有一些文件和文件夹，如下图所示：



```

riguke@riguke:/opt/petalinux/2023.1$ ls -l
总计 40
-rw-r--r-- 1 root root 4236 6月 8 13:01 environment-setup-cortexa9t2hf-neon-x
ilinx-linux-gnueabi
-rw-r--r-- 1 root root 22387 6月 8 13:01 site-config-cortexa9t2hf-neon-xilinx-
linux-gnueabi
drwxr-xr-x 4 root root 4096 6月 8 12:59 sysroots
-rw-r--r-- 1 root root 114 6月 8 13:01 version-cortexa9t2hf-neon-xilinx-linu
x-gnueabi

```

图 5-8 SDK 安装目录

该目录下有一个环境变量配置文件 `environment-setup-cortexa9t2hf-neon-xilinx-linux-gnueabi` 每次使用之前都需要使用 `source` 来运行这个配置文件，例如：

命令为：`source environment-setup-cortexa9t2hf-neon-xilinx-linux-gnueabi`

当 SDK 环境变量配置完成之后，我们可以在终端输入 `arm`，然后按两次 `TAB` 键查看交叉编译相关的命令，如果配置成功则会显示如下信息，表示环境变量已经配置成功了：

```

riguke@riguke:/opt/petalinux/2023.1$ source environment-setup-cortexa9t2hf-neon-
xilinx-linux-gnueabi
riguke@riguke:/opt/petalinux/2023.1$ arm
Display all 150 possibilities? (y or n)
arm2hpd1
arm-linux-gnueabihf-addr2line
arm-linux-gnueabihf-ar
arm-linux-gnueabihf-as
arm-linux-gnueabihf-c++filt
arm-linux-gnueabihf-cpp
arm-linux-gnueabihf-dwp
arm-linux-gnueabihf-elfedit
arm-linux-gnueabihf-g++
arm-linux-gnueabihf-gcc
arm-linux-gnueabihf-gcc-ar
arm-linux-gnueabihf-gcc-nm
arm-linux-gnueabihf-gcc-ranlib

```

图 5-9 查看交叉编译工具链

输入如下命令查看交叉编译工具所在位置，命令如下所示：

命令为：`whereis arm-xilinx-linux-gnueabi-gcc`

结果如下图所示：

```

riguke@riguke:/opt/petalinux/2023.1$ whereis arm-xilinx-linux-gnueabi-gcc
arm-xilinx-linux-gnueabi-gcc: /opt/petalinux/2023.1/sysroots/x86_64-petalinux-li
nux/usr/bin/arm-xilinx-linux-gnueabi/arm-xilinx-linux-gnueabi-gcc

```

图 5-10 查看交叉编译工具位置

此时，我们就得到了 `linux` 交叉编译工具链，可以摆脱 `Petalinux`，直接使用 `linux` 交叉编译工具链编译 `uboot`、`linux` 内核和 `linux` 应用。

注意，如果不想每次都手动设置 SDK 环境变量，可以在终端输入如下命令：

命令为：`echo '. /opt/petalinux/2023.1/environment-setup-cortexa9t2hf-neon-xilinx-linux-gnueabi' | tee -a ~/.bashrc`

输出结果：`/opt/petalinux/2023.1/environment-setup-cortexa9t2hf-neon-xilinx-linux-gnueabi`，这样设置以后，我们就可以直接使用 `arm` 的交叉编译器，无需再设置 SDK 的工作环境。

## 4. 编写“Hello World”代码

按上一章重新制作一个 Petalinux 工程，使用的硬件环境仍是上一章中 vivado 制作的硬件环境，在这就不多重复介绍了。

需要注意的是：如果想将交叉编译的工程直接拷贝的工程中运行，需要在配置 petalinux 根文件系统，配置 Linux 系统为自启动。配置根文件命令如下：

命令为：petalinux-config -c rootfs

```
riguke@riguke:~/petalinux_prj/04_Hello_World/arm_hello_world$ petalinux-config --get-hw-description ../hardware/
[INFO] Sourcing buildtools
[INFO] Getting hardware description...
INFO: Renaming petalinux_config_wrapper.xsa to system.xsa
[INFO] Extracting yocto SDK to components/yocto. This may take time!
[INFO] Generating Kconfig for project
[INFO] Menuconfig project
[INFO] Generating kconfig for rootfs
[INFO] Silentconfig rootfs
[INFO] Adding user layers
[INFO] Generating machine conf file
[INFO] Generating plnxtool conf file
[INFO] Generating workspace directory
riguke@riguke:~/petalinux_prj/04_Hello_World/arm_hello_world$ petalinux-config -c rootfs
```

图 5-12 配置根系统文件

如果不希望每次启动 linux 后都得输入密码验证，可以使能该菜单下的“serial-autologin-root”选项。如下图所示：

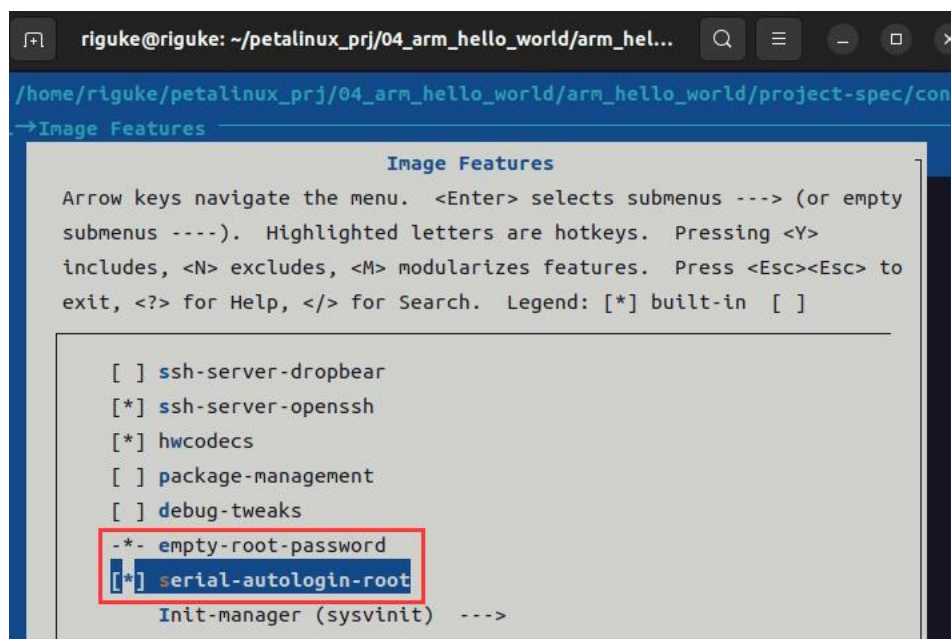


图 5-13 配置为自动启动

这样配置后，会自动登录，不用再手动输入用户名和密码，方便调试。

(1) 在“/home/riguke/petalinux\_prj/04\_hello\_world”路径中创建一个目录“app\_hello\_world”来存放本章的应用程序。如下图：

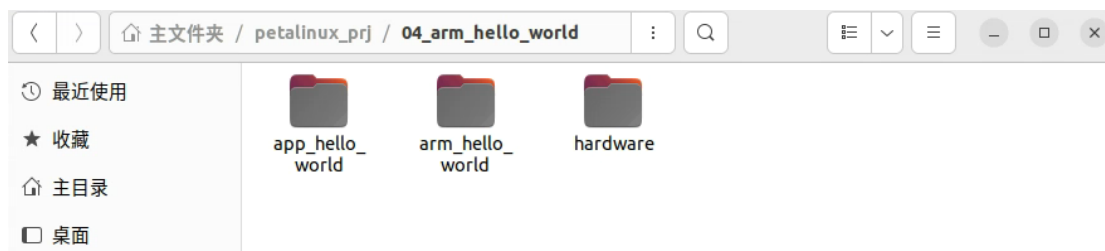


图 5-11=4 新建一个 app 目录

(2) 在 app\_hello\_world 目录中打开中端，使用下面的命令  
命令为: touch main.c

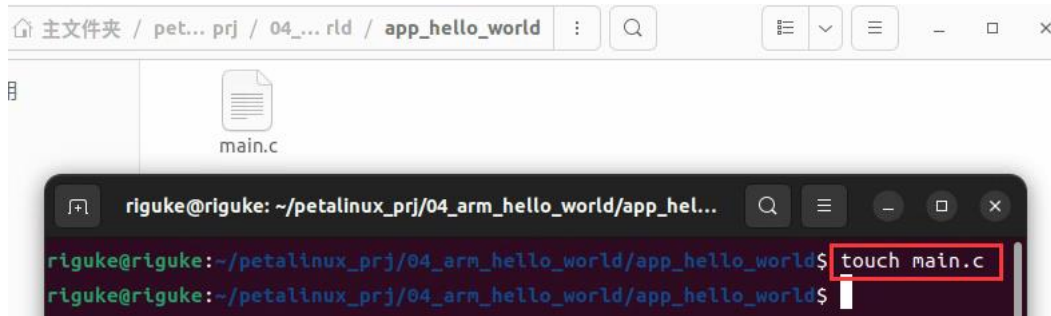


图 5-15 新建一个 main.c 文件

(3) 编辑 main.c 文档，输入如下内容并保存。

```
#include <stdio.h>
int main(void)
{
    printf("hello world\r\n");
    return 0;
}
```

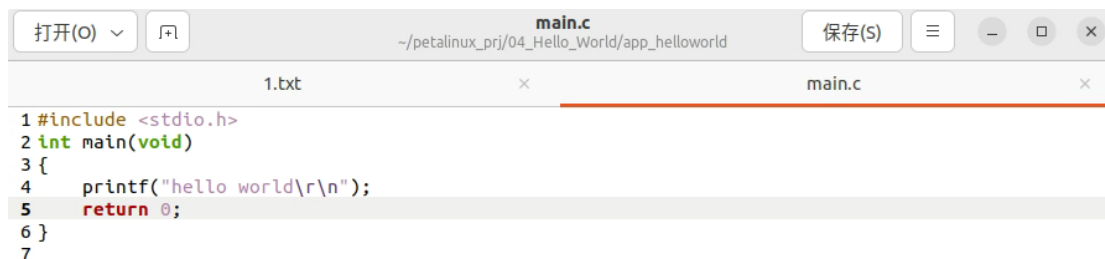


图 5-16 main.c 文件中进行代码编写

(4) 在终端中使用下面的命令编译程序，“home/riguke/petalinux\_prj/04\_hello\_world/app\_helloworld” 路径中会生成一个名为 “helloworld” 的可执行文件。

命令为: gcc -o helloworld main.c

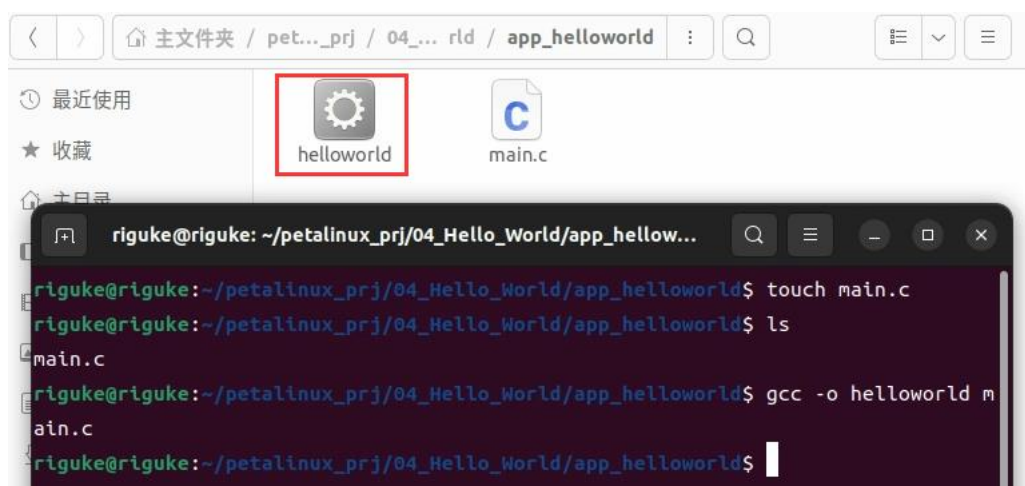


图 5-17 编译得到可执行文件

(5) 在终端中输入下面的命令，直接运行程序，可以看到打印信息“hello world”，说明程序已经在电脑上正确执行。

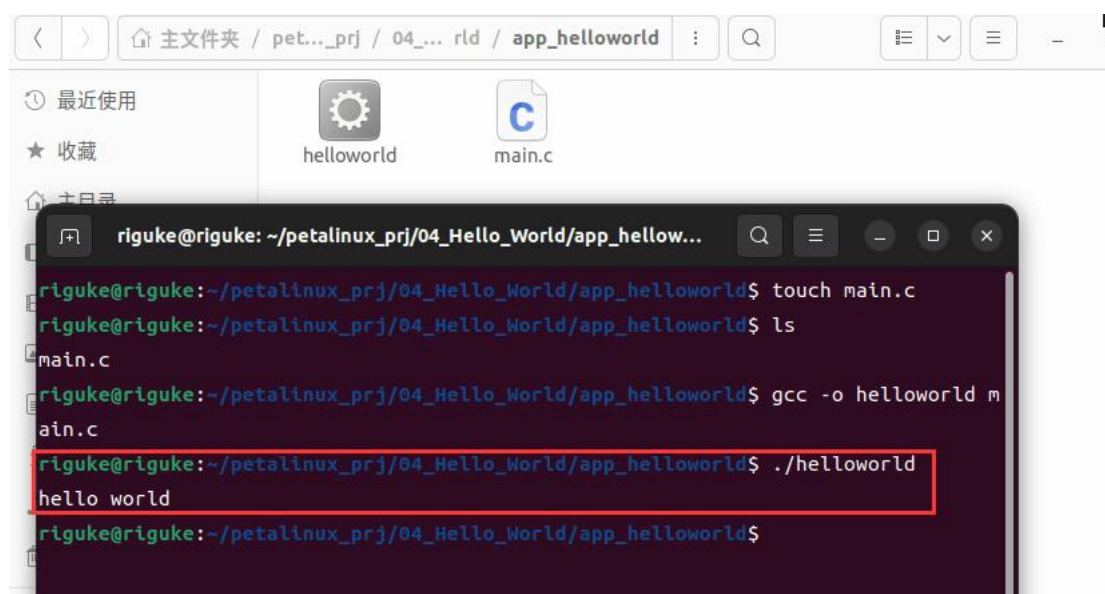


图 5-18 终端中运行程序

## 5. 交叉编译 arm 程序

刚才使用“gcc”编译器来编译程序在 pc 机上运行。但程序最终是需要运行在板卡上的，我们需要使用 arm 的交叉编译器编译程序才能在板卡上运行。交叉编译器以及如何使用在前面已经安装好，那么在使用交叉编译的时候，只需要运行交叉编译的运行环境。

(1) 使用下面的命令交叉编译程序，生成板卡上可执行程序，名字是“zynq\_app”。

命令为：\$CC main.c -o zynq\_app



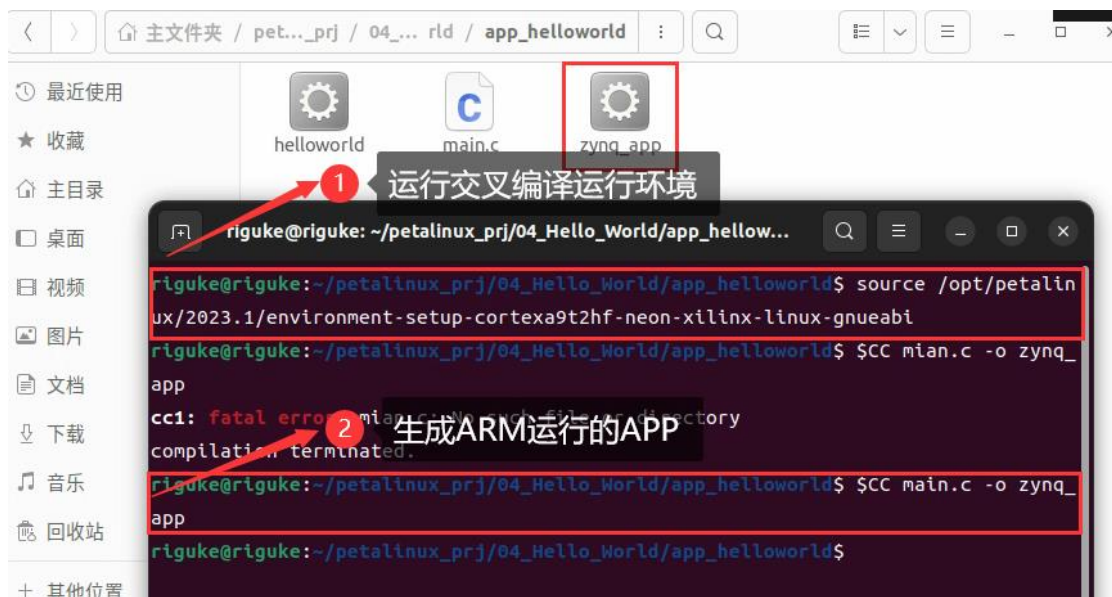


图 5-19 生成 ZYNQ 能使用的 APP 程序

(2) 将产生的 APP 程序拷贝至第一章第四节中的 NFS 服务器中，连接好串口，将板卡设置为 SD 卡启动，然后上电。输入账号密码，进入 Linux 系统，将板卡挂载至 NFS 服务器，随后进入 NFS 服务器，运行 `zynq_app`。如下图所示：

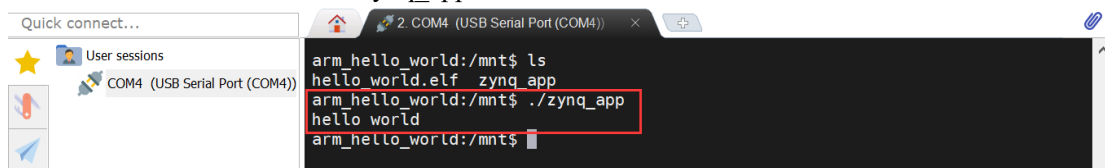


图 5-20 板卡上运行 zynq\_app

(3) 上述是通过 NFS 服务器运行的 APP 程序，在配置 Petalinux 时，将其设置为自动登录。运行该 APP 还有一种方式就是将编译产生的 APP 文件复制到 SD 卡中，如下图所示：

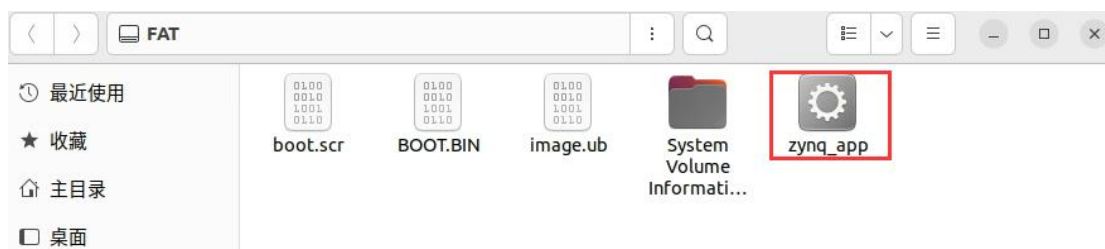


图 5-21 拷贝 zynq\_app 至 SD 卡中

(4) 当 Linux 系统启动后，使用下面的命令搜索一下“zynq\_app”程序所在的路径。  
命令为：`find / -name zynq_app`

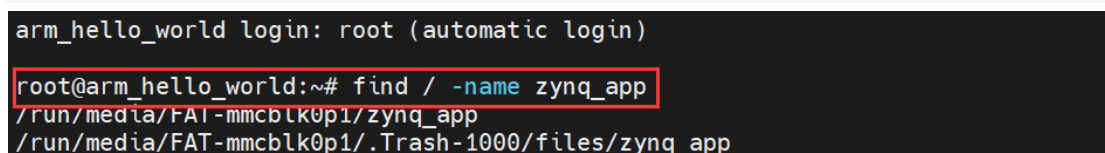


图 5-22 找到 APP 所在的位置

(5) 运行程序，可以看到打印信息“hello world!”,说明程序已经在板卡上正确执行。如下图所示：

```
root@arm_hello_world:~# find / -name zynq_app
/run/media/FAT-mmcbk0p1/zynq_app
/run/media/FAT-mmcbk0p1/.Trash-1000/files/zynq_app
root@arm_hello_world:~# /run/media/FAT-mmcbk0p1/zynq_app
hello world!
root@arm_hello_world:~#
```

1 运行APP

2 运行结果

图 5-23 板卡上运行 APP



## 六. GPIO 控制 LED

在Xilinx提供的Linux的内核中，已经包含了很多模块的驱动，因此可以直接使用Xilinx提供的内核驱动。ZYNQ的GPIO可以分为2种，一种是PS端自带的GPIO，一种是使用PL实现的GPIO，在建立Vivado工程时添加了Xilinx的GPIO IP，大部分Xilinx提供的IP核在Linux下都已经有了驱动，而且很多默认配置都是可以用的，像AXI GPIO驱动不需要在内核中再配置就可以使用。此次则用PS端自带的GPIO来驱动LED。

对Linux内核中包含的驱动，我们需要将这些驱动，与我们板卡上的硬件关联起来，才能在系统下生成我们可以使用的设备。我们可以使用设备树来关联我们的设备与驱动。使用petalinux工具去定制Linux系统的话，在“petalinux-bulid”编译之后petalinux会自动生成一些设备树，在petalinux工程路径下的“components/plnx\_workspace/device\_tree/device\_tree”的路径中，如下图：

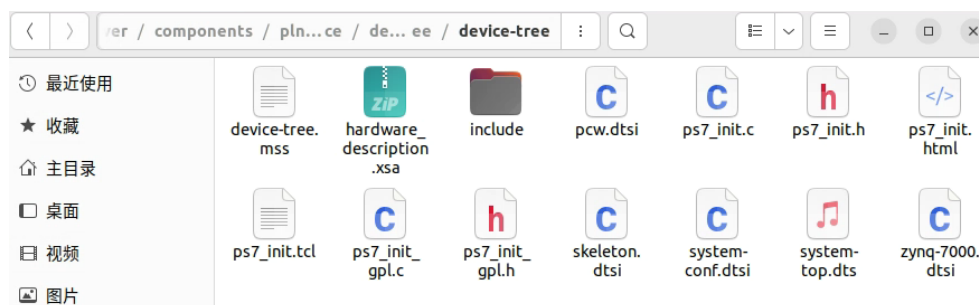


图 6-1 PS 端设备树

**注意：**这里的文件都是修改无效的。

其中“zynq-7000.dtsi”是PS端的设备树，“pl.dtsi”是PL端的设备树(因为这里使用的XSA硬件描述文件中没有bitstream，也就是Vivado工程中没有设置PL端的内容，所以没有生成“pl.dtsi”文件)。打开zynq-7000.dtsi文件，在这个文件中可以找到PS端的GPIO设备树节点，如下图所示：

```

132         can1: can@e0009000 {
133             compatible = "xlnx,zynq-can-1.0";
134             status = "disabled";
135             clocks = <&clk 20>, <&clk 37>;
136             clock-names = "can_clk", "pclk";
137             reg = <0xe0009000 0x1000>;
138             interrupts = <0 51 4>;
139             interrupt-parent = <&intc>;
140             tx-fifo-depth = <0x40>;
141             rx-fifo-depth = <0x40>;
142         };
143
144         gpt0: gpio@e000a000 {
145             compatible = "xlnx,zynq-gpio-1.0";
146             #gpio-cells = <2>;
147             clocks = <&clk 42>;
148             gpio-controller;
149             interrupt-controller;
150             #interrupt-cells = <2>;
151             interrupt-parent = <&intc>;
152             interrupts = <0 20 4>;
153             reg = <0xe000a000 0x1000>;
154         };
155
156         i2c0: i2c@e0004000 {
157             compatible = "cdns,i2c-rip10";
158             status = "disabled";
159             clocks = <&clk 38>;
160             interrupt-parent = <&intc>;
161             interrupts = <0 25 4>;
162             reg = <0xe0004000 0x1000>;
163             #address-cells = <1>;
164             #size-cells = <0>;
165         };
166
167         i2c1: i2c@e0005000 {

```

图 6-2 PS 端 LED 设备树

如果需要修改设备树，可以编辑petalinux工程路径中“project-spec/meta-user/recipes-bsp/device-tree/files”路径中的“system-user.dtsi”这个文件。

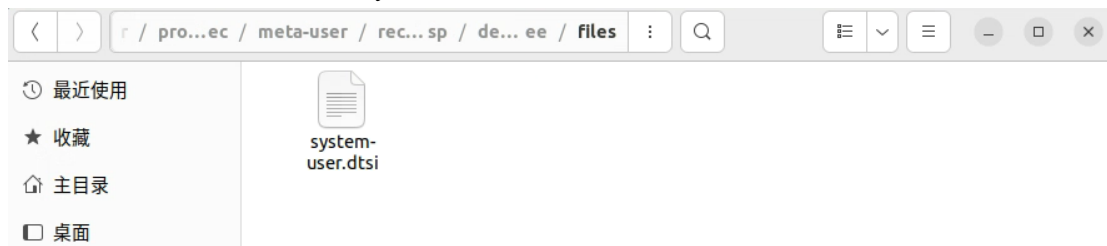


图 6-3 可修改的设备树

这一章我们使用的GPIO，因为PS端的GPIO设备树节点已经由petalinux中正确的生成，所以我们不需要在这里作任何修改。

## 1. 使用 Shell 控制 LED 亮灭

(1) 在 petalinux 工程编译完毕后，在开发板中启动后，通过以下命令可以查看 GPIO 编号，命令如下：

命令为：`cd /sys/class/gpio`

`ls`

`cat gpiochip906/label`

```
root@arm_hello_world:~# cd /sys/class/gpio/
root@arm_hello_world:/sys/class/gpio# ls
export      gpiochip906  unexport
root@arm_hello_world:/sys/class/gpio# cat gpiochip906/label
zynq_gpio
root@arm_hello_world:/sys/class/gpio#
```

图 6-4 板卡查询 GPIO 控制器编号

通过上述命令可以看出当前 GPIO 控制器编号为 906。

上述内容表明，GPIO 的第 0 号引脚从 906 开始，到 1023 结束（对于 ZYNQ 7 来说，GPIO 共有 118 个引脚）。

GPIO 编号的计算公式为：

GPIO 编号 = base\_gpio + other\_gpios

other\_gpio 为 PS 端 LED 的管脚号，比如 RK 7020 开发板中一个 LED 管脚在 MIO 8，那么对于 GPIO 的编号为：906 + 8 = 914；

(2) 接下来则通过控制台对 MIO 8 管脚的 LED 进行点亮驱动。确认 LED 的编号后，流程如下：

- a. 导出 LED GPIO 的管脚。
- b. 从该管脚读出控制方向及控制的值。
- c. 设置该管脚为输出并依据原理图的控制设置值至该管脚。

将 LED GPIO 管脚导出的命令如下：

命令为：`echo 914 > /sys/class/gpio/export`

注意：上述命令中红色的 914 是依据原理图计算的 GPIO 编号。

```
root@arm_hello_world:~# echo 914 > /sys/class/gpio/export
root@arm_hello_world:~#
```

图 6-5 导出对应的 GPIO 管脚

(3) 首先读出当前该管脚下的文件，命令为：

命令为：ls /sys/class/gpio/gpio914/

```
root@arm_hello_world:~# echo 914 > /sys/class/gpio/export
root@arm_hello_world:~# ls /sys/class/gpio/gpio914/
active_low  direction  power      uevent
device      edge       subsystem  value
root@arm_hello_world:~#
```

图 6-6 读出当前管脚文件中的文件

(4) 其次，读出当前该管脚的是输入还是输出，命令为：

命令为：cat /sys/class/gpio/gpio914/direction

```
root@arm_hello_world:~# echo 914 > /sys/class/gpio/export
root@arm_hello_world:~# ls /sys/class/gpio/gpio914/
active_low  direction  power      uevent
device      edge       subsystem  value
root@arm_hello_world:~# cat /sys/class/gpio/gpio914/direction
in
root@arm_hello_world:~#
```

图 6-7 读出当前管脚的方向

从上图中可以看出，该管脚的输入方向为输入。

(5) 其次，读出当前该管脚的值，命令为：

命令为：cat /sys/class/gpio/gpio914/value

```
root@arm_hello_world:~# cat /sys/class/gpio/gpio914/direction
in
root@arm_hello_world:~# cat /sys/class/gpio/gpio914/value
0
root@arm_hello_world:~#
```

图 6-8 读出当前管脚的值

从上图中可以看出，当前管脚的值为 0。

(6) 将该管脚方向设置为输出，并将值设置为 1。

命令为：echo out > /sys/class/gpio/gpio914/direction

echo 1 > /sys/class/gpio/gpio914/value

```
root@arm_hello_world:~# echo out > /sys/class/gpio/gpio914/direction
root@arm_hello_world:~# echo 1 > /sys/class/gpio/gpio914/value
root@arm_hello_world:~#
```

图 6-9 将 GPIO 设置为输出并设置为 1

输入完上述指令后，可以发现 RK7020 开发板上 PS 端 LED1 已经亮起，如下图所示：

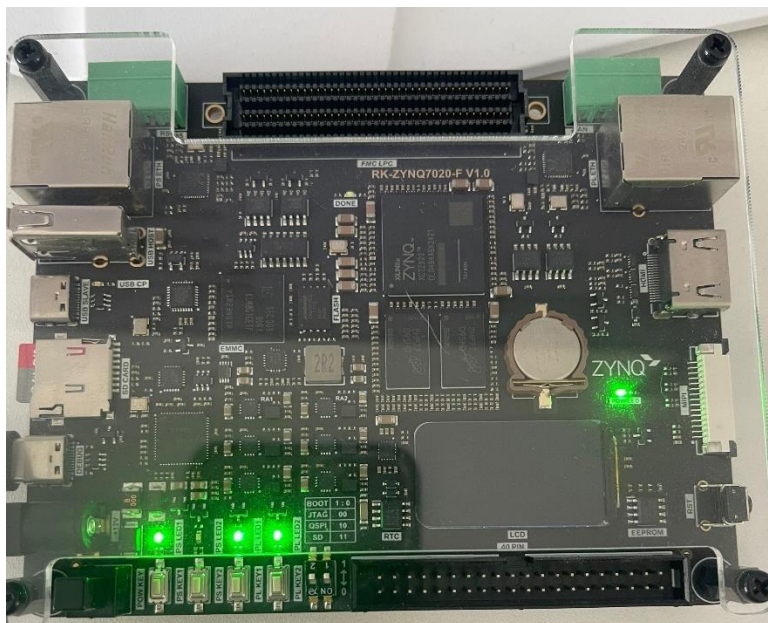


图 6-10 PS LED 端亮起

如果要将 LED 关闭的话，可以输入以下指令，将 LED 熄灭。

命令为：`echo 0 > /sys/class/gpio/gpio914/value`

输入完上述指令，可以熄灭 LED。上述 Shell 指令，也可以通过脚本实现。

## 2. 使用 C 语言控制 LED 的亮灭

在嵌入式的开发中，大部分情况都是使用 C 语言来控制外设，下面介绍一种使用 C 语言控制 LED 亮灭的方式。

(1) 在“/home/riguke/petalinux\_prj/05\_LED\_Driver”路径中新建一个 led\_driver\_app 文件，并使用以下命令创建一个名为“led\_driver.c”的文件：

命令为：`touch led_driver.c`

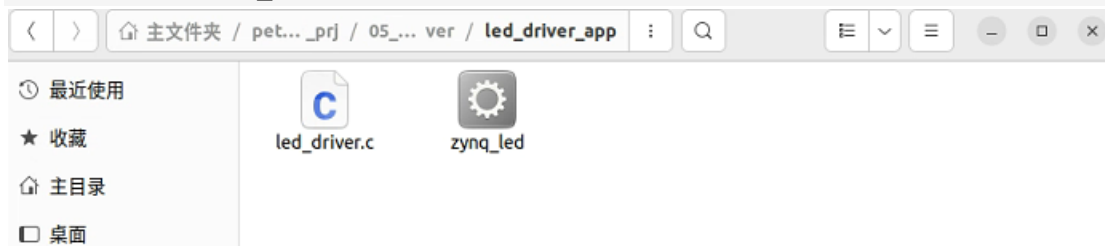


图 6-11 新建一个名为“led\_driver.c”的 C 语言文件

(2) 对新建的 led\_driver.c 文件进行代码编写，编写代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdbool.h>
#include <string.h>

#define PS_LED_BASE 904
```

```

#define PS_LED_INDEX 8
#define PS_LED_ADDR (PS_LED_BASE+PS_LED_INDEX)

// 安全的 write 包装函数
static int safe_write(int fd, const char *buf) {
    ssize_t ret = write(fd, buf, strlen(buf) + 1); // +1 包含 null 终止符
    if (ret == -1) {
        perror("write error");
        return -1;
    }
    return 0;
}

void led_ctl_init(void) {
    int lGpioFd;
    char lCache[100];

    // 导出 GPIO
    lGpioFd = open("/sys/class/gpio/export", O_WRONLY);
    if (lGpioFd < 0) {
        perror("open export fail");
        return;
    }

    int len = snprintf(lCache, sizeof(lCache), "%d", PS_LED_ADDR);
    if (safe_write(lGpioFd, lCache) < 0) {
        close(lGpioFd);
        return;
    }
    close(lGpioFd);

    // 设置方向
    snprintf(lCache, sizeof(lCache), "/sys/class/gpio/gpio%d/direction",
PS_LED_ADDR);
    lGpioFd = open(lCache, O_RDWR);
    if (lGpioFd < 0) {
        perror("open direction fail");
        return;
    }

    if (safe_write(lGpioFd, "out") < 0) {
        close(lGpioFd);
    }
}

```

```

        return;
    }
    close(lGpioFd);
}

void led_ctl_on(bool isON) {
    int lGpioFd;
    char lCache[100];

    snprintf(lCache, sizeof(lCache), "/sys/class/gpio/gpio%d/value",
PS_LED_ADDR);
    lGpioFd = open(lCache, O_RDWR);
    if (lGpioFd < 0) {
        perror("open value fail");
        return;
    }

    if (safe_write(lGpioFd, isON ? "1" : "0") < 0) {
        close(lGpioFd);
        return;
    }
    close(lGpioFd);
}

int main() {
    led_ctl_init();
    printf("=====start led ctrl=====\\n");

    while (1) {
        led_ctl_on(true);
        printf("on\\n");
        usleep(500000);

        led_ctl_on(false);
        printf("off\\n");
        usleep(500000);
    }

    return 0;
}

```

代码中“led\_ctl\_init()”函数主要功能为导出 GPIO 的管脚，并将其初始化为输出管脚。“led\_ctl\_on()”函数控制管脚的高低电平，实现 LED 灯的亮灭。



(3) 通过 RK7020 的原理图可以看到 PS\_LED 连接的管脚为 MIO 7 和 MIO 8，例程选择 MIO 8 作为演示。

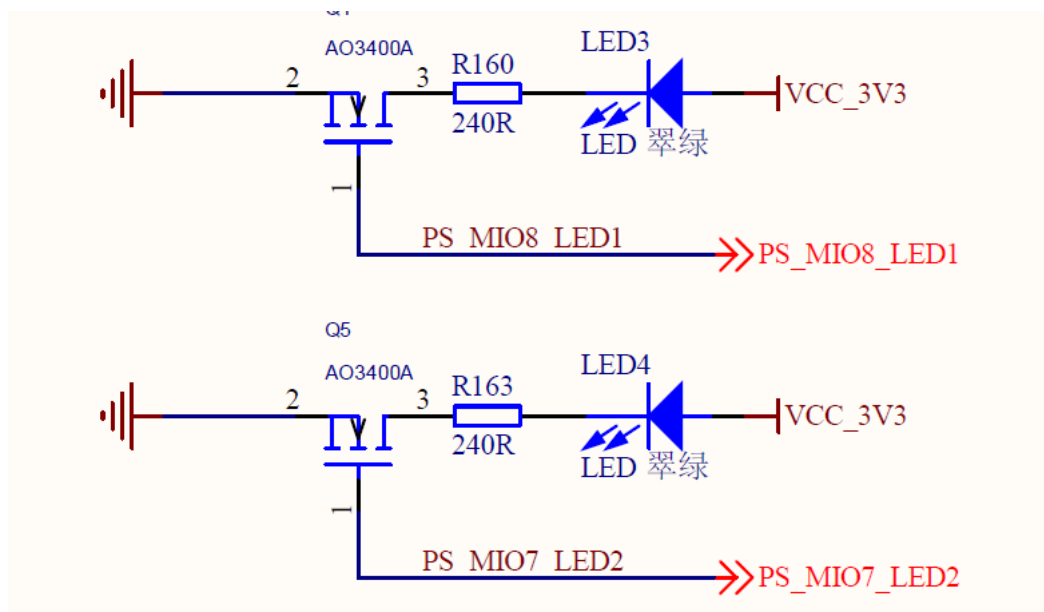


图 6-12 RK 7020 LED 管脚

MIO 8 也就是说这个管脚在这个 GPIO 组中的偏移值为 0，那么依据上一节中计算的 GPIO 编号公式可以知道，操作的编号就是 904+8，也就对应代码中的宏定义：

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdbool.h>
#include <string.h> // 添加strlen()支持

#define PS_LED_BASE 906
#define PS_LED_INDEX 8
#define PS_LED_ADDR (PS_LED_BASE+PS_LED_INDEX)
```

图 6-13 GPIO 编号与硬件对应

(4) 使用第五章中交叉编译 ARM 程序的教程，先设置交叉编译的环境，随后使用交叉编译，将上述 C 语言代码输出为 ARM 能够识别的程序。

命令为：\$CC led\_driver.c -o zynq\_led

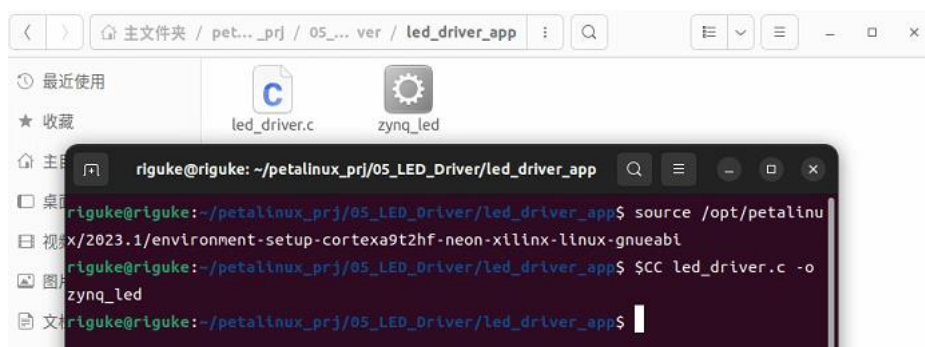


图 6-14 将 C 语言程序编译为 ARM 能识别的应用程序

(5) 使用第四章教程中通过 NFS 共享运行的方式运行教程编译完毕的程序，运行结果如

下:

```
=====start led ctrl=====
on
off
on
off
on
off
on
off
on
^C
root@arm_hello_world:/mnt#
```

图 6-15 终端输出

随着终端输出，可以看到板卡上 PS 端 LED 1 在闪烁。

## 七. Petalinux 下的 HDMI 显示

前面的教程中体验了使用 Petalinux 开发嵌入式 Linux 系统，但使用的功能仅仅是 Petalinux 冰山一角，Petalinux 的强大功能需要我们不断的探索，本实验讲解如何使用一个自己的内核来做 Linux，这样我们可以在内核里添加很多驱动，例如 HDMI 显示。

开发板是没有使用的没有 HDMI 接口芯片，但是使用了 PFPGA 去完成编码，笔者对 Xilinx 提供的 Linux 内核中加入了 HDMI 编码 IP 的驱动。请使用其他版本软件开发者注意，本教程只提供修改版 linux-xlnx-xlnx\_rebase\_v6.1\_LTS 内核，不提供其他版本修改版，也不提供修改说明。

### 1. 建立 vivado 工程

本节教程是使用了 PL 端 HDMI 接口，而前面章节中 vivado 工程只对 PS 端外设进行配置，而配置 PL 端 HDMI 是比较复杂的，因此在本章开始前，对 Vivado 工程的配置进行详细配置。

此工程除了 HDMI 的驱动，也添加了 PL 端的按键与 LED 外设。

(1) 将本章例程中提供的 HDMI 的编码 IP 先拷贝到开发环境中。(vivado 工程是在 Linux 环境想配置，windows 工程是一样的，几乎没有区别。)

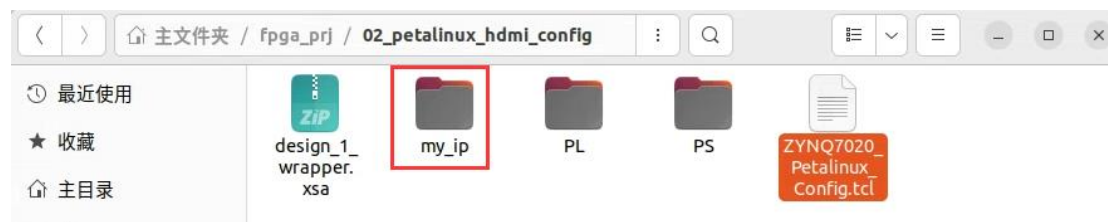


图 7-1 拷贝 my\_ip 文件至目录中

上图中文件内容如下图所示：

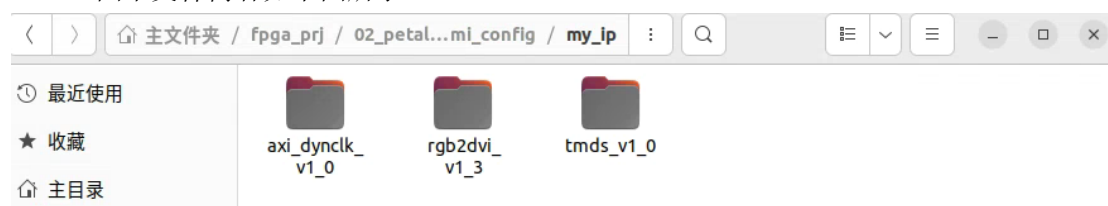


图 7-2 my\_ip 文件内容

上述自定义 IP 主要是添加了一个同步时钟以及添加了一个 RGB 转换为 HDMI 时序的 IP。

在 vivado 首页通过 Tools → setting → ip → Repository 界面中将 IP 文件添加进去，如下图所示：

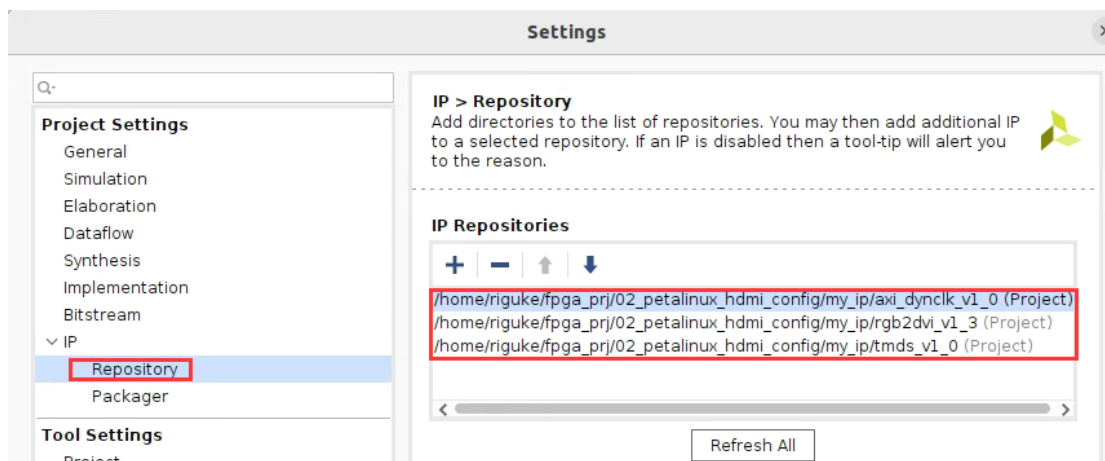


图 7-3 添加 IP

(2) 首先配置 ZYNQ 的外设，这个主要根据原理图来，如配置串口 UART，QSPI，DDR 等，需要注意的是，该外设需要与原理图对应，这一张中配置了 UART0，USB，IIC 等，配置的外设如下：

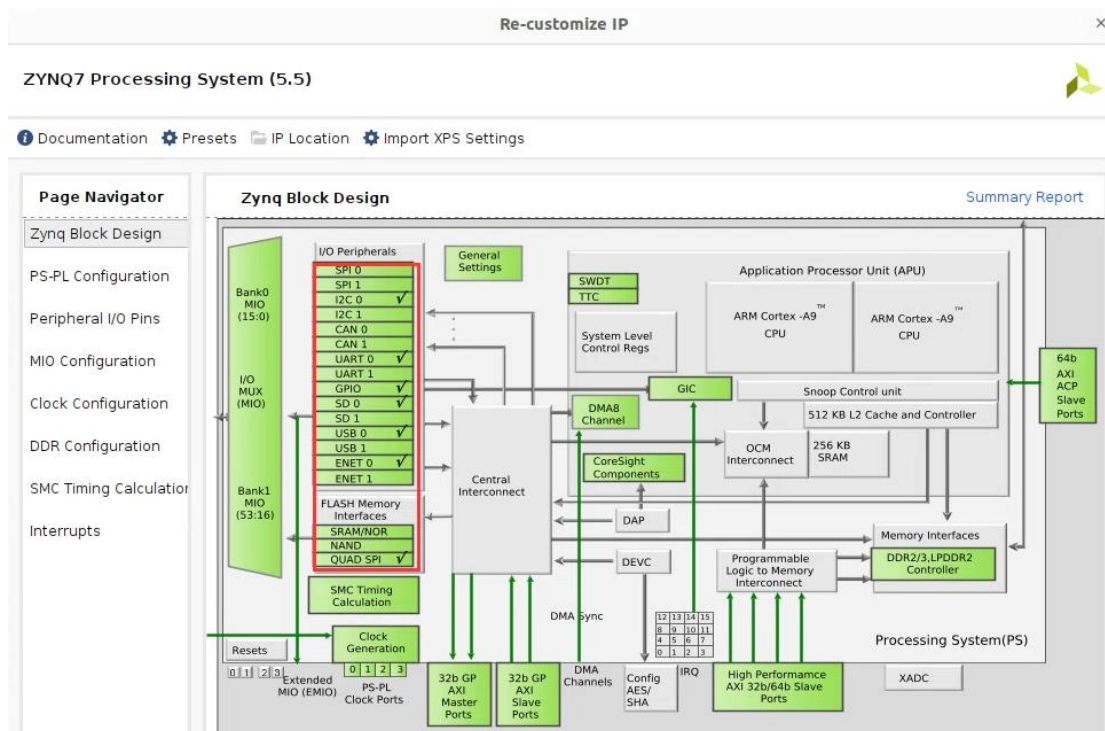


图 7-4 配置 ZYNQ 外设

需要注意的是，上述几乎所有的外设都是依据原理图进行的配置，但 IIC 0 这个外设是用于与显示器进行通信的，因此该外设是配置为 EMIO 模式，如下图所示：

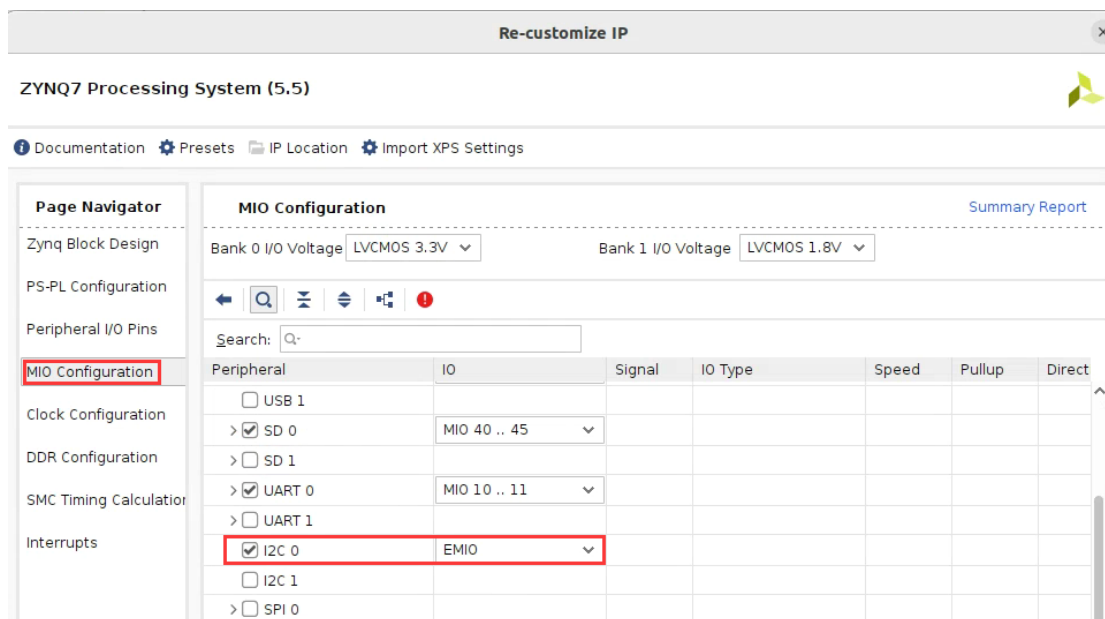


图 7-5 将 IIC 0 配置为 EMIO 模式

其他关于 DDR，以太网等外设与 Vitis 教程中是一致的，因此这一块不多说，如果有疑问的话，可以直接使用提供的 TCL 文件，快速恢复 ZYNQ 的配置。

(3) 配置时钟，由于需要使用 PS 端给 PL 端提供两组时钟，一组 100M，一组 150M，因此配置如下：

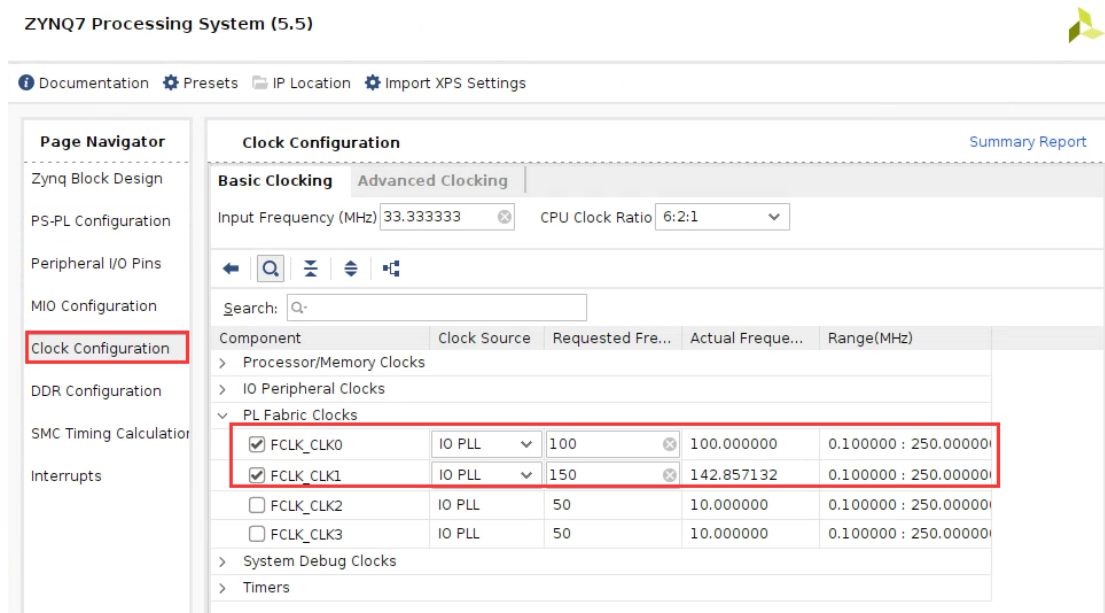


图 7-6 配置 PS 端输出 PL 端的时钟

(4) 打开 GP 与 HP 接口，主要是将图像数据流通过 HP 端口进入 PS 端 DDR，同时使用 GP 接口配置 AXI Interface，配置如下图所示：

## ZYNQ7 Processing System (5.5)



Documentation Presets IP Location Import XPS Settings

**Page Navigator**

- Zynq Block Design
- PS-PL Configuration**
- Peripheral I/O Pins
- MIO Configuration
- Clock Configuration
- DDR Configuration
- SMC Timing Calculation
- Interrupts

**PS-PL Configuration** [Summary Report](#)

Search: Q-

| Name                            | Select | Description                                           |
|---------------------------------|--------|-------------------------------------------------------|
| > General                       |        |                                                       |
| ✓ AXI Non Secure Enablement     | 0      | Enable AXI Non Secure Transaction                     |
| > GP Master AXI Interface       |        |                                                       |
| > M AXI GP0 Interface           | ✓      | Enables General purpose AXI master interface 0        |
| > M AXI GP1 Interface           |        | Enables General purpose AXI master interface 1        |
| > GP Slave AXI Interface        |        |                                                       |
| > HP Slave AXI Interface        |        |                                                       |
| > S AXI HP0 Interface           | ✓      | Enables AXI high performance slave interface 0        |
| > S AXI HP1 Interface           |        | Enables AXI high performance slave interface 1        |
| > S AXI HP2 Interface           |        | Enables AXI high performance slave interface 2        |
| > S AXI HP3 Interface           |        | Enables AXI high performance slave interface 3        |
| > ACP Slave AXI Interface       |        |                                                       |
| > DMA Controller                |        |                                                       |
| > PS-PL Cross Trigger interface |        | Enables PL cross trigger signals to PS and vice-versa |

图 7-7 打开 GP 与 HP 接口

(5) 最后开启 PL 至 PS 端的中断接口，如下图所示：

**Re-customize IP**

**ZYNQ7 Processing System (5.5)**

Documentation Presets IP Location Import XPS Settings

**Page Navigator**

- Zynq Block Design
- PS-PL Configuration
- Peripheral I/O Pins
- MIO Configuration
- Clock Configuration
- DDR Configuration
- SMC Timing Calculation
- Interrupts**

**Interrupts** [Summary Report](#)

Search: Q-

| Interrupt Port          | ID               | Description                                                                            |
|-------------------------|------------------|----------------------------------------------------------------------------------------|
| ✓ Fabric Interrupts     |                  | Enable PL Interrupts to PS and vice versa                                              |
| > PL-PS Interrupt Ports |                  |                                                                                        |
| ✓ IRQ_F2P[15:0]         | [91:84], [68:61] | Enables 16-bit shared interrupt port from the PL. MSB is assigned to PS and vice versa |
| Core0_nFIQ              | 28               | Enables fast private interrupt signal for CPU0 from the PL                             |
| Core0_nIRQ              | 31               | Enables private interrupt signal for CPU0 from the PL                                  |
| Core1_nFIQ              | 28               | Enables fast private interrupt signal for CPU1 from the PL                             |
| Core1_nIRQ              | 31               | Enables private interrupt signal for CPU1 from the PL                                  |
| > PS-PL Interrupt Ports |                  |                                                                                        |

图 7-8 开启中断接口



图 7-9 ZYNQ 最小系统配置完毕



(6) 添加自定义的参考时钟 IP 以及 RGB 转 TMDS 的 IP，如下图所示：

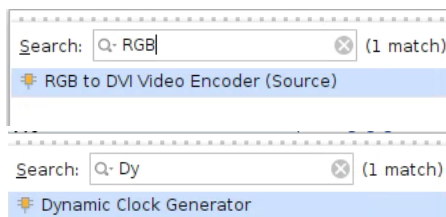


图 7-10 添加 RGB 转 DVIIP 以及同步时钟产生模块 IP

上述添加的 IP 是自定义产生的 IP，是不用配置的。

(7) 添加 Xilinx 的 VDMA IP，并对 IP 进行配置。

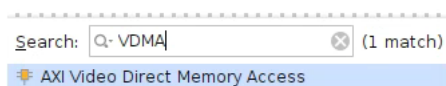


图 7-11 添加 VDMA IP

添加完 IP 后，对 VDMA 进行配置，配置如下图所示：

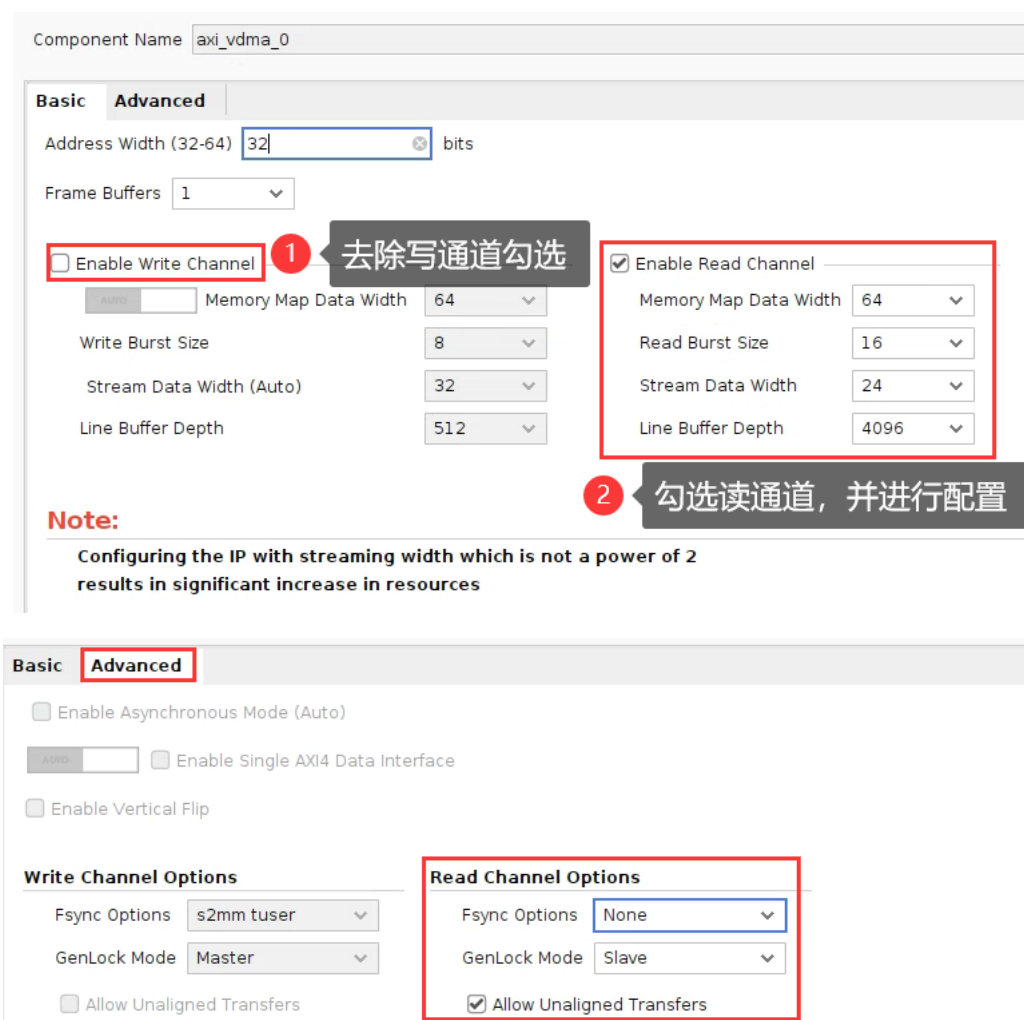


图 7-12 配置 VDMA

(8) 添加视频时序控制模块。

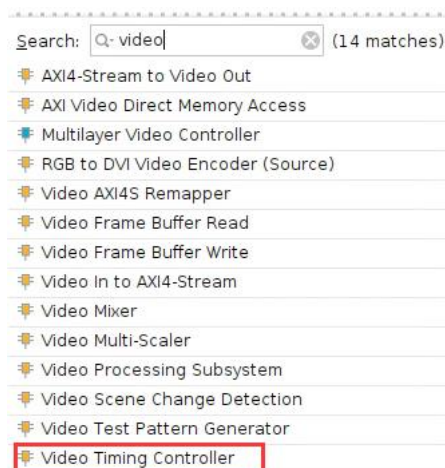


图 7-13 添加视频时序控制 IP

IP 只需要配置第一个界面，其他界面不需要修改，如下图所示：

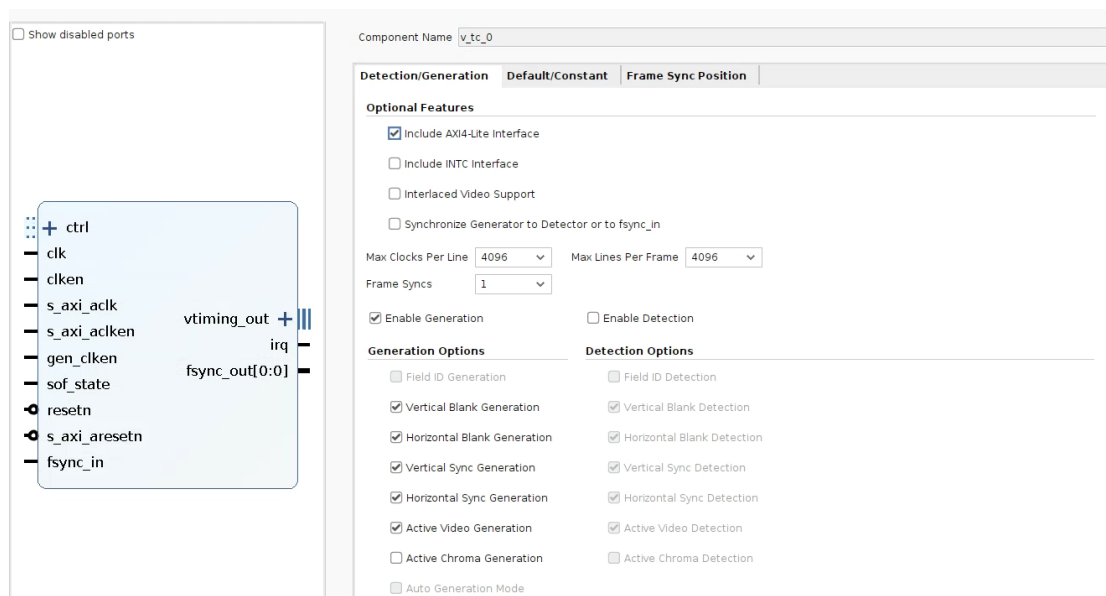


图 7-14 配置时序控制 IP

## (9) 添加 AXI4-Stream Subset Converter IP, IP 配置如下:

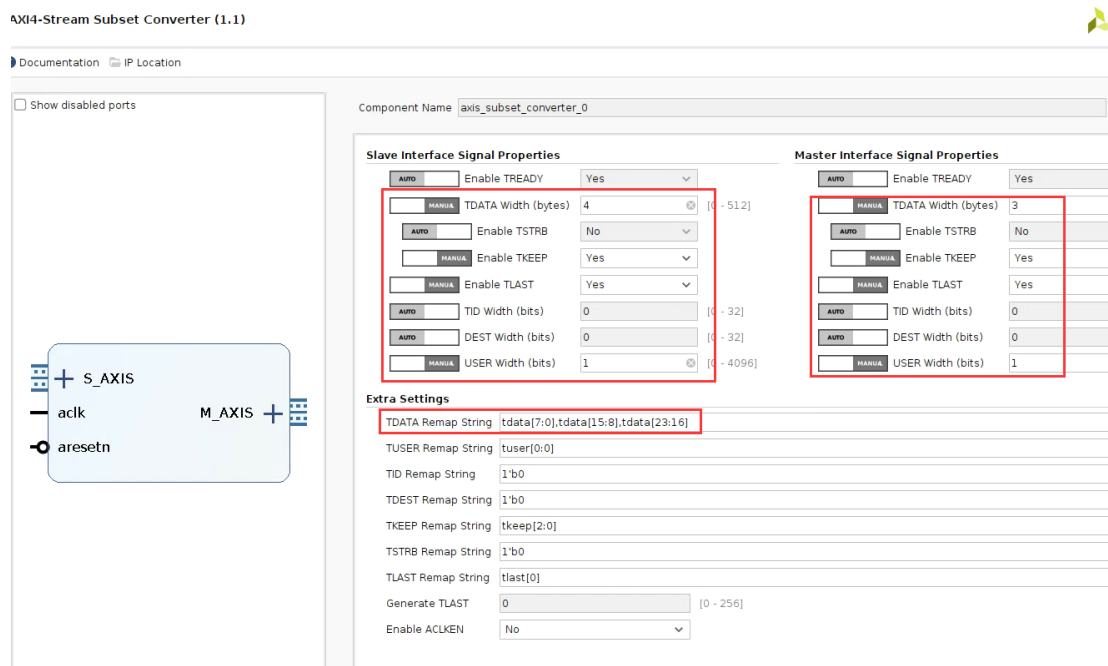


图 7-15 添加 AXI4-Stream Subset Converter IP 并进行配置

## (10) 添加 AXI4-Stream to Video Out IP, IP 配置如下:

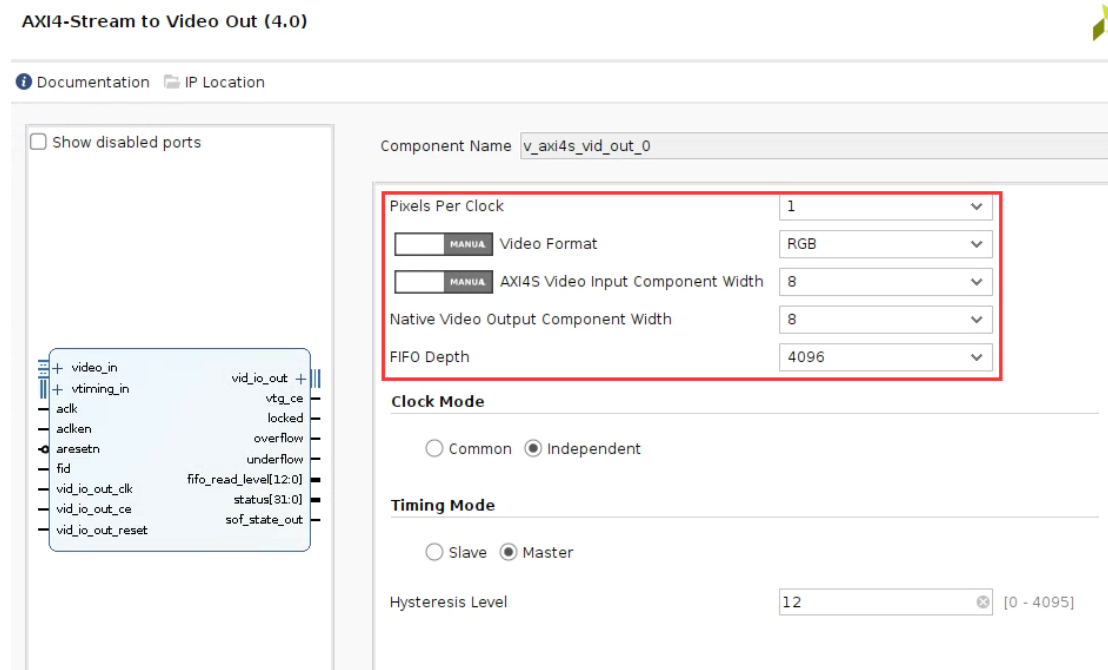


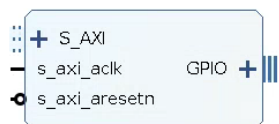
图 7-16 添加 AXI4-Stream to Video Out IP 并进行配置

(11) 最后添加两个 AXI GPIO IP, 用于控制 PL 端 LED 和按键, 由于 RK 7020 PL 端按键与 LED 都只有两个因此, AXI GPIO 的位宽设置为 2, 按键 GPIO 设置为输入, LED GPIO 设置为输出, 如下图所示:

## AXI GPIO (2.0)

Documentation IP Location

☐ Show disabled ports



Component Name axi\_gpio\_btn

## GPIO

☒ All Inputs

☐ All Outputs

GPIO Width 2 [1 - 32]

Default Output Value 0x00000000 [0x00000000,0xFFFFFFFF]

Default Tri State Value 0xFFFFFFFF [0x00000000,0xFFFFFFFF]

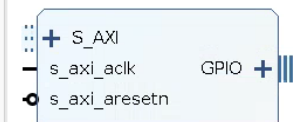
☐ Enable Dual Channel

## GPIO 2

图 7-17 按键 GPIO 设置

Documentation IP Location

☐ Show disabled ports



Component Name axi\_gpio\_led

## GPIO

☐ All Inputs

☒ All Outputs

GPIO Width 2 [1 - 32]

Default Output Value 0x00000000 [0x00000000,0xFFFFFFFF]

Default Tri State Value 0xFFFFFFFF [0x00000000,0xFFFFFFFF]

☐ Enable Dual Channel

## GPIO 2

图 7-18 LED GPIO 设置

(12) 上述基本就配置好 IP，接下来则是进行连线，只要搞明白数据流向，这些都比较简单的。

将 RGB to DIV 的 RGB 输入与 AXI4-Stream to Video Out 的输出进行连接，如下图所示：

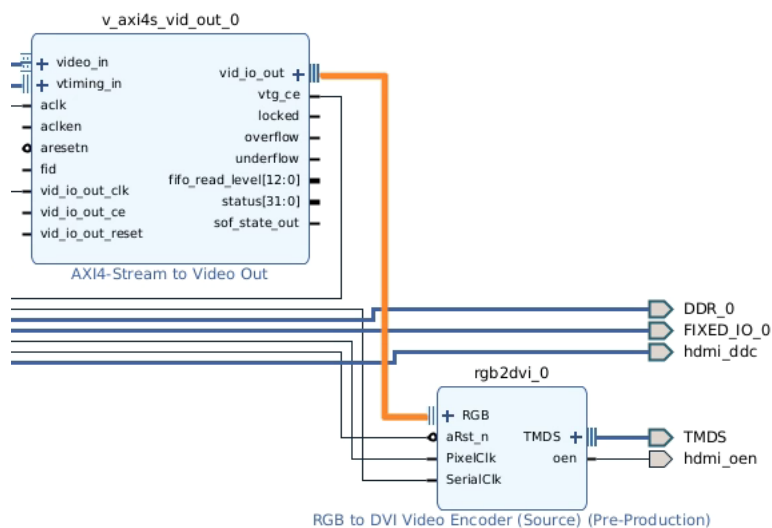


图 7-19 连接 AXI4-Stream to Video Out 的输出与 RGB to DIV 的 RGB 输入

连接完毕后，将 RGB to DIV 的 TMDS 与 HDMI\_OEN 管脚引出，HDMI\_OEN 管脚是检测热拔插的，对应原理图中 HDMI\_HPDI。

随后将 AXI4-Stream to Video Out 的 video\_in 与 vtiming\_in 分别连接至 AXI4-Stream Subset Converter 与 Video Timing Controeller 模块，连接如下图所示：

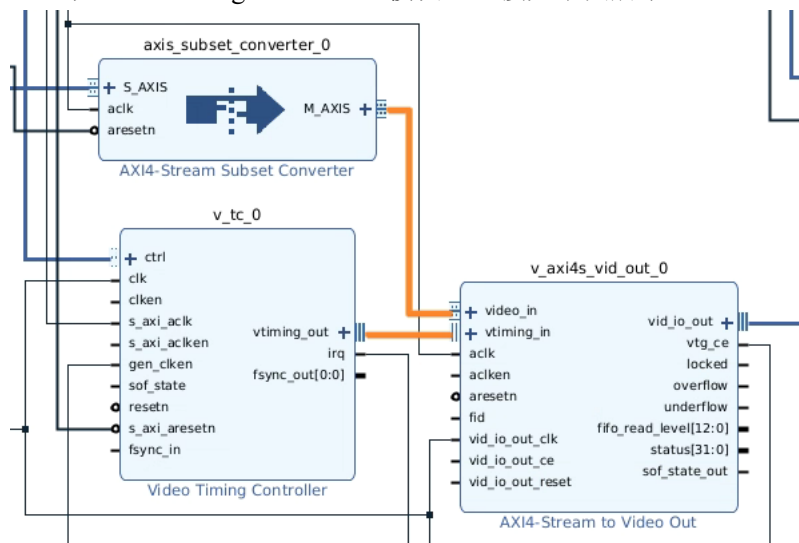


图 7-20 连接时序控制与视频控制输入

随后将 AXI4-Stream Subset Converter 与 VDMA 模块进行连接，如下图所示：

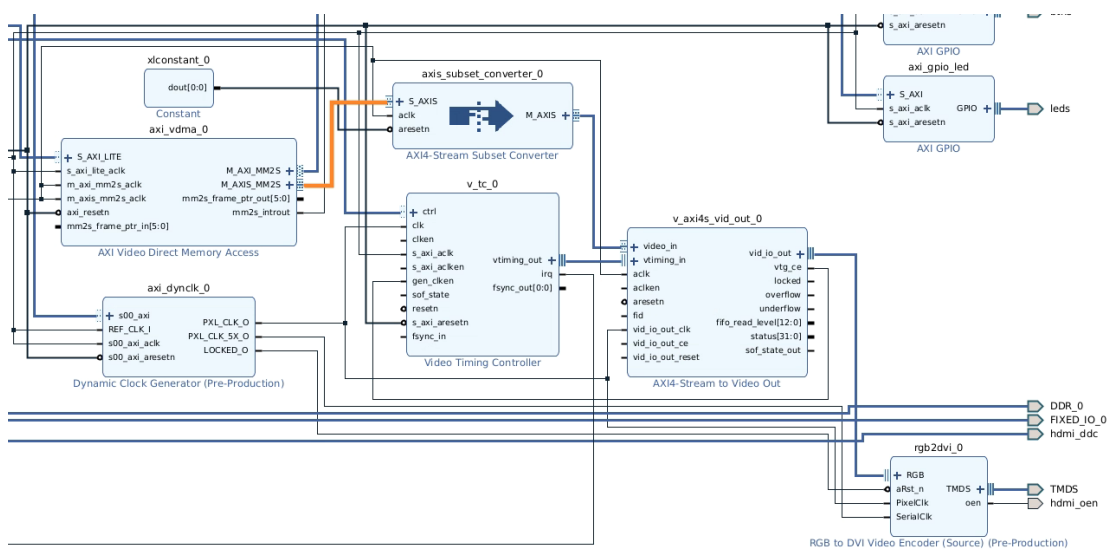


图 7-20 连接 AXI4-Stream Subset Converter 与 VDMA 模块

随后将同步时钟模块与 RGB to DVI 模块的时钟进行连接，如下图所示：

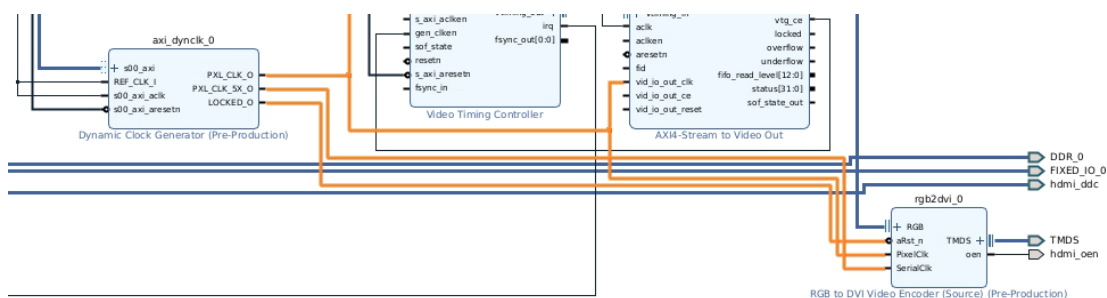


图 7-21 连接 RGB to DVI 模块的时钟与复位

通过上述已经将整个数据流已经连接完毕了，接下来就是通过 AXI Internet 模块以及同步复位模块将时钟信号连接，这些可以通过 vivado 自动连接。整个连接完毕图如下图所示：

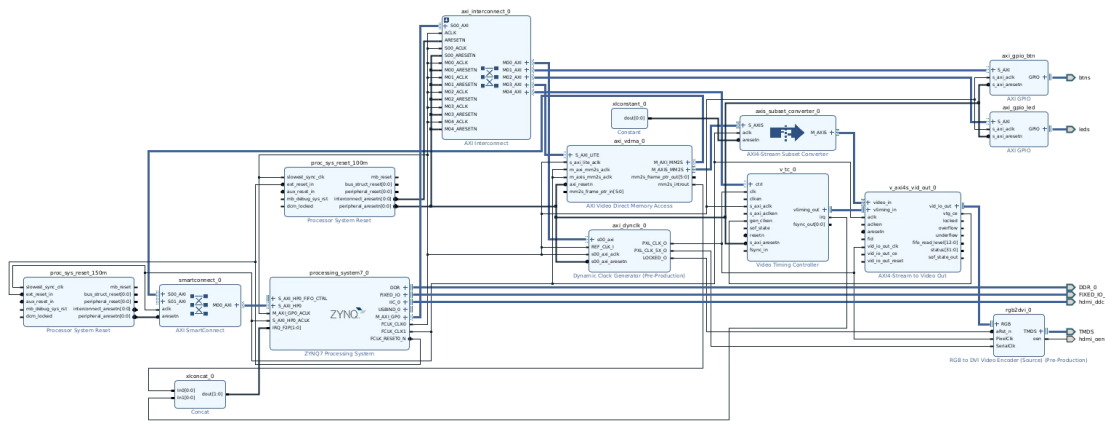


图 7-22 整个系统连接图

连接完毕后，就将进行管脚约束，在这就不多说了。管脚约束完毕后，生成 Bit，并导出 XSA，导出 XSA 时需要选择包括 BIT 文件。

## 2. Petalinux 配置

本实验还是在前面实验的 Petalinux 工程修改，需要先掌握前面的实验内容。

(1) 通过上一节中导出的 XSA 文件，新建一个 Petalinux 工程，新建工程，并进入 Petalinux 配置，进入配置如下：

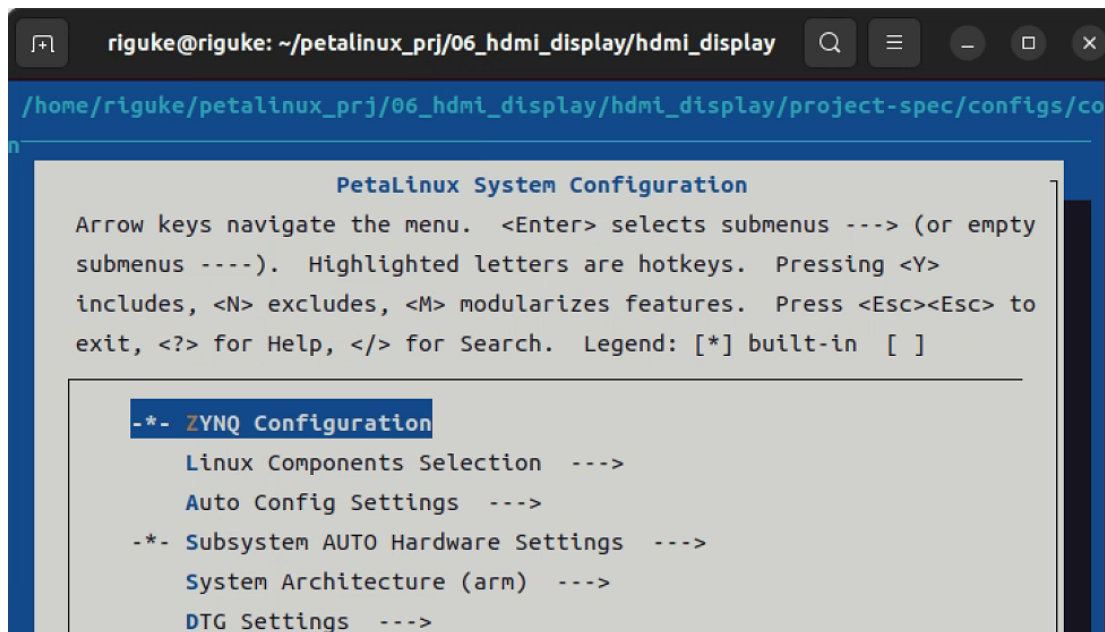


图 7-23 进入 Petalinux 配置

(2) 配置离线编译包的地址等，这些配置流程与前面章节是一样的，不进行详细的配置说明。若有疑问可以参考第二章熟悉整个工程建立的流程。



### 3. 配置 Linux 内核

(13) 运行下面的命令配置 Linux 内核：

命令为：petalinux-config -c kernel

```
riguke@riguke:~/petalinux_prj/06_hdmi_display/hdmi_display$ petalinux-config -c
kernel
[INFO] Sourcing buildtools
[INFO] Silentconfig project
[INFO] Silentconfig rootfs
[INFO] Generating workspace directory
```

图 7-24 配置 Linux 内核

一般进入 Linux 内核配置的时间都特别长，进行等待即可。

**注意：**在配置内核之前一定要配置离线编译包，不然时间会特别长，还会存在进入配置内核失败的可能。

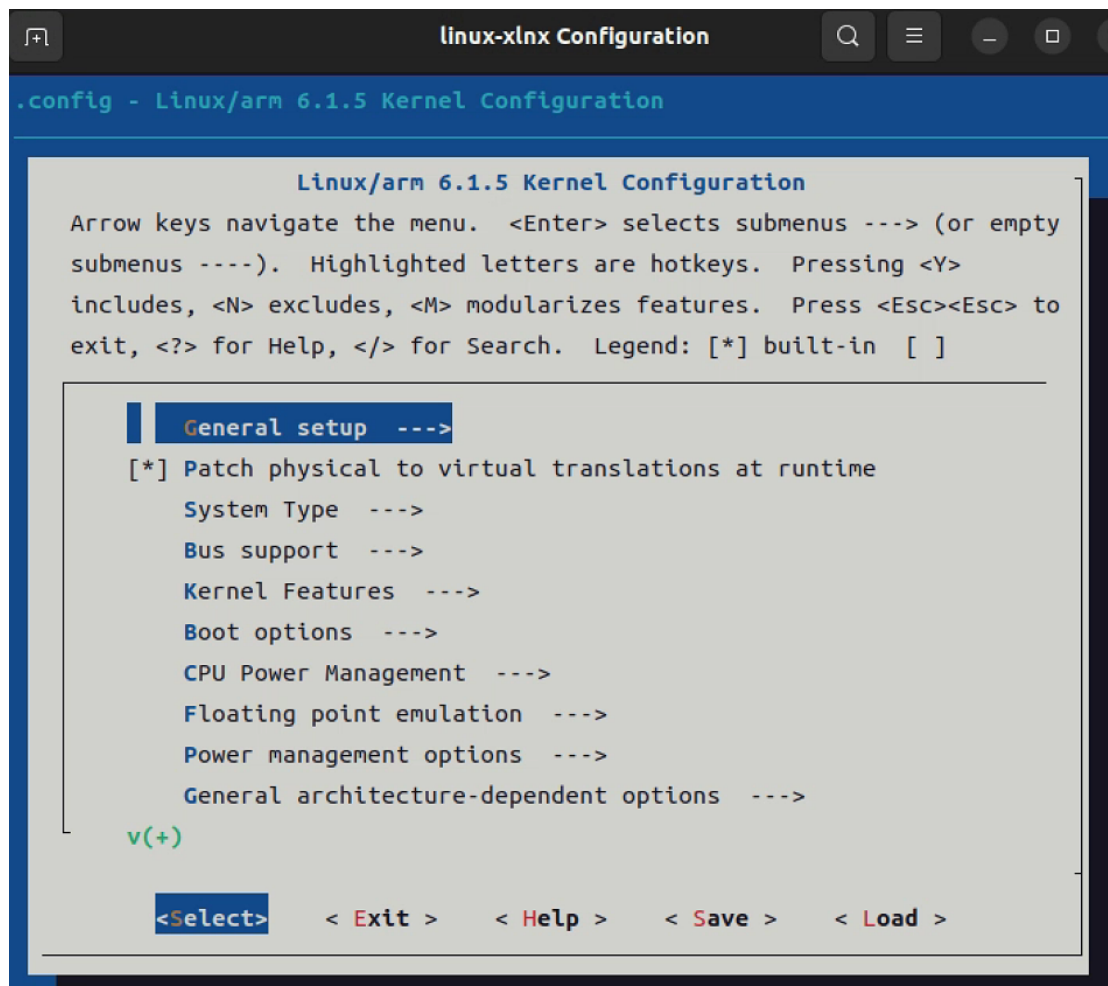


图 7-25 进入 Linux 内核配置

(14) 在弹出的窗口中，进入 Device Drivers → Graphics support，分别选择以下 5 项，如下图所示：

Xilinx DRM KMS Driver

Xilinx DRM KMS bridge  
 Digilent DRM Driver  
 Xilinx DRM PL display driver  
 Xilinx DRM VTC Driver

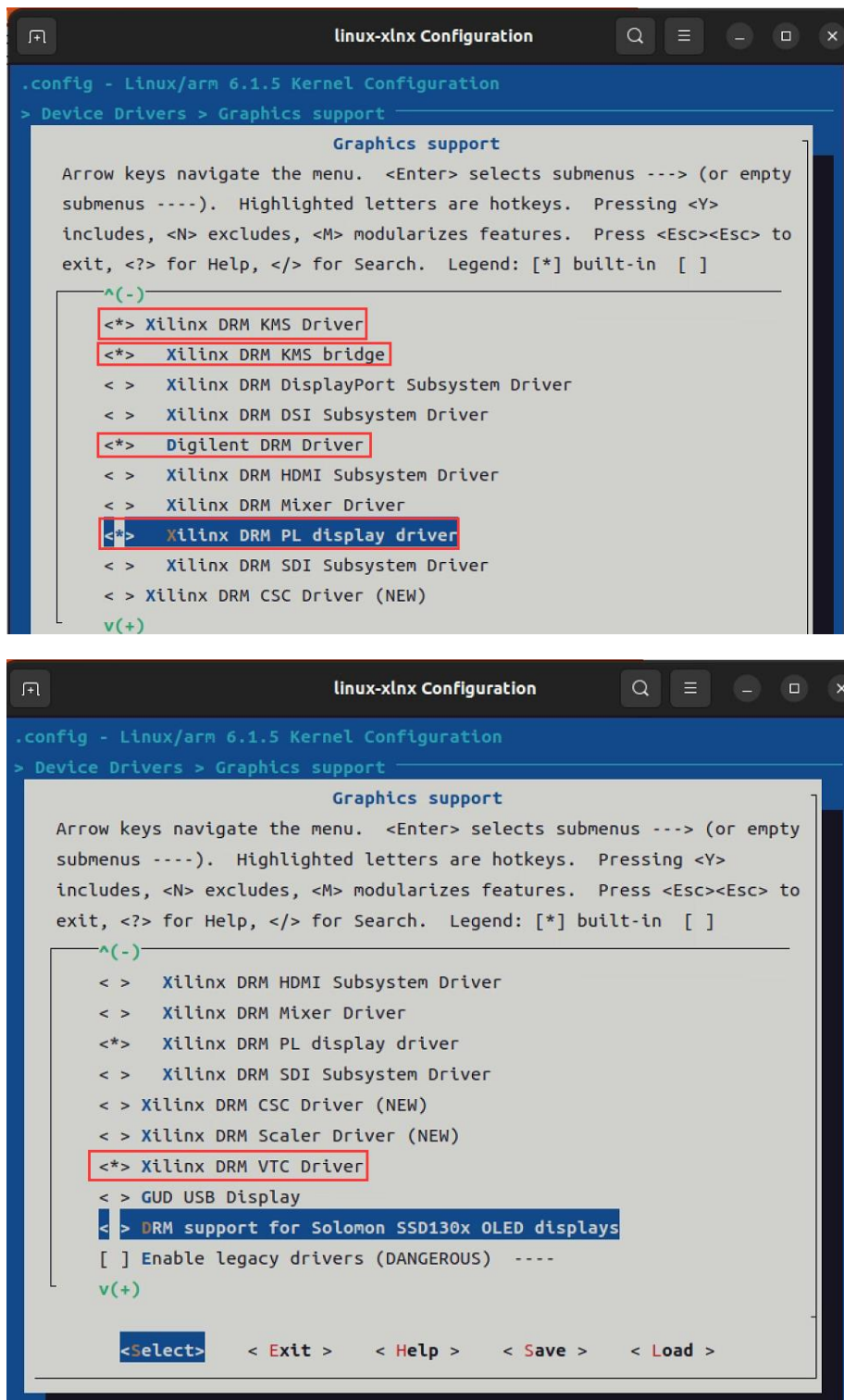


图 7-26 配置 HDMI 驱动

(15) 在 Device Drivers → Graphics support → Frame buffer Devices 中选择 Support for frame buffer devices 按 y

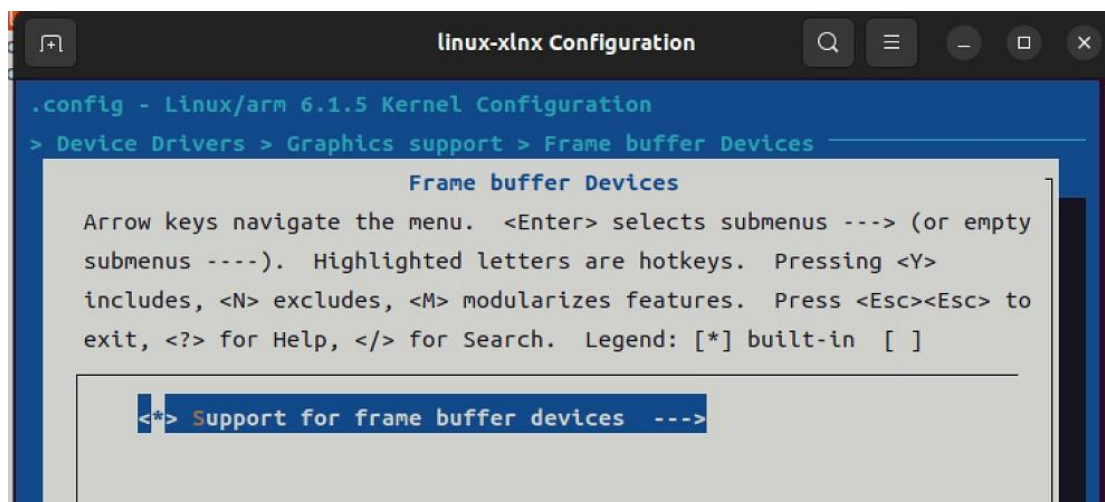


图 7-27 配置 buffer

(16) 在 Device Drivers → Graphics support → Console display driver support 选项中选择 Framebuffer Console support 按 y

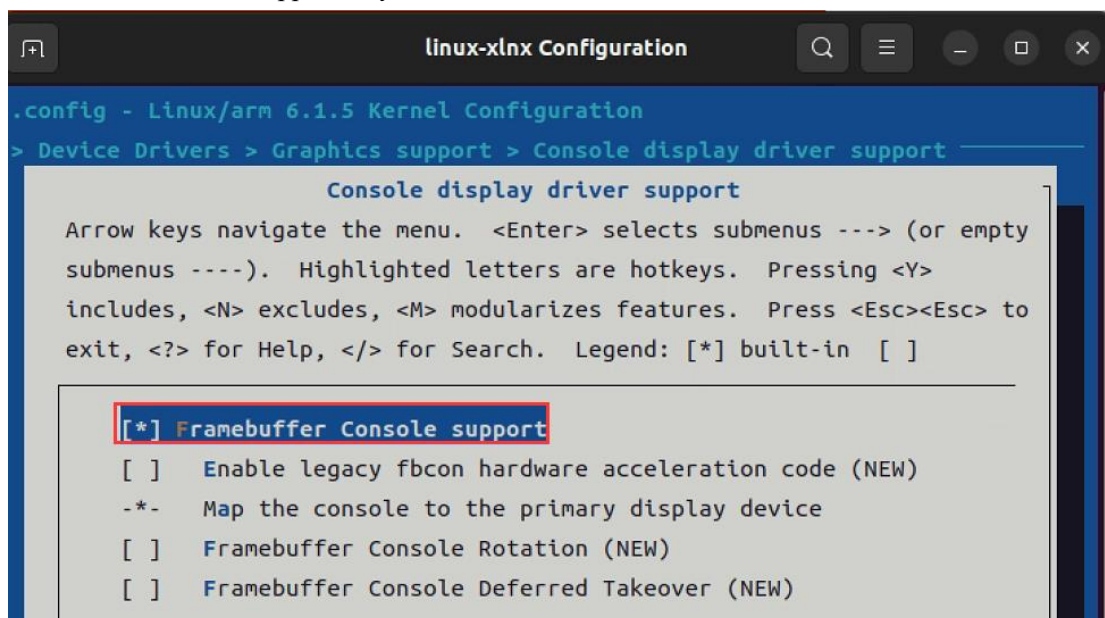


图 7-28 配置 buffer

(17) 在 Device Drivers → Common Clock Framework 选项中选择 Digilent axi\_dynclk Driver 按 y

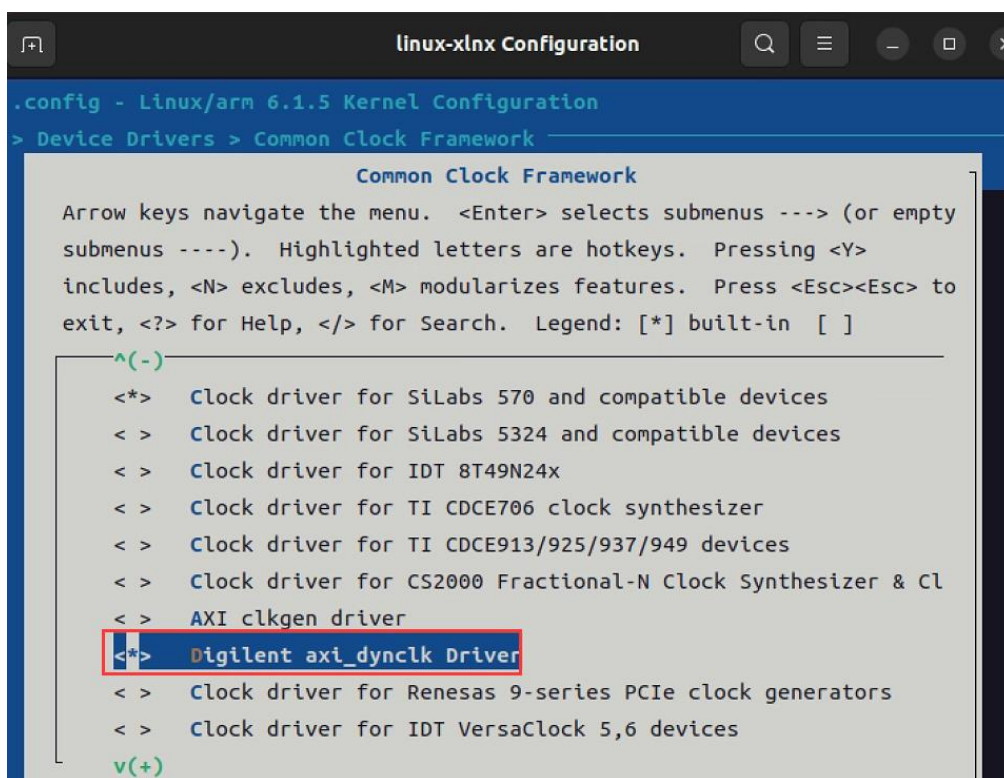


图 7-29 配置时钟

(18) 保存并退出

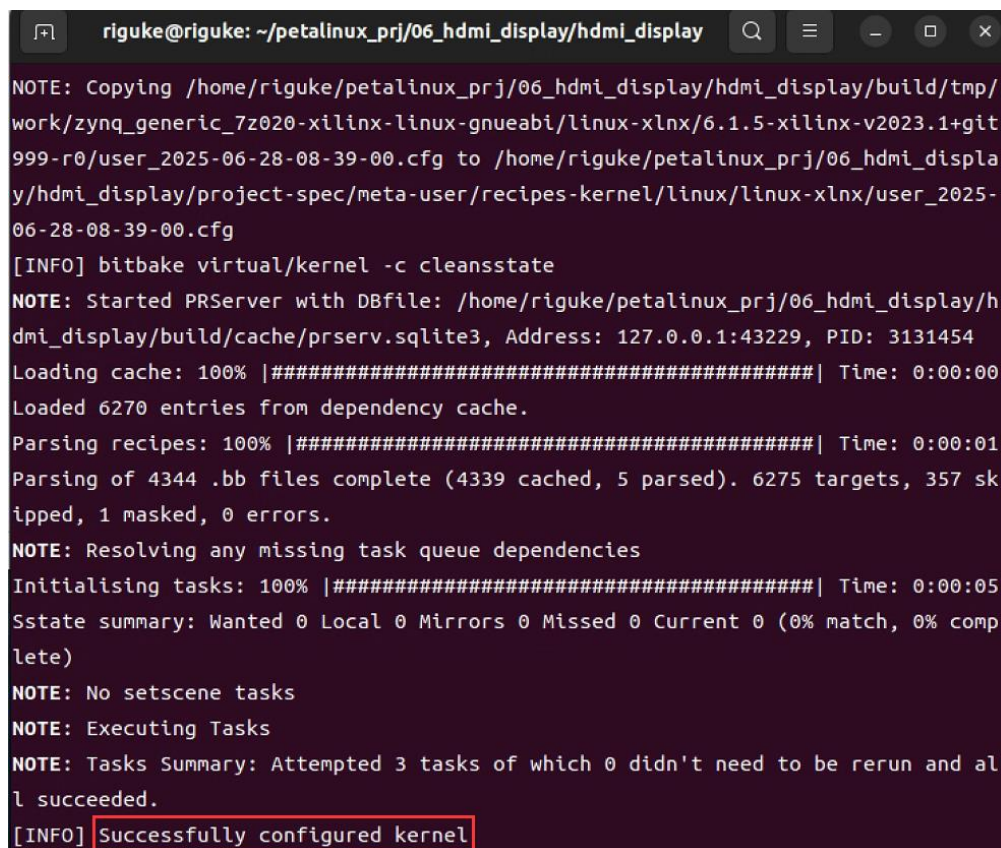


图 7-30 配置内核成功



## 4. 编写设备树

设备树是描述设备信息的一种格式化文本，这种文本结构对类似于 XML 和 JSON，每个驱动要求的设备树节点也是不同的，对于没接触过设备的开发人员，需要慢慢熟悉，在一次一次的使用中慢慢掌握。

(1) 打开 petalinux 工程文件中的名为 system-user.dtsi 的文件，如下图所示：

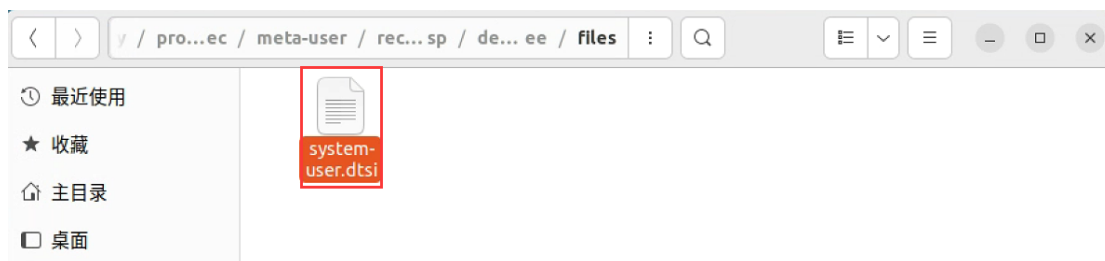


图 7-31 打开设备树

(2) 将设备树内容如下：

```
/include/ "system-conf.dtsi"

/ {
    model = "Zynq RIGUKE Development Board";
    compatible = "riguke,zynq", "xlnx,zynq-7000";
    aliases {
        ethernet0 = "&gem0";
        serial0 = "&uart0";
    };

    usb_phy0: usb_phy@0 {
        compatible = "ulpi-phy";
        #phy-cells = <0>;
        reg = <0xe0002000 0x1000>;
        view-port = <0x0170>;
        drv-vbus;
    };
};

&i2c0 {
    clock-frequency = <100000>;
};
```

```

&usb0 {
    dr_mode = "host";
    usb-phy = <&usb_phy0>;
};

&sdhci0 {
    u-boot,dm-pre-reloc;
};

&uart0 {
    u-boot,dm-pre-reloc;
};

&flash0 {
    compatible = "micron,m25p80", "w25q256", "spi-flash";
};

&gem0 {
    phy-handle = <&ethernet_phy>;
    ethernet_phy: ethernet-phy@1 {
        reg = <1>;
        device_type = "ethernet-phy";
    };
};

&amba_pl {

    digilent_hdmi {
        compatible = "digilent,hdmi";

        clocks = <&axi_dyncclk_0>;
        clock-names = "clk";

        digilent_edid-i2c = <&i2c0>;
        digilent_fmax = <150000>;

        port@0 {
            #address-cells = <1>;

```



```

        #size-cells = <0>;

        hdmi_ep: endpoint {
            remote-endpoint = <&pl_disp_ep>;
        };
    };

xlnx_pl_disp {
    compatible = "xlnx,pl-disp";
    dmas = <&axi_vdma_0 0>;
    dma-names = "dma0";

    xlnx,vformat = "RG24";
    xlnx,bridge = <&v_tc_0>;

    port@0 {
        reg = <0>;
        pl_disp_ep: endpoint {
            remote-endpoint = <&hdmi_ep>;
        };
    };
};

&axi_dyncclk_0 {
    compatible = "dglnt,axi-dyncclk";
    #clock-cells = <0>;
    clocks = <&clkc 15>;
};

&v_tc_0 {
    compatible = "xlnx,bridge-v-tc-6.1";
    xlnx,pixels-per-clock = <1>;
};

```

修改完设备树后，则编译 uboot、内核、根文件系统、设备树等

(3) 编译完毕后，进入工程文件下的 image/linux 文件夹下，并运行下面命令生成 BOOT 文件，注意空格和短线，这次由于用到 PL 端资源，因此需要注意打包命令，命令如下所示：

命令为：petalinux-package --boot --fsbl --fpga --u-boot --force

```

riguke@riguke:~/petalinux_prj/06_hdmi_display/hdmi_display/images/linux$ petalinux-package --boot --fsbl --fpga --u-boot --force
[INFO] Sourcing buildtools
INFO: File in BOOT BIN: "/home/riguke/petalinux_prj/06_hdmi_display/hdmi_display/images/linux/zynq_fsbl.elf"
INFO: File in BOOT BIN: "/home/riguke/petalinux_prj/06_hdmi_display/hdmi_display/project-spec/hw-description/design_1_wrapper.bit"
INFO: File in BOOT BIN: "/home/riguke/petalinux_prj/06_hdmi_display/hdmi_display/images/linux/u-boot.elf"
INFO: File in BOOT BIN: "/home/riguke/petalinux_prj/06_hdmi_display/hdmi_display/images/linux/system.dtb"
INFO: Generating zynq binary package BOOT.BIN...

***** Bootgen v2023.1
**** Build date : Apr 7 2023-10:18:04
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
** Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.

[WARNING]: Partition zynq_fsbl.elf.0 range is overlapped with partition design_1_wrapper.bit.0 memory range
[WARNING]: Partition design_1_wrapper.bit.0 range is overlapped with partition system.dtb.0 memory range

[INFO] : Bootimage generated successfully

INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
riguke@riguke:~/petalinux_prj/06_hdmi_display/hdmi_display/images/linux$

```

图 7-32 打包工程输出

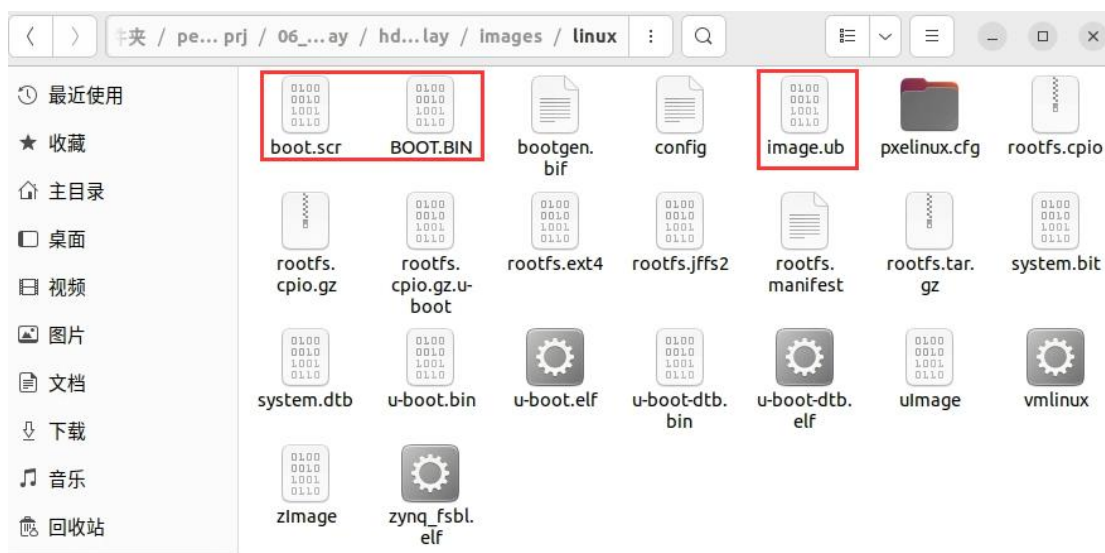


图 7-33 输出打包文件

## 5. 运行结果

将图 7-33 中的文件拷贝到 SD 卡中，随后将 RK 7020 的 HDMI 接口与显示器连接，随后启动开发板，等待一端时间后，可以看出显示器显示以下结果：

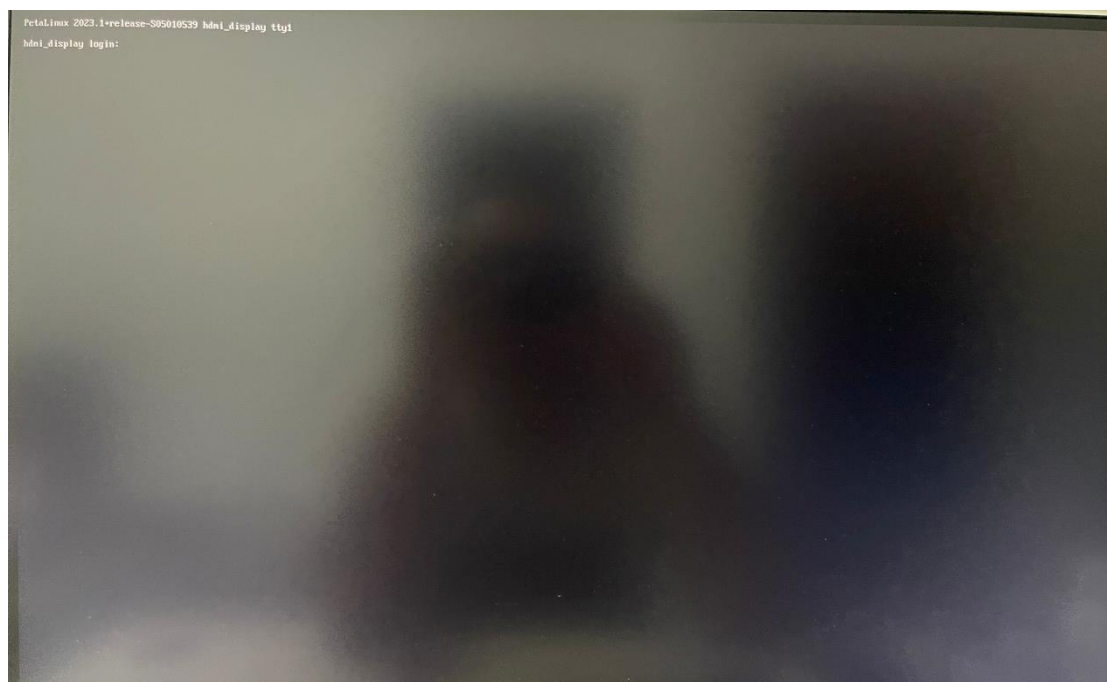


图 7-32 显示器输出

## 八. SD 卡根文件系统

前面的教程中我们学习了如何使用 Petalinux 定制一个嵌入式 Linux 系统以及 HDMI 显示, 本实验将做一个使用 SD 卡的根文件系统, 根文件系统的制作比较复杂, 本章不再讲解, 可以直接使用 Petalinux 工程生成的根文件系统, 或者直接使用制作好的 Debian 根文件系统。

由于根文件系统比较大, QSPI flash 是无法放下的, 只能放在 SD 卡或 EMMC 里, 所以我们要配置 Petalinux。

本实验还是在前面 HDMI 实验的 Vivado 的工程基础上进行修改的, 关于 HDMI 部分的 Petalinux 配置不在本章重复, 如果不清楚的, 参考第七章的配置。

### 1. Petalinux 配置

(1) 使用下列命令重新配置 petalinux, 命令如下:

命令为: `petalinux-config`

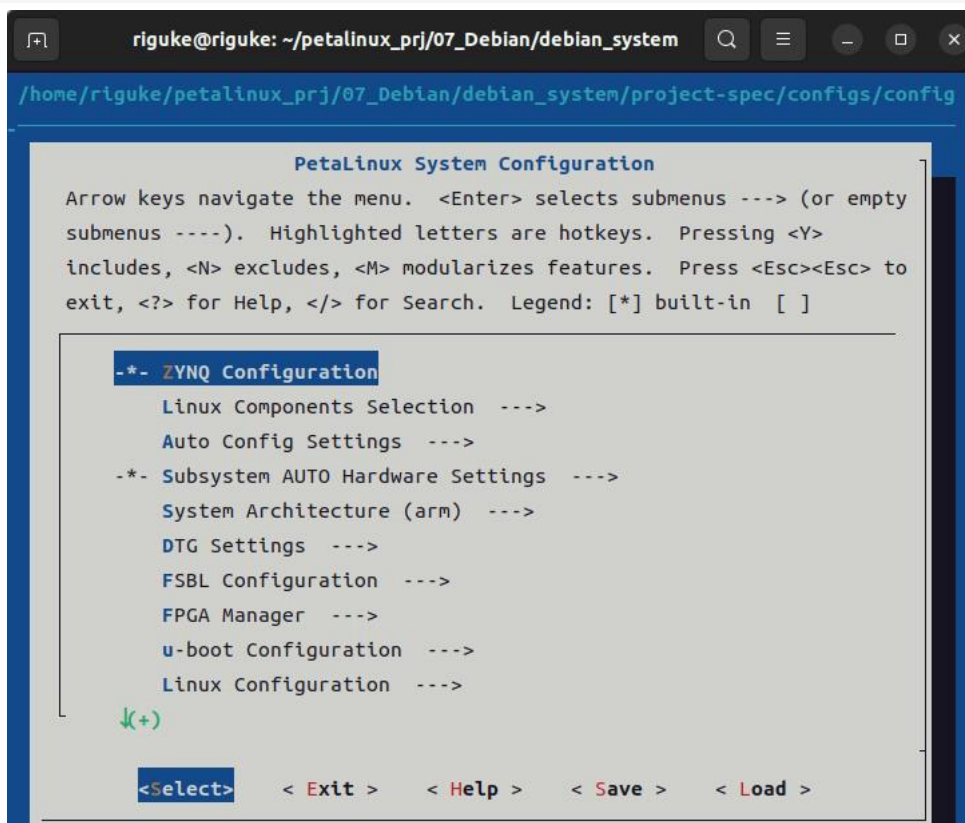


图 8-1 重新配置 petalinux

(2) 在 Image Packaging Configuration → Root filesystem type 选项中选择 SD card, 把根文件系统放在 SD 卡中。

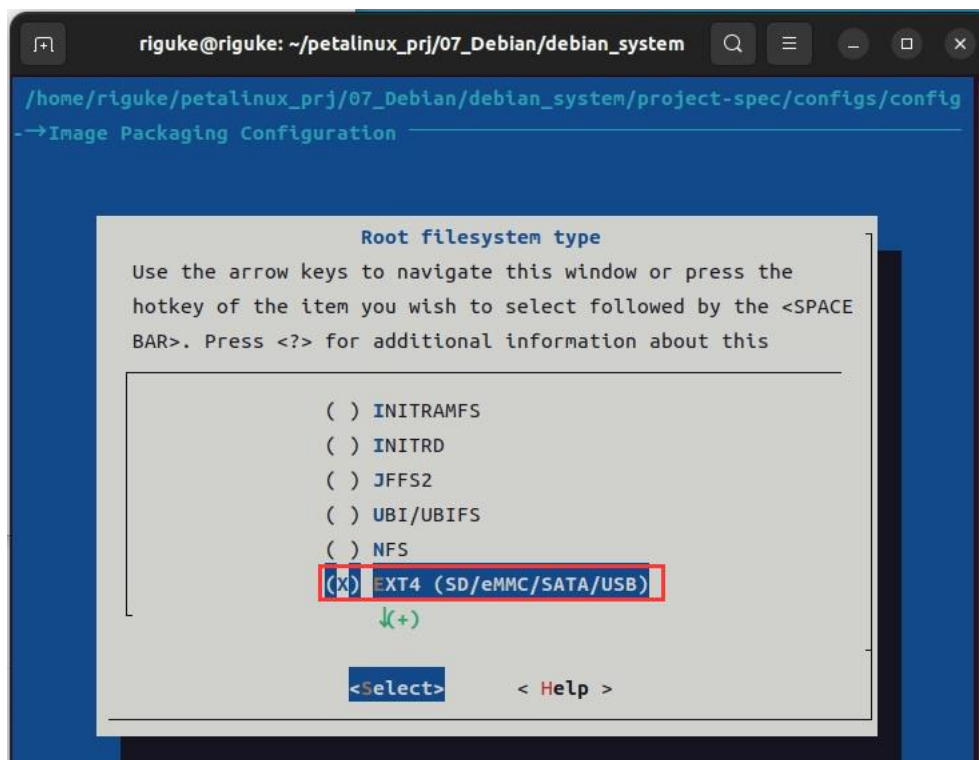


图 8-2 配置将根文件系统放在 SD 卡中

配置完毕后保存并退出。

(3) 配置 Linux 内核，在选项 Networking Support -> Wireless -> 选项中先选择 cfg80211 - wireless configuration API 再选择 Generic IEEE 802.11 Networking Stack (mac80211)

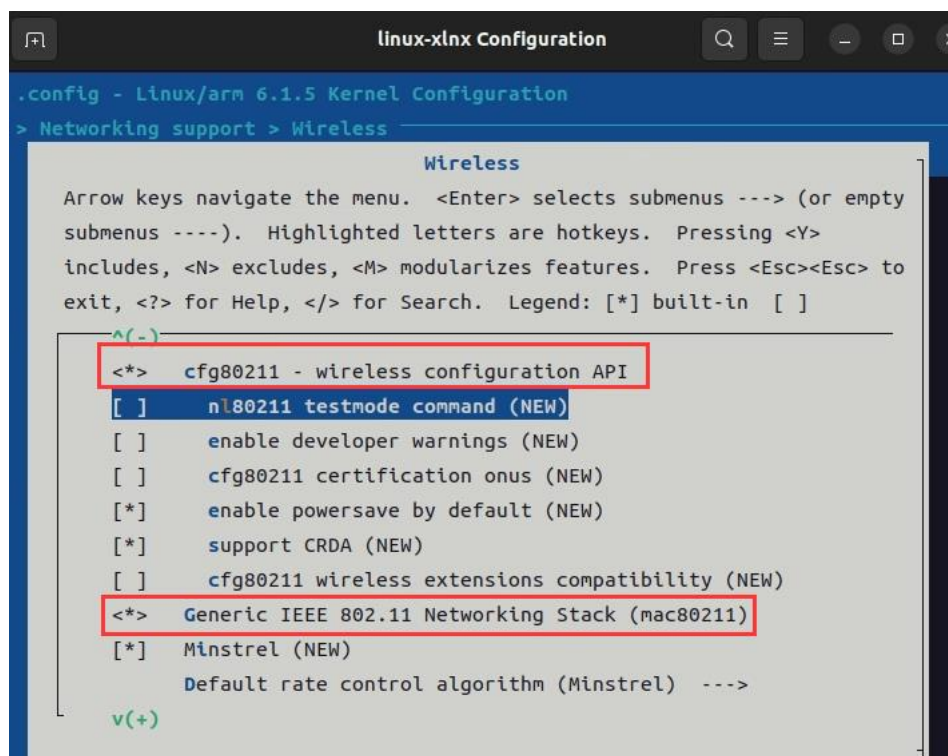


图 8-3 配置网络文件

(4) 在选项 Device Drivers -> Network device support -> Wireless LAN -> Realtek rtlwifi



family of devices 选择 Realtek RTL8192CU/RTL8188CU USB Wireless Network Adapter

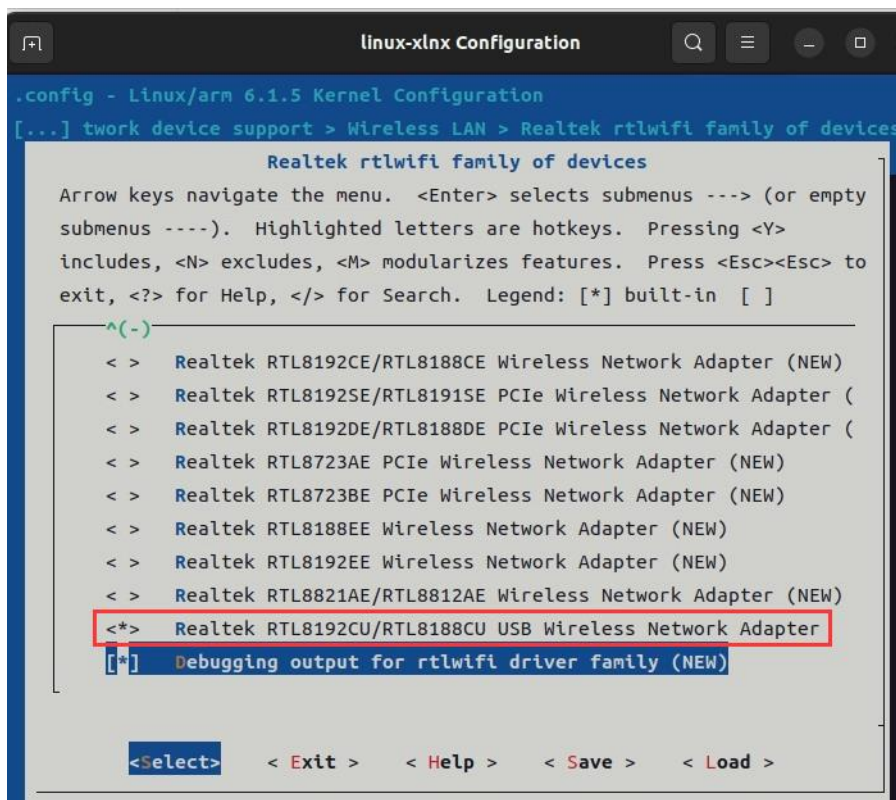


图 8-3 配置 USB 网络驱动

(5) 配置 USB 摄像头驱动, 在选项 Device Drivers → Multimedia support → Media drivers → Media USB Adapters → 中使能 USB Video Class (UVC)

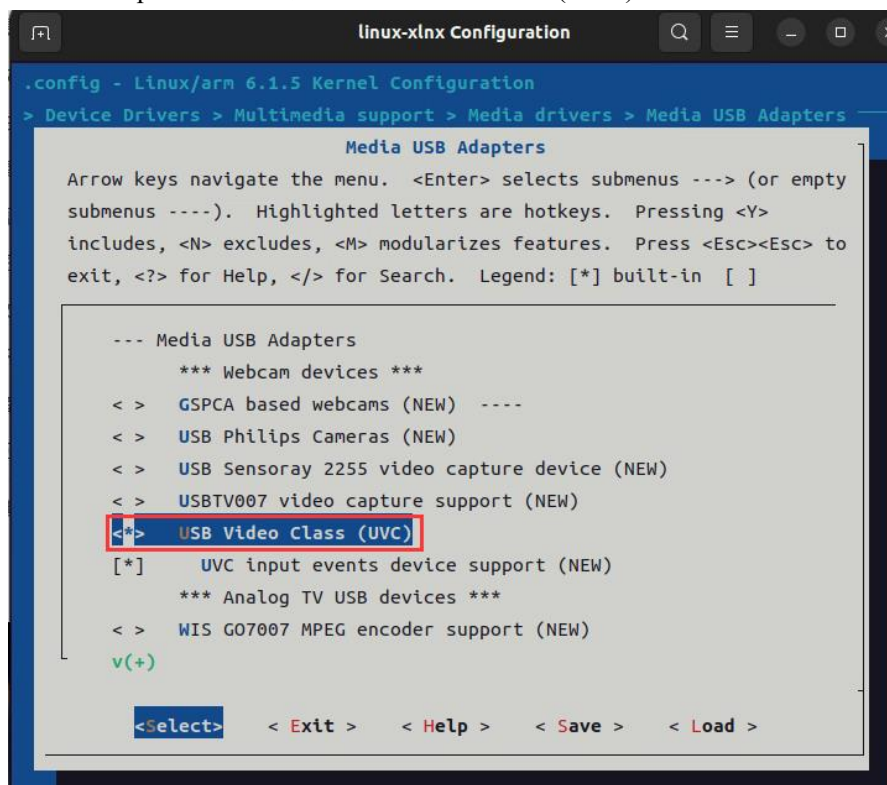


图 8-4 配置 USB 摄像头



配置完毕后保存并退出。随后修改完设备树后，则编译 uboot、内核、根文件系统、设备树等。

(6) 编译完毕后，进入工程文件下的 image/linux 文件夹下，并运行下面命令生成 BOOT 文件，注意空格和短线，这次由于用到 PL 端资源，因此需要注意打包命令，命令如下所示：

命令为：petalinux-package --boot --fsbl --fpga --u-boot --force

```

riguke@riguke: ~/petalinux_prj/07_Debian/debian_sy...
INFO: File in BOOT BIN: "/home/riguke/petalinux_prj/07_Debian/debian_syste
m/images/linux/system.dtb"
INFO: Generating zynq binary package BOOT.BIN...

***** Bootgen v2023.1
**** Build date : Apr  7 2023-10:18:04
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
** Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserve
d.

[WARNING]: Partition zynq_fsbl.elf.0 range is overlapped with partition de
sign_1_wrapper.bit.0 memory range
[WARNING]: Partition design_1_wrapper.bit.0 range is overlapped with parti
tion system.dtb.0 memory range

[INFO]   : Bootimage generated successfully

INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
riguke@riguke:~/petalinux_prj/07_Debian/debian_system/images/linux$

```

图 8-5 打包工程输出

## 2. 制作 SD 卡文件系统

制作 SD 卡文件系统会导致 SD 卡里内容丢失，请先做好备份。

(1) 开发板的 SD 卡插入读卡器，然后插入电脑的 USB 口。连接上 Linux 后，在 Ubuntu 的搜索路径中，输入 disk，会出现 Disks 的图标，如下图所示：



图 8-6 搜索磁盘

(2) 鼠标点击 Disks 图标，出现“Disks”的对话框，本实验中 SD 卡已经分为 1 个分区，名为 FAT，这里要重新分区。

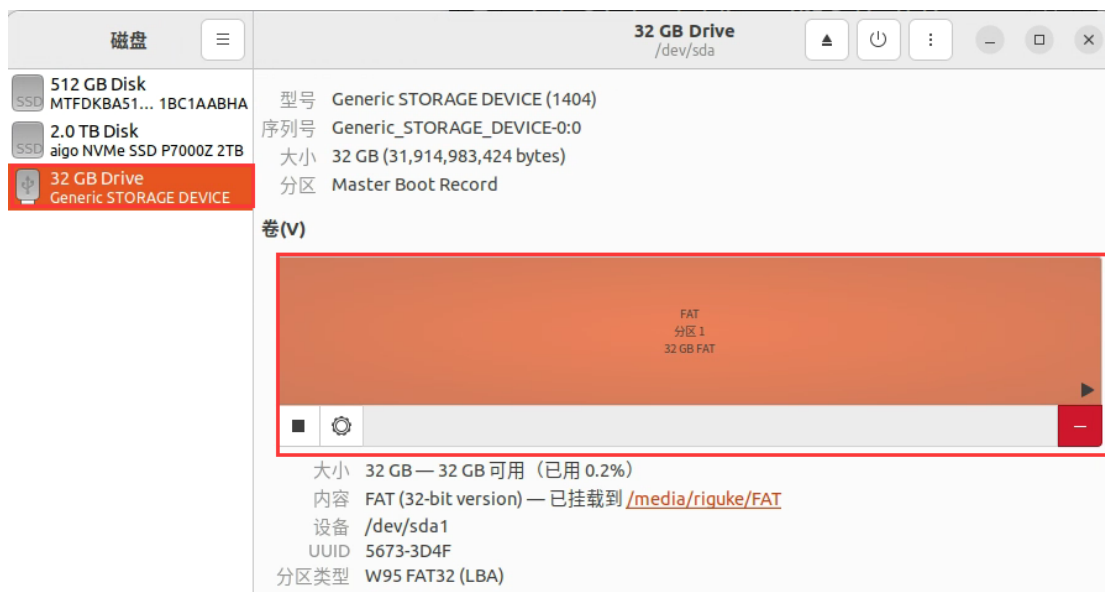


图 8-7 SD 卡分区

(3) 选择每个分区下面的删除分区图标，删除所有分区。



图 8-8 删除 SD 卡分区



图 8-9 SD 卡删除分区后

(4) 点击添加分区的图标，添加第一个分区，本实验填写 5GB，格式为 FAT，用于存放 ZYNQ 的启动文件 BOOT.bin, image.ub 和 boot.scr，名称为 FAT

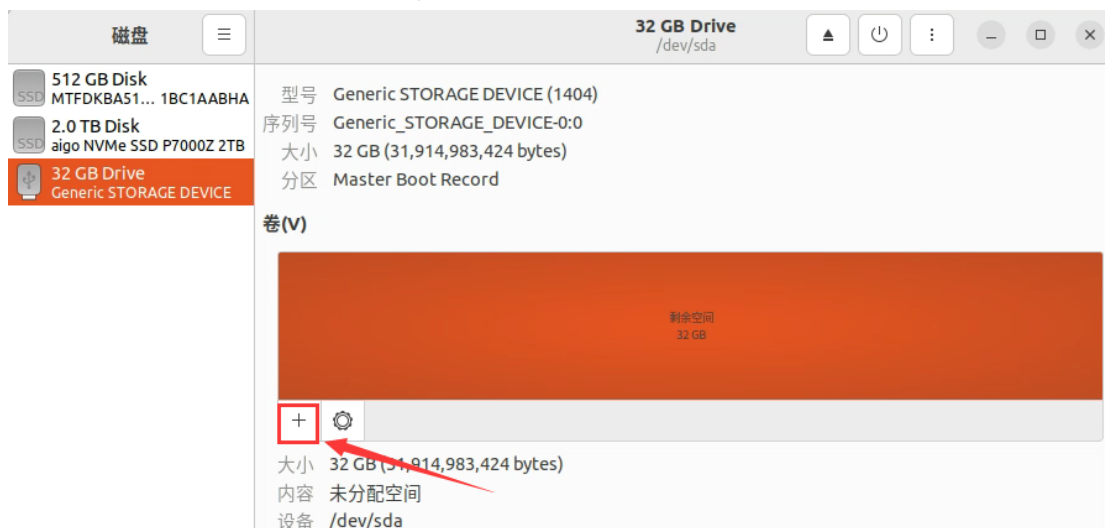


图 8-10 添加 SD 卡分区



图 8-11 FAT 区域分区 5GB



图 8-12 格式化 FAT 分区

(5) 创建第二个分区，用于存放根文件系统，格式为 EXT4，名称为 EXT



图 8-13 添加第二个分区



图 8-14 第二个分区大小



图 8-15 格式化 EXT 分区

(6) 创建分区完成后重新插入 SD 卡，系统会自动挂载分区，并弹出窗口，如下图所示：

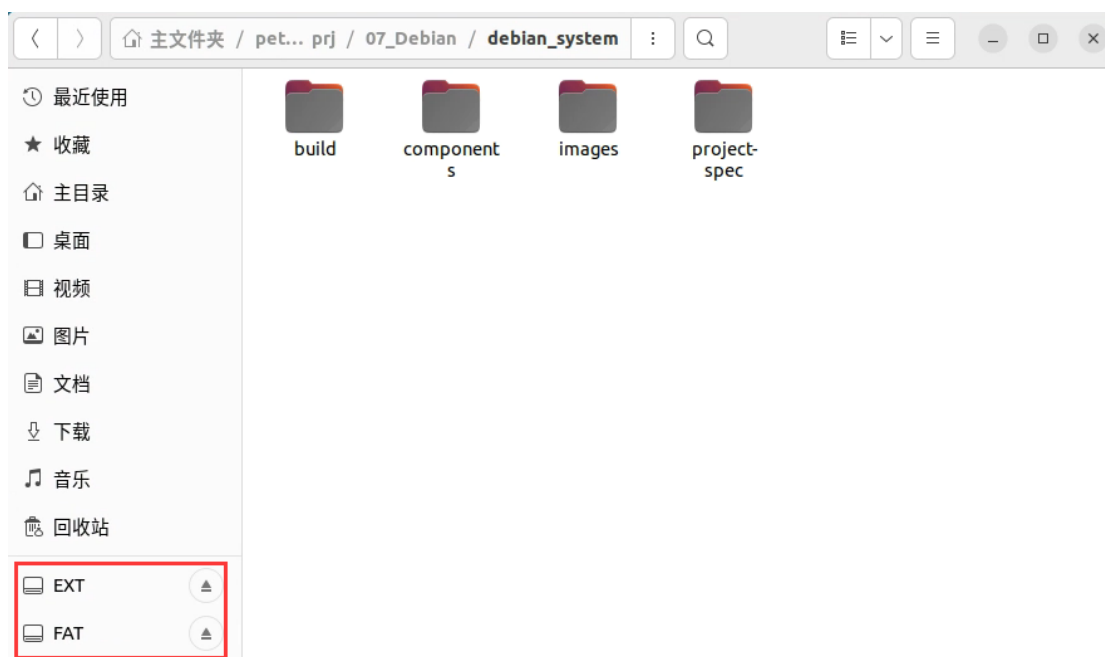


图 8-16 自动挂载分区

### 3. 同步根文件系统到 SD 卡 EXT 分区

#### 使用 Petalinux 工程编译后生成的根文件系统

(1) 输入下面的解压缩命令，解压根文件系统到 SD 卡 EXT4 分区。请注意解压命令格式，必须和教程一致，“/media/riguke/EXT”为当前 Linux 环境下 SD 卡 EXT 分区的路径，如果没有完全按照教程执行可能会有不同，请根据自己实际情况修改，petalinux 工程路径“<petalinuxpath>”是需要你自己补全你 images 前面的全部路径。

命令为: `sudo tar -zxvf /home/riguke/petalinux_prj/07_Debian/debian_system/images/linux/rootfs.tar.gz -C /media/riguke/EXT/`

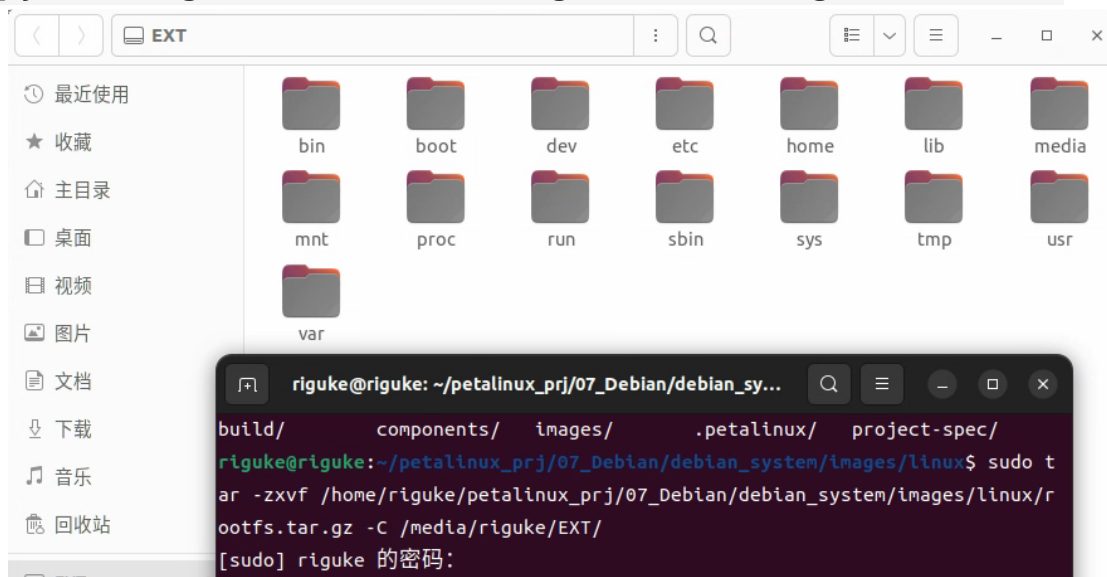


图 8-17 将 petalinux 工程中根文件系统同步到 SD 卡的 EXT 分区

- (2) 在命令窗口输入"sudo sync"同步缓存数据，同步可能需要比较长的时间。
- (3) 把 BOOT.bin、boot.scr 和 image.ub 复制到 SD 的 FAT 分区中。

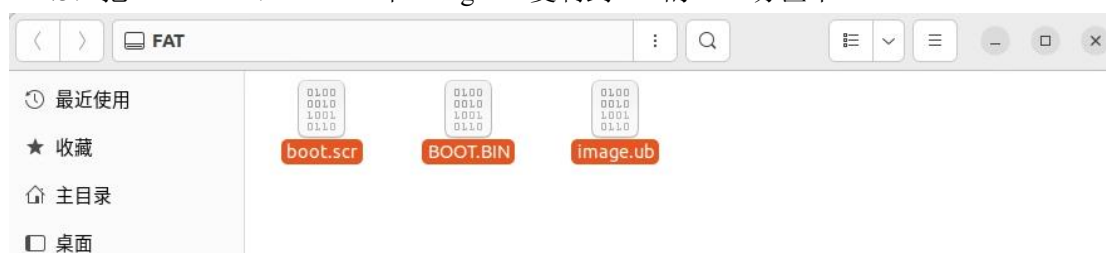


图 8-18 启动镜像拷贝到 FAT 分区



(4) 设置开发板为 SD 卡模式启动，连接好终端工具后，启动开发板。正常启动如下图所示：

```
PetaLinux 2023.1+release-S05010539 debian_system ttyPS0
debian_system login: root
Password:

Login incorrect
debian_system login: petalinux
Password:
debian_system:~$ pwd
/home/petalinux
debian_system:~$ ls
debian_system:~$ cd /
debian_system:/$ ls
bin          etc          lost+found   proc         sys          var
boot         home        media        run          tmp
dev          lib         mnt         sbin         usr
debian_system:/$
```

图 8-19 正常启动

由于是一个全新的系统，当前用户目录下是没有文件的，所以 ls 没有结果，不要为此以为出了什么问题，可以进入根目录看看，后面的系统也是一样的。

## 使用 Debian 根文件系统

(1) 首先需要先清除掉 SD 卡中原有的文件，FAT 分区直接图形化界面删除较快，EXT 分区需要使用命令（注意确保当前路径在 EXT 分区下!!! 建议直接在 EXT 分区右键打开终端）：

命令为：`sudo rm -rf *`

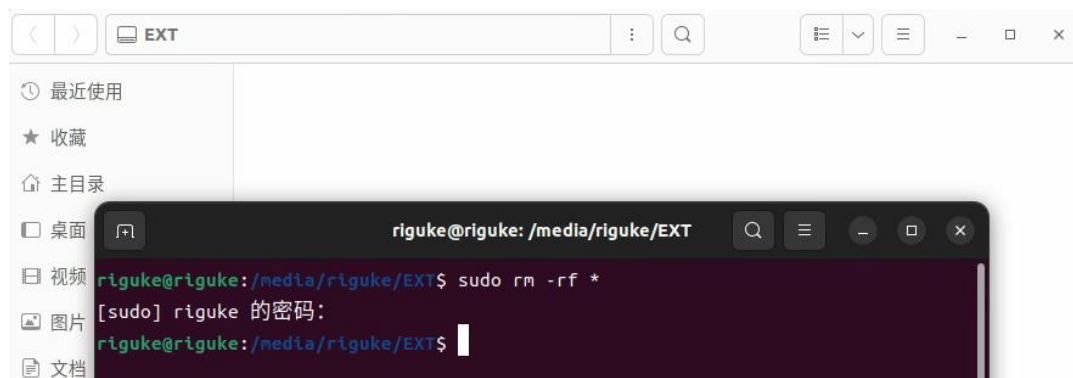


图 8-20 清除 EXT 分区的内容

(2) 复制资料中的 debian 根文件系统的压缩包到 Linux 主机中。

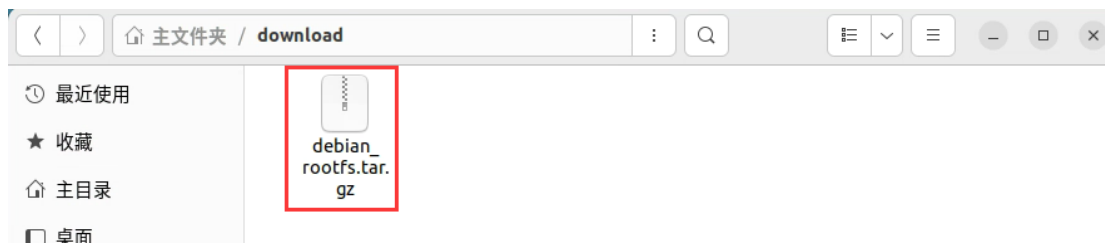


图 8-21 下载 debian 根文件系统至 Linux 主机中

(3) 输入解压缩命令“`sudo tar -xzvf debain_rootfs.tar.gz -C /media/riguke/EXT/`”，解压缩文件系统，-C 选项后面的路径是 EXT 分区所在的路径。解压缩可能需要几分钟的时间。请注意解压方式，红色部分为自己 SD 卡 EXT 的位置。解压完毕如下图所示：

命令为：`sudo tar -xzvf debain_rootfs.tar.gz -C /media/riguke/EXT /`

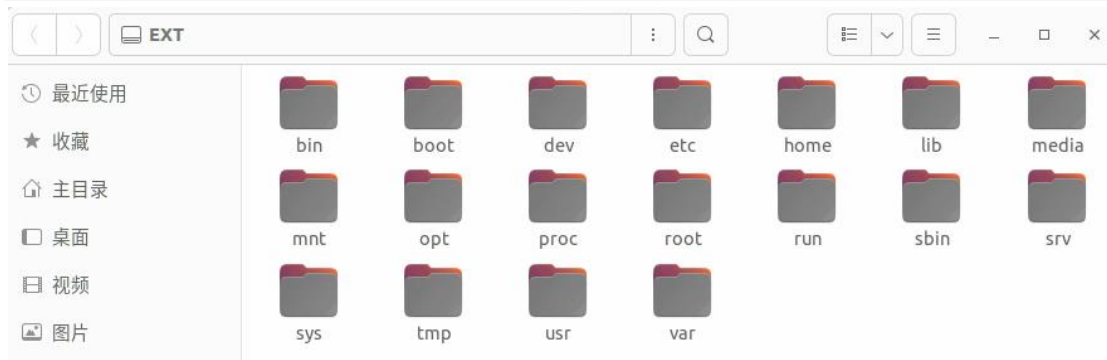


图 8-22 解压该文件系统至 SD 卡 EXT 分区中

(4) 输入 `sync` 命令，等待系统把解压后的文件同步到 SD 卡中，需要等待一段时间，当命令行中重新出现命令提示符时，表示同步的过程结束。

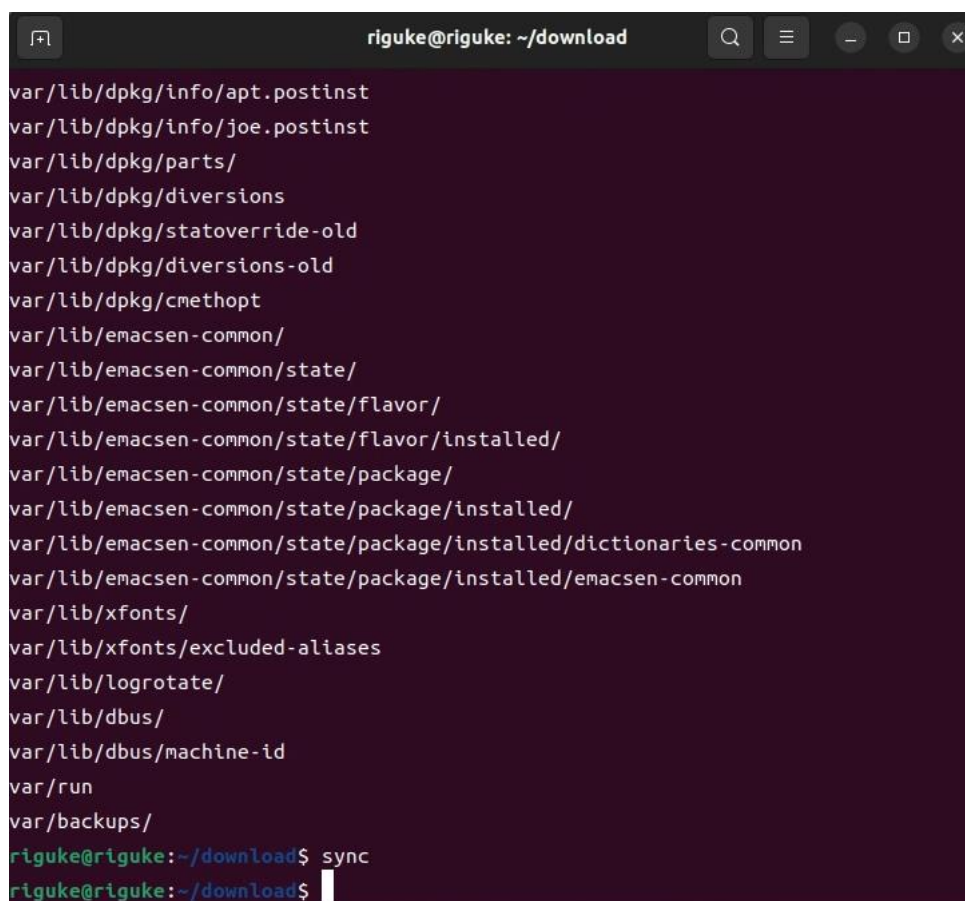


图 8-23 同步根文件

(5) 把 BOOT.bin, iimage.ub, boot.scr 复制到 SD 的 FAT32 分区（第一分区）中，设置开发板 SD 卡模式启动，插上 HDMI 显示器，启动开发板。该步骤在上一节中已经实现，因此可以不用拷贝了。

(6) SD 卡制作完成后，把制作好后的 SD 卡插入到开发板的 SD 卡插槽内。连接 USB 串

口线，连接 HDMI 显示器，开发板上电后在 HDMI 显示器上会显示 Debian 的操作系统的界面。另外在串口工具里我们可以看到操作系统启动的过程，运行 u-boot 之后开始运行 Linux，账号:root，密码: root。

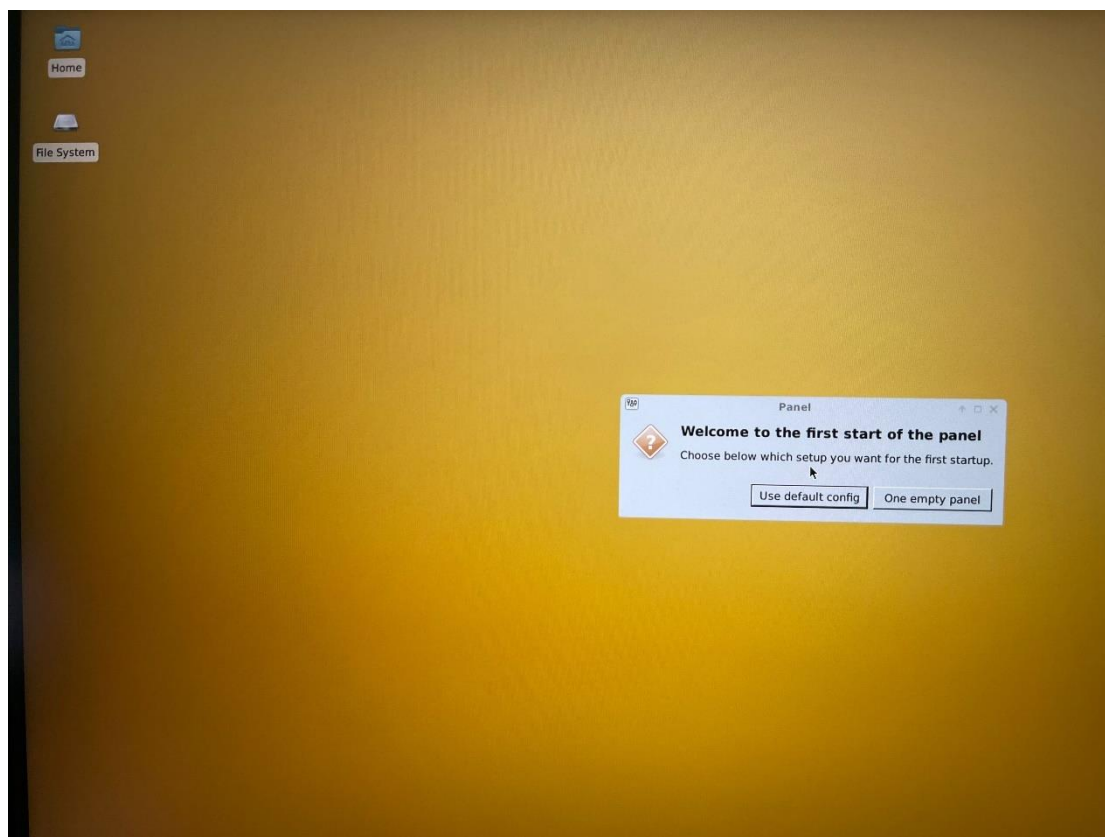


图 8-23 debain 界面

```
Debian GNU/Linux 8 zynq ttyPS0
zynq login: root
Password:
Last login: Thu Jan  1 08:02:27 CST 1970 on ttyPS0
Linux zynq 6.1.5-xilinx-v2023.1 #1 SMP PREEMPT Mon Dec  5 22:16:50 UTC 2022 armv7l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@zynq:~# ls
Desktop
root@zynq:~# cd \
> ls
-bash: cd: ls: No such file or directory
root@zynq:~# cd /
root@zynq:~# ls
bin  dev  home  media  opt  root  sbin  srv  tmp  var
boot  etc  lib  mnt  proc  run  sd  sys  usr
root@zynq:~#
```

图 8-24 终端界面输出

## 九. 制作 QSPI Flash 启动的 Linux

前面教程讲解的 Linux 都是在 SD 里启动，本实验讲解如何使用 Petalinux 制作一个从 QSPI Flash 启动的 Linux。

前面的 HDMI 教程中创建了一个 BSP 包，这里我们可以直接使用该工程，后面如果需要创建新的工程，工程配置又和之前工程差别不大的情况下，就可以直接使用之前的工程。这样不需要重复编译，加快编译速度。

为了熟悉流程，这边还是重新开始配置 petalinux 工程

### 1. Petalinux 配置

(1) 新建工程完毕后，对 petalinux 工程进行配置，在 Subsystem AUTO Hardware Settings → Flash Settings → Primary flash 选项中选择要使用的 flash 设备



图 9-1 选择要使用的 Flash 设备

(2) Subsystem AUTO Hardware Settings → Flash Settings 中可以修改 QSPI flash 的分区，需要按照下图调整一下大小，注意：u-boot 的默认加载地址为 0x9C0000。

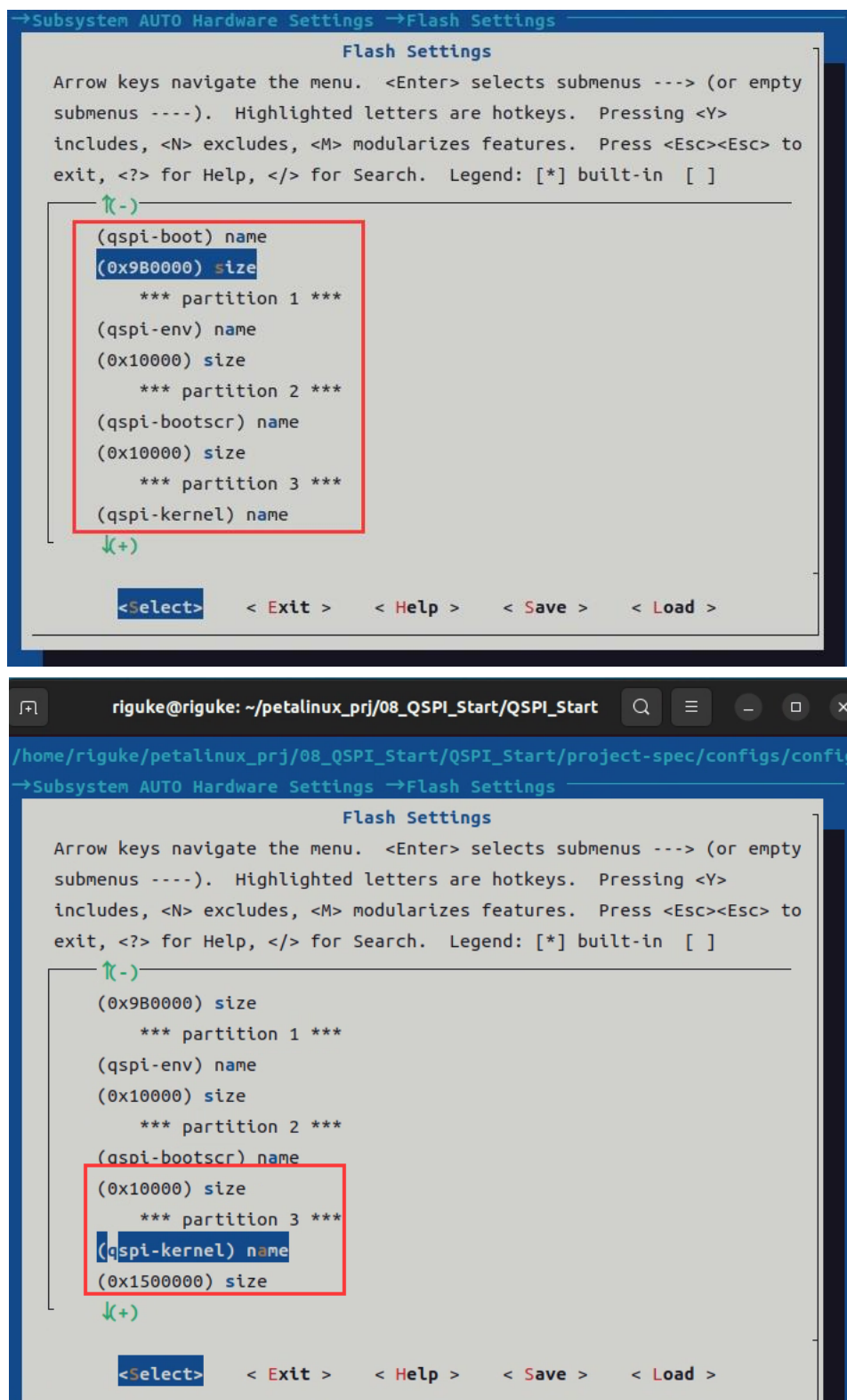


图 9-2 配置每个文件在 QSPI 中占用的大小

注：如果缺少 qspi-kernel，可以点击 name 输入名字，并配置大小。

- boot 分区中存放描述 FPGA 的 .bit 文件、u-boot、fsbl、设备树.dtb 等文件。boot 分区的大小需要大于这些文件大小总和，需要根据实际情况修改。
- bootenv 分区存放 u-boot 的环境变量，根据实际情况修改。
- kernel 分区存放 image.ub，分区大于 image.ub 即可，需要根据实际情况修改。



- bootscr 分区存放 boot.scr，分区大于 boot.scr 即可，需要根据实际情况修改。
- 另外需要注意，所有打包文件的大小总和，不能超过 flash 的大小。

(3) 在 u-boot Configuration → u-boot script configuration → QSPI/OSPI image offsets 中更改下图中的 QSPI/OSPI fit image offset 选项，将偏移量改为 0x9D0000，放在 boot.scr 文件的后面，需要与上一步中的地址对应。

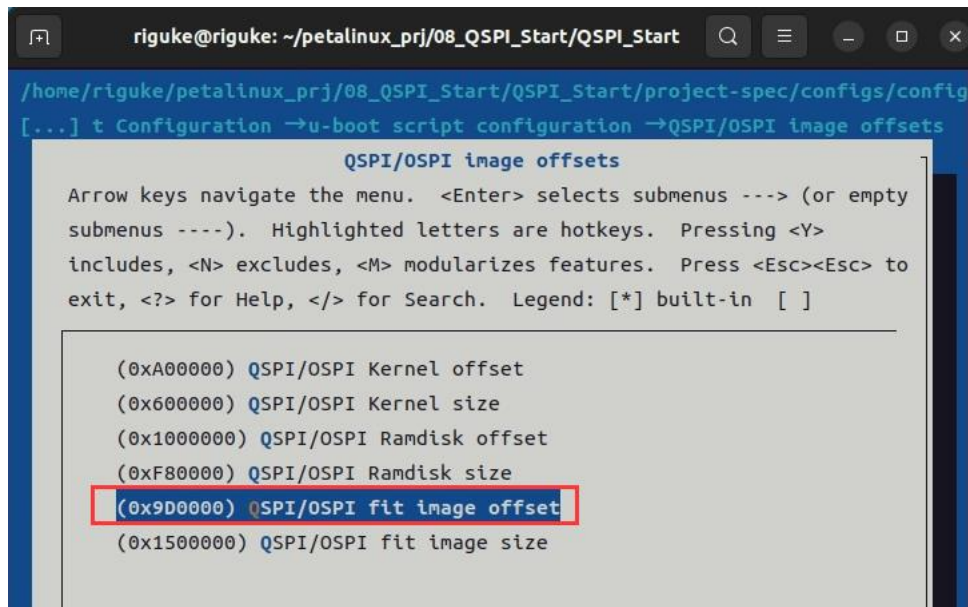


图 9-3 设置 image 的偏移地址

(4) 在 Image Packaging Configuration → Root filesystem type 选择 INITRAMFS，使用 RAM 类型的根文件系统，这样就可以方便打包烧写到 QSPI Flash。

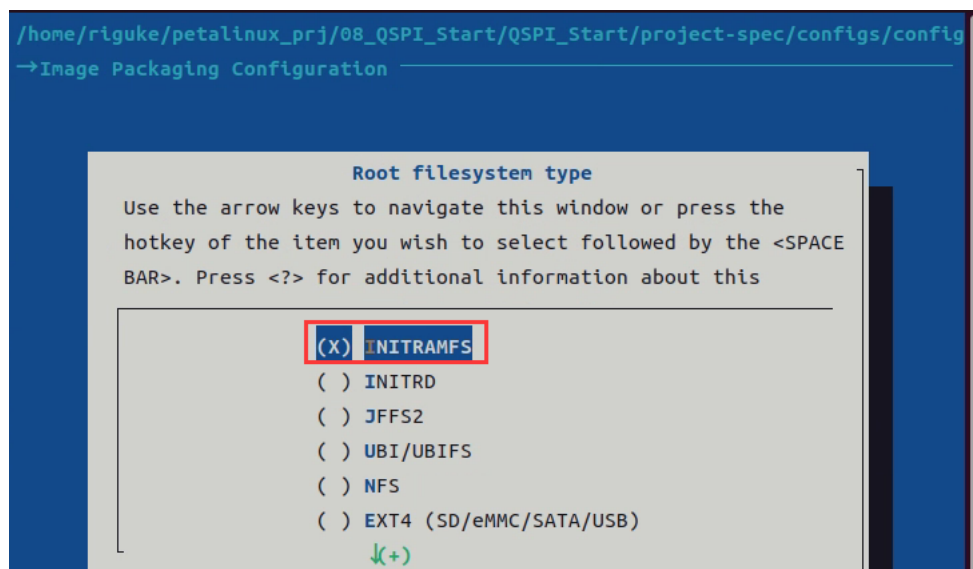


图 9-4 设置 RAM 类型的根文件系统

(5) 在 Firmware Version Configuration → 中可以修改 Host name 等信息，这里按需修改，教程中保持原状。



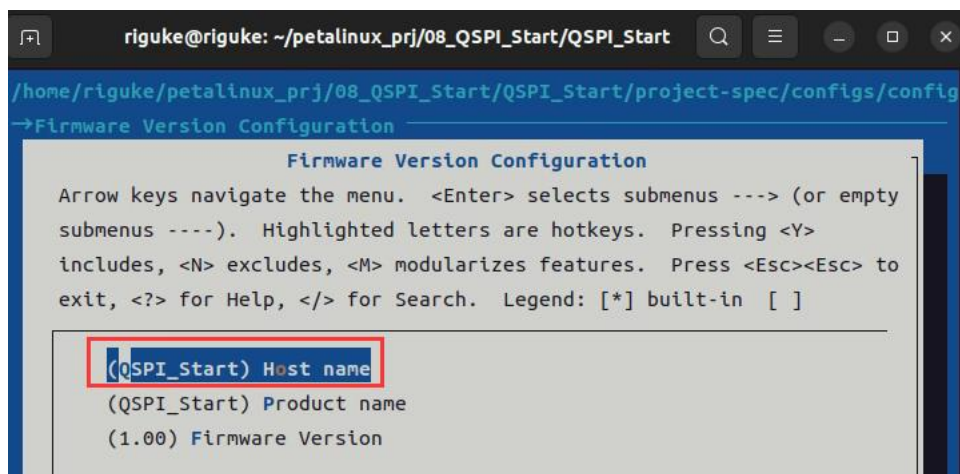


图 9-5 按需修改 Host name

(6) 编译成功后,使用下面命令合成 BOOT.bin,使用--kernel 选项把内核编译进 BOOT.bin。  
命令为: petalinux-package --boot --u-boot --kernel --fsbl --fpga --force

## 2. 下载至 Flash 验证 Linux 启动

(1) 编译完成后,将下图中 BOOT.BIN 与 Zynq\_fsbl.elf 从 Linux 环境中拷贝至 Windows 中,使用 Vitis 下载至 Flash 中。若 Linux 环境中有 Vitis 软件,也可以直接下载。

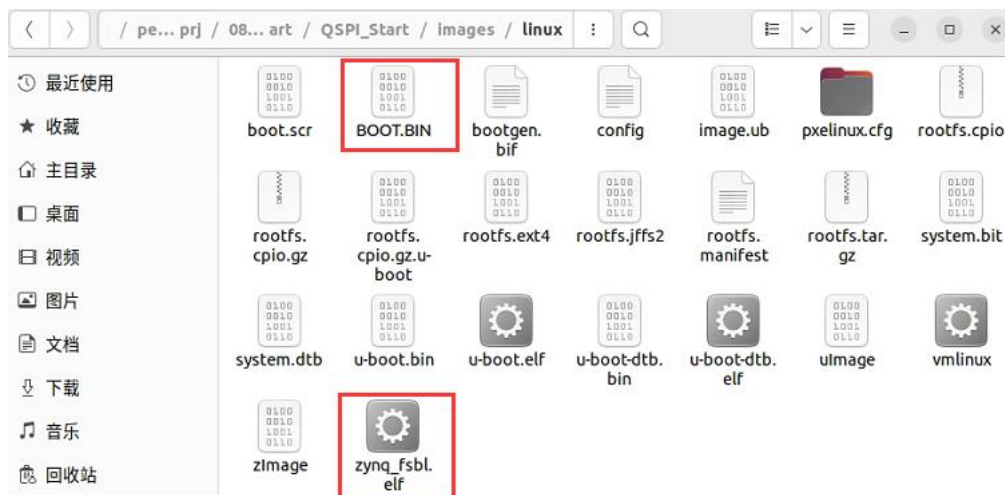


图 9-6 将红框文件拷贝至 Windows 环境中,使用 Vitis 下载

(2) 启动 Vitis，并在 Vitis 中选择拷贝出来的两个文件，直接下载至 Flash 中，如下图所示：

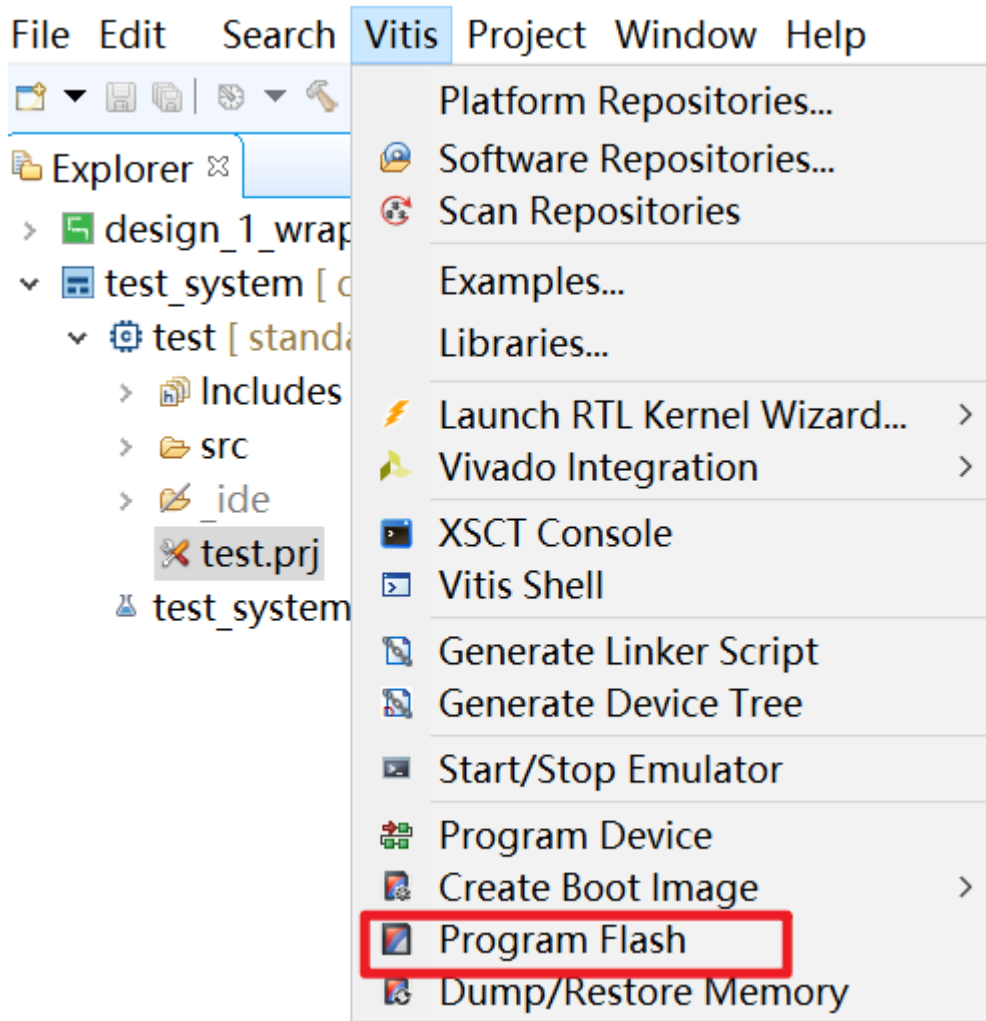


图 9-8 Vitis 中选择 Program Flash

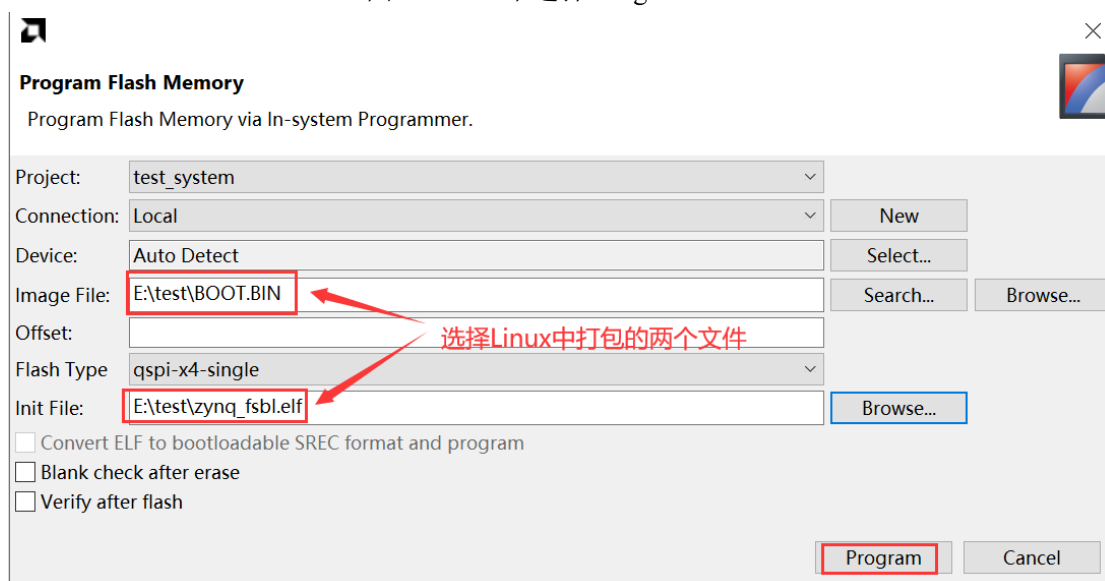


图 9-8 选择从 Linux 环境中拷贝的两个文件

选择完毕后，直接点击 Program，等待下载完毕即可。

(3) 下载完毕后，将 RK ZYNQ 7020 调整为 Flash 启动，并且在终端中观察启动信息，启动信息如下：

```
done.
Starting rpcbind daemon...done.
Starting statd: done
hwclock: can't open '/dev/misc/rtc': No such file or directory
Starting internet superserver: inetd.
NFS daemon support not enabled in kernel
Starting syslogd/klogd: done
Starting tcf-agent: OK

PetaLinux 2023.1+release-S05010539 QSPI_Start ttyPS0

QSPI_Start login: petalinux
You are required to change your password immediately (administrator enforced).
New password:
Retype new password:
Sorry, passwords do not match.

Authentication token manipulat
PetaLinux 2023.1+release-S05010539 QSPI_Start ttyPS0

QSPI_Start login: petalinux
You are required to change your password immediately (administrator enforced).
New password:
Retype new password:
QSPI_Start::~$ ls
QSPI_Start::~$ cd /
QSPI_Start::~$ ls
bin  boot  dev  etc  home  init  lib  media  mnt  proc  run  sbin  sys  tmp  usr  var
QSPI_Start::~$
```

图 9-10 从 QSPI Flash 中启动 Linux

通过从上述实验，就成功将 Linux 系统从 Flash 中启动了。

注：也可以采用 Vitis 实验教程中采用 Bat 脚本的方式，将 BIN 与 elf 文件下载至 Flash 中。

## 十. Petalinux 板载 LCD 显示 Linux 命令行实验

SPI LCD 是连接在 RK 7020 的 PL 端，因此需要重新配置 Vivado 工程，Vivado 工程完成配置后，再配置 Petalinux 工程。随后实现在板载 LCD 显示屏上显示 Linux 命令行的实验。

### 1. 建立 vivado 工程

(1) 此次新建的 Vivado 工程是在一开始只有 PS 端的 Vivado 工程上添加 PL 端 SPI 端口，因此可以直接使用保存的 TCL 文件快速恢复 ZYNQ 配置，这样可以节省大量的配置时间。工程恢复如下图所示：

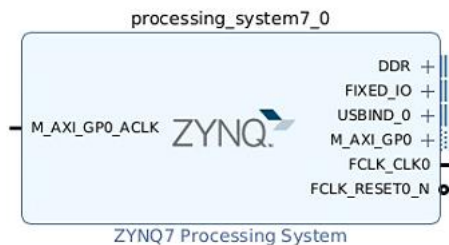


图 10-1 恢复 ZYNQ 的配置

(2) 由于原来的 ZYNQ 配置中已经配置了 UART, SD 卡等外设，因此这边不进行详细介绍，因为屏幕的接口是连接到 ZYNQ 的 PL 口上，所以这里调用 EMIO 的 SPI 功能（即使用 PS 的 SPI 功能，但是内部连接映射到 PL 的 GPIO 口上），MIO Configuration→SPI 0 勾上 并选择 EMIO，如下图所示：

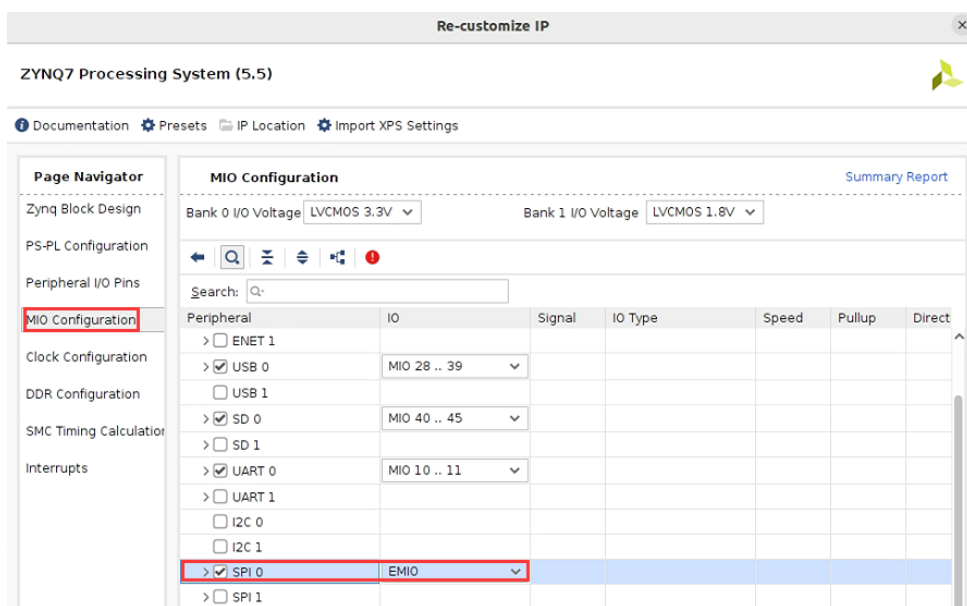


图 10-2 配置 SPI0 为 EMIO 模式

(3) 由于屏幕的驱动除了 SPI 外, 还需要 3 个额外的 GPIO 口来分别控制复位 RES, D/C 和 LED 信号, 所以这里增加 3 个 GPIO 口, 这里 GPIO 资源选择 EMIO GPIO 位宽 Width 选择 3 (因为是 3 个 IO 口), 如下图所示:

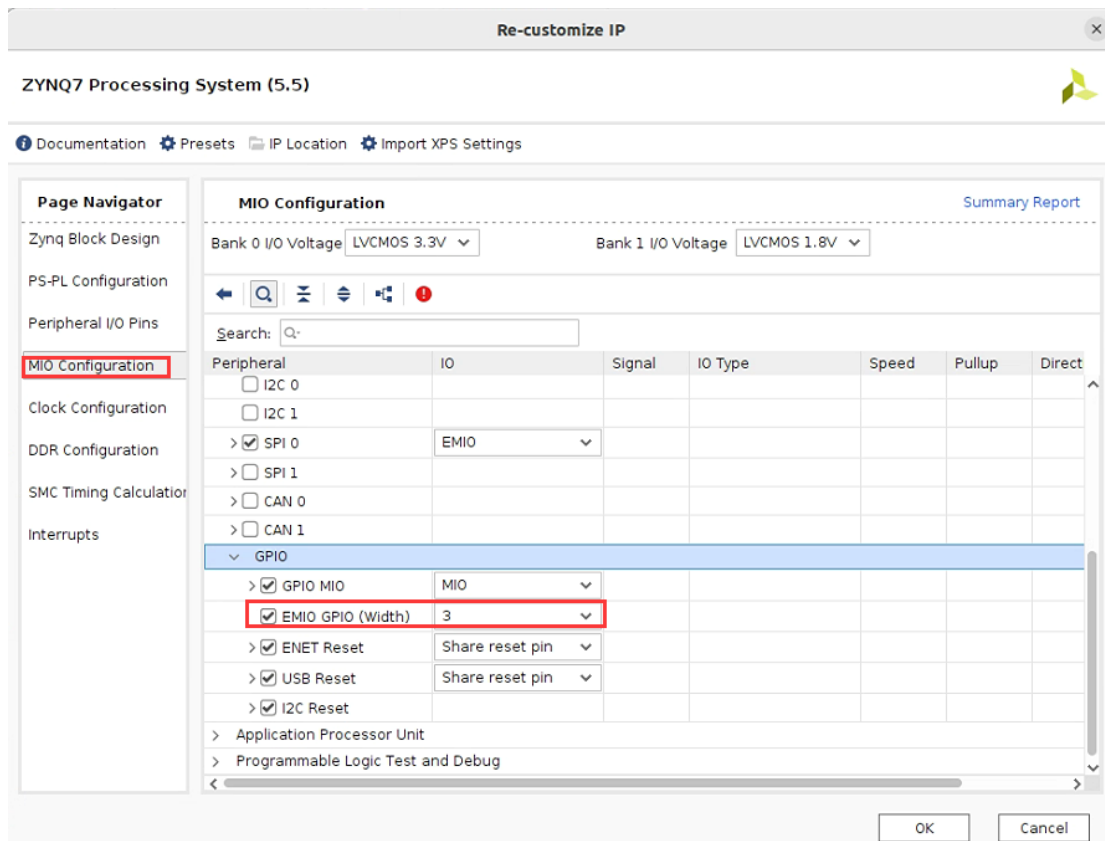


图 10-3 配置 EMIO 位宽为 3

(4) 因为工程暂时用不到 AXI 功能, 所以可以先禁用 AXI 功能。

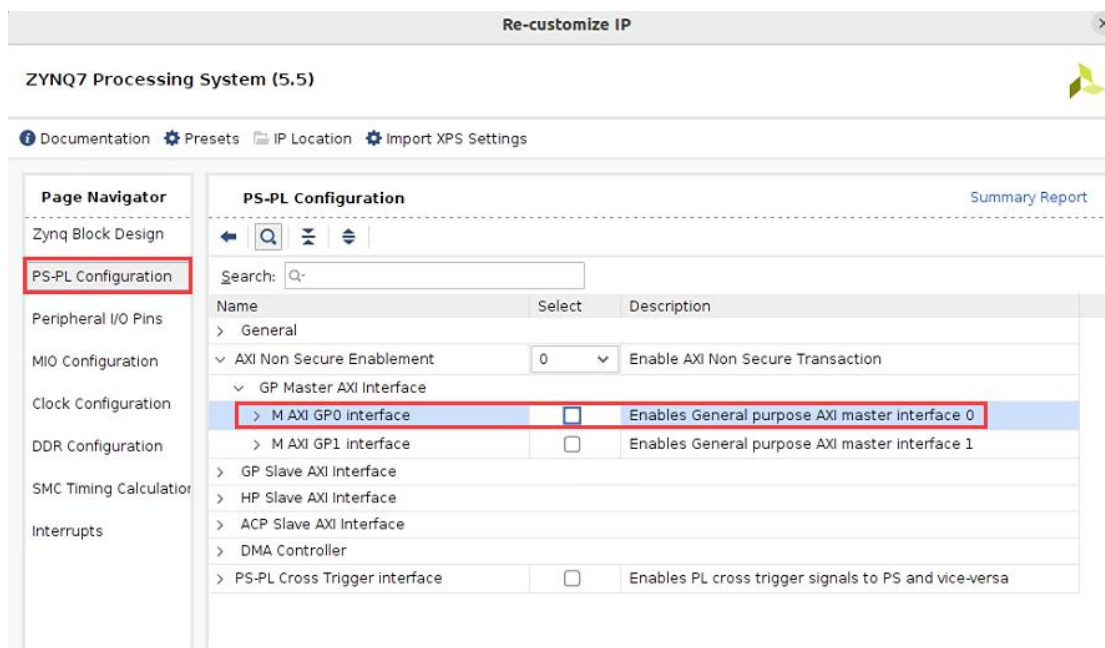


图 10-4 禁用 AXI 功能

(5) 完成上述操作后, 点击“Run Block Automation”如下图所示。

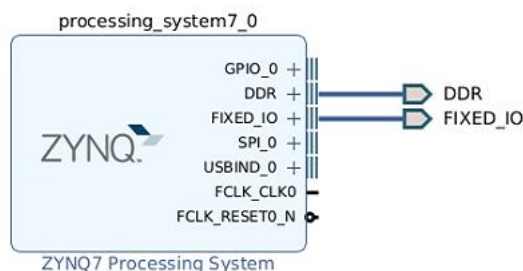


图 10-5 Run Block Automation

(6) 在上图中分别点击 IO 口进行以下操作, 右键 GPIO\_0, SPI\_0, 选择 Make External, 引出端口。

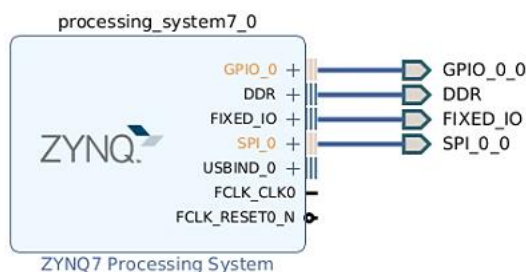


图 10-6 引出端口

(7) 最后则为创建硬件描述, 这时候可以看到软件的 TOP 层的 Verilog 文件, 打开后观察管脚信号, 如下图所示:

```
inout [3:0]DDR_dm;
inout [31:0]DDR_dq;
inout [3:0]DDR_dqs_n;
inout [3:0]DDR_dqs_p;
inout DDR_odt;
inout DDR_ras_n;
inout DDR_reset_n;
inout DDR_we_n;
inout FIXED_IO_ddr_vrn;
inout FIXED_IO_ddr_vrp;
inout [53:0]FIXED_IO_mio;
inout FIXED_IO_ps_clk;
inout FIXED_IO_ps_porb;
inout FIXED_IO_ps_srstb;
inout [1:0]GPIO_0_0_tri_io;
inout SPI_0_0_io0_io;
inout SPI_0_0_io1_io;
inout SPI_0_0_sck_io;
output SPI_0_0_ss1_o;
output SPI_0_0_ss2_o;
inout SPI_0_0_ss_io;
```

图 10-7 TOP 层 Verilog 文件

(8) 圈内的几个管脚是定义的 SPI 管脚和 GPIO 管脚, 其中 SPI\_0\_0\_io0\_io 为 SPI 的 MISO 信号, SPI\_0\_0\_io1\_io 为 MOSI 信号, SPI\_0\_0\_sck\_io 为 SPI 时钟信号, SPI\_0\_0\_ss\_io、SPI\_0\_0\_ss1\_io、SPI\_0\_0\_ss2\_io 分别为三组 SPI 的 CS 信号, GPIO\_0\_0\_tri\_io 为刚才定义的 3 个 GPIO(复位, D/C 和 LED)。

(9) 因为 SPI LCD 屏幕只用到了 一组 CS, 所以删除多余的 CS, 屏幕实际只用到了 SPI 的数据 SDA 和时钟 SCL 和 CS 三个信号, 所以 SPI 只需要引出 MOSI(SDA)和 SCK(SCL), 和 CS(SS)即可。删除如下框图中的信号的位置:



```

inout [3:0]DDR_dm;
inout [31:0]DDR_dq;
inout [3:0]DDR_dqs_n;
inout [3:0]DDR_dqs_p;
inout DDR_odt;
inout DDR_ras_n;
inout DDR_reset_n;
inout DDR_we_n;
inout FIXED_IO_ddr_vrn;
inout FIXED_IO_ddr_vrp;
inout [53:0]FIXED_IO_mio;
inout FIXED_IO_ps_clk;
inout FIXED_IO_ps_porcb;
inout FIXED_IO_ps_srstb;
inout [1:0]GPIO_0_0_tri_io;
inout SPI_0_0_io0_io;
inout SPI_0_0_io1_io;
inout SPI_0_0_sck_io;
output SPI_0_0_ss1_o;
output SPI_0_0_ss2_o;
inout SPI_0_0_ss_io;

```

图 10-8 删除 TOP 层中部分定义

(10) 同样也需要删除内部 IOBUF(否则外部管脚不调用会报错), 如下图所示:

```

IOBUF SPI_0_0_io0_iobuf
(.I(SPI_0_0_io0_o),
.IO(SPI_0_0_io0_io),
.O(SPI_0_0_io0_i),
.T(SPI_0_0_io0_t));
IOBUF SPI_0_0_io1_iobuf
(.I(SPI_0_0_io1_o),
.IO(SPI_0_0_io1_io),
.O(SPI_0_0_io1_i),
.T(SPI_0_0_io1_t));
IOBUF SPI_0_0_sck_iobuf
(.I(SPI_0_0_sck_o),
.IO(SPI_0_0_sck_io),
.O(SPI_0_0_sck_i),
.T(SPI_0_0_sck_t));
IOBUF SPI_0_0_ss_iobuf
(.I(SPI_0_0_ss_o),
.IO(SPI_0_0_ss_io),
.O(SPI_0_0_ss_i));

```

图 10-9 删除 IOBUF

(11) 修改 GPIO 管脚定义和 SPI 管脚定义, 如下图所示, 修改后保存(如弹出窗口需要, 则在窗口中输入约束文件名, 然后保存)。管脚约束如下图所示:

|                     |       |      |  |    |           |       |    |      |      |           |
|---------------------|-------|------|--|----|-----------|-------|----|------|------|-----------|
| GPIO_0_0_54576 (3)  | INOUT |      |  | 33 | LVCMS533* | 3.300 | 12 | NONE | NONE | FP_VTT_50 |
| GPIO_0_0_tri_io (3) | INOUT |      |  | 33 | LVCMS533* | 3.300 | 12 | NONE | NONE | FP_VTT_50 |
| GPIO_0_0_tri_io[2]  | INOUT | Y13  |  | 33 | LVCMS533* | 3.300 | 12 | NONE | NONE | FP_VTT_50 |
| GPIO_0_0_tri_io[1]  | INOUT | AA18 |  | 33 | LVCMS533* | 3.300 | 12 | NONE | NONE | FP_VTT_50 |
| GPIO_0_0_tri_io[0]  | INOUT | W13  |  | 33 | LVCMS533* | 3.300 | 12 | NONE | NONE | FP_VTT_50 |
| Scalar ports (0)    |       |      |  |    |           |       |    |      |      |           |
| SPI_0_0_54576 (3)   | INOUT |      |  | 33 | LVCMS533* | 3.300 | 12 | NONE | NONE | FP_VTT_50 |
| Scalar ports (3)    |       |      |  |    |           |       |    |      |      |           |
| SPI_0_0_io0_io      | INOUT | U19  |  | 33 | LVCMS533* | 3.300 | 12 | NONE | NONE | FP_VTT_50 |
| SPI_0_0_sck_io      | INOUT | V18  |  | 33 | LVCMS533* | 3.300 | 12 | NONE | NONE | FP_VTT_50 |
| SPI_0_0_ss_io       | INOUT | AA13 |  | 33 | LVCMS533* | 3.300 | 12 | NONE | NONE | FP_VTT_50 |

图 10-10 IO 约束

等待生成 BIT, 并导出 XSA 文件。

## 2. Petalinux 中配置

(1) 新建 Petalinux 工程, 这与前面的过程是一样的, 因此不多重复。只需要配置离线编

译包，其他不用配置。

(2) 配置 Linux 内核，命令为：

命令为：petalinux-config -c kernel

配置内核界面弹出的时间过长，等待界面跳出。

(3) 选择 Device Drivers → Graphics support → Frame buffer Devices，确保 Support for frame buffer devices 启用，如下图所示：

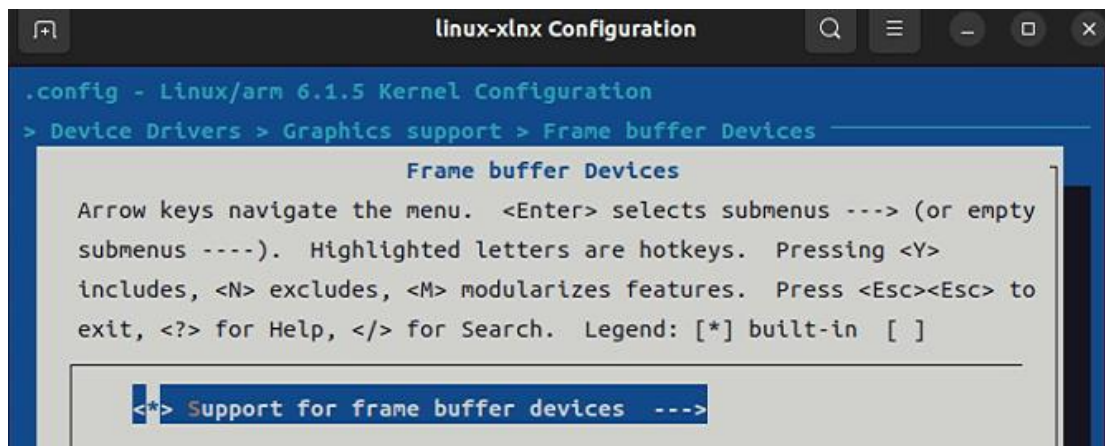


图 10-11 启用 Support for frame buffer devices

(4) 选择 Device Drivers，选中 Staging drivers 并且按下键盘上的 Y 键，对 Staging drivers 进行使能，并按下回车键进入

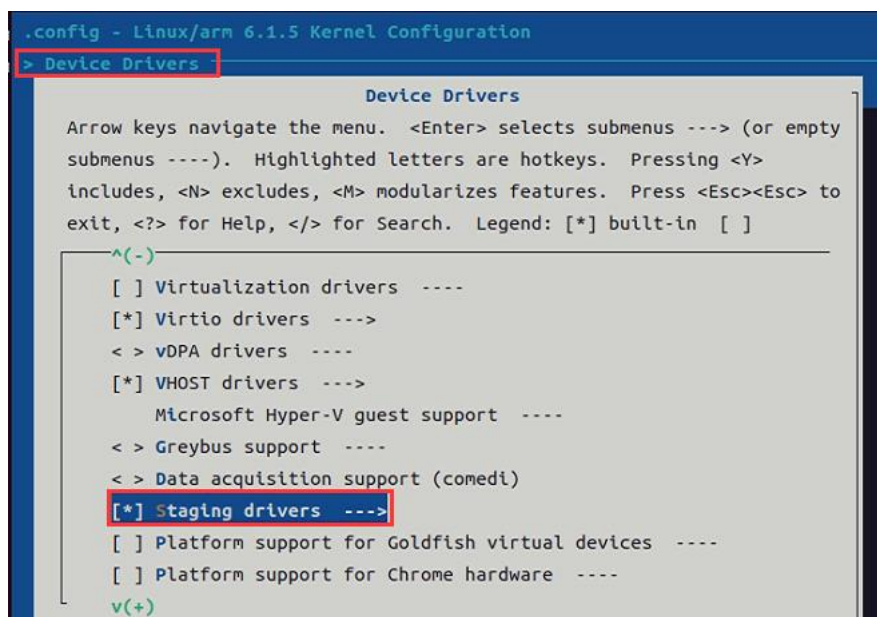


图 10-11 进入 Staging drivers

(5) 选中 Support for small TFT LCD display modules 然后按下键盘上的 Y 进行使能，并按回车键进入。

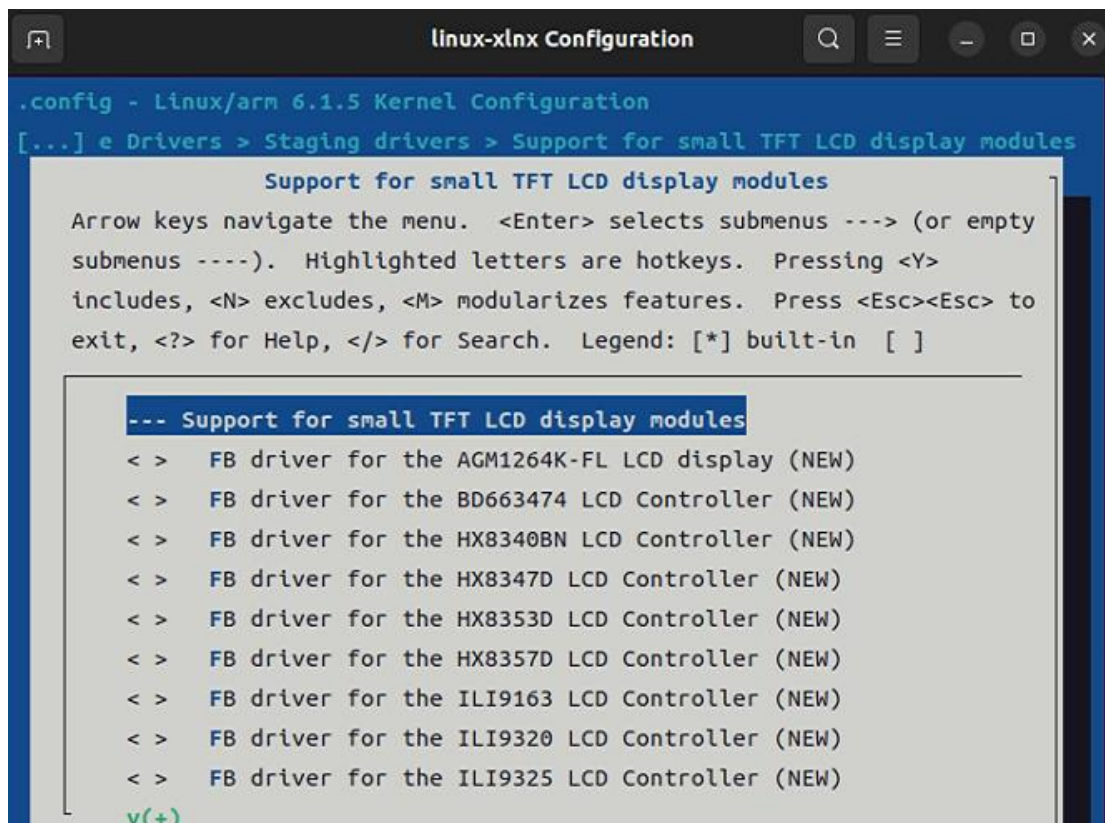


图 10-12 进入 Support for small TFT LCD display modules

(6) 选中 FB driver for the ST7789V LCD Controller, 并按下键盘上的 Y 键进行使能。

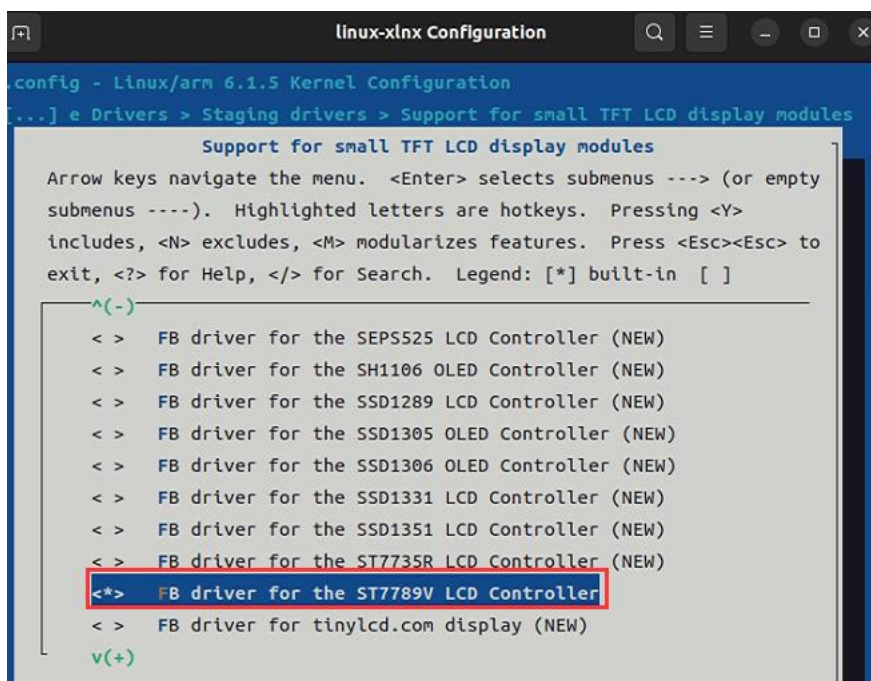


图 10-13 勾选 FB driver for the ST7789V LCD Controller

(7) 如果需要显示开机的 Linux 企鹅 logo 可以在 Device Drivers 中选择 Graphics support, 在 Bootup logo 的地方按下键盘上的 Y 键进行使能, 如下图所示:

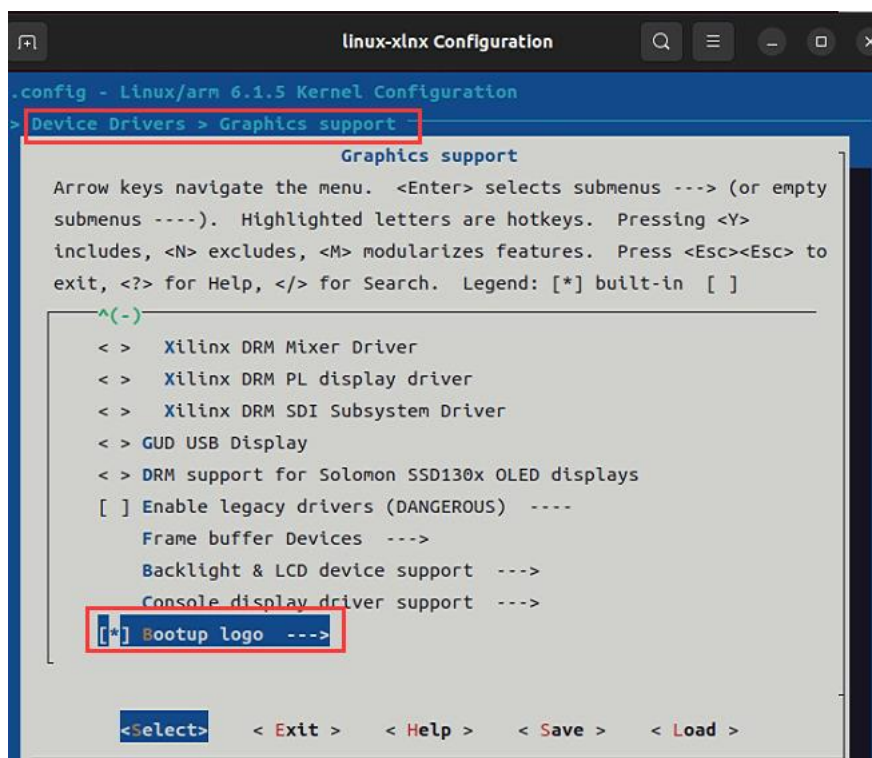


图 10-14 选择显示 Logo

(8) 内核启用 framebuffer 控制台 (fbcon) 模块, 如果不启用的话, 即使/dev/fb0 工作、驱动加载、penguin logo 支持都配置完毕, 也永远不会显示 penguin, 因为没有任何机制去把控制台输出“映射”到屏幕。

通过 petalinux 搜索功能可以发现 framebuffer 的位置如下:

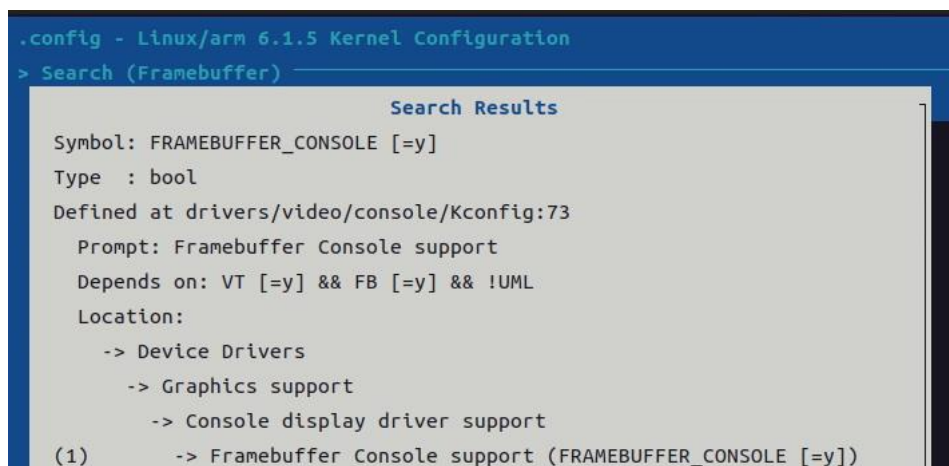


图 10-15 查找 framebuffer 的位置



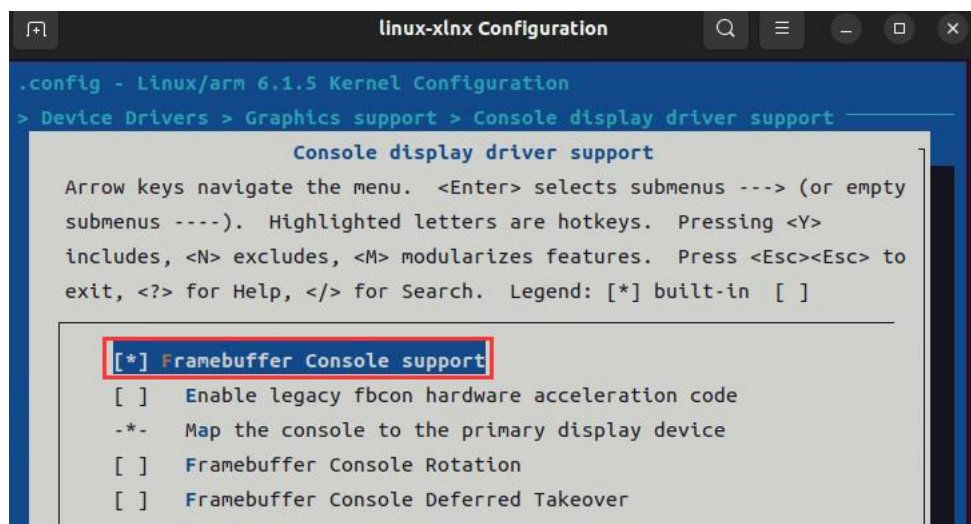


图 10-16 启动 Framebuffer Console support

(9) 最后退出并保存 Linux 内核

### 3. 编写设备树

为了让 LCD 功能正常使用，需要在设备树上增加 LCD 的相关内容，设备树所在位置在前面教程中已经说了很多次了，如果不熟悉可以去找找前面的教程，这边不重复进行查找，设备树添加内容如下：

```
/include/ "system-conf.dtsi"
/ {
};
#include <dt-bindings/gpio/gpio.h>
&spi0 {
    st7789v@0 {
        status = "okay";
        compatible = "sitronix,st7789v";
        reg = <0>;
        spi-max-frequency = <20000000>;
        rotate = <270>;
        spi-cpol;
        spi-cpha;
        rgb;
        fps = <30>;
        buswidth = <8>;
        reset-gpios = <&gpio0 55 GPIO_ACTIVE_LOW>;
        dc-gpios = <&gpio0 54 GPIO_ACTIVE_HIGH>;
        width = <172>;
        height = <320>;
        debug = <0>;
    };
};
```

```
};
};
```



图 10-16 编写设备树

设备树编写完毕则可以编译工程，等待编译完毕即可。

## 4. SPI LCD 显示命令行输出测试

当 Petalinux 工程编译完毕并完成打包后，将 Linux 系统的拷贝至 SD 卡中，随后插入 RK ZYNQ 7020 的 SD 卡槽中，随后设置 ZYNQ 为 SD 卡启动，等待设备完全启动后，串口终端会打印当前的日志。打印的日志如下图所示：

```
/etc/rc3.d/S99rcm-postcmds
INIT: Entering runlevel: 5
Configuring network interfaces... macb e000b000.ethernet eth0: PHY [e000b000.ethernet-ffffffff:00] driver [RTL8211F
Gigabit Ethernet] (irq=POLL)
macb e000b000.ethernet eth0: configuring for phy/rgmii-id link mode
udhcpc: started, v1.35.0
udhcpc: broadcasting discover
udhcpc: broadcasting discover
udhcpc: broadcasting discover
udhcpc: no lease, forking to background
done.
Starting OpenBSD Secure Shell server: sshd
generating ssh RSA host key...
generating ssh ECDSA host key...
generating ssh ED25519 host key...
done.
Starting rpcbind daemon...done.
starting statd: done
hwclock: can't open '/dev/misc/rtc': No such file or directory
Starting internet superserver: inetd.
NFS daemon support not enabled in kernel
Starting syslogd/klogd: done
Starting tcf-agent: OK

PetaLinux 2023.1+release-S05010539 SPI_LCD ttyPS0
SPI_LCD login: █
```

图 10-17 串口终端输出日志

经过大概 20 秒钟的启动之后，屏幕会显示 Linux 的启动 Logo 企鹅，以及终端命令行，如下图所示：



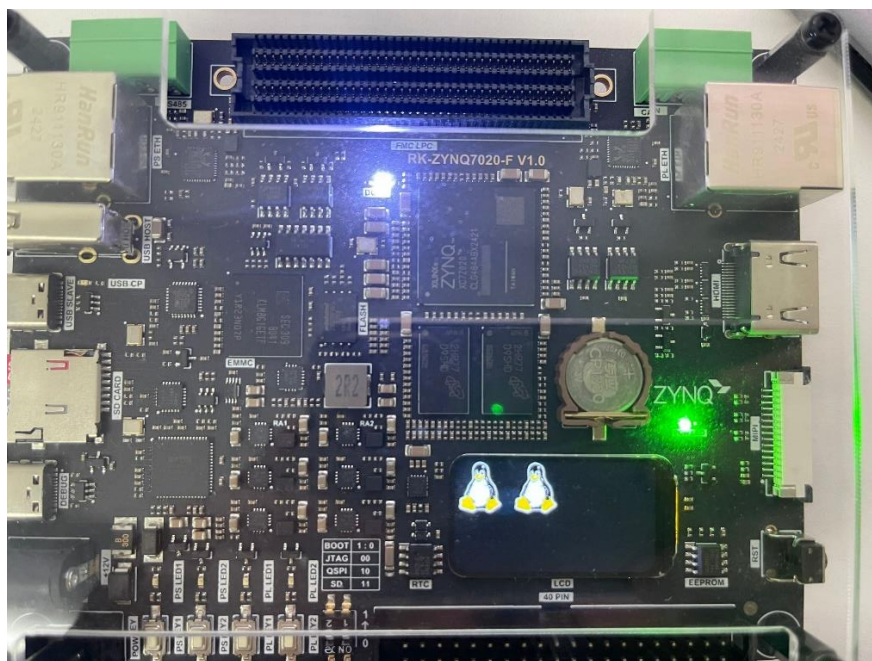


图 10-18 SPI 显示屏显示 Linux Logo

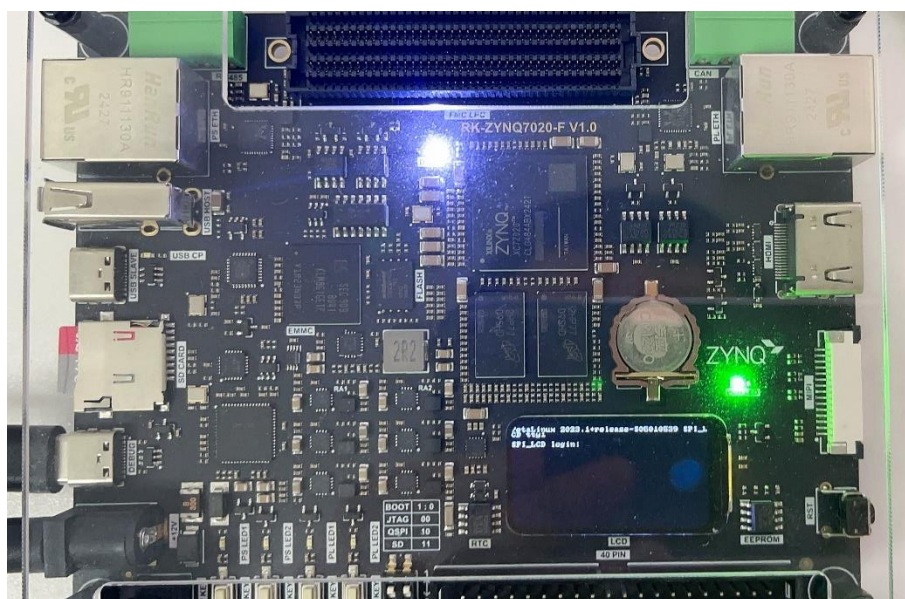


图 10-19 SPI 显示屏显示终端命令行

这里的命令行和串口的命令行显示的内容并不相通，所以通过电脑端串口在命令行中输入的指令这边并不会显示（可以通过 USB HOST 插入键盘来进行指令的输入和显示，但是本章节并没有做这部分内容，大家可以自行尝试）。

# 十一. Petalinux 中 USB HOST 实验

本章实验，介绍如何在 Petalinux 下调用 USB 资源实现 USB HOST 的功能,并最终实现 U 盘的识别和读取，

与上一章一样，首先需要在 Vivado 下完成工程建立，工程建立完毕后，最后在 Petalinux 中完成 USB HOST 的工程编译与实现。

## 1. 建立 vivado 工程

在前面章节中，配置了基础的 petalinux 工程，其中也配置了 USB 外设的管脚，在配置工程的时候，采用 TCL 文件的方式快速恢复基础工程，恢复工程后，快速检查配置的 PS 端的管脚，是否与原原理图上管脚一致，在 Block Design 的 ZYNQ 设置页面里使能 USB 0 资源，如下图所示：

| Peripheral                                   | IO           | Signal | IO Type | Speed | Pullup | Direction | Polarity |
|----------------------------------------------|--------------|--------|---------|-------|--------|-----------|----------|
| I/O Peripherals                              |              |        |         |       |        |           |          |
| > <input type="checkbox"/> ENET 0            |              |        |         |       |        |           |          |
| > <input type="checkbox"/> ENET 1            |              |        |         |       |        |           |          |
| > <input checked="" type="checkbox"/> USB 0  | MIO 28 .. 39 |        |         |       |        |           |          |
| <input type="checkbox"/> USB 1               |              |        |         |       |        |           |          |
| > <input checked="" type="checkbox"/> SD 0   | MIO 40 .. 45 |        |         |       |        |           |          |
| > <input type="checkbox"/> SD 1              |              |        |         |       |        |           |          |
| > <input checked="" type="checkbox"/> UART 0 | MIO 10 .. 11 |        |         |       |        |           |          |

图 11-1 使能 PS 端 USB 0 资源

USB Phy 的复位部分是接在 MIO 13 管脚上的，所以这里还需要使能 ZYNQ 的 MIO 功能，并且在下方的 USB 0 RESET 选项里勾选 MIO 13 复位方式选择低电平复位。（因为我们没有用到 I2C 功能，所以 I2C 复位的选项去除），如下图所示：

|                                                 |                 |       |              |      |        |     |            |
|-------------------------------------------------|-----------------|-------|--------------|------|--------|-----|------------|
| GPIO                                            |                 |       |              |      |        |     |            |
| > <input checked="" type="checkbox"/> GPIO MIO  | MIO             |       |              |      |        |     |            |
| <input type="checkbox"/> EMIO GPIO (Width)      |                 |       |              |      |        |     |            |
| > <input type="checkbox"/> ENET Reset           |                 |       |              |      |        |     |            |
| > <input checked="" type="checkbox"/> USB Reset | Share reset pin |       |              |      |        |     |            |
| <input checked="" type="checkbox"/> USB0 Reset  | MIO 13          |       |              |      |        |     |            |
| <input type="checkbox"/> USB1 Reset             |                 |       |              |      |        |     |            |
| USB Reset                                       | MIO 13          | reset | LVC MOS 3.3V | slow | enable | out | Active Low |
| > <input type="checkbox"/> I2C Reset            |                 |       |              |      |        |     |            |

图 11-2 设置 USB Phy 的复位

其他 AXI 总线及复位功能并未使用，因此可以去除，去除 AXI 总线及复位的配置如下图所示：

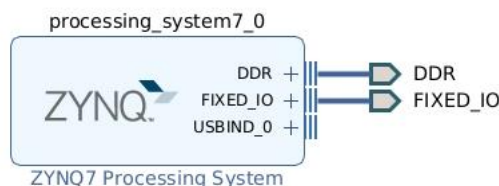


图 11-3 配置完 ZYNQ IP 核

配置完毕后，由于没有使用 PL 端资源，因此可以直接通过 Vivado 生成 XSA 文件，不需要生成 Bit。这部分教程在前面使用的次数比较多，因此在这不详细进行说明。

## 2. Petalinux 中配置

(1) 新建 Petalinux 工程，这与前面的过程是一样的，因此不多重复。只需要配置离线编译包，其他不用配置。

(2) 在 Linux Kernel 中增加 USB 驱动，与 Xilinx WIKI 网站上的内容作对比，Kernel 的配置页默认已经包含了 USB HOST 的所有必须的内容，所以不需要额外再单独配置了。根据 Xilinx WIKI 网站上的介绍，如果需要配置成 USB HOST 模式则需要将 kernel 按照下图进行设置：

```
WARNING
We could not get USB 3.0 to work with the USB Micro-B to A adapter that came with the ZCU102. It only worked with a direct cable from our hard drive enclosure to the Micro-B connector on the eval board without requiring an adaptor. With the adaptor in place it always fell back to USB 2.0.

Configuration
CONFIG_USB_DWC3=y
CONFIG_USB_DWC3_DUAL_ROLE=y
CONFIG_USB_LBCOMPOSITE=m
CONFIG_USB_F_MASS_STORAGE=m
CONFIG_USB_CONFIGFS=m
CONFIG_USB_CONFIGFS_MASS_STORAGE=y
CONFIG_USB_SUPPORT=y
CONFIG_USB_COMMON=y
CONFIG_USB_ARCH_HAS_HCD=y
CONFIG_USB=y
CONFIG_USB_XHCI_HCD=y
CONFIG_USB_XHCI_PLATFORM=y
```

图 11-4 Xilinx WIKI 中推荐的配置 USB 设备树

(3) 在 Petalinux2023.1 中，Linux 内核中 USB 的设备树配置都是默认配置的，因此关于这个设备树是不用配置，保持不变即可，但是可以通过进入 Linux 内核查看配置，进入内核命令如下：

命令为：petalinux-config -c kernel

配置 Linux 内核界面弹出的时间过长，等待界面跳出。找到在 Linux 设备树中对 USB 的配置，观察是否进行配置，一般都是保持默认的，并不需要仔细确认。

(4) Linux 内核配置完毕后，需要对设备树进行编写，主要是编写 USB Phy 以及串口的设备树，设备树内容如下：

```
/include/ "system-conf.dtsi"

/ {
    model = "Zynq RIGUKE Development Board";
    compatible = "riguke,zynq", "xlnx,zynq-7000";
    aliases {
        ethernet0 = "&gem0";
        serial0 = "&uart0";
    };

    usb_phy0: usb_phy@0 {
        compatible = "ulpi-phy";
        #phy-cells = <0>;
        reg = <0xe0002000 0x1000>;
        view-port = <0x0170>;
        drv-vbus;
    };
};
```

```

&uart0 {
    u-boot,dm-pre-reloc;
};

&usb0 {
    dr_mode = "host";
    usb-phy = <&usb_phy0>;
};

```

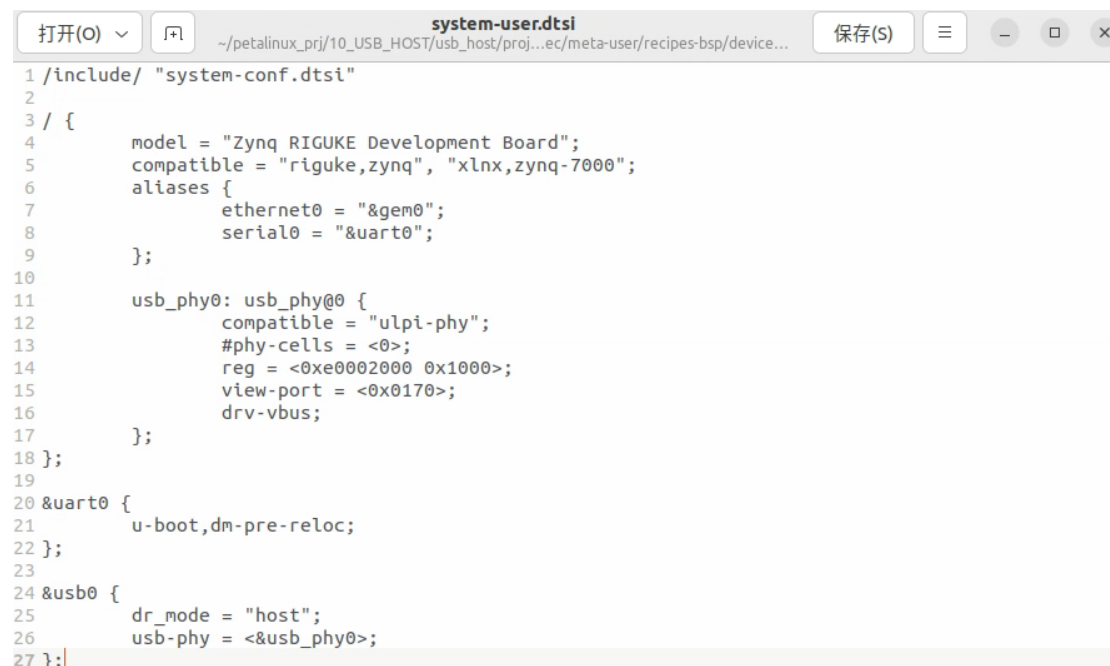


图 11-5 编写 USB 的设备树

需要注意的是，一定要在设备树中编写 USB 的节点，并且在节点中将 USB 配置为 host 模式，不然可能存在 ZYNQ 后检测不到设备的情况。

(5) 编写完设备树后，则可以对整个 Petalinux 工程进行编译，编译完后进行打包，并将 Linux 工程拷贝至 SD 卡中，上述操作在前面 Petalinux 基础操作中有详细介绍，如果不明白可以回到前端 Petalinux 基础操作过程去查找如何进行操作。

### 3. USB HOST 功能测试

测试准备的是一个 64GB 的 U 盘（U 盘大小可以随意，依据自己手中 U 盘进行实验即可），但是需要将 U 盘格式化为 FAT32 格式，但是 64G 的 U 盘在 Windows 下是没法直接格式化为 FAT32 格式的，因此在 Linux 下进行格式化。对整个 U 盘格式化 8GB 的空间为 FAT32 格式。

(1) 将 U 盘插入 Linux 系统中，并在 Linux 系统下搜索 Disk，随后会扫到挂载的 U 盘，挂载的 U 盘大小为 64GB，如下图所示：

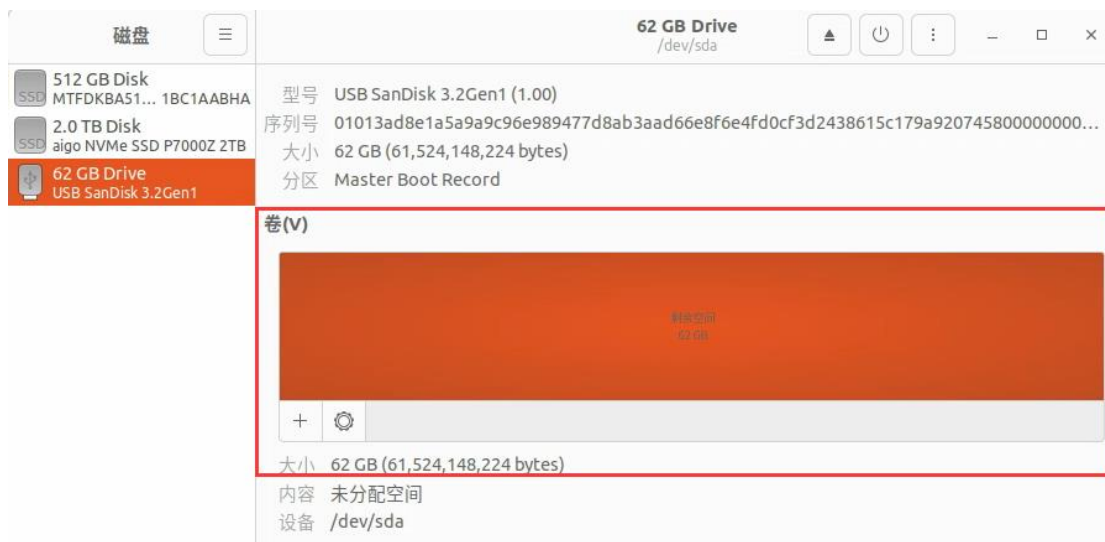


图 11-6 挂载 U 盘

(2) 将 U 盘分配一个 8GB 大小的分区，并选择当前分区的类型，如下图所示：



图 11-7 分配 8GB 大小的分区



图 11-8 选择当前分区的名字以及分区类型



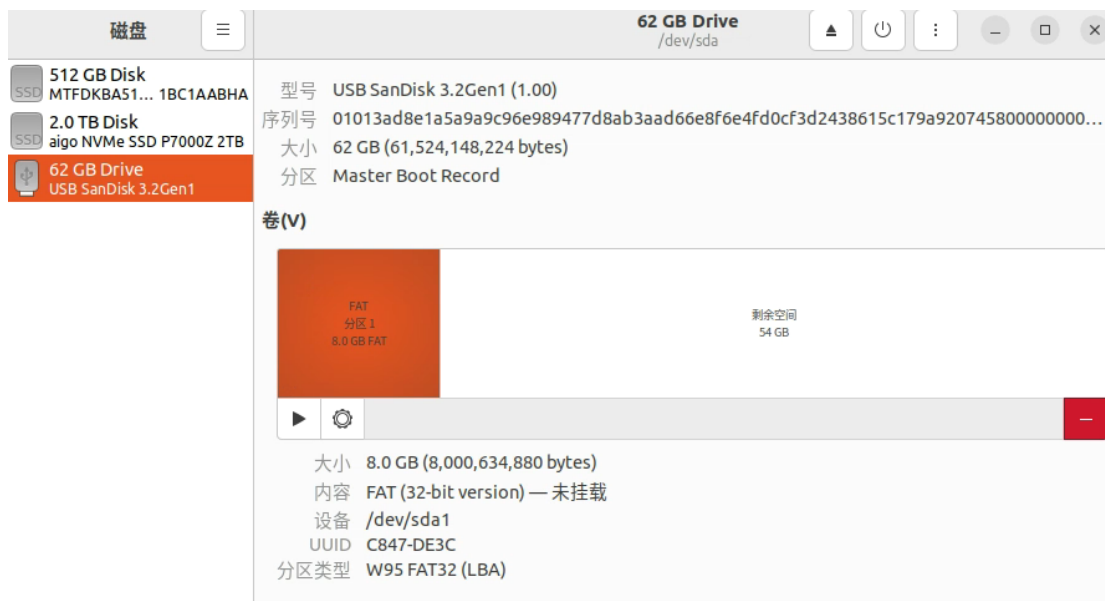


图 11-9 U 盘分配空间完毕

(3) 随后在该分区内新建两个 txt 文件，用于实验测试，txt 文件什么都不用写，是一个空文件，如下图所示：

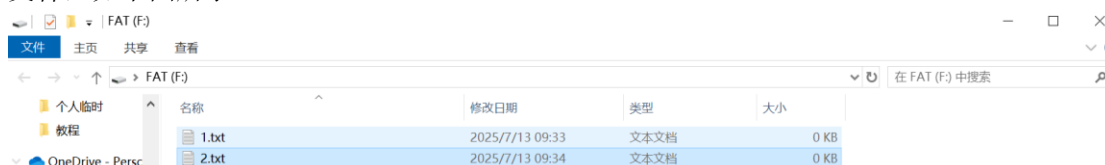


图 11-10 新建空文件

(4) 将 RK ZYNQ 7020 通过 SD 卡启动，等待 Linux 启动，串口打印完毕，随后将 U 盘插入 RK ZYNQ 7020 的 USB HOST 接口，如下图所示：

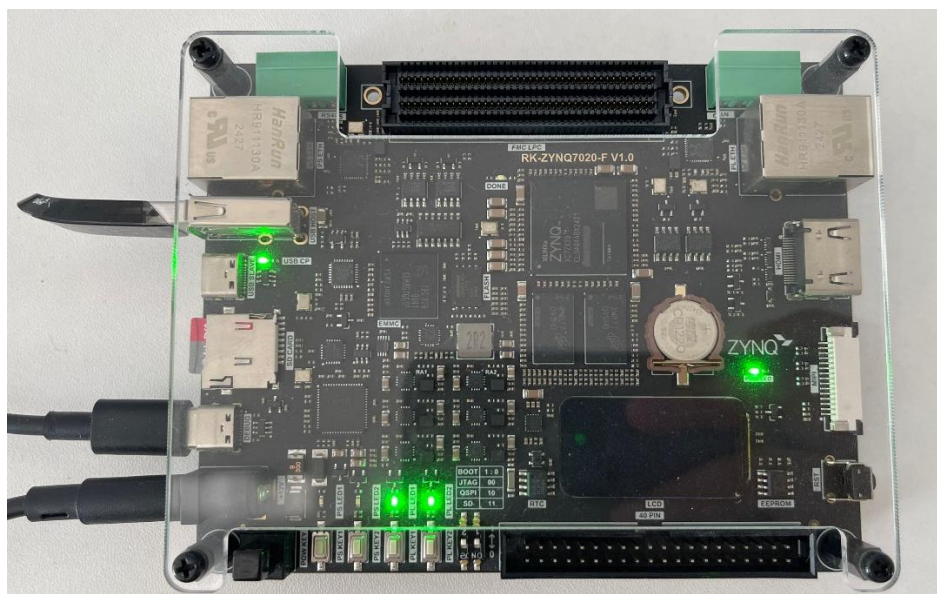


图 11-11 将 U 盘插入开发板



U 盘插入开发板后，串口会打印如下信息，如下图所示：

```

generating ssh ECDSA host key...
generating ssh ED25519 host key...
done.
Starting rpcbind daemon...done.
starting statd: done
hwclock: can't open '/dev/misc/rtc': No such file or directory
Starting internet superserver: inetd.
NFS daemon support not enabled in kernel
Starting syslogd/klogd: done
Starting tcf-agent: OK

Petalinux 2023.1+release-S05010539 usb_host ttyPS0

usb_host login: petalinux
You are required to change your password immediately (administrator enforced).
New password:
Retype new password:
usb_host:~$ usb 1-1: new high-speed USB device number 2 using c_hndrc
usb-storage 1-1:1.0: USB Mass Storage device detected
scsi host0: usb-storage 1-1:1.0
scsi 0:0:0:0: Direct-Access    USB          SanDisk 3.2Gen1 1.00 PQ: 0 ANSI: 6
sd 0:0:0:0: Attached scsi generic sg0 type 0
sd 0:0:0:0: [sda] 120164352 512-byte logical blocks: (61.5 GB/57.3 GiB)
sd 0:0:0:0: [sda] Write Protect is off
sd 0:0:0:0: [sda] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
sda: sda1
sd 0:0:0:0: [sda] Attached SCSI removable disk
ext3: Unknown parameter 'umask'
ext2: Unknown parameter 'umask'
ext4: Unknown parameter 'umask'
  
```

图 11-12 串口打印信息

上述输出的内容中可以看出，当前已经检测到 U 盘信息了，没有硬件层面的信息，最下面的 Unknown 是挂载的分区问题，这个不影响当前的实验。

(5) 在串口终端软件中，输入 `sudo fdisk -l` 可以查看 U 盘的信息，如下图所示：

```

sd 0:0:0:0: [sda] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
sda: sda1
sd 0:0:0:0: [sda] Attached SCSI removable disk
ext3: Unknown parameter 'umask'
ext2: Unknown parameter 'umask'
ext4: Unknown parameter 'umask'

sudo fdisk -l

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

#1) Respect the privacy of others.
#2) Think before you type.
#3) With great power comes great responsibility.

Password:
Sorry, try again.
Password:

Disk /dev/mmcblk0: 30 GB, 31914983424 bytes, 62333952 sectors
122223 cylinders, 255 heads, 2 sectors/track
Units: sectors of 1 * 512 = 512 bytes

Device Boot StartCHS EndCHS StartLBA EndLBA Sectors Size Id Type
/dev/mmcblk0p1 4,4,1 1023,254,2 2048 9768959 9766912 4769M c Win95 FAT32 (LBA)
Disk /dev/sda: 57 GB, 61524148224 bytes, 120164352 sectors
235616 cylinders, 255 heads, 2 sectors/track
Units: sectors of 1 * 512 = 512 bytes

Device Boot StartCHS EndCHS StartLBA EndLBA Sectors Size Id Type
/dev/sda1 4,4,1 1023,254,2 2048 15628287 15626240 7630M c Win95 FAT32 (LBA)
  
```

图 11-13 查看当前的 U 盘信息

也可以通过输入 `df -h` 查看 U 盘 `/dev/sda1` 挂载的目录为 `/run/media/FAT-sda1`。

```

usb_host:~$ df -h
Filesystem      Size      Used Available Use% Mounted on
devtmpfs        487.3M    4.0K    487.3M   0% /dev
tmpfs           502.1M   236.0K   501.9M   0% /run
tmpfs           502.1M    52.0K   502.1M   0% /var/volatile
/dev/mmcblk0p1   4.6G     18.1M    4.6G    0% /run/media/FAT-mmcblk0p1
/dev/sda1        7.4G     16.0K    7.4G    0% /run/media/FAT-sda1
usb_host:~$
  
```

图 11-14 查看当前的 U 盘挂载目录

(6) 按照上面的图上 U 盘的位置, 输入 `cd /run/media/FAT-sda1` 进入 U 盘或者 `cd /dev/sda1` 也可以进入 U 盘。如果提示没有权限, 可以采用 root 模式, 如下图所示:

```

Quick connect...
User sessions
COM4 (USB Ser
Device Boot StartCHS EndCHS StartLBA EndLBA Sectors Size Id Type
/dev/mmcblk0p1 4,4,1 1023,254,2 2048 9768959 9766912 4769M c Win95 FAT32 (LBA)
Disk /dev/sda: 57 GB, 61524148224 bytes, 120164352 sectors
235616 cylinders, 255 heads, 2 sectors/track
Units: sectors of 1 * 512 = 512 bytes

Device Boot StartCHS EndCHS StartLBA EndLBA Sectors Size Id Type
/dev/sda1 4,4,1 1023,254,2 2048 15628287 15626240 7630M c Win95 FAT32 (LBA)
usb_host::~$ df -h
Filesystem Size Used Available Use% Mounted on
devtmpfs 487.3M 4.0K 487.3M 0% /dev
tmpfs 502.1M 236.0K 501.9M 0% /run
tmpfs 502.1M 52.0K 502.1M 0% /var/volatile
/dev/mmcblk0p1 4.6G 18.1M 4.6G 0% /run/media/FAT-mmcblk0p1
/dev/sda1 7.4G 16.0K 7.4G 0% /run/media/FAT-sda1
usb_host::~$ cd /run/media/FAT-sda1/
-sh: cd: /run/media/FAT-sda1/: Permission denied
usb_host::~$ cd /dev/
Display all 130 possibilities? (y or n)
usb_host::~$ cd /dev/
usb_host:/dev$ cd /dev/s
sda sda1 sg0 shm/ snd/
usb_host:/dev$ cd /dev/sda
sda sda1
usb_host:/dev$ cd /dev/sda1
-sh: cd: /dev/sda1: Not a directory
usb_host:/dev$ sudo -i
Password:
root@usb_host::~$ cd /run/media/FAT-sda1/
root@usb_host:/run/media/FAT-sda1#
  
```

图 11-15 进入 U 盘目录

(7) 随后在 U 盘目录中, 输入 `ls` 检索文件夹下的信息, 可以看到新建的两个 TXT 文件, 如下图所示:

```

root@usb_host:/run/media/FAT-sda1# ls
1.txt      2.txt
root@usb_host:/run/media/FAT-sda1#
  
```

图 11-16 进入 U 盘目录

从上图中可以看出, 文件夹下有 1.txt 和 2.txt 两个文件, 这之前新建的 TXT 文件是一致的。

(8) 在 `mnt` 文件夹下创建一个 USB 目录, 命令如下:

命令为: `mkdir -p /mnt/USB`

输入指令后, 终端中并不会有什么输出, 只是在 `mnt` 中新建了一个名为 USB 的目录, 如下图所示:

```

root@usb_host:/run/media/FAT-sda1# mkdir -p /mnt/USB
root@usb_host:/run/media/FAT-sda1#
  
```

图 11-17 新建一个名为 USB 的目录

(9) 将 `/dev/sda1` 也就是 U 盘挂载到 `/mnt/USB` 文件夹下, 命令如下:

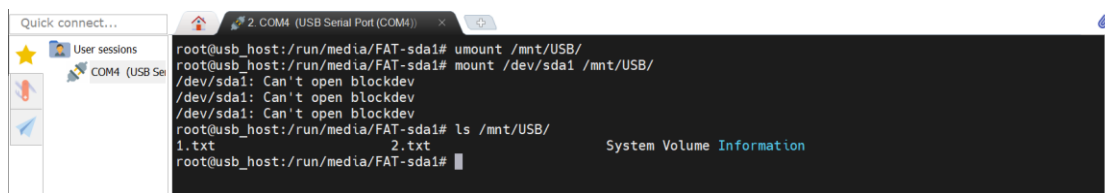
命令为: `mount /dev/sda1 /mnt/USB`

```

root@usb_host:/run/media/FAT-sda1# umount /mnt/USB/
root@usb_host:/run/media/FAT-sda1# mount /dev/sda1 /mnt/USB/
/dev/sda1: Can't open blockdev
/dev/sda1: Can't open blockdev
/dev/sda1: Can't open blockdev
root@usb_host:/run/media/FAT-sda1#
  
```

图 11-18 将 U 盘挂载到 `/mnt/USB` 文件夹

(10) 用 ls 检索挂载的目录，可以看到目录下存在 1.txt 和 2.txt 证明挂载成功了，如下图所示：



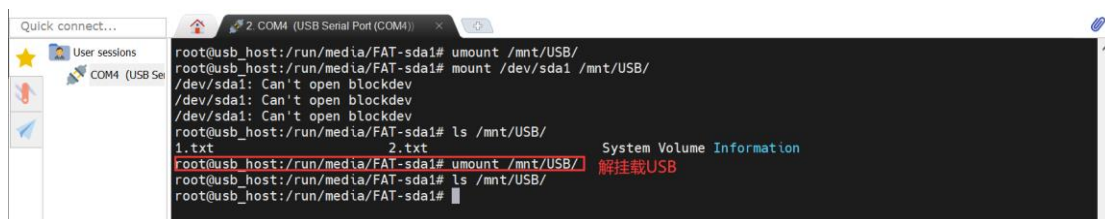
```

root@usb_host:/run/media/FAT-sda1# umount /mnt/USB/
root@usb_host:/run/media/FAT-sda1# mount /dev/sda1 /mnt/USB/
/dev/sda1: Can't open blockdev
/dev/sda1: Can't open blockdev
/dev/sda1: Can't open blockdev
root@usb_host:/run/media/FAT-sda1# ls /mnt/USB/
1.txt      2.txt      System Volume Information
root@usb_host:/run/media/FAT-sda1#

```

图 11-19 在挂载目录中看到存在 U 盘中的文件

(11) 解挂载用 umount 指令解除之前的挂载，之后再用 ls 命令已经看不到 U 盘下的文件了，说明解除挂载成功，如下图所示：



```

root@usb_host:/run/media/FAT-sda1# umount /mnt/USB/
root@usb_host:/run/media/FAT-sda1# mount /dev/sda1 /mnt/USB/
/dev/sda1: Can't open blockdev
/dev/sda1: Can't open blockdev
/dev/sda1: Can't open blockdev
root@usb_host:/run/media/FAT-sda1# ls /mnt/USB/
1.txt      2.txt      System Volume Information
root@usb_host:/run/media/FAT-sda1# umount /mnt/USB/
root@usb_host:/run/media/FAT-sda1# ls /mnt/USB/
root@usb_host:/run/media/FAT-sda1#

```

图 11-20 解挂载 USB 并验证其中文件内容

以上是本次用 U 盘来验证开发板的 USB HOST 功能，实际上 USB HOST 还可以接鼠标和键盘之类的外设，如果有兴趣可以自行挖掘如何使用。